Designing Data-Intensive Applications — Vol 2 (Ch 7–14)

Martin Kleppmann: beginner-friendly study notes

8 parts · every chapter explained in plain English, with redrawn figures

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes written from the book. Figure numbers refer to the book. Not for redistribution.

Contents

1. **Chapter 7 — Transactions**
   - Overview
   - Key terms
   - The Slippery Concept of a Transaction
   - Weak Isolation Levels
   - Serializability
   - Chapter summary
   - Interview angle
   - Self-check
2. **Chapter 8 — The Trouble with Distributed Systems**
   - Overview
   - Key terms
   - Faults and Partial Failures
   - Unreliable Networks
   - Unreliable Clocks
   - Knowledge, Truth, and Lies
   - Chapter summary
   - Interview angle
   - Self-check
3. **Chapter 9 — Consistency and Consensus**
   - Overview
   - Key terms
   - Consistency Guarantees
   - Linearizability
   - Ordering Guarantees
   - Distributed Transactions and Consensus
   - Chapter summary
   - Interview angle
   - Self-check
4. **Chapter 10 — Batch Processing**
   - Overview
   - Key terms
   - Batch Processing with Unix Tools
   - MapReduce and Distributed Filesystems
   - Beyond MapReduce
   - Chapter summary
   - Interview angle
   - Self-check
5. **Chapter 11 — Stream Processing**
   - Overview
   - Key terms
   - Transmitting Event Streams
   - Databases and Streams
   - Processing Streams
   - Chapter summary
   - Interview angle
   - Self-check
6. **Chapter 12 — The Future of Data Systems**
   - Overview
   - Key terms
   - Data Integration
   - Unbundling Databases
   - Aiming for Correctness
   - Doing the Right Thing
   - Chapter summary
   - Interview angle
   - Self-check
7. **Glossary**
   - Every other term used in these notes (A–Z)
8. **Chapter 14 — Interview Question Bank**
   - How to use this bank
   - Reliability, scalability and maintainability
   - Data models and query languages
   - Storage engines and indexes (LSM vs B-tree)
   - OLTP vs OLAP and column storage
   - Encoding and schema evolution
   - Replication
   - Partitioning (sharding)
   - Transactions and isolation anomalies
   - The trouble with distributed systems
   - Consistency and consensus
   - Batch processing
   - Stream processing, CDC and event sourcing
   - Exactly-once, idempotency and data integration
   - Scenario and design questions
   - Sources of the question lists

Chapter 7 — Transactions

Book: Chapter 7, pp. 221–272

Overview

The question: when many things can fail (crashes, network cuts, users touching the same data), how can a database let applications act as if most of those problems don't happen? The industry's long-standing answer is the **transaction**.

What goes wrong in real data systems:

- Database software or hardware can fail at any moment, even mid-write.
- The application can crash partway through a series of operations.
- The network can cut the app off from the database, or one database node from another.
- Several clients can write at once and overwrite each other's changes.
- A client can read half-updated, meaningless data.
- Race conditions between clients cause odd bugs.

Hand-building fault tolerance for all this takes careful thought and testing. A **transaction** groups reads and writes into one logical unit that either **commits** (succeeds) or **aborts** / **rolls back** (fails, leaving no trace), so the app can simply retry — no **partial failure**.

Framing points:

- Transactions were *designed* to simplify programming; their **safety guarantees** let the database handle certain faults and concurrency problems for the app.
- Not every app needs them; sometimes they are weakened or dropped for performance or availability, and some safety properties are achievable without them.
- Deciding requires knowing exactly what they guarantee and cost; the details are subtle.
- Most of the chapter covers **concurrency control**: the race conditions possible and the isolation levels that prevent them (read committed, snapshot isolation, serializability).
- Applies to single-node and distributed databases alike; Chapter 8 covers distributed-only problems.

The epigraph (Google Spanner paper, 2012): better for programmers to deal with performance problems from *overusing* transactions than to always code around their *absence*.

Key terms

Term	Plain-English meaning
Transaction	Group of reads and writes treated as one unit: all or nothing.
Commit / abort (rollback)	Succeeded, writes permanent and visible / failed, all writes discarded.
Partial failure	Some operations in a group succeeded, others not.
Safety guarantees	Promises (atomicity, isolation…) letting the app ignore certain errors.
ACID / BASE	Atomicity, Consistency, Isolation, Durability (coined 1983) / Basically Available, Soft state, Eventual consistency — effectively "not ACID".
Atomicity (ACID)	If a transaction can't finish, all its writes are undone ("abortability").
Consistency (ACID)	Application-defined **invariants** (statements that must always hold) stay true — an app property.
Isolation (ACID)	Concurrent transactions don't interfere.
Serializability	Usually the strongest isolation: same result as some one-at-a-time order.
Durability (ACID)	Committed data isn't lost even if the machine crashes.
Write-ahead log (WAL)	Append-only log of changes used for crash recovery.
Race condition	Bug depending on unlucky timing of concurrent operations.
Multi-object transaction / single-object operation	Changes several rows/documents together / atomic change to one object.
Denormalization	Storing a redundant copy or derived value (e.g. counter) that must be kept in sync.
Compare-and-set (CAS)	Write only if the value is still what you read.
Isolation level	How strongly concurrent transactions are separated.
Read uncommitted / read committed	Prevents dirty writes only / prevents dirty reads and dirty writes.
Dirty read / dirty write	Seeing / overwriting another transaction's uncommitted data.
Row-level lock	Lock on one row/document held by a writer until commit or abort.
Read skew (nonrepeatable read)	Reading parts of the database at different moments, giving an inconsistent picture.
Snapshot isolation	Each transaction reads a consistent snapshot as of its start.
MVCC	Multi-version concurrency control: several committed versions per object.
Transaction ID (txid)	Unique, ever-increasing number per transaction.
Garbage collection (MVCC)	Removing old versions no one can see.
Append-only / copy-on-write B-tree	B-tree never overwriting pages; each write creates new pages and a new root.
Repeatable read	SQL-standard level name, meaning varies by vendor (often snapshot isolation).
Lost update / read-modify-write cycle	Concurrent read-modify-write cycles where one clobbers the other / read, compute in app, write back.
Atomic write operation	Database-provided safe update (e.g. `value = value + 1`).
Cursor stability	Atomic updates via an exclusive lock taken at read time.
Explicit locking	App locks specific rows (e.g. `SELECT ... FOR UPDATE`).

Term	Plain-English meaning
Siblings	Conflicting versions kept side by side in a replicated DB, merged later.
Commutative operation	Result independent of order (increment, add to set).
Last write wins (LWW)	Keep only the "latest" write — loses updates.
Write skew	Two transactions read the same data, then update *different* objects, together breaking a rule.
Phantom	One transaction's write changes the result of another's search.
Materializing conflicts	Creating artificial rows just to have something to lock.
Actual serial execution	Running one transaction at a time on one thread.
Stored procedure	Transaction code submitted to and run inside the database.
OLTP	Online transaction processing: many short user-facing reads/writes.
Anti-caching	Abort a transaction needing on-disk data, load it in the background, restart.
Cross-partition transaction	Touches several partitions; needs coordination.
Two-phase locking (2PL)	Readers and writers block each other; locks held until transaction end.
Shared / exclusive lock	Many holders (readers) / one holder blocking all others.
Deadlock	Two transactions each waiting for the other's lock; the DB aborts one.
Predicate lock / index-range (next-key) lock	Lock on all objects matching a condition, including future ones / cheaper approximation attached to an index range.
Pessimistic / optimistic concurrency control	Wait whenever something might go wrong (2PL, serial) / proceed, check at commit, abort if needed.
Serializable snapshot isolation (SSI)	Optimistic algorithm on snapshot isolation that detects conflicts and aborts to ensure serializability.
Premise	A fact a transaction read and acted on (e.g. "two doctors on call").
Contention	Many transactions competing for the same objects.

The Slippery Concept of a Transaction

Nearly all relational and some non-relational databases support transactions in a style from **IBM System R** (**1975**, the first SQL database); **MySQL, PostgreSQL, Oracle and SQL Server** transactions still closely resemble it 40 years later.

Late-2000s **NoSQL** databases brought new data models (Chapter 2) and built-in replication (Chapter 5) and partitioning (Chapter 6), but often dropped transactions or redefined the word as something much weaker.

Two exaggerated views followed: "transactions kill scalability; large systems must abandon them for performance and availability" vs "serious applications with valuable data *need* transactions" (some vendors' line). The book calls both hype: transactions have benefits and limits that must be weighed in detail.

The Meaning of ACID

ACID = **Atomicity, Consistency, Isolation, Durability**, coined by **Theo Härder and Andreas Reuter** in **1983** to name database fault-tolerance mechanisms.

> **Common confusion:** implementations differ widely, especially on isolation, so "ACID compliant" says little — largely a marketing term.

> **BASE.** Non-ACID systems are sometimes called BASE (*Basically Available, Soft state, Eventual consistency*), which is even vaguer; it reliably means only "not ACID".

Atomicity

What it is. "Atomic" means indivisible, but differs by field:

- In **multi-threaded programming**, an atomic operation can't be seen half-done by another thread.
- In **ACID**, atomicity is **not** about concurrency (that's isolation) but about **a fault occurring after only some of a client's writes were applied**.

How it works. A client groups writes into a transaction; partway through, something fails (process crash, network drop, full disk, violated integrity constraint such as uniqueness); the transaction can't commit, so the database aborts it and **discards or undoes all its writes so far**.

Why it matters. Without atomicity you can't tell which changes landed, and retrying might duplicate or corrupt data. With it, an aborted transaction changed nothing, so retrying is safe. The book suggests **"abortability"** as a better name.

Consistency

"Consistency" is heavily overloaded:

Where	What "consistency" means there
Replication (Chapter 5)	Replica consistency, and *eventual consistency* in asynchronously replicated systems.
Partitioning (Chapter 6)	*Consistent hashing*, a rebalancing technique.
CAP theorem (Chapter 9)	Linearizability (every read sees the latest write, as if one copy).
ACID	The database being in an application-defined "good state".

ACID consistency. You have **invariants**, e.g. in accounting, credits and debits across all accounts must balance. If each transaction starts valid and preserves validity, invariants always hold.

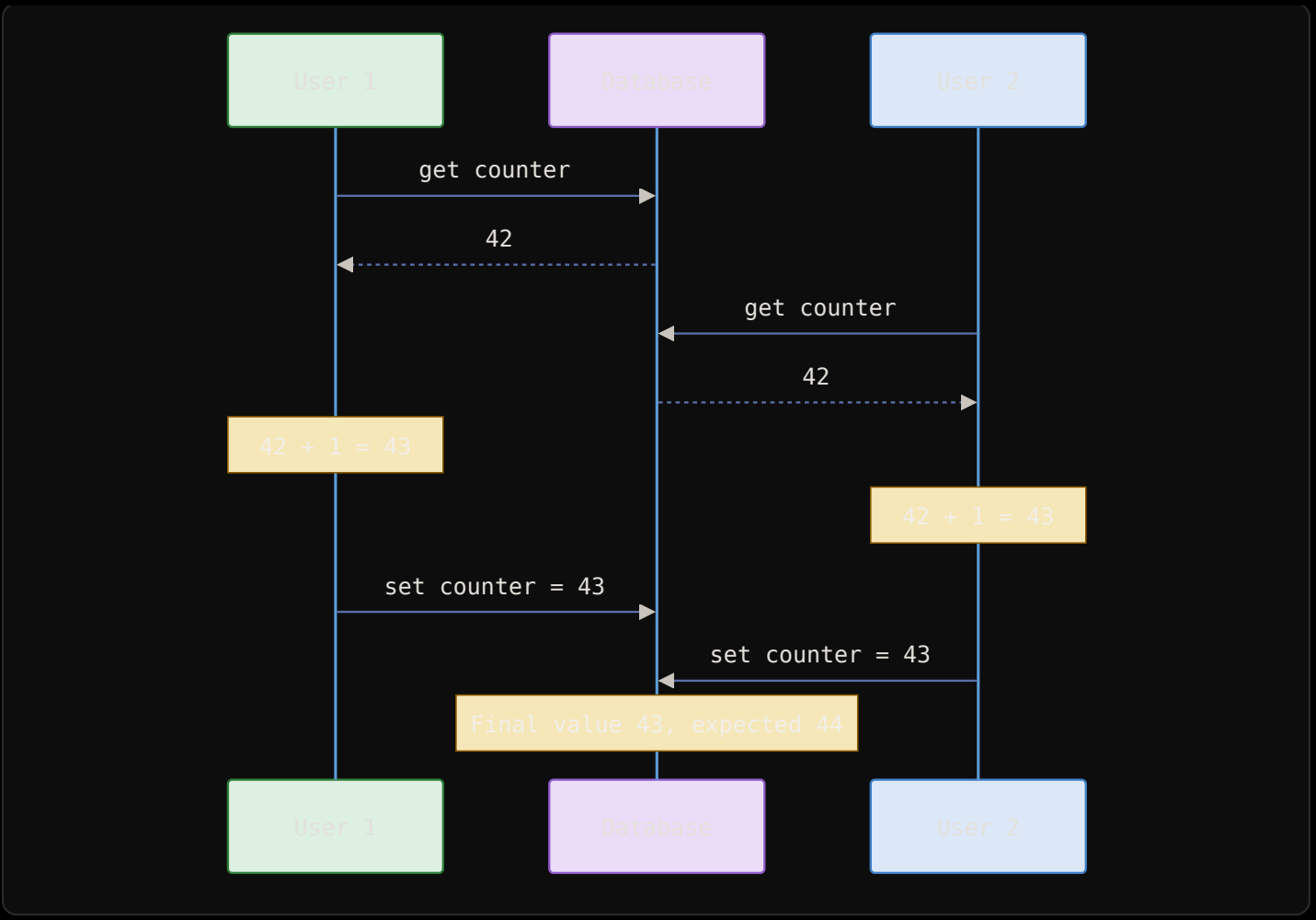
The catch. Only the **application** knows its invariants and must write transactions that preserve them; the database can't stop data that violates your rules. Some rules can be enforced by the database (**foreign key constraints**, **uniqueness constraints**), but generally the app defines validity. So A, I and D are **database properties**, while consistency is an **application property** relying on A and I — the "C" doesn't really belong. (Joe Hellerstein remarked that the C was added mainly to make the acronym work and wasn't considered important in the original paper.)

Isolation

The problem. Clients accessing the **same records** concurrently cause **race conditions**.

The book's example (Figure 7-1). Two clients increment a counter by reading, adding 1 in code, and writing back (no built-in increment). From 42, two increments should give 44, but both read 42 first, so it ends at **43**.

Figure 7-1 (redrawn): two clients increment the same counter concurrently and one increment is lost



ACID isolation: concurrent transactions don't interfere. Textbooks formalise it as **serializability**: each transaction can act as if it runs alone, and the committed result equals *some* serial order, though they actually overlapped.

Common confusion. Serializable isolation is **rarely used** in practice due to its performance cost; some databases lack it: **Oracle 11g**'s "serializable" is actually the weaker **snapshot isolation** (covered below).

Durability

What it is. Once committed, data won't be forgotten despite hardware failure or crashes.

How: on a **single node**, written to **non-volatile storage** (hard drive or SSD), usually with a **write-ahead log** (Chapter 3) to recover corrupted structures; in a **replicated** database, copied to **some number of nodes**. Either way, the database must **wait** for this before reporting a commit.

Limitation. **Perfect durability doesn't exist**: if all disks and backups are destroyed at once, nothing helps.

Replication and Durability (sidebar). Durability historically meant archive tape, then disk/SSD, now replication. None is perfect:

- A dead machine's disk is safe but *unreachable* until repaired or moved; replicated systems stay available.
- A **correlated fault** (power outage, a bug crashing all nodes on one input) can take out all replicas at once, losing anything only in memory — so disk writes matter even for in-memory databases.
- With **asynchronous replication**, recent writes can be lost if the leader fails.
- On sudden power loss **SSDs** sometimes violate their guarantees; even `fsync` isn't guaranteed to work correctly; disk **firmware has bugs**.

- Subtle storage-engine/**filesystem** interactions can cause hard-to-find bugs and corrupted files after a crash.
- Disk data can **silently rot**; if unnoticed, replicas and recent backups may be corrupt too, requiring an older backup.
- One study: **30% to 80% of SSDs** develop at least one bad block in their **first four years**; magnetic disks have fewer bad sectors but fail completely more often.
- An **unpowered** SSD may start losing data within **a few weeks**, depending on temperature.

Takeaway: no absolute guarantees, only combinable risk-reduction techniques (disk writes, remote replication, backups). Treat theoretical "guarantees" sceptically.

Single-Object and Multi-Object Operations

For **several writes in one transaction**: **atomicity** = all-or-nothing on failure; **isolation** = another transaction sees **all or none** of the writes, never a subset. These assume a **multi-object transaction**, needed whenever several pieces of data must stay in sync.

The book's email example. Show a user's unread count with a query (ours):

```sql
SELECT COUNT(*) FROM messages
WHERE owner_id = 7 AND is_read = false;
```

With many messages this is too slow, so you **denormalize** with an `unread_count` field, incremented on arrival and decremented on read — now two things must change together.

Figure 7-2 — isolation violated. User 1 inserts an email then increments the counter; between those writes, User 2 sees the new unread email but a counter of 0 — a **dirty read**. Isolation guarantees both changes or neither. (Trivial for a counter; serious for an account balance and a payment.)

Figure 7-2 (redrawn): a reader sees the new email but not the updated counter (dirty read)

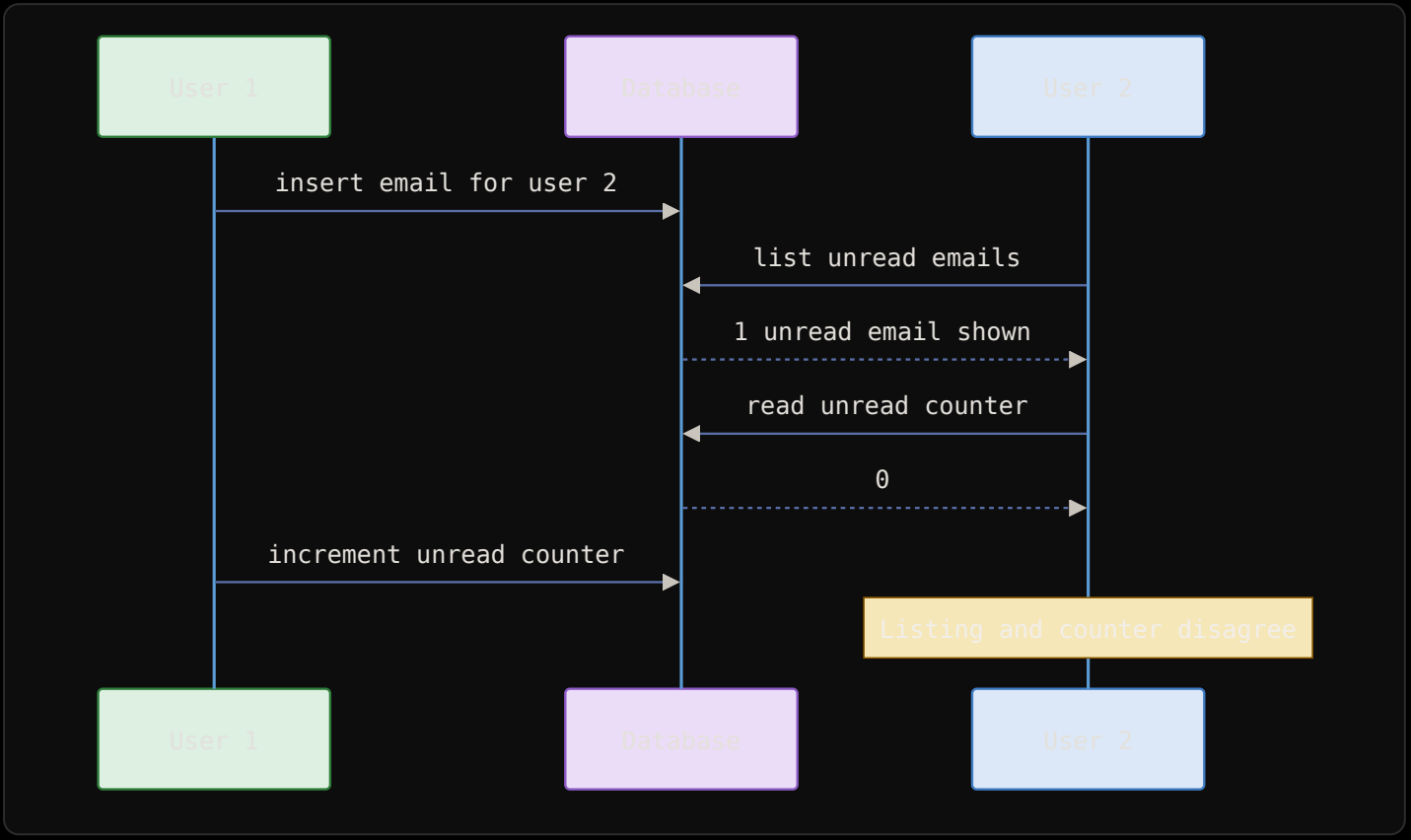
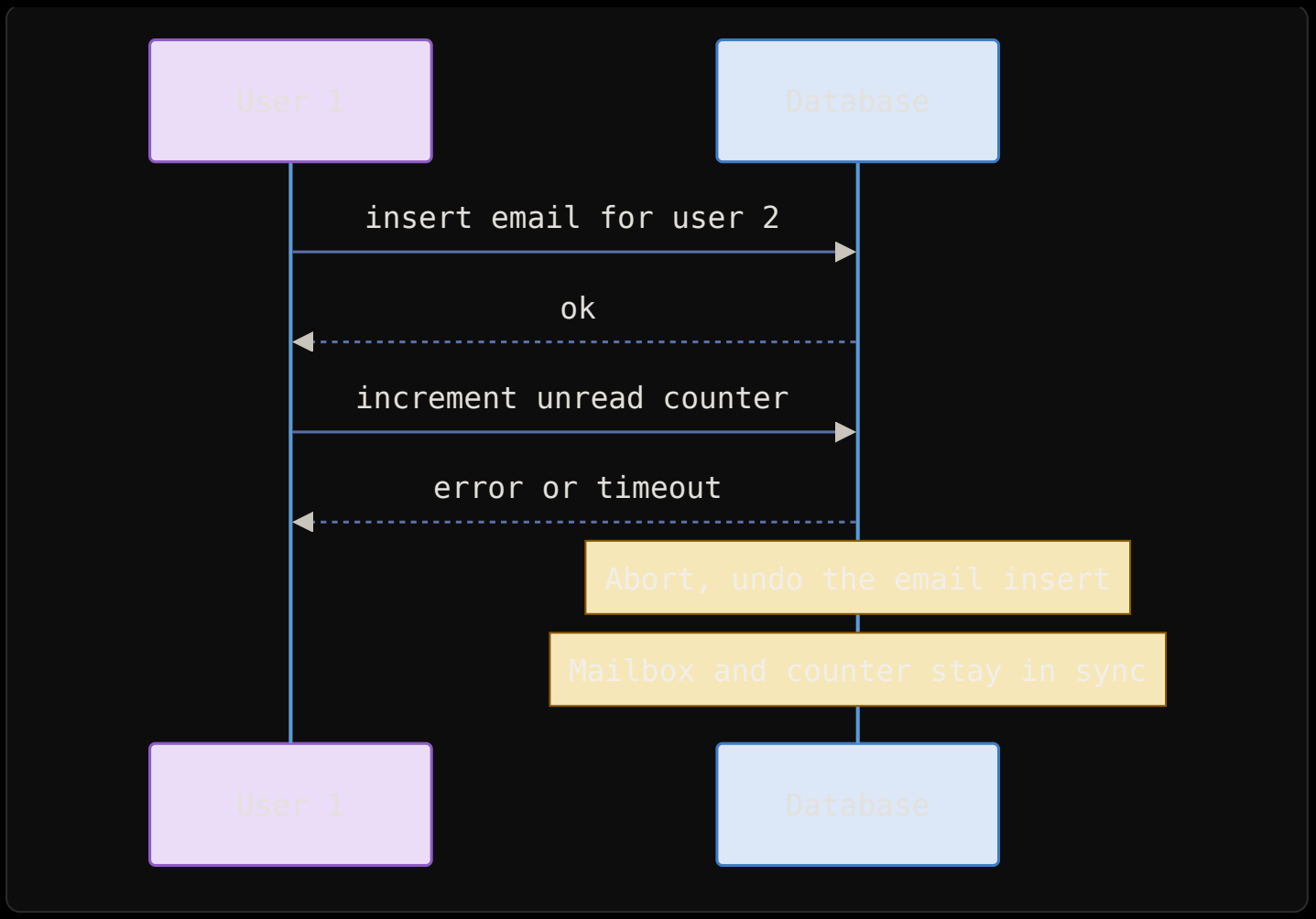


Figure 7-3 — atomicity needed. The email insert succeeds but the counter update fails; with atomicity the whole transaction aborts and the insert is **rolled back**, keeping them in sync.

Figure 7-3 (redrawn): an error during the counter update causes the earlier email insert to be undone



Grouping operations. In relational databases a transaction is usually tied to the client's **TCP connection**: everything between `BEGIN TRANSACTION` and `COMMIT` on that connection. (Not ideal: if the connection drops after the commit request but before confirmation, the client can't tell whether it committed. A transaction manager can use a connection-independent **transaction ID** — Chapter 12, "The End-to-End Argument for Databases".)

Many **non-relational** databases can't group operations; even a **multi-put** may lack transaction semantics, leaving some keys updated and others not.

Single-object writes

Atomicity and isolation matter even for **one** object. Writing a **20 KB JSON document**: what if the network drops after **10 KB**, power fails mid-overwrite (old and new bytes spliced?), or another client reads mid-write? So storage engines almost always give **single-object atomicity** (via a **crash-recovery log**, Chapter 3) and **isolation** (via a **per-object lock**, one thread at a time) on one node.

Some databases add **more complex single-object atomic operations**: an **increment** (avoiding Figure 7-1's read-modify-write), and **compare-and-set** (write only if unchanged). These prevent lost updates, but calling them **transactions** — as vendors do with "lightweight transactions" or even "ACID" — is misleading: a transaction normally groups **several operations on several objects**. (Strictly, "atomic increment" uses the threading sense; in ACID terms it would be *isolated* or *serializable* — a nitpick.)

The need for multi-object transactions

Many distributed datastores dropped multi-object transactions because they are **hard across partitions** and can hurt very-high-availability or performance needs — but distributed transactions aren't *impossible* (Chapter 9). Cases needing several objects to change together:

1. **Foreign keys / graph edges:** records referencing each other must have correct references.
2. **Denormalized documents:** single-document updates need no multi-object transaction, but document databases without joins encourage denormalization, and updating copies (like Figure 7-2) touches several documents.

3. **Secondary indexes:** must be updated with every value change; each index is a separate object, so without isolation a record may appear in one index but not another.

These apps can work without transactions, but error handling gets much harder without atomicity, and missing isolation causes concurrency bugs (next; Chapter 12 explores alternatives).

Handling errors and aborts

Philosophy. Transactions **can be aborted and safely retried**; ACID databases abandon a transaction entirely rather than leave it half done.

Not universal. **Leaderless replication** datastores (Chapter 5) work on a **"best effort"** basis: they won't undo what already happened after an error; recovery is the app's job.

Common mistake. Developers code for the happy path. Popular **ORM** frameworks (**ActiveRecord** in Rails, **Django**) **don't retry** aborted transactions: the exception bubbles up, user input is lost, and an error page is shown — defeating the purpose of aborts.

Retrying is useful but imperfect (also see System Design Fundamentals):

Problem with retry	Why / what to do
The commit actually succeeded, but the acknowledgement was lost on the network	Retrying runs it twice, unless the app has its own **deduplication**.
The error was caused by **overload**	Retrying makes it worse. Limit retries, use **exponential backoff** (wait 1 s, then 2 s, then 4 s, … between attempts), and handle overload errors specially.
The error is **permanent** (e.g. constraint violation)	Retrying is pointless. Only retry **transient** errors: deadlock, isolation violation, brief network glitches, failover.
The transaction has **side effects outside the database** (e.g. sending an email)	They may happen even if it aborts; don't resend on each retry. For multi-system commit/abort, see **two-phase commit** (Chapter 9).
The **client process dies** while retrying	Whatever it was trying to write is lost.

Weak Isolation Levels

When concurrency issues arise. Transactions on disjoint data can run in parallel safely; trouble starts when one **reads data another is modifying**, or both **modify the same data**.

Why they're nasty. **Hard to find by testing** (only with unlucky, rare, hard-to-reproduce timing) and **hard to reason about**, especially in large codebases where you don't know what else touches the data.

Serializable isolation would let you ignore concurrency entirely, but costs performance, so many systems use **weaker isolation levels** that prevent some issues but not all — harder to understand, causing subtle bugs, yet widely used.

Real damage. Weak-isolation bugs have caused **large financial losses** (e.g. bitcoin stolen from the Poloniex exchange in 2014), **financial auditor investigations**, and **corrupted customer data**. "Use an ACID database for money!" misses the point: many popular relational databases default to weak isolation and wouldn't have prevented these bugs.

So **understand the kinds of race conditions** and how to prevent them. The book covers weak (non-serializable) levels informally; rigorous definitions are in the literature.

Read Committed

The most basic level. Two guarantees:

1. **No dirty reads:** reads see only committed data.
2. **No dirty writes:** writes overwrite only committed data.

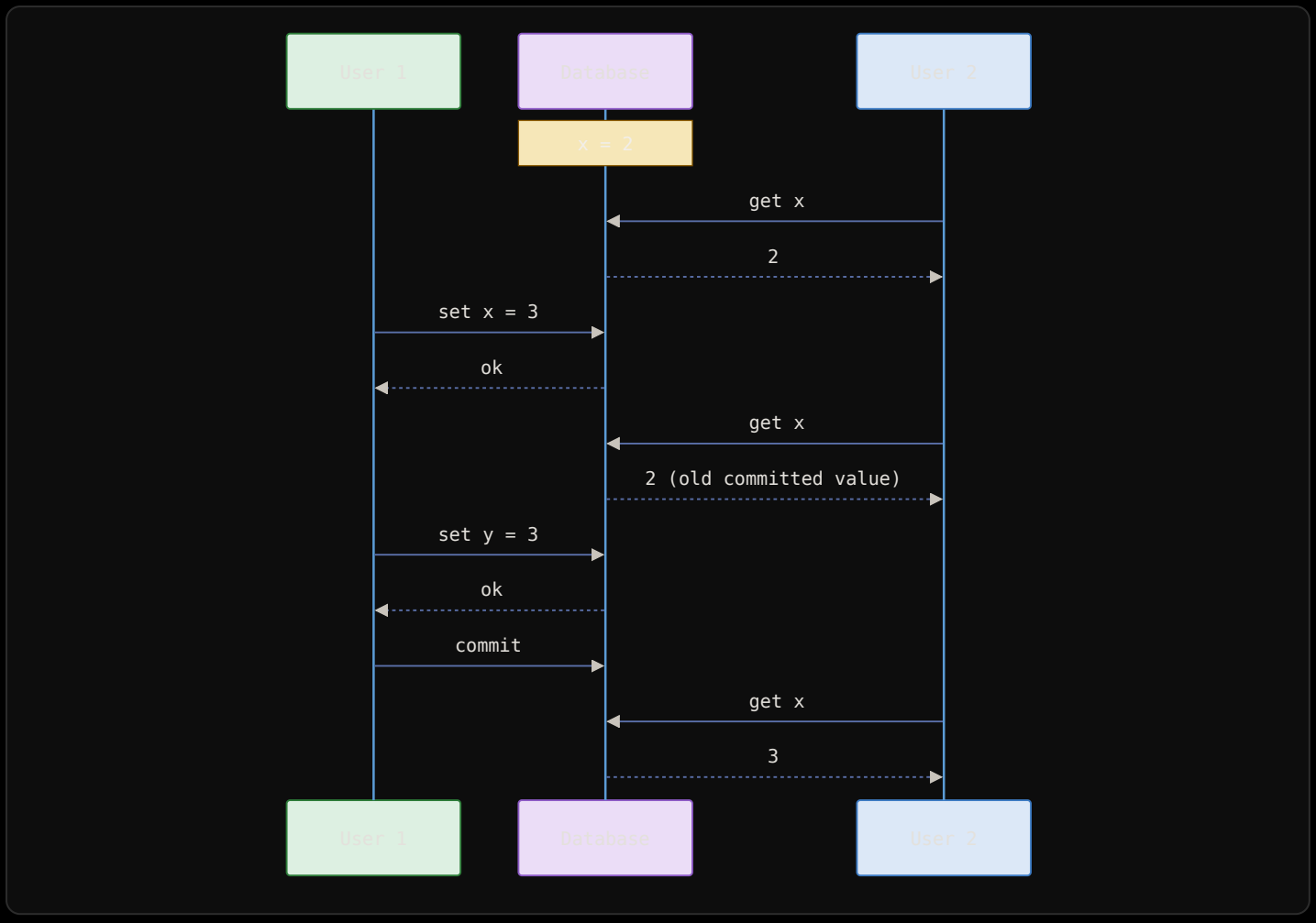
(Some databases offer a weaker **read uncommitted**, preventing dirty writes but allowing dirty reads.)

No dirty reads

What it is. A **dirty read** sees another transaction's uncommitted writes. Under read committed, writes become visible **only at commit, all at once**.

Figure 7-4. User 1 sets `x = 3` and `y = 3` without committing; User 2 keeps reading the old value **2** until User 1 commits.

Figure 7-4 (redrawn): no dirty reads, the new x becomes visible to user 2 only once user 1 commits



Why prevent them? A dirty read can show **some of a transaction's updates but not others** (Figure 7-2's mismatch), confusing users and misleading other transactions. And if the transaction **aborts** (Figure 7-3), a dirty reader saw data **never committed**, which is very hard to reason about.

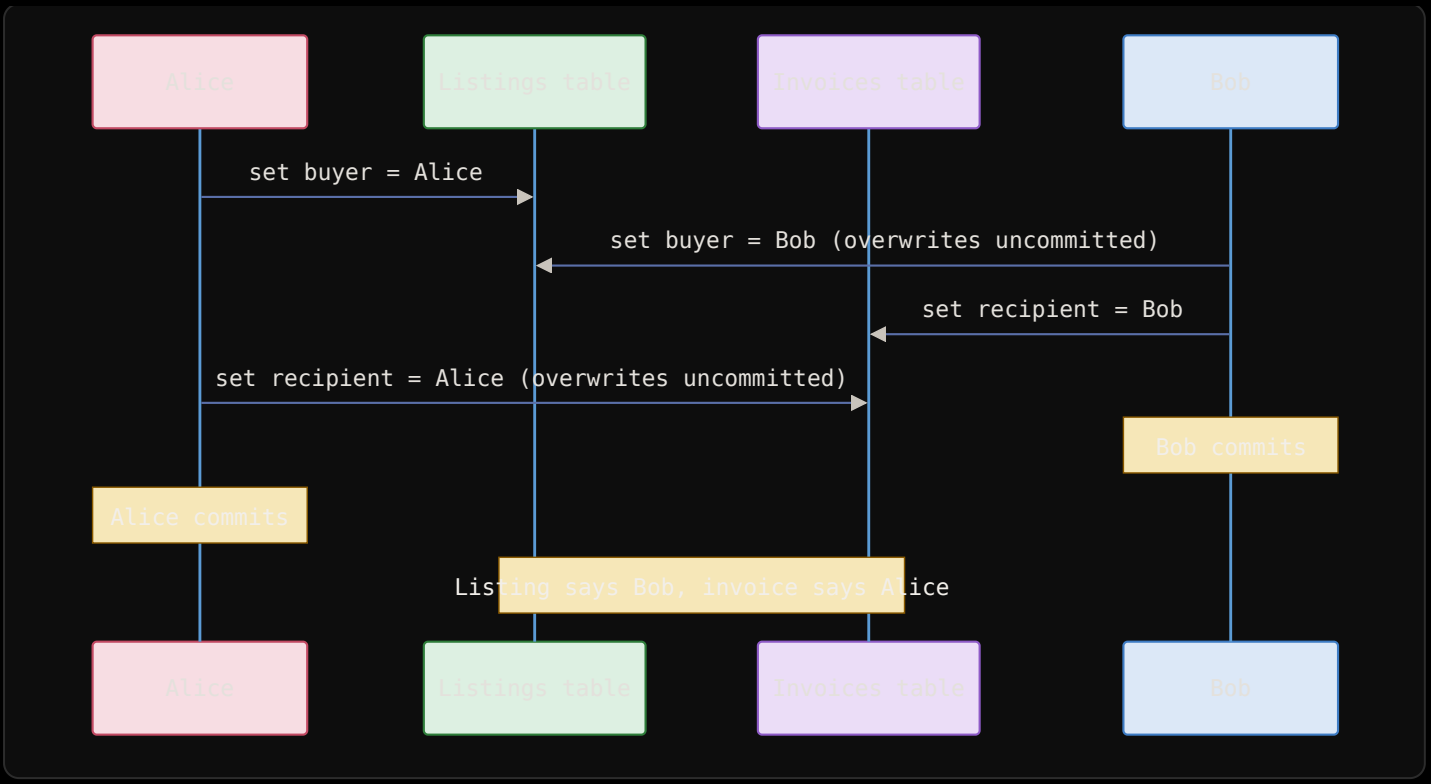
No dirty writes

What it is. A **dirty write** overwrites a value written by a transaction that **hasn't committed**. Read committed prevents it, usually by **delaying the second write** until the first transaction commits or aborts.

What it prevents: **mixing up multi-object writes.** Book example (Figure 7-5): Alice and Bob buy the same car; a purchase updates the **listing** and sends the **invoice**. With dirty writes, Bob wins the listing but Alice gets the invoice.

What it doesn't prevent: Figure 7-1's counter race — the second write comes after the first transaction commits, so it isn't a dirty write; it's a **lost update** (below).

Figure 7-5 (redrawn): with dirty writes, Bob ends up with the listing while Alice gets the invoice



Implementing read committed

Very popular: the default in **Oracle 11g**, **PostgreSQL**, **SQL Server 2012**, **MemSQL** and others.

Dirty writes — row-level locks: a transaction **acquires a lock** on an object before modifying it and **holds** it until commit or abort; only one transaction can hold it, others **wait**. Done **automatically** at read committed and above.

Dirty reads — option 1: read locks (rarely used). Readers briefly take the same lock. **Problem:** one long write transaction makes many read-only transactions wait, hurting response times and operability (slowdowns spread across the app). When the book was written, only **IBM DB2** and **Microsoft SQL Server with** `read_committed_snapshot=off` did this.

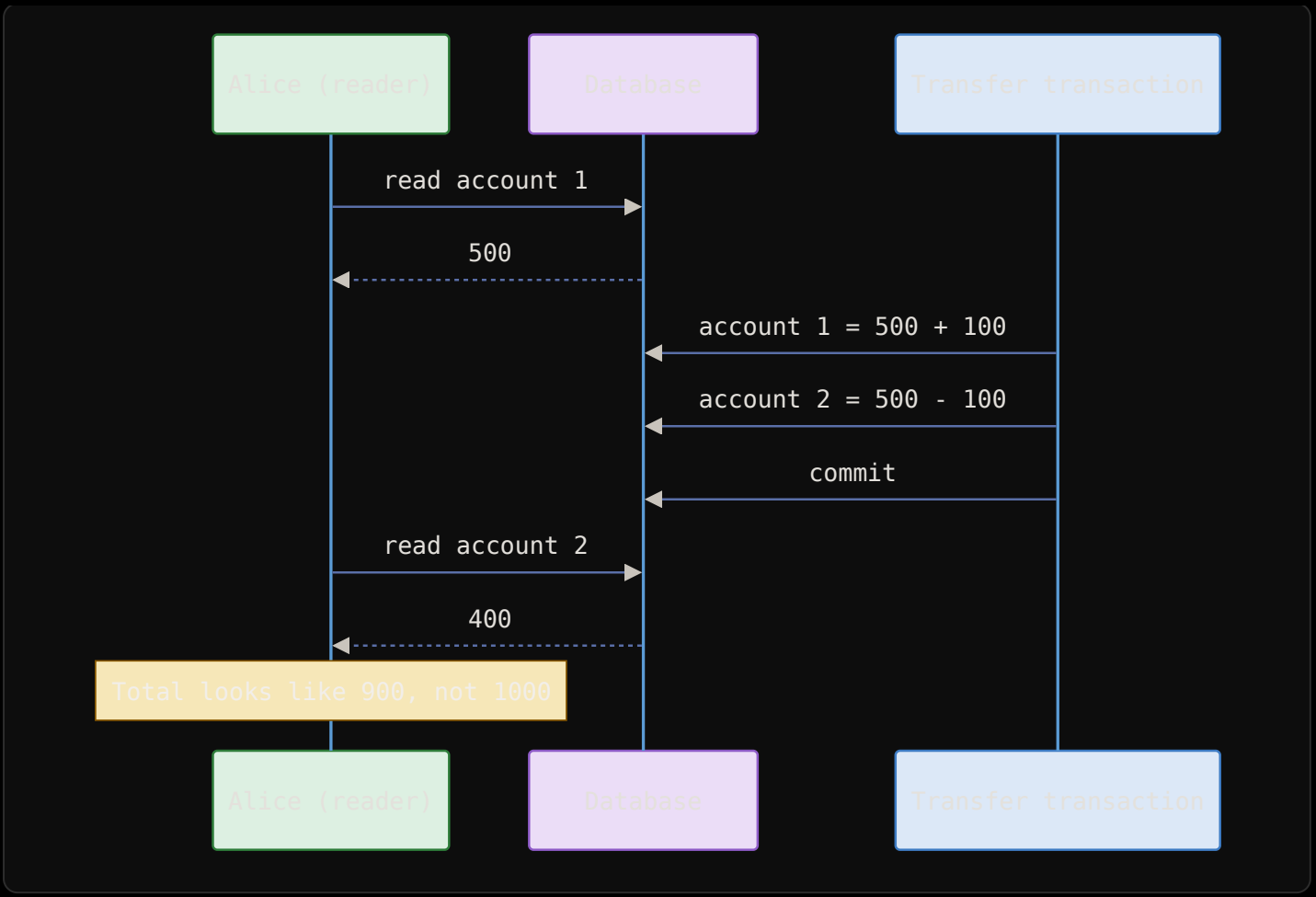
Dirty reads — option 2: remember old values (most databases). For each object being written, keep **the old committed value and the new uncommitted one**; others read the **old** value until the writer commits, then the **new** one (Figure 7-4).

Snapshot Isolation and Repeatable Read

Read committed supports aborts, hides incomplete transactions, and prevents interleaved writes — real improvements — but many concurrency bugs remain.

Figure 7-6 — read skew. Alice has **$1,000** in two accounts, **$500** each. A transfer moves **$100** from account 2 to account 1. She reads account 1 *before* the transfer (**$500**) and account 2 *after* (**$400**): apparently only **$900**.

Figure 7-6 (redrawn): Alice reads the two accounts at different moments during a transfer and sees only 900 dollars total



This is **read skew** (**nonrepeatable read**): re-reading account 1 at the end would give **$600**. Read committed **allows** it, since each value was committed when read. ("Skew" here is a **timing anomaly**, unlike Chapter 6's unbalanced load.)

For Alice it's temporary, but some cases **can't tolerate** temporary inconsistency:

- **Backups:** copying a large database takes **hours** while writes continue; a restored backup makes inconsistencies (vanished money) **permanent**.
- **Analytic queries and integrity checks:** large scans give **nonsense** if they see different parts at different times.

The fix — snapshot isolation: each transaction reads a **consistent snapshot** of everything committed **when it started**, unaffected by later changes. Ideal for long read-only jobs like backups and analytics, which are hard to reason about on changing data. Supported by **PostgreSQL, MySQL with InnoDB, Oracle, SQL Server**, and others.

Implementing snapshot isolation

Writes: like read committed, typically **write locks** prevent dirty writes (a writer can block another writer on the same object).

Reads: **no locks**. Key principle:

> **A reader never has to wait for a writer, and a writer never has to wait for a reader.**

So long reads on a consistent snapshot run alongside normal writes without lock contention.

MVCC. Generalising Figure 7-4: since in-progress transactions may need the database at **different points in time**, keep **several committed versions** per object: **multi-version concurrency control (MVCC)**.

Level	Versions needed	Snapshot scope (typical MVCC implementation)
Read committed	Two are enough: committed + not-yet-committed	A new snapshot for **each query**
Snapshot isolation	Many committed versions	One snapshot for the **whole transaction**

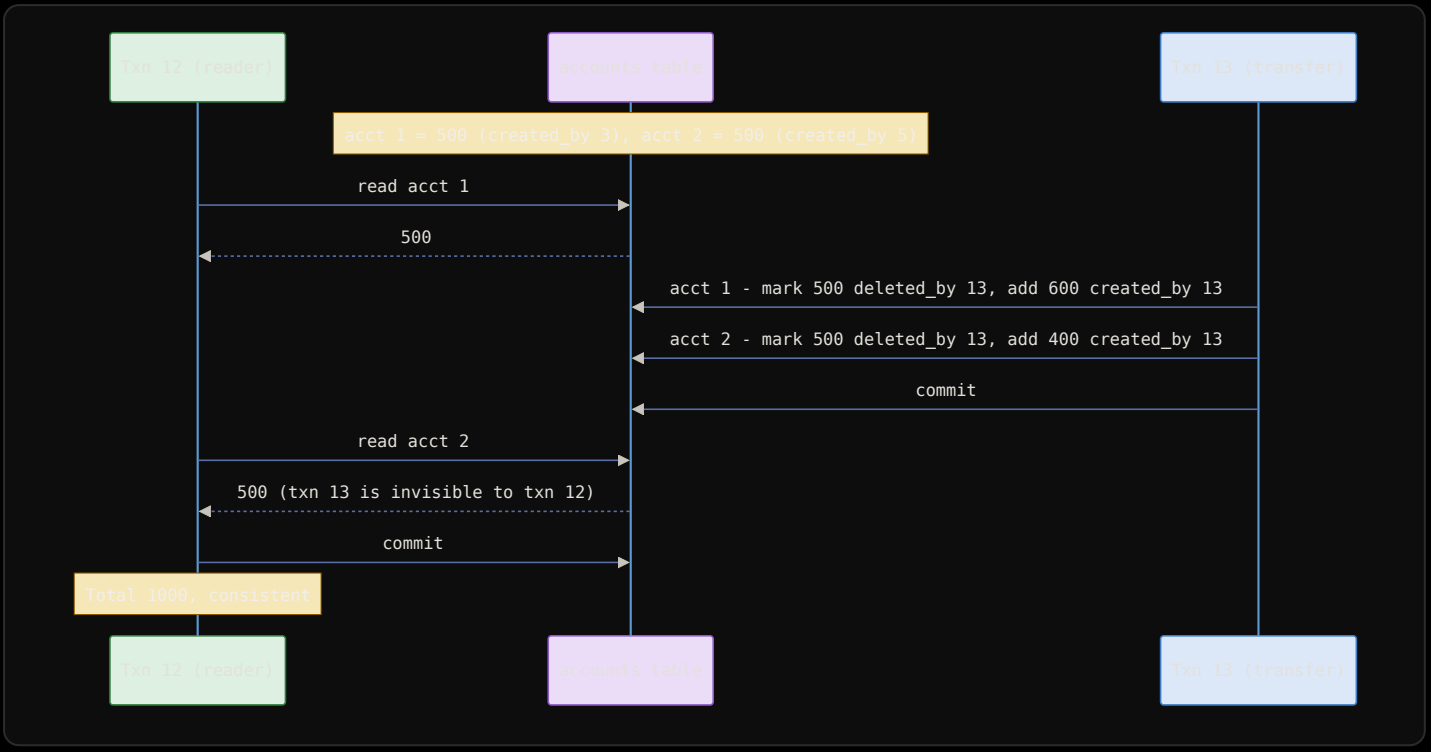
Engines supporting snapshot isolation usually implement read committed with MVCC too.

PostgreSQL's implementation (Figure 7-7; others similar):

1. Each transaction gets a unique, **always-increasing transaction ID (txid)**; its writes are **tagged with it**.
2. Each row has `created_by` (inserting txid) and `deleted_by` (initially empty).
3. **Delete** sets `deleted_by` instead of removing the row.
4. **Update** = **delete + create**: in Figure 7-7, transaction 13 takes $100 from account 2, leaving the **$500** row with `deleted_by = 13` and a new **$400** row with `created_by = 13`.
5. **Garbage collection** removes deleted rows once no transaction can see them.

(PostgreSQL txids are **32-bit**, overflowing after about **4 billion** transactions; its **vacuum** process prevents this from affecting data.)

Figure 7-7 (redrawn): MVCC in PostgreSQL, an update creates a new row version and marks the old one deleted



Visibility rules for observing a consistent snapshot

1. At transaction **start**, list all transactions **in progress**; ignore their writes even if they later commit.
2. Ignore writes by **aborted** transactions.
3. Ignore writes by transactions with a **later txid**, **whether or not** committed.
4. **Everything else** is visible.

These apply to creations and deletions. In Figure 7-7, transaction 12 sees **$500** for account 2: rule 3 hides both transaction 13's deletion and its new $400 row.

Equivalently, an object is **visible** if **both**: its creator **had committed** when the reader started; and it is **not deleted**, or its deleter **hadn't committed** when the reader started.

A long transaction may keep reading long-overwritten versions; since values are never updated in place (each update creates a new version), consistent snapshots are cheap.

Indexes and snapshot isolation

- **Approach 1:** the index points to **all versions**; lookups **filter out** invisible ones, and garbage collection removes index entries with old versions. Implementation details matter, e.g. **PostgreSQL** skips index updates when an object's versions fit on the **same page**.
- **Approach 2 — append-only / copy-on-write B-trees** (**CouchDB, Datomic, LMDB**): pages are never overwritten; a changed page is **copied**, and its parents up to the **root** are copied to point at it, while untouched pages are shared. Each write transaction (or batch) creates a **new root**, which *is* a consistent snapshot — no txid filtering needed, since later writes only create new roots. Requires background **compaction and garbage collection**.

Repeatable read and naming confusion

Database	Name used for snapshot isolation
Oracle	"serializable"
PostgreSQL, MySQL	"repeatable read"
IBM DB2	uses "repeatable read" to mean **serializability** (not snapshot isolation)

Why the mess? The **SQL standard**'s isolation definitions come from **System R (1975)**, before snapshot isolation existed. It defines **repeatable read**, superficially similar, so PostgreSQL and MySQL use that name to claim compliance. The standard's definitions are **flawed** (vague, imprecise, implementation-dependent), so "repeatable read" databases give very different guarantees; a **formal** research definition exists, but most implementations don't meet it. Punchline: **no one really knows what "repeatable read" means.**

Preventing Lost Updates

So far: what **read-only** transactions see during concurrent writes. Concurrent **writing** transactions (beyond dirty writes) bring the best-known conflict, the **lost update** (Figure 7-1).

What it is. In a **read-modify-write cycle** (read, change in code, write back), two concurrent cycles let the later write **clobber** the earlier, since it didn't include it.

Where it occurs: incrementing a counter or updating a balance; editing part of a complex value (appending to a list inside a JSON document); two users editing a **wiki page** and each saving the **whole page**.

Atomic write operations

Atomic update operations remove the read-modify-write cycle from app code; when they fit, they are usually the **best solution**. Ours:

```
UPDATE page_views SET hits = hits + 1 WHERE page = '/home';
```

Concurrency-safe in most relational databases. **MongoDB** offers atomic partial-document updates; **Redis** atomic operations on structures like **priority queues**. Not everything fits (wiki edits are arbitrary text; expressing them as atomic mutations is possible but complex — "Automatic Conflict Resolution", Chapter 5).

Implementation: take an **exclusive lock** on the object when read so no one else reads it until the update is applied (**cursor stability**), or run all atomic operations on a **single thread**.

Common pitfall. **ORM frameworks** make it easy to write unsafe read-modify-write code instead of using atomic operations — a source of subtle bugs tests rarely catch.

Explicit locking

If built-in atomic operations don't suffice, the **application explicitly locks** objects to be updated; other transactions reading them wait until the cycle finishes.

Book example: a multiplayer game where several players may move the **same piece**; the move must be **validated against game rules** (not expressible as a query), so lock the row. Ours — `FOR UPDATE` locks every row the query returns until commit:

```
BEGIN TRANSACTION;

SELECT * FROM tokens
  WHERE label = 'knight' AND match_id = 51
  FOR UPDATE;

-- app checks the move is legal, then:
UPDATE tokens SET square = 'e5' WHERE id = 908;

COMMIT;
```

Works, but forgetting a lock anywhere reintroduces the race.

Automatically detecting lost updates

Instead of forcing cycles to run **one at a time**, let them **run in parallel**; when the transaction manager **detects** a lost update, it **aborts** the offending transaction for retry. This check can run **efficiently with snapshot isolation**.

Database / level	Detects lost updates automatically?
PostgreSQL repeatable read	Yes
Oracle serializable	Yes
SQL Server snapshot isolation	Yes
MySQL/InnoDB repeatable read	**No**

Some argue a database must prevent lost updates to count as snapshot isolation; by that definition **MySQL doesn't provide snapshot isolation**. Benefit: **no special app code**, so **less error-prone** than remembering locks or atomic operations.

Compare-and-set

Databases **without** transactions may offer atomic **compare-and-set**: update only if **unchanged since read**, else retry the cycle. Ours — check the affected row count afterwards; zero means someone else edited it, so re-read and retry:

```
-- Safe or not depends on the database implementation
UPDATE notes SET body = 'edited text'
  WHERE id = 77 AND body = 'text I originally read';
```

Warning: if the `WHERE` clause may read from an **old snapshot**, the condition can pass despite a concurrent write, so lost updates are **not** prevented. Verify your database's compare-and-set is safe.

Conflict resolution and replication

With replicas on several nodes changed concurrently:

- **Locks and compare-and-set assume one up-to-date copy.** Multi-leader and leaderless replication allow concurrent async writes with no single up-to-date copy, so these **don't apply** (more under linearizability, Chapter 9).
- **Common approach:** let concurrent writes create conflicting versions (**siblings**) and merge them later in app code or with special data structures (Chapter 5).
- **Atomic operations work well** if **commutative** (any order on different replicas gives the same result), e.g. incrementing a counter, adding to a set. **Riak 2.0 datatypes** merge concurrent updates automatically so none are lost.
- **Last write wins (LWW)** is **prone to lost updates**, yet it is the **default** in many replicated databases.

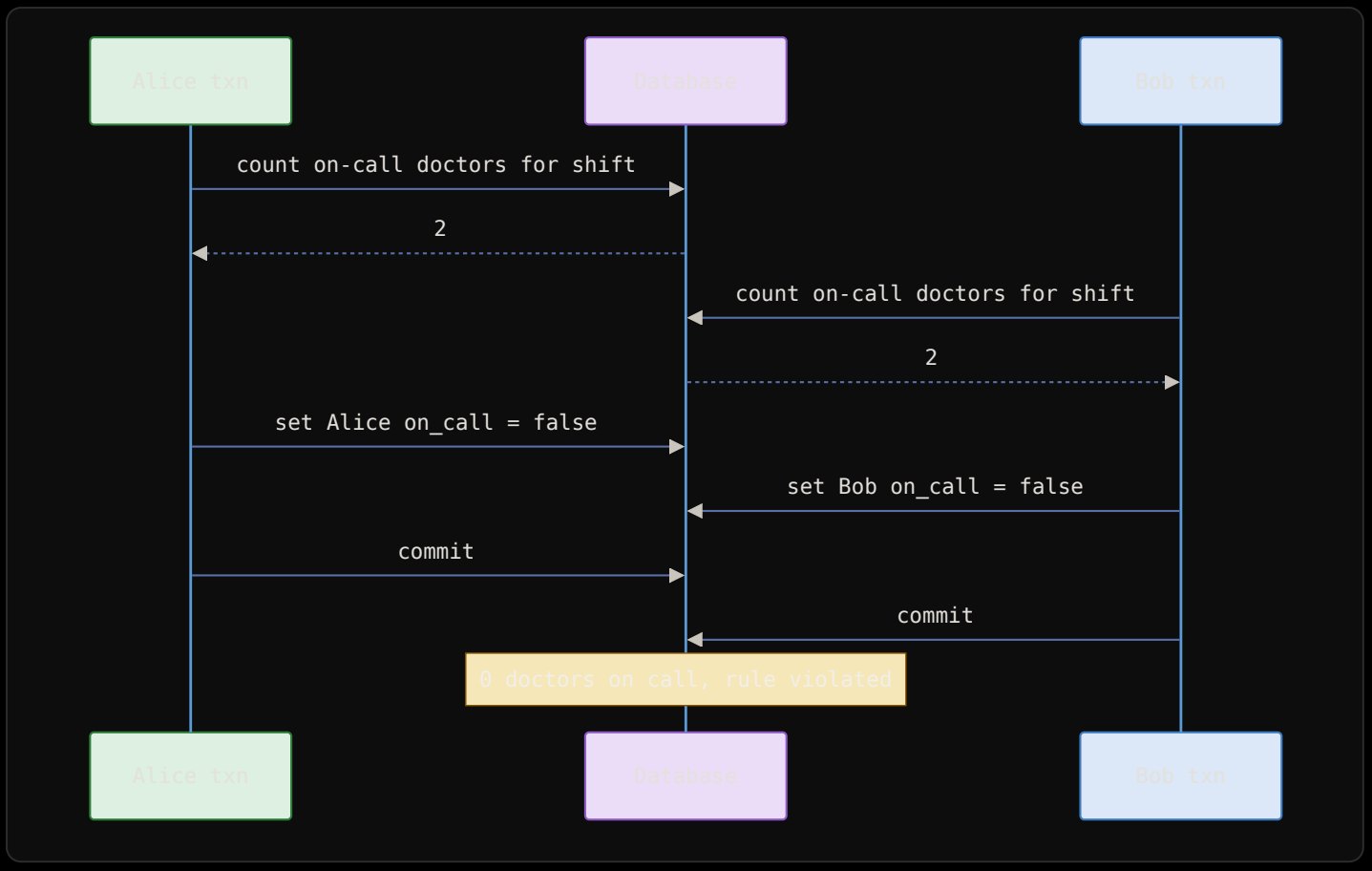
Write Skew and Phantoms

Dirty writes and lost updates involve writes to the **same** object; subtler conflicts exist.

Hospital example (Figure 7-8). Doctors manage **on-call shifts**; at least **one** must be on call, and a doctor can go off call if a colleague remains. **Alice and Bob**, the only two on call, both feel ill and request off call almost simultaneously:

1. Each transaction counts on-call doctors; under **snapshot isolation both see 2**.
2. Both decide it's safe.
3. Alice's transaction updates **her** record, Bob's **his**.
4. Both commit: **zero** doctors on call — rule broken.

Figure 7-8 (redrawn): Alice and Bob both see two doctors on call, both go off call, leaving nobody



Characterizing write skew

What it is. **Write skew**: not a dirty write or lost update, since the transactions update **different** objects (Alice's and Bob's rows), but clearly a race — run serially, the second would have been refused.

Mental model: a **generalisation of lost update**: two transactions **read an overlapping set of objects** and each **modifies some of them** (possibly different ones). If they modify the same object, it's a dirty write or lost update depending on timing.

Fewer options than for lost updates:

Technique	Helps with write skew?
Atomic single-object operations	**No** — several objects are involved.
Automatic lost-update detection in snapshot isolation	**No** — not detected in PostgreSQL repeatable read, MySQL/InnoDB repeatable read, Oracle serializable, or SQL Server snapshot isolation. Only **true serializable** isolation prevents it automatically.
Database constraints	Only for a **multi-object** constraint ("at least one on call"), which most databases lack; **triggers** or **materialized views** may work, depending on the database.
Explicitly locking the rows the transaction depends on	**Yes** — the second-best option if serializable is unavailable.

Our locking fix for an on-call rota — the `FOR UPDATE` locks every available engineer in slot 42, so a concurrent check waits; the app verifies at least 2 before updating:

```
BEGIN TRANSACTION;

SELECT * FROM rota
  WHERE available = true
  AND slot_id = 42 FOR UPDATE;

UPDATE rota
  SET available = false
  WHERE engineer = 'Priya'
  AND slot_id = 42;

COMMIT;
```

More examples of write skew

- **Meeting room booking.** No two overlapping bookings for one room: check for overlaps, then insert. Snapshot isolation doesn't stop a concurrent clashing insert; serializable isolation is needed. Ours — an existing booking overlaps if it ends after our start *and* starts before our end; two concurrent transactions can both count zero and both insert:

  ```
  BEGIN TRANSACTION;

  -- Any booking for desk 9 that overlaps 09:00-11:00?
  SELECT COUNT(*) FROM desk_bookings
    WHERE desk_id = 9 AND
      ends_at > '2024-03-04 09:00' AND starts_at < '2024-03-04 11:00';

  -- If the count was zero:
  INSERT INTO desk_bookings (desk_id, starts_at, ends_at, person_id)
    VALUES (9, '2024-03-04 09:00', '2024-03-04 11:00', 314);

  COMMIT;
  ```

  (PostgreSQL's **range types** make this more elegant but aren't widely supported elsewhere.)

- **Multiplayer game.** The earlier lock stops two players moving the **same** piece, not moving **different pieces to the same square** or other rule violations; a unique constraint may help for some rules, otherwise write skew is possible.
- **Claiming a username.** Two simultaneous sign-ups each check the name is free; unsafe under snapshot isolation. **Fix:** a **unique constraint** aborts the second insert.
- **Preventing double-spending.** Insert a tentative spend, list all items, check the balance stays positive; two concurrent spends can drive it negative unnoticed.

Phantoms causing write skew

The common pattern:

1. A `SELECT` checks a **requirement** via rows matching a condition (≥ 2 doctors on call, no overlapping booking, square empty, username free, enough money).
2. App code **decides** to proceed or abort based on the result.
3. It **writes** (`INSERT`, `UPDATE`, `DELETE`) and commits, **changing the precondition** of step 2 (rerunning step 1 would now differ).

Steps may be reordered (e.g. write first, then query, then decide).

Why locking doesn't always work. In the doctors case, the updated row was **among those step 1 returned**, so `SELECT FOR UPDATE` could lock it. The other examples check for **absence** of rows and then **add** one; if step 1 returns nothing, there is **nothing to lock**.

Definition. A **phantom**: one transaction's write changes the result of another's **search query**. Snapshot isolation prevents phantoms in **read-only** queries; in **read-write** transactions they can cause especially tricky write skew.

Materializing conflicts

If the problem is **no object to lock**, **create one artificially**. For meeting rooms: a table of **time slots × rooms** (e.g. **15-minute** slots), **pre-populated** for, say, the **next six months**. A booking transaction first locks (`SELECT FOR UPDATE`) the slot rows for its room and time range, then checks overlaps and inserts. The slots table holds **no booking data** — it's purely locks.

This is **materializing conflicts**: turning a phantom into a lock conflict on concrete rows. **Downsides:** hard and error-prone to design, and leaks concurrency control into the data model. A **last resort**; serializable isolation is usually far better.

Serializability

Read committed and snapshot isolation prevent some races but not others (write skew, phantoms). The situation is grim:

- Isolation levels are **hard to understand** and **inconsistently implemented** (e.g. "repeatable read").
- It's **hard to tell from app code** whether it's safe at a given level, especially in large apps.
- **No good tools** detect race conditions; static analysis might help in principle but hasn't reached practice, and testing is hard because problems are **nondeterministic**.

True since weak isolation appeared in the **1970s**; researchers' answer has always been **serializable isolation**.

What it guarantees. Usually the **strongest** level: parallel transactions produce the same result as running **one at a time** in some order, so transactions correct alone stay correct concurrently — **all** race conditions prevented.

Why not everyone uses it: implementation cost. Three techniques: **actual serial execution**; **two-phase locking (2PL)** (for decades the only viable option); **optimistic concurrency control** such as **serializable snapshot isolation (SSI)**. This chapter covers single nodes; Chapter 9 generalises.

Actual Serial Execution

The idea. Eliminate concurrency: run **one transaction at a time, in serial order, on a single thread** — serializable **by definition**.

Why only recently? Only around **2007** did designers decide a single-threaded loop was feasible; for **30 years** before, multi-threading was considered essential. What changed:

- **RAM got cheap** enough that the **active dataset often fits in memory** (Chapter 3), so transactions run much faster without disk waits.
- **OLTP transactions are usually short** (few reads and writes); long analytic queries are usually read-only and can run on a **consistent snapshot outside** the serial loop.

Who: **VoltDB/H-Store, Redis, Datomic**. Single-threaded systems can **outperform** concurrent ones by avoiding locking overhead, but throughput is limited to **one CPU core**, and transactions must be structured differently.

Encapsulating transactions in stored procedures

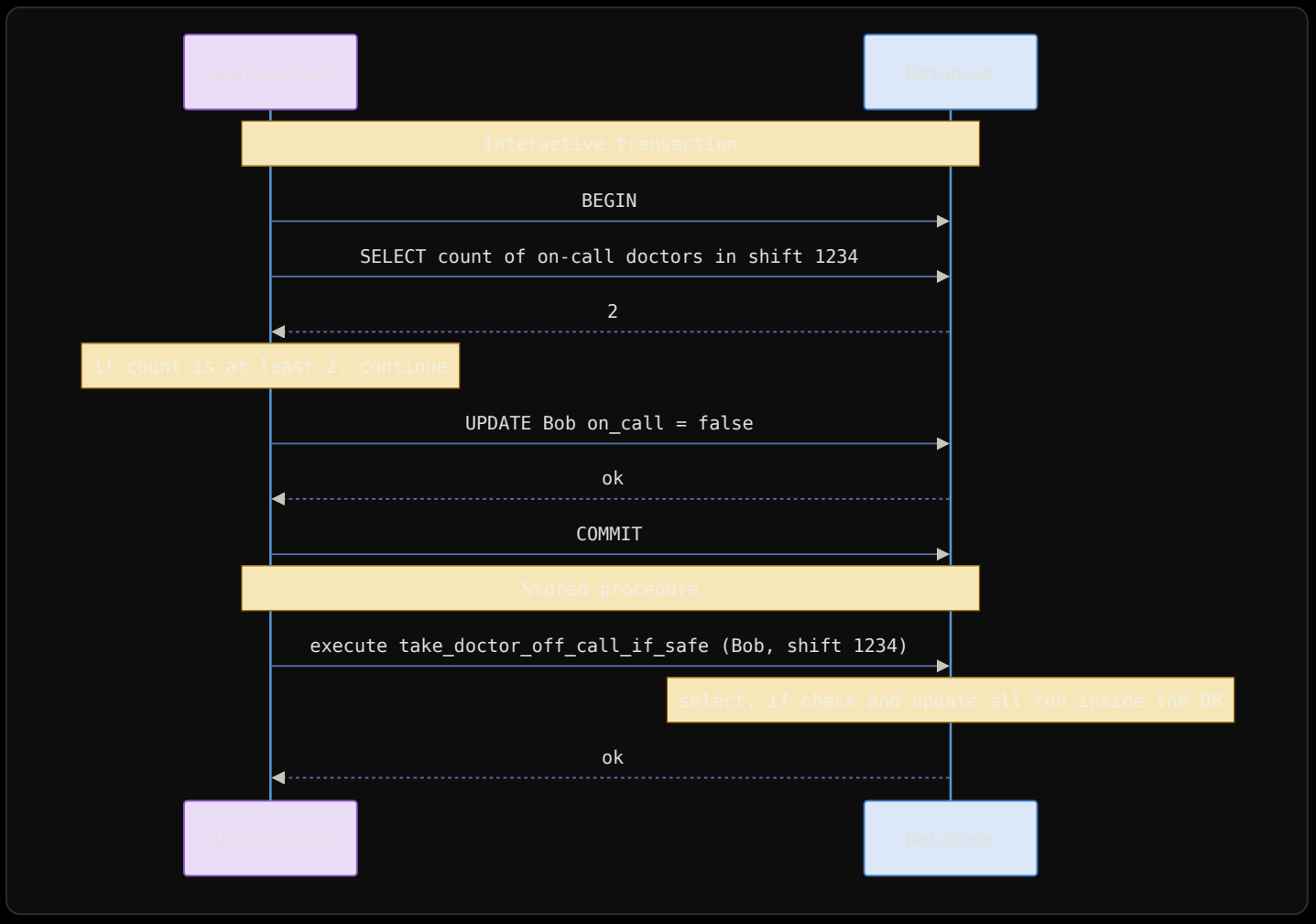
History. Early designers hoped a transaction could cover a **whole user flow** (e.g. booking a flight: search, choose, book legs, passenger details, pay).

Problem: **humans are slow**; databases can't efficiently handle huge numbers of mostly **idle** open transactions. So OLTP apps keep transactions short and never wait for users inside one; on the web, a transaction **commits within one HTTP request**.

Still interactive. Statements are usually sent **one at a time**: app sends a query, reads the result, sends another — many **network round trips**. A one-transaction-at-a-time database would mostly sit **idle waiting** for the app, giving terrible throughput, which is why such databases run many transactions concurrently.

The fix. Single-threaded serial systems **disallow interactive multi-statement transactions**; the app submits the **entire transaction code up front** as a **stored procedure**, which runs very fast if all data is in memory (no network or disk waits).

Figure 7-9 (redrawn): interactive transaction with many round trips vs a stored procedure sent in one call (using the on-call example)



Pros and cons of stored procedures

Stored procedures are long-standing (in the SQL standard as **SQL/PSM** since **1999**) but have a poor reputation:

- **Vendor-specific languages** — **Oracle: PL/SQL**, **SQL Server: T-SQL**, **PostgreSQL: PL/pgSQL** — dated, with weak library ecosystems.
- **Code in the database is hard to manage**: debugging, version control, deployment, testing, and metrics/monitoring are harder.
- **Databases are more performance-sensitive** than app servers (one database serves many app servers), so a badly written procedure does more damage.

These are surmountable with **general-purpose languages**:

System	Stored procedure language
VoltDB	Java or Groovy
Datomic	Java or Clojure
Redis	Lua

With stored procedures and **in-memory** data, single-threaded execution avoids I/O waits and concurrency-control overhead, giving good throughput on one thread.

Replication with procedures. **VoltDB** replicates by running the **same stored procedure on each replica** rather than copying writes, so procedures must be **deterministic**; current date/time must come from special **deterministic APIs**.

Partitioning

Problem. Serial execution limits throughput to **one CPU core on one machine**; read-only transactions can run elsewhere on snapshots, but **write-heavy** apps bottleneck.

Solution: partition the data (Chapter 6), as **VoltDB** supports: if **each transaction touches a single partition**, each partition gets its **own single-threaded transaction processor**, and assigning one per CPU core makes throughput **scale linearly** with cores.

Catch — cross-partition transactions need **coordination** across all touched partitions, running the procedure **in lock-step** on each to stay serializable. They are **vastly slower**: VoltDB reports about **1,000 cross-partition writes per second**, orders of magnitude below single-partition throughput, and **not improved** by adding machines.

Single-partition feasibility depends on **data structure**: simple key-value data partitions easily; data with **multiple secondary indexes** likely needs much cross-partition coordination (Chapter 6).

Summary of serial execution

Serial execution achieves serializable isolation **under constraints**:

- **Every transaction must be small and fast** — one slow one stalls all.
- **The active dataset must fit in memory**; rarely accessed data could be on disk, but a single-threaded transaction touching it would grind everything to a halt.
- **Write throughput must fit one CPU core**, or transactions must partition **without** cross-partition coordination.
- **Cross-partition transactions** are possible only up to a hard limit.

> **Anti-caching (footnote).** If a transaction needs data not in memory, **abort** it, **load** the data in the background while others continue, then **restart** it.

Two-Phase Locking (2PL)

For about **30 years**, **two-phase locking (2PL)** was the only widely used serializability algorithm.

> **2PL is not 2PC.** Two-phase locking and two-phase commit (2PC, Chapter 9) are completely different. (The variant here is also called **strong strict two-phase locking (SS2PL)**.)

What it is. Locks already prevent dirty writes; 2PL makes them **much stronger**. Concurrent **reads** of an object are allowed if no one writes it, but **writes** need **exclusive** access:

- If **A has read** an object and **B wants to write** it, B waits until A commits or aborts.
- If **A has written** an object and **B wants to read** it, B waits until A commits or aborts (reading an old version isn't allowed).

So **writers block writers *and* readers, and readers block writers** — the opposite of snapshot isolation's principle, and the key difference. In return, 2PL is **serializable**, preventing **all** races including lost

updates and write skew.

Implementation of two-phase locking

Used by **MySQL (InnoDB)** and **SQL Server** for **serializable**, and **DB2** for its level called **repeatable read**.

Mechanism: each object's lock is in **shared** or **exclusive** mode.

1. **Read** → acquire **shared** (many may share; wait if someone holds exclusive).
2. **Write** → acquire **exclusive** (wait if any lock exists).
3. **Read then write** → **upgrade** shared to exclusive.
4. Locks are **held until the transaction ends**: **phase one** acquires locks during execution, **phase two** releases them all at the end.

Deadlocks. A waits for B while B waits for A: the database **detects** this and **aborts one**, which the **app must retry**.

Performance of two-phase locking

The big downside (why it isn't universal since the 1970s): throughput and response times are **significantly worse** than under weak isolation.

- Overhead of acquiring and releasing many locks.
- More importantly, **reduced concurrency**: any potential race makes one transaction **wait**.
- Traditional databases **don't limit transaction duration** (designed for interactive apps waiting on humans), so waits are **unbounded**; even short transactions can **queue** on a popular object.
- Hence **unstable latencies** and very slow **high percentiles** under contention; one slow transaction, or one grabbing many locks, can **halt** the system.
- **Deadlocks** occur under lock-based read committed too, but **far more often** under 2PL serializable; each abort wastes work.

Predicate locks

The gap. The above ignores **phantoms**, which a serializable database **must** prevent. Meeting rooms: if one transaction searched a room's bookings in a time window, another must not concurrently **insert or update** an overlapping booking for that room (other rooms or non-overlapping times are fine).

Predicate lock: like a shared/exclusive lock, but on **all objects matching a search condition** — e.g. (ours) the whole `WHERE` clause below, "desk 9, overlapping 09:00–11:00":

```
SELECT * FROM desk_bookings
  WHERE desk_id = 9 AND
    ends_at > '2024-03-04 09:00' AND
    starts_at < '2024-03-04 11:00';
```

Rules:

1. To **read** matching objects, A takes a **shared-mode predicate lock**; if B holds an **exclusive lock** on any matching object, A waits.
2. Before A **inserts, updates or deletes**, it checks the object's **old and new values** against existing predicate locks; if B's matches, A waits.

Key idea: predicate locks cover objects that **don't exist yet** (phantoms). **2PL plus predicate locks** prevents all write skew and races: truly **serializable**.

Index-range locks

Problem: predicate locks **perform poorly** (checking many active locks is slow), so most 2PL databases use **index-range locking** (**next-key locking**), a **simplified approximation**.

Why safe: approximate a predicate with a **broader** one — "room 123, noon–1pm" → "room 123, any time" or "all rooms, noon–1pm"; any write matching the original matches the broader one.

Mechanism: the bookings table likely has indexes on `room_id` and/or `start_time`/`end_time`. Using the `room_id` **index**, attach a **shared lock to that index entry** ("someone searched room 123"); using a **time index**, lock a **range of index values** ("bookings overlapping noon–1pm on **January 1, 2018**"). Any

transaction inserting/updating/deleting a matching booking must **update the same index part**, hits the lock, and **waits**.

Trade-off. Prevents phantoms and write skew; **less precise** (may lock more than necessary) but **much lower overhead** — a good compromise.

Fallback. With **no suitable index**, take a **shared lock on the whole table** — safe but bad for performance, blocking all other writers.

Serializable Snapshot Isolation (SSI)

So far: serializability that **performs poorly** (2PL) or **doesn't scale** (serial execution), vs weak levels that perform well but allow races. Must we choose?

Perhaps not: **serializable snapshot isolation (SSI)** offers full serializability with only a **small performance penalty** over snapshot isolation. First described in **2008** (**Michael Cahill**'s PhD thesis). Used in **PostgreSQL's serializable level since 9.1** (single-node) and by **FoundationDB** (distributed, a similar algorithm). Still proving itself, but may become the **default**.

Pessimistic versus optimistic concurrency control

	Pessimistic	Optimistic
Idea	If anything *might* go wrong (someone else holds a lock), **wait** until safe	**Carry on**, hoping all is well; at **commit**, check whether isolation was violated and **abort** if so
Analogy	**Mutual exclusion** (only one thread at a time may enter a protected section, usually via a lock or *mutex*) in multi-threaded programming	"Ask forgiveness, not permission"
Examples	2PL; serial execution (extremely pessimistic)	SSI
Weakness	Waiting and reduced concurrency	Performs badly under **high contention** — many aborts; retries add load when near max throughput
When it does well	Serial execution: when every transaction is tiny and fast. (For 2PL the book mainly stresses the costs: blocking, unstable latency, deadlocks.)	With **spare capacity** and **low contention**, usually **faster** than pessimistic methods

Serial execution as extreme pessimism: like each transaction holding an exclusive lock on the whole database (or partition), compensated by making transactions **very fast**.

Optimistic concurrency control is an old, long-debated idea. **Commutative atomic operations** reduce contention: concurrent increments to a counter don't conflict (as long as the counter isn't also read in the same transaction).

What's new in SSI: it builds on **snapshot isolation** (all reads from a consistent snapshot), unlike earlier optimistic techniques, adding an algorithm to **detect serialization conflicts among writes** and choose **which transactions to abort**.

Decisions based on an outdated premise

In write skew, a transaction reads, examines the result, and **acts** on it; under snapshot isolation, by commit time the result may be **stale**. It acts on a **premise** (e.g. "two doctors are on call") that may have become **false**.

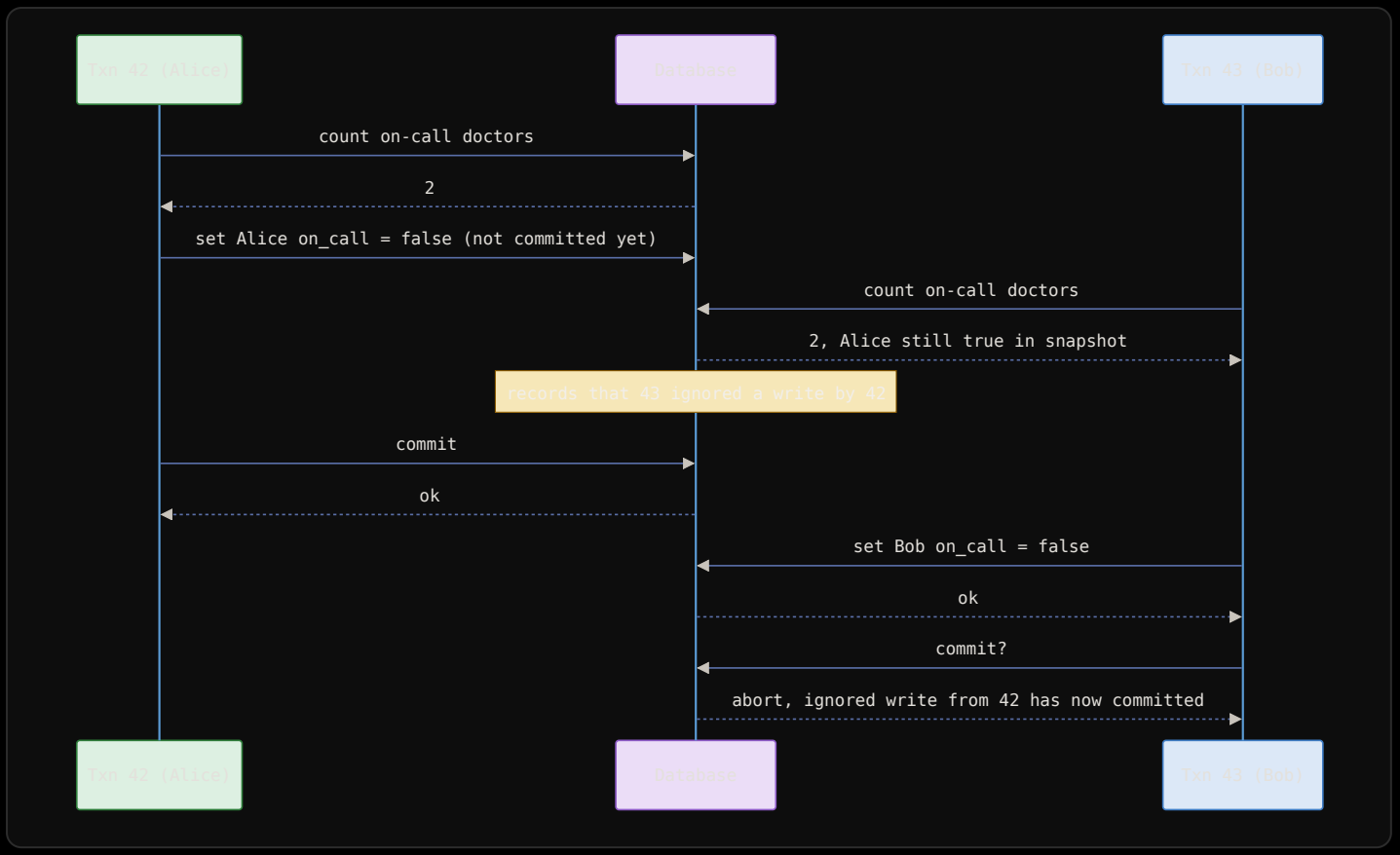
The database can't see how app logic uses query results, so it must assume **any change to the premise may invalidate the writes** (a possible **causal dependency** between reads and writes). For serializability it must **detect** transactions that may have acted on an outdated premise and **abort** them. Two cases:

1. **Reads of a stale MVCC version** (an uncommitted write occurred **before** the read).
2. **Writes that affect earlier reads** (the write occurs **after** the read).

Detecting stale MVCC reads

Situation (Figure 7-10). MVCC snapshots ignore writes by transactions uncommitted at snapshot time. Transaction **43** sees Alice `on_call = true` because transaction **42** (which changed her) hasn't committed; by the time 43 commits, **42 has committed**, so 43's premise is false.

Figure 7-10 (redrawn): txn 43 read a value that txn 42 had changed but not committed, so 43 aborts at commit. In the book's table, shift 1234 holds two versions of Alice (on_call true, created_by 1, deleted_by 42; and on_call false, created_by 42), plus Bob (true) and Carol (false); after the abort, 43 retries.



Detection: the database **records** whenever MVCC visibility makes a transaction **ignore** another's write; at **commit**, it checks whether any ignored writes **have since committed**, and if so **aborts**.

Why wait until commit? If 43 is **read-only**, there's no write skew risk, and at read time the database can't know whether 43 will write. Also 42 might still **abort** or be **uncommitted** when 43 commits, so the read may not be stale. Avoiding needless aborts preserves snapshot isolation's support for **long-running reads**.

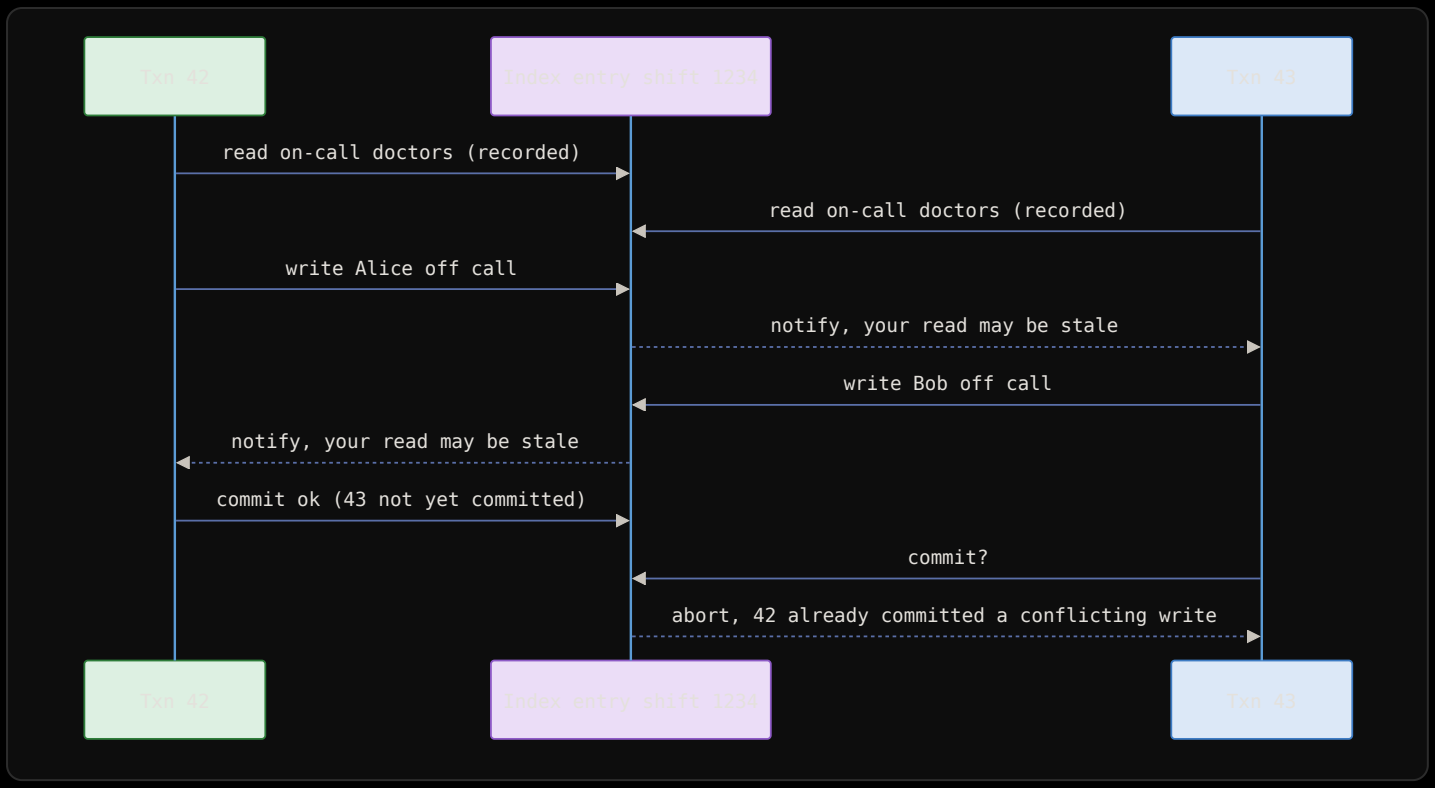
Detecting writes that affect prior reads

Situation (Figure 7-11): another transaction modifies data **after** it was read.

Detection borrows **index-range locks** from 2PL, but they **don't block**:

1. Transactions 42 and 43 both query on-call doctors for **shift 1234**; with a `shift_id` index, the database records on **index entry 1234** that 42 and 43 **read** it (without an index, track at **table** level).
2. These records are kept only until the transaction ends **and all transactions concurrent with it have ended**.
3. A **writing** transaction looks in the indexes for transactions that **recently read** the affected data — like taking a write lock on the range, but as a **tripwire** that **notifies** readers their data may be stale instead of blocking.
4. In Figure 7-11, 43 notifies 42 and 42 notifies 43.
5. **42 commits first** successfully: 43's write hasn't taken effect since 43 hasn't committed.
6. When **43 commits**, 42's conflicting write is already committed, so **43 is aborted**.

Figure 7-11 (redrawn): SSI records reads on index entry shift 1234 and aborts the second committer



Performance of serializable snapshot isolation

Tracking granularity trade-off: **detailed** tracking aborts precisely but costs bookkeeping; **coarser** tracking is faster but may abort **more than necessary**.

Smarter analysis: sometimes reading overwritten data is provably still serializable; **PostgreSQL** uses this theory to **reduce unnecessary aborts**.

Versus 2PL: transactions **don't block on locks**; as in snapshot isolation, **readers and writers never wait for each other**, making latency **more predictable and less variable**, and **read-only queries run lock-free on a snapshot** — great for read-heavy workloads.

Versus serial execution: **not limited to one CPU core**; **FoundationDB** distributes conflict detection over **multiple machines** for very high throughput, and transactions can span **multiple partitions** while staying serializable.

Abort rate strongly affects performance: long-running read-write transactions likely conflict and abort, so SSI needs **short read-write transactions** (long **read-only** ones are fine). Still, slow transactions probably hurt SSI **less** than 2PL or serial execution.

Approach	How it serializes	Main strengths	Main weaknesses
Actual serial execution	One transaction at a time, single thread, stored procedures, in-memory	Simple; no lock overhead	Limited to one core per partition; transactions must be tiny; data must fit in RAM; cross-partition is slow (~1,000 writes/s in VoltDB)
Two-phase locking	Shared/exclusive locks held to end; predicate or index-range locks for phantoms	Decades of use; prevents all anomalies	Readers and writers block each other; unstable, high tail latency; frequent deadlocks
SSI	Snapshot isolation + detect conflicts, abort at commit	No blocking; predictable latency; scales across machines (FoundationDB)	Aborts under high contention; long read-write transactions likely to abort; relatively new

Chapter summary

(Book section: Summary)

- **Transactions are an abstraction** letting apps pretend certain concurrency problems and hardware/software faults don't exist: many errors reduce to a simple **abort**, and the app retries.
- Apps with very simple access patterns (single-record reads/writes) may not need them; for complex access patterns they **greatly reduce** the error cases to consider.
- Without them, crashes, network interruptions, power outages, full disks and unexpected concurrency can leave data inconsistent (e.g. **denormalized data drifting** from its source).
- Three widely used isolation levels: **read committed**, **snapshot isolation** (sometimes called repeatable read), **serializable**, characterised by these races:

Anomaly	What happens	What prevents it
Dirty reads	Reading another client's uncommitted writes	Read committed and stronger
Dirty writes	Overwriting another client's uncommitted write	Almost all transaction implementations
Read skew (nonrepeatable read)	Seeing different parts of the DB at different moments	Snapshot isolation, usually via MVCC
Lost updates	Two concurrent read-modify-write cycles; one overwrites the other	Some snapshot isolation implementations detect it automatically; others need a manual lock (`SELECT FOR UPDATE`)
Write skew	A decision based on a read whose premise is no longer true by write time	Only serializable isolation
Phantom reads	Another client's write changes the result of your search	Snapshot isolation handles simple phantoms; phantoms causing write skew need things like index-range locks

- Weak levels leave some anomalies to the app (e.g. explicit locking); **only serializable isolation** covers all. Implementations: **literal serial execution** (if transactions are very fast and throughput fits one core); **two-phase locking** (standard for decades, often avoided for performance); **SSI** (new, optimistic, non-blocking, checks at commit and aborts non-serializable transactions).
- Examples were relational, but transactions matter **for any data model**.
- All of this assumed a **single machine**; distributed transactions bring new challenges (Chapters 8, 9).

Interview angle

1. What does ACID mean, and which letter is misleading? *30-second answer:* Atomicity = abort on failure and undo everything ("abortability"). Consistency = app-defined invariants hold — the app's job, so the C barely belongs. Isolation = concurrent transactions don't interfere; the ideal is serializability, but most DBs default weaker. Durability = committed data survives crashes, via disk + WAL or replication — never perfect.

- *Follow-up: Is "ACID compliant" meaningful?* Not really; implementations differ, especially on isolation. Oracle's "serializable" is snapshot isolation.
- *Follow-up: ACID atomicity vs threading atomicity?* Threading: no one sees a half-done state (concurrency). ACID: undoing partial work after a fault.

2. Explain read committed vs snapshot isolation. *30-second answer:* Read committed prevents dirty reads and writes (row locks for writes; readers see the last committed value) but allows read skew. Snapshot isolation gives each transaction a consistent snapshot from its start via MVCC (txid-tagged versions plus visibility rules); readers and writers never block each other — great for backups and analytics.

- *Follow-up: Why not read locks against dirty reads?* A long write would stall many readers and spread slowdowns.
- *Follow-up: "Repeatable read"?* Interpreted differently: snapshot isolation in PostgreSQL/MySQL, serializability in DB2 (and Oracle calls snapshot isolation "serializable").

3. What is a lost update and how do you prevent it? *30-second answer:* Concurrent read-modify-write cycles where one write clobbers the other. Fixes: atomic updates (`SET x = x + 1`), explicit locks (`SELECT ... FOR UPDATE`), automatic detection (PostgreSQL repeatable read, Oracle serializable, SQL Server snapshot isolation — not MySQL/InnoDB repeatable read), compare-and-set, and in replicated systems commutative operations or sibling merging. LWW loses updates.

- *Follow-up: Is compare-and-set always safe?* No — if its WHERE reads an old snapshot it can pass despite a concurrent write.

4. What is write skew? Give an example and fix. *30-second answer:* Two transactions read the same data, decide, then write *different* objects, together breaking an invariant — e.g. two on-call doctors both going off call. Snapshot isolation misses it. Fixes: serializable isolation; `FOR UPDATE` on the rows depended on; multi-object constraints via triggers/materialized views; for absence checks (bookings, usernames), unique constraints, materializing conflicts, or serializable.

- *Follow-up: Why can't FOR UPDATE fix bookings?* The check looks for rows that don't exist — nothing to lock: a phantom.

5. Compare the three ways to implement serializability. *30-second answer:* Serial execution — single thread, stored procedures, in-memory; simple but one core per partition and slow cross-partition. 2PL — shared/exclusive locks held to commit plus predicate or index-range locks; correct but blocking, deadlock-prone, bad tail latency. SSI — snapshot isolation plus conflict detection and abort at commit; non-blocking, predictable latency, but aborts rise with contention and long read-write transactions.

- *Follow-up: 2PL vs 2PC?* Unrelated: 2PL is isolation on one DB; 2PC is atomic commit across nodes (Chapter 9).
- *Follow-up: When is optimistic worse?* Under high contention near max throughput — aborts and retries add load.

6. Why should apps retry aborted transactions, and when is retrying dangerous? *30-second answer:* Aborts exist to enable safe retries, yet many ORMs (ActiveRecord, Django) don't retry. Dangers: commit succeeded but the ack was lost (double execution — dedupe); overload (backoff, limits); permanent errors (pointless); external side effects like emails; client dying mid-retry.

Self-check

1. **Why does the book suggest "abortability" as a better name for atomicity?** The defining feature is that a failed transaction is aborted with all writes discarded, making retry safe — not concurrency.
2. **Which ACID property is the application's job rather than the database's, and why?** Consistency: only the app knows its invariants; the DB can check some (foreign keys, uniqueness) but generally stores what it's given.
3. **Under read committed, can a transaction see $900 instead of $1,000 during a transfer? What is this called and what fixes it?** Yes — read skew (nonrepeatable read). Snapshot isolation fixes it.
4. **In PostgreSQL's MVCC, what happens physically when a row is updated?** The old row gets `deleted_by = txid` and a new row with `created_by = txid` is added; garbage collection later removes invisible versions.
5. **List the four visibility rules for a snapshot.** Ignore writes of transactions in progress at start; ignore aborted transactions' writes; ignore writes of later txids; everything else is visible.
6. **Scenario:** your app adds a tag to a JSON array by reading the document, appending in code, and writing it back. Two users do it at once and one tag disappears. Name the anomaly and two fixes. Lost update. Use an atomic document operation (e.g. MongoDB's) or lock with `SELECT FOR UPDATE`; or rely on automatic lost-update detection if available.
7. **Why is materializing conflicts a last resort?** Hard and error-prone to design, and it leaks concurrency control into the data model; serializable isolation is usually better.
8. **Trade-off:** why does 2PL have unstable high-percentile latency? Readers and writers block each other, waits are unbounded, queues form on hot objects, one big or slow transaction stalls many, and deadlocks force redone work.

9. **Why does SSI wait until commit to abort a transaction that read a stale value?** It may be read-only (no risk), and the other writer may still abort or be uncommitted; waiting avoids needless aborts and keeps long reads working.
10. **Scenario:** a workload has high write throughput, many secondary indexes and frequent cross-entity transactions. Would actual serial execution be a good fit? Probably not: one core per partition, many secondary indexes imply cross-partition coordination, and cross-partition transactions are slow (~1,000 writes/s in VoltDB) and don't scale with machines. SSI or 2PL fit better.

Chapter 8 — The Trouble with Distributed Systems

Book: Chapter 8, pp. 273–320

Overview

The question this chapter answers: once software runs on many machines connected by a network, what can go wrong, what can you rely on, and what can you *never* be sure of?

Chapter 5 (replica failover, replication lag) and Chapter 7 (concurrency control) already dealt with faults, but the book now calls them *too optimistic* and adopts Murphy's law: **whatever can fail, will fail** (experienced operators consider this reasonable). The one exception: faults are assumed **non-Byzantine** — a **node** (one machine or process in the system) may fail but does not lie; Byzantine faults come near the end. (The epigraph is a Kyle Kingsbury 2013 joke rewriting a pop lyric about laggy networks.)

The chapter surveys four enemies; Chapter 9 gives algorithms that cope with them:

1. **Unreliable networks** — messages can be lost or delayed, and you cannot tell which.
2. **Unreliable clocks** — every machine's time is a bit (or a lot) wrong.
3. **Process pauses** — a program can freeze for seconds or minutes without noticing.
4. **Knowledge, truth and lies** — how a node reasons when it cannot trust its own view, and how *system models* define "correct".

> **Analogy:** coordinating a project by post — letters get lost, arrive late or out of order, silence is ambiguous, every wristwatch differs, and friends occasionally fall asleep mid-sentence.

Key terms

Terms defined in the body are listed here only as a quick index; full definitions are in Chapter 13.

Term	Meaning
Distributed system	Software on several computers that communicate only by messages over a network.
Deterministic / nondeterministic	Same input always gives the same result / may work or fail unpredictably.
Partial failure	Some parts broken while others work, unpredictably.
HPC vs cloud computing	Supercomputers for scientific batch jobs vs multi-tenant datacenters of commodity machines with elastic resources and metered billing.
Checkpoint	Saving a long computation's state to durable storage so it can restart from there.
Commodity machines	Cheap ordinary servers; better cost/performance, higher failure rates.
RDMA	Remote direct memory access — reading another machine's memory directly.
Clos topology	Datacenter network layout with high bisection bandwidth.
Rolling upgrade	Restarting nodes one at a time to upgrade without downtime.
Shared-nothing	Each machine has its own CPU, memory, disk; cooperation only via network.
Asynchronous packet network	Promises nothing about when or whether a packet arrives.
Timeout	Giving up waiting for a reply after a chosen time.
Network partition / netsplit	Part of the network cut off from the rest (book prefers "network fault").
Cascading failure	Overloaded nodes wrongly declared dead push load onto others, which then fail too.
Unbounded delay	No upper limit on how long a packet or request can take.
Queueing delay / congestion	Time waiting in switch, OS or VM buffers because a resource is busy.
Flow control (congestion avoidance, backpressure)	Sender slows itself to avoid overloading the network or receiver.
UDP	Transport without flow control or retransmission; for when late data is useless.
Noisy neighbor	Another tenant hogging shared resources, making your latency vary.
Jitter	Variability of delays/response times.
Phi Accrual failure detector	Adapts timeouts to the measured response-time distribution.
Synchronous network / circuit switching	Bandwidth reserved along a route per connection, giving bounded delay (telephone).
Packet switching	Packets share bandwidth opportunistically (Ethernet, IP).
Bursty traffic	Transfers with no fixed bandwidth need that want to finish ASAP.
QoS / admission control	Prioritising and scheduling packets / rate-limiting senders.
Duration vs point in time	A time interval vs a specific date and time.
NTP / PTP	Network Time Protocol / the more precise Precision Time Protocol for clock sync.
Time-of-day clock	Wall-clock date and time; can jump.
Monotonic clock	Always moves forward; for durations; absolute value meaningless.
Epoch	00:00 UTC, 1 January 1970 — reference point of Unix timestamps.
Leap second / smearing	A 59- or 61-second minute / spreading that adjustment over a day.

Term	Meaning
Slewing	NTP adjusting a clock's rate slightly instead of jumping it.
Clock drift / skew	A clock running fast or slow / difference between two machines' clocks.
Last write wins (LWW)	Keep the write with the highest timestamp, discard the rest.
Logical vs physical clock	Counter that orders events vs a clock measuring real time.
TrueTime	Spanner's clock API returning `[earliest, latest]`.
Lease	A lock with a timeout; the holder must renew it.
Stop-the-world GC pause	Garbage collector halting all application threads.
Steal time	CPU time a VM loses to other VMs on the same host.
Thrashing	OS spending nearly all its time swapping pages.
Hard real-time system / RTOS	Must meet deadlines or fail / OS guaranteeing CPU time in set intervals.
System model	Formal statement of what an algorithm may assume about timing and failures.
Quorum	Decision requiring a minimum number of votes (often a majority).
Split brain	Two nodes both believing they are leader.
Fencing token	Increasing number issued with each lock/lease; the resource rejects stale ones.
Byzantine fault	A node sending arbitrary or deliberately misleading messages.
Supermajority	More than two-thirds of nodes.
Safety / liveness property	"Nothing bad happens" (violation is permanent) / "something good eventually happens".
Limping node	Degraded (e.g. very slow) but not dead.

Faults and Partial Failures

Single computer. With working hardware, the same operation always gives the same result — it is **deterministic**. When hardware fails (memory corruption, loose connector), the result is usually **total** failure: kernel panic, "blue screen of death", failure to boot. Computers are designed to **crash rather than return a wrong answer**, hiding physics behind an **idealised system model** of perfect instructions and intact memory — a goal dating back to the **very first digital computer**. A flaky single machine usually means bad software.

Many computers. The idealisation breaks down. Coda Hale's anecdote lists long network partitions inside one datacenter, **PDU** (power distribution unit) failures, switch failures, whole racks accidentally power-cycled, whole-datacenter backbone and power failures, and a driver with low blood sugar crashing a **Ford pickup truck** into a datacenter's **HVAC** system — and he is not even an ops person.

Partial failure: some parts break unpredictably while others work. It is **nondeterministic** — an operation spanning nodes may work sometimes and fail other times — and you may not even **know whether an operation succeeded**, because message transit time is itself nondeterministic. That combination is what makes distributed systems hard.

Cloud Computing and Supercomputing

	High-performance computing (HPC)	Cloud computing
Typical use	Compute-heavy science: **weather forecasting**, **molecular dynamics**	Online internet services
Hardware	**Specialised**, reliable nodes; **shared memory** and **RDMA**	**Commodity** machines: cheaper (economies of scale), **higher failure rates**
Network	**Multi-dimensional meshes** and **toruses** suited to HPC patterns	**IP and Ethernet** in **Clos topologies** (high bisection bandwidth)
Other traits	Nodes close together	**Multi-tenant**, **elastic/on-demand**, **metered billing**

Traditional enterprise datacenters sit in between.

Supercomputer fault handling: periodically **checkpoint** to durable storage; on node failure **stop the whole workload**; after repair **restart from the last checkpoint**. This turns partial failure into **total failure** — it behaves like a single computer.

Why internet services differ:

- **Online, low latency, always** (latency = time for one request). Stopping the service to fix a node is unacceptable; a batch weather simulation can just be rerun.
- **Commodity hardware** fails more often; **different networks** (IP/Ethernet vs meshes).
- **Something is always broken:** at **thousands of nodes**, assume some part is faulty at any moment. If error handling is "give up", a large system spends most of its time recovering.
- **Fault tolerance aids operations: rolling upgrades** (Chapter 4); a badly performing VM (**virtual machine**, a software-simulated computer sharing a physical host) can be killed and replaced.
- **Geographic distribution:** keeping data near users means crossing the slower, less reliable **internet**.

Consequence: **accept partial failure** and build **fault tolerance into software** — a **reliable system from unreliable components**. Even small systems eventually hit faults, so fault handling must be **part of the design**, operators must know what the software does in a fault, and faults should be **deliberately created in testing**. Being **suspicious, pessimistic and paranoid is rewarded**.

> **Building a Reliable System from Unreliable Components (sidebar).** A system *can* be more reliable than its parts: **error-correcting codes** carry data across a channel that flips bits (e.g. radio interference); **TCP** over unreliable **IP** retransmits lost packets, removes duplicates and restores order. There is always a **limit** — codes handle a few bit errors, not a swamped signal; TCP **cannot remove network delay** — but the higher layer leaves fewer, simpler faults to reason about (revisited in Chapter 12's end-to-end argument).

Unreliable Networks

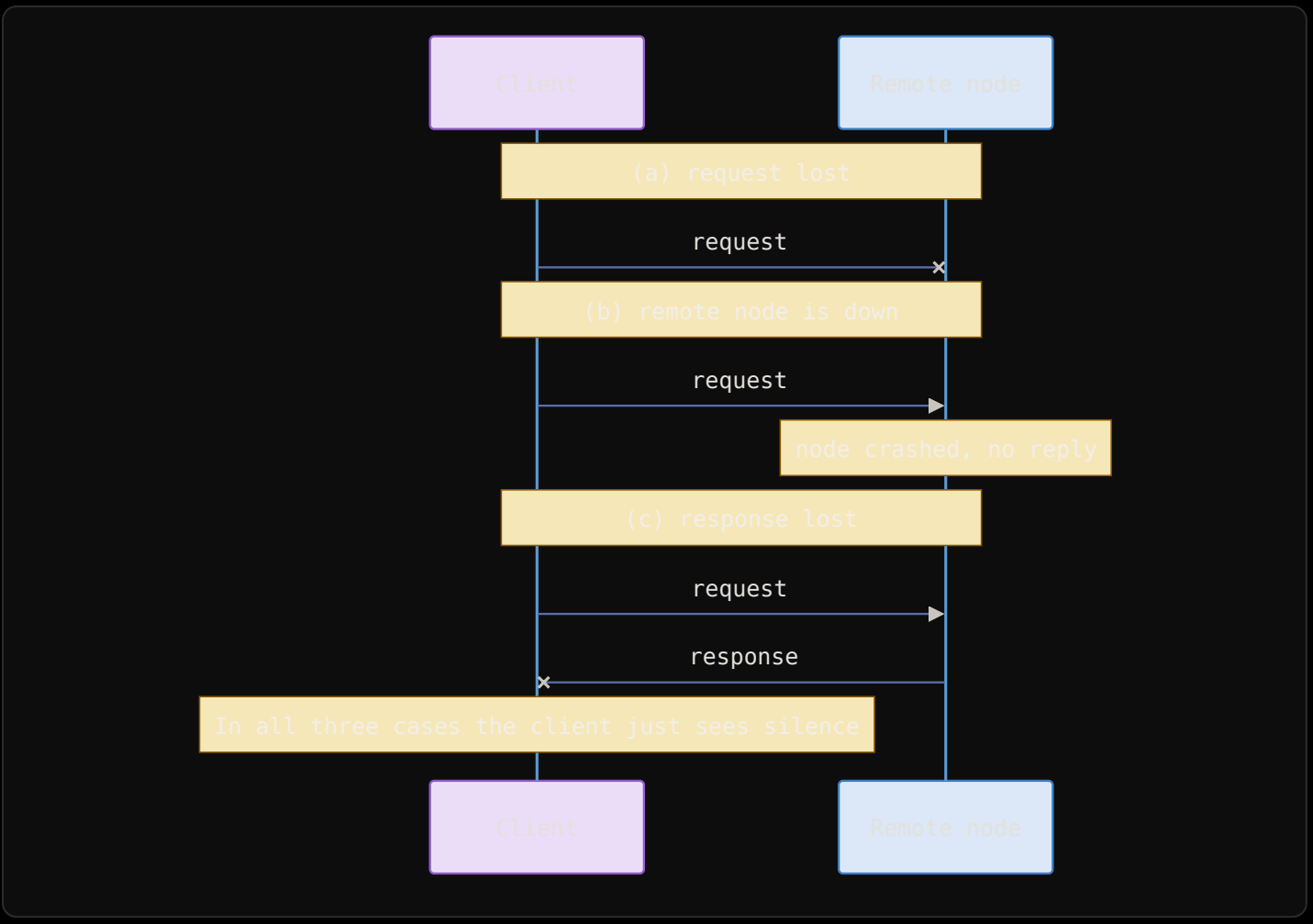
The book focuses on **shared-nothing** systems: each machine has its **own memory and disk**, reachable only via network requests. It dominates because it is **cheap** (no special hardware), uses **commoditised cloud services**, and gains reliability through **redundancy across geographically distributed datacenters**.

The internet and most datacenter networks (often **Ethernet**) are **asynchronous packet networks**: **no guarantee when — or whether — a packet arrives**. After sending a request, any of these may have happened (Figure 8-1):

1. The **request was lost** (unplugged cable).
2. The request is **stuck in a queue** (network or recipient overloaded).
3. The **remote node failed** (crashed or powered down).
4. The remote node **temporarily stopped responding** (e.g. a long **garbage collection pause** — the runtime freezing the program to reclaim memory) but will resume.
5. The node **processed it, but the response was lost** (e.g. misconfigured switch).

6. The node processed it, but the **response is delayed**.

Figure 8-1 (redrawn): after sending a request with no reply, the sender cannot tell which of these happened



These cases are **indistinguishable**: you only know you have **not received a response yet**. The usual approach is a **timeout** — but when it fires you **still do not know** whether the request arrived, and a queued request may be delivered later anyway.

Network Faults in Practice

Networks are still not reliable after decades of engineering:

- A **medium-sized datacenter** saw about **12 network faults a month**: **half** disconnected a **single machine**, **half** an **entire rack**.
- A study of **load balancers**, **aggregation switches** (connect racks) and **top-of-rack switches** found **redundant gear** helps less than hoped, because it doesn't protect against **human error** (misconfigured switches), a **major cause** of outages.
- **Public clouds such as EC2** are notorious for **transient network glitches**; well-run private networks can be more stable.
- A **switch software upgrade** triggered a topology reconfiguration delaying packets **more than a minute** (the book's references cite a GitHub post-mortem; the index lists it as split brain from a 1-minute delay).
- **Sharks** bite **undersea cables**.
- A NIC **dropped all inbound packets while sending outbound fine** — one working direction says nothing about the other.

> **Network partitions.** A fault isolating one part of the network is a **network partition** or **netsplit**; the book says **network fault** to avoid confusion with database partitions (shards, Chapter 6). See

> also System Design Fundamentals.

Without **defined and tested** handling, anything can happen: a cluster may **deadlock** and stay unusable after the network recovers, or even **delete all your data**. Handling ≠ tolerating: on a usually reliable network it is fine to **show users an error**, but you must **know how the software reacts** and ensure it **recovers**. Deliberately **causing** network failures — the **Chaos Monkey** idea (Chapter 1) — is worthwhile.

Detecting Faults

Systems must detect faulty nodes: a **load balancer** takes dead nodes **out of rotation**; in **single-leader replication** (one leader takes writes, followers copy) a follower must be **promoted** when the leader fails (Chapter 5). Sometimes there is explicit feedback:

Situation	Feedback	Limitation
Machine reachable, **no process listening** on the port	OS refuses/closes the TCP connection with **RST** or **FIN**	If it crashed *mid-request*, you don't know how much was processed
Process crashed but **OS alive**	A script **notifies other nodes** to take over without waiting for a timeout — **HBase** does this	Needs the OS alive
Access to **switch management interface**	Query for **hardware-level link failures** (machine powered off)	Not over the internet, in shared datacenters, or if the problem blocks the interface
Router sure the IP is unreachable	**ICMP Destination Unreachable**	Routers have no magic detection either

Lessons: fast feedback is useful but **cannot be relied on**. A **TCP ack** doesn't mean the **application** handled the request — only an **application-level success reply** does. Plan for **silence**: retry a few times (TCP retries invisibly too), wait for a timeout, then **declare the node dead**.

Timeouts and Unbounded Delays

Timeout	Benefit	Cost
Long	Fewer false alarms	Slow detection; users wait or see errors
Short	Faster detection	Node wrongly declared dead during a brief slowdown

Declaring a node dead too early is dangerous: if it was mid-action (e.g. **sending an email**), the takeover node may **do it twice**. Its load shifts to others; if it was only **slow due to overload**, this can cause a **cascading failure** — at worst every node declares every other dead and the system halts.

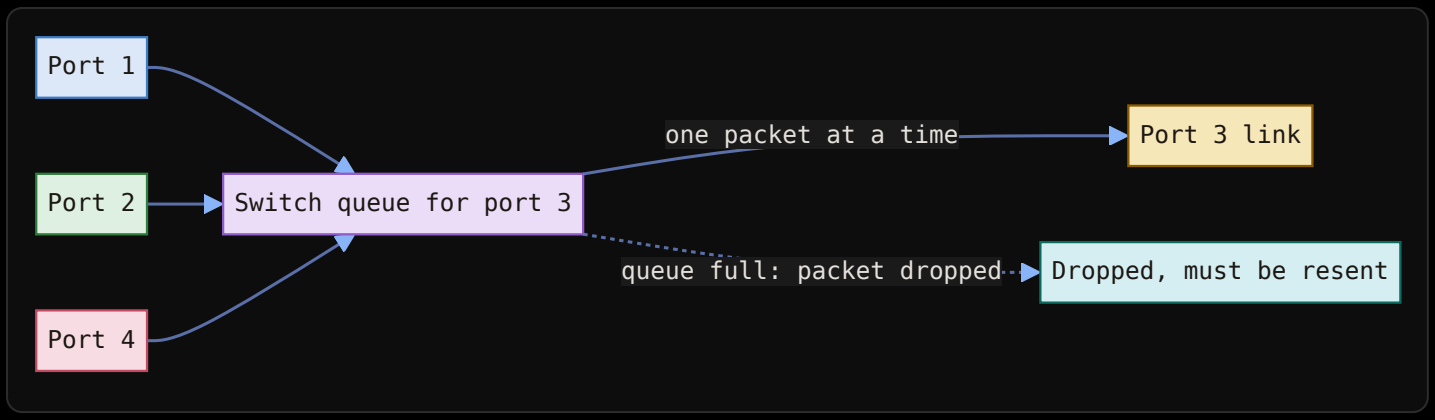
Ideal vs reality. If every packet were delivered within **d** or lost, and every live node answered within **r**, then **2d + r** would be a perfect timeout. Real systems have **neither** guarantee: asynchronous networks have **unbounded delays**, and most servers can't bound handling time. Being fast *most* of the time isn't enough — one **transient spike** in round-trip time can destabilise a system with a low timeout.

Network congestion and queueing

Delay variability, like road traffic, comes mostly from **queueing**:

1. **Switch queues:** several senders to one destination are fed onto its link **one at a time** (Figure 8-2) — **network congestion**. A **full queue drops** the packet, which must be resent though the network "works".
2. **OS queues:** if all **CPU cores are busy**, the incoming request waits for the application.
3. **VM pauses:** a guest is often **frozen for tens of milliseconds** while another VM uses the core; the **virtual machine monitor** (hypervisor) **buffers** incoming data meanwhile.
4. **TCP flow control** (**congestion avoidance**, **backpressure**): the sender limits its own rate, queueing **at the sender** before data enters the network.

Figure 8-2 (redrawn): ports 1, 2 and 4 all send to port 3, so the switch queue for port 3 fills up



TCP retransmission: a packet not acknowledged within a timeout (computed from observed round-trip times) is **retransmitted**; the app doesn't see the loss but **does see the delay**.

> **TCP Versus UDP. Videoconferencing** and **VoIP** use **UDP**: **no flow control**, **no retransmission**, so less delay variability (switch queues and scheduling still apply). It fits when **late data is worthless** — a lost VoIP packet becomes a moment of **silence**, and the retry happens at the **human layer** ("could you repeat that?").

Utilisation: queueing delays vary **most** near **maximum capacity**. *Shared environments:* in **public clouds and multi-tenant datacenters**, customers share links, switches, NICs and CPUs; **batch jobs like MapReduce** (Chapter 10) can **saturate** links, and a **noisy neighbor** makes delays vary wildly.

Choosing timeouts: measure the **round-trip distribution** over time and machines, then trade **detection delay** against **premature timeouts**. Better: **continuously measure response times and jitter and adapt** — the **Phi Accrual failure detector** does this in **Akka** (JVM message-passing toolkit) and **Cassandra**; **TCP retransmission timeouts** work similarly.

Synchronous Versus Asynchronous Networks

Why not build a network with **fixed maximum delay** that **never drops** packets? Compare the **fixed-line telephone network**, which is **extremely reliable**: dialling sets up a **circuit** — a **guaranteed slice of bandwidth** reserved along the **entire route** until the call ends. **ISDN** runs at **4,000 frames per second** and gives each call **16 bits per frame** in each direction, i.e. **16 bits every 250 microseconds**. Since space in the next hop is already reserved, nothing queues and the **worst-case latency is a known constant** — a **synchronous** network with **bounded delay**.

Can we not simply make network delays predictable?

Circuit (telephone)	TCP connection (packet-switched)
Fixed reserved bandwidth **nobody else can use**	**Opportunistically uses whatever bandwidth is available**
Suits constant-rate audio/video	Suits variable-sized blocks (email, web page) sent ASAP
Uses bandwidth for the whole call	**No bandwidth when idle** (except occasional **TCP keepalive**)

Ethernet and IP are packet-switched — queueing, **unbounded delays**, **no concept of a circuit** — because they are optimised for **bursty traffic**. A file transfer over a circuit needs a **bandwidth guess**: too low wastes capacity and is slow; too high and the network **refuses the circuit**. TCP instead **adapts its rate**.

Hybrids: **ATM** supported both. **InfiniBand** does **end-to-end flow control at the link layer**, reducing queueing (link congestion still delays). Careful **QoS** and **admission control** can **emulate circuit switching** or give **statistically bounded delay**.

> **Footnote — ATM.** Asynchronous Transfer Mode competed with Ethernet in the **1980s** but survived mainly in **telephone core switches**; unrelated to cash machines. The book muses video calls would be more

reliable in a universe where the internet ran on it.

> **Latency and Resource Utilization (sidebar).** Variable delay comes from **dynamic resource partitioning**. A cable for **10,000 simultaneous calls** is split **statically**: your circuit gets the same bandwidth even if **9,999 slots are empty**. The **internet shares bandwidth dynamically** — switches pick the next packet moment by moment: **queueing** in exchange for **maximum utilisation**, making each byte **cheaper**. **CPUs** are the same: dynamic sharing of a core means threads wait in the OS **run queue**, but hardware is used better (also a big reason for **VMs**). **Static partitioning** gives latency guarantees at lower utilisation (**more expensive**). Variable delay is **not a law of nature**, just a **cost/benefit trade-off**.

Reality: such QoS is **not enabled** in multi-tenant datacenters, public clouds or the internet, so **assume congestion, queueing and unbounded delays**; there is **no "correct" timeout** — determine it **experimentally**.

> **Footnote — BGP.** ISP peering and **Border Gateway Protocol** routes resemble circuit switching more than IP does (you can buy **dedicated bandwidth**), but at the level of **networks**, not host connections, and on a **much longer timescale**.

Unreliable Clocks

Clocks answer questions such as: (1) has this request timed out? (2) what is the **99th percentile** response time (only the slowest 1% take longer)? (3) average QPS over the **last five minutes**? (4) how long did a visitor stay? (5) when was this article published? (6) when should the reminder email go out? (7) when does this cache entry expire? (8) what is this log error's timestamp? Items **1–4 measure durations**; **5–8 are points in time**.

Why time is tricky: messages arrive after they're sent, but by a **variable** amount, so cross-machine **ordering** is hard. Each machine has its own **quartz crystal oscillator**, which is **not perfectly accurate**. Clocks are synchronised only **to a degree**, usually via the **Network Time Protocol (NTP)**, which takes time from a group of servers that in turn use an accurate source such as a **GPS receiver**.

Monotonic Versus Time-of-Day Clocks

Time-of-day clocks

Returns the calendar date and time (**wall-clock time**): `clock_gettime(CLOCK_REALTIME)` on Linux, `System.currentTimeMillis()` in Java — seconds/ms since the **epoch** (**00:00 UTC, 1 January 1970**, Gregorian), **ignoring leap seconds**. (The "real-time" in `CLOCK_REALTIME` is unrelated to real-time OSes.)

- Usually **NTP-synced**, so timestamps are ideally comparable across machines.
- A clock far **ahead** of the NTP server may be **forcibly reset**, appearing to **jump back in time**. Jumps plus ignored leap seconds make it **a poor choice for measuring durations**.
- Historically **coarse resolution** (e.g. **10 ms** steps on older Windows).

Monotonic clocks

For **durations** (timeouts, response times): `clock_gettime(CLOCK_MONOTONIC)` on Linux, `System.nanoTime()` in Java. Guaranteed to **never go backward**. Read it, do work, read again — the **difference** is elapsed time.

- **Absolute value is meaningless** (e.g. ns since boot) — **never compare across computers**.
- On multi-**socket** servers each CPU may have its own unsynchronised timer; the OS compensates, but treat that with **some scepticism**.
- **NTP slewing** may change its **rate** by **up to 0.05%**, but **cannot make it jump**.
- Resolution is usually **microseconds or finer**.

Monotonic clocks are **generally fine** for timeouts in distributed systems — no cross-node sync needed.

	Time-of-day clock	Monotonic clock
Answers	"What time is it?" (point in time)	"How much time passed?" (duration)
Linux / Java	`CLOCK_REALTIME` / `System.currentTimeMillis()`	`CLOCK_MONOTONIC` / `System.nanoTime()`
Can jump backward?	Yes (NTP reset)	No (only slewed up to 0.05%)
Comparable across machines?	Ideally, if synced	No
Needs NTP?	Yes	No

Clock Synchronization and Accuracy

Getting the right time is **far less reliable** than hoped:

- **Quartz drift** varies with **temperature**. **Google assumes 200 ppm — 6 ms** if resynced every **30 seconds**, **17 seconds** if resynced **once a day**. Drift caps accuracy even when all works.
- **Forced resets:** if a clock differs too much from NTP, the client may **refuse to sync** or **reset** it; apps see time **jump backward or forward**.
- **Firewalled from NTP:** a node may silently lose NTP access for a while.
- **Network delay limits NTP:** one experiment got a **minimum error of 35 ms** over the internet, with spikes to **about a second**; with large delays the client may **give up**.
- **Wrong NTP servers** can be off by **hours**. Clients query **several servers and ignore outliers**, but correctness still rests on **a stranger on the internet**.
- **Leap seconds** (59- or 61-second minutes) have **crashed many large systems** (the book's references include a 2012 Linux-bug leap second that crashed "half the internet", and Cloudflare DNS in 2017). Best fix may be NTP servers **"lying"** by **smearing** the second over a day; real behaviour varies.
- **VMs:** the hardware clock is **virtualised**; a VM paused **tens of milliseconds** sees its clock **jump forward**.
- **Devices you don't control:** don't trust the clock — some users **set wrong dates on purpose** (e.g. to dodge **game** time limits).

High accuracy costs effort: the **MiFID II** draft European regulation requires **high-frequency trading funds** to be **within 100 microseconds of UTC**, to investigate **"flash crashes"** and detect **market manipulation**. Achievable with **GPS receivers**, the **Precision Time Protocol (PTP)**, careful deployment and monitoring — but a misconfigured NTP daemon or a firewall blocking NTP lets drift grow fast.

Relying on Synchronized Clocks

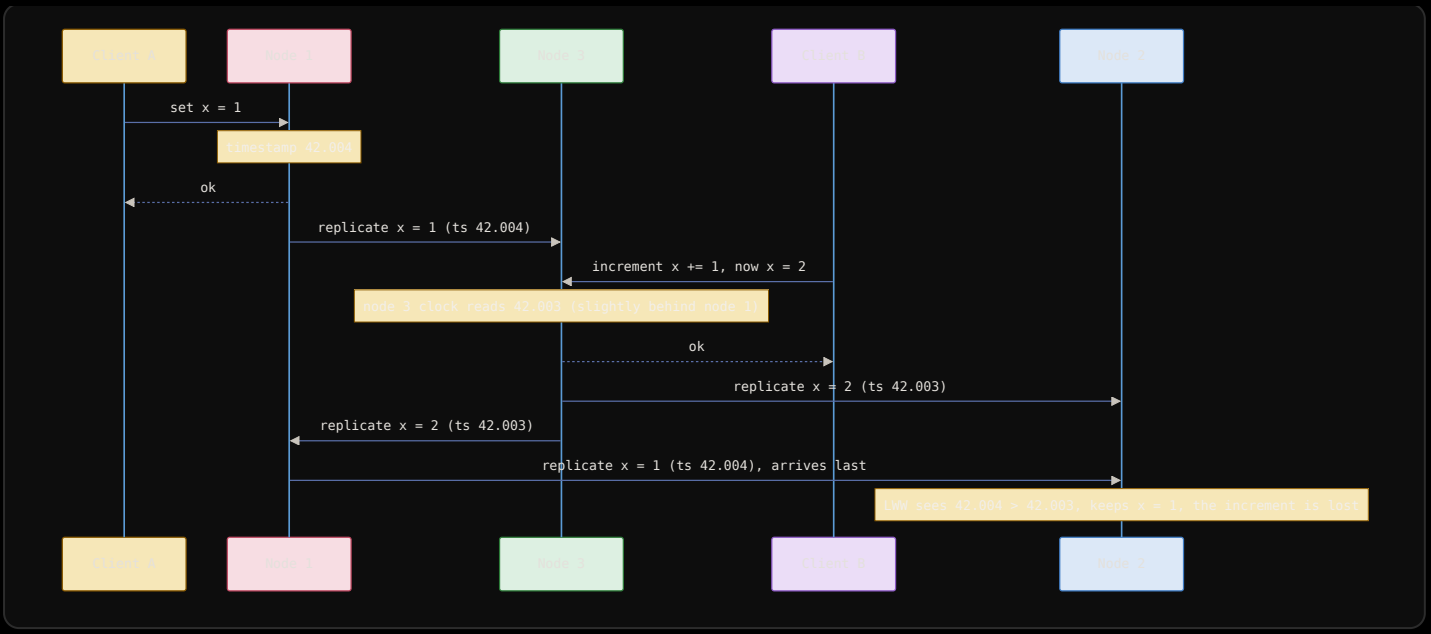
A day is **not always 86,400 seconds**, time-of-day clocks can **run backward**, and nodes' clocks can differ a lot. As with networks, **robust software must be prepared for wrong clocks**.

Clock faults are sneaky: a bad CPU or network gets noticed fast, but a bad quartz or NTP config leaves things **looking fine** while the clock **drifts further**, causing **silent, subtle data loss** rather than a crash. If you depend on synced clocks, **monitor offsets between all machines** and **kick out** any node that drifts too far.

Timestamps for ordering events

Figure 8-3 — a **multi-leader** database (several nodes accept writes and replicate to each other; cf. Figure 5-9): **client A** sets `x = 1` on **node 1**; it replicates to **node 3**; **client B** increments x on node 3 to `x = 2`; both replicate to **node 2**. Each write carries its **origin node's time-of-day timestamp**. Even with **< 3 ms** skew (better than typical), `x = 1` gets **42.004 s** and `x = 2` gets **42.003 s**. Node 2 keeps `x = 1` and **drops B's increment**.

Figure 8-3 (redrawn): client B's write is causally later but gets an earlier timestamp, so node 2 discards it under LWW



This is **last write wins (LWW)**, used in multi-leader and leaderless databases (any replica accepts writes) such as **Cassandra** and **Riak**. Generating timestamps on the **client** doesn't fix its problems:

- **Writes mysteriously vanish:** a node whose clock is **behind** can't overwrite values from a node **ahead** until the skew elapses — data **silently dropped**.
- **Can't tell sequential from concurrent writes** — that needs **causality tracking** such as **version vectors** (Chapter 5).
- **Identical timestamps** (likely at **millisecond** resolution) need a **tiebreaker** (e.g. random number), which can also **violate causality**.

"Most recent" is decided by **one machine's possibly wrong clock**: a packet sent at **100 ms** may be received at **99 ms** by the receiver's clock. NTP can't fix this — its accuracy is bounded by **network round-trip time**, the very thing you'd need to beat.

Safer: ***logical clocks*** — **incrementing counters** that capture only **relative order**, not time. Time-of-day and monotonic clocks are **physical clocks**. (More in Chapter 9.)

Clock readings have a confidence interval

Resolution ≠ accuracy. Drift can be **several ms** even with **local NTP every minute**; via **public-internet** NTP, **tens of ms** at best, spiking **over 100 ms** under congestion. Treat a reading as an **interval with a confidence level** (e.g. **95% sure** it's **10.3–10.5 s** past the minute); with **±100 ms** uncertainty the microsecond digits mean nothing.

Uncertainty: a directly attached **GPS receiver** or **atomic (caesium) clock** has a manufacturer-stated error; from a **server**, it is roughly **drift since last sync + server uncertainty + round-trip time**. Most systems **don't expose** it — `clock_gettime()` can't tell you if it's off by **five ms or five years**.

Exception — Google's ***TrueTime*** *API in* ***Spanner*** returns `[earliest, latest]`; the true time lies inside. Width depends on **time since the last sync** with an accurate source.

Synchronized clocks for global snapshots

Snapshot isolation (Chapter 7 — each transaction reads a consistent snapshot as of its start) lets **small read-write** transactions coexist with **large read-only** ones (backups, analytics) without locking. It usually uses a **monotonically increasing transaction ID**: writes with a larger ID are **invisible** to the snapshot. A **counter** works on one node.

Distributed problem: a **global** ID across partitions and datacenters needs **coordination** and must respect **causality** (if B read A's write, B's ID must be larger). With many small transactions this becomes an **untenable bottleneck**.

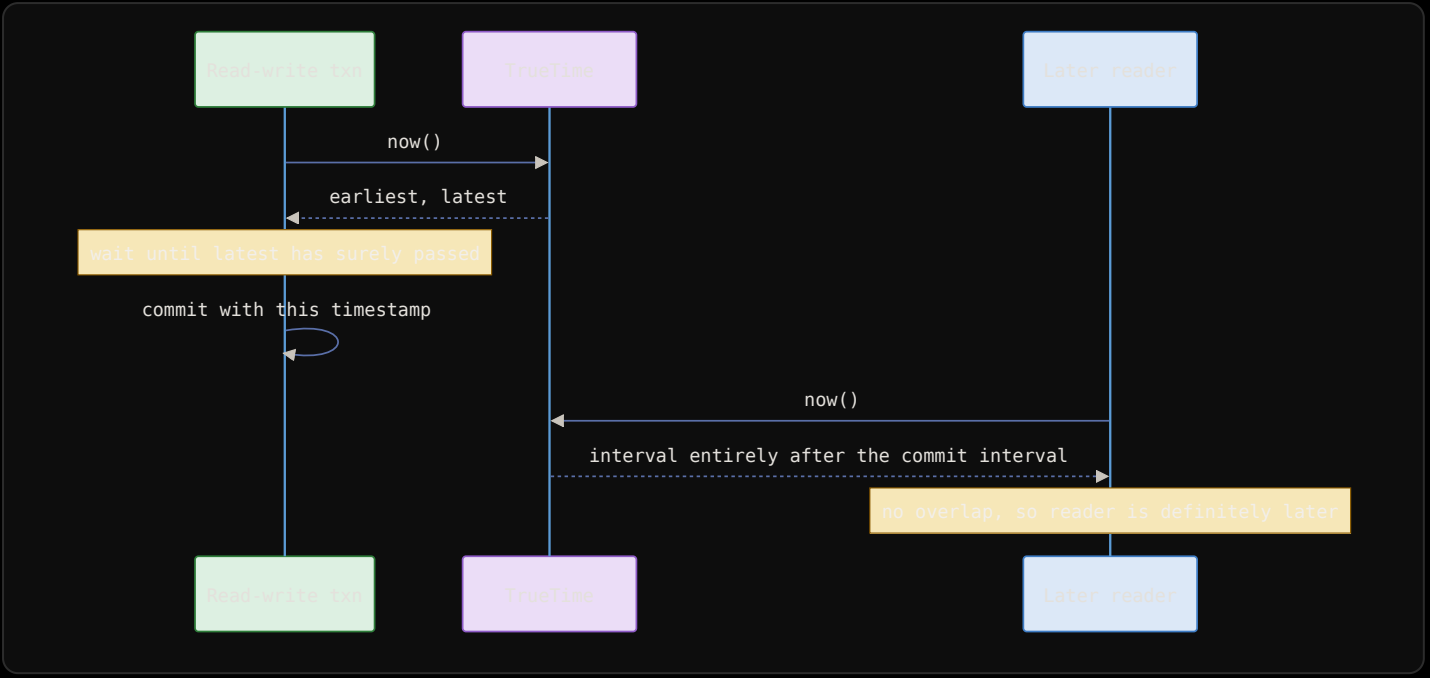
> **Footnote — Snowflake.** Generators like **Twitter's Snowflake** scalably produce **approximately** increasing IDs (e.g. by handing **ID ranges** to nodes), but usually **can't guarantee causal order**,

> since blocks are allocated on a longer timescale than reads/writes.

Use clock timestamps as transaction IDs? Works if sync is good enough — the obstacle is uncertainty. **Spanner's approach** (snapshot isolation **across datacenters**):

1. Each timestamp is an interval: A = [A_earliest, A_latest], B = [B_earliest, B_latest].
2. If they **don't overlap** (A_earliest < A_latest < B_earliest < B_latest), **B definitely followed A**; if they overlap, order is uncertain.
3. Spanner **waits for the interval width before committing a read-write transaction**, so any later reader's interval cannot overlap.
4. To keep the wait short, Google puts a **GPS receiver or atomic clock in every datacenter**, keeping uncertainty around **7 ms**.

Spanner commit-wait (illustrative, not a book figure)



Clock-based distributed transaction semantics are an **active research area**, **not yet in mainstream databases outside Google**.

Process Pauses

How does a partition **leader** know it is **still leader** and may accept writes? It holds a **lease** from the other nodes — a **lock with a timeout**. Only one node holds it; the holder must **renew** before expiry, and if it fails, another node takes over after expiry.

A request loop in that style (my own version; the book's uses a **10-second** buffer and expiry computed elsewhere as e.g. **now + 30 s**):

```
while (running) {
    Job job = queue.nextJob();

    // Renew early: keep at least 5 seconds of lease left
    long remaining = myLease.expiresAtMillis() - System.currentTimeMillis();
    if (remaining < 5000) {
        myLease = myLease.extend();
    }

    if (myLease.stillValid()) {
        handle(job);
    }
}
```

What is wrong with it?

1. **It relies on synchronised clocks:** the expiry was set by **another machine** but compared with the **local** clock; a few seconds of skew breaks it.
2. **Even with a local monotonic clock**, it assumes little time passes between **checking** and **handling**. If the thread **freezes for 15 seconds** right after the validity check (the book's `lease.isValid()`), the lease has lapsed and **another node may be leader** — and the thread won't notice until the **next iteration**, after doing something **unsafe**.

Are 15-second pauses realistic? Yes:

- **Garbage collection: stop-the-world** pauses in runtimes like the **JVM** have lasted **several minutes**. Even "concurrent" collectors like **HotSpot JVM's CMS** (Concurrent Mark Sweep) stop the world sometimes. Tuning helps, but **assume the worst**.
- **VM suspend/resume:** a VM can be suspended (memory saved to disk) for any length of time — used for **live migration**, where pause length depends on how fast processes write memory.
- **End-user devices** suspend arbitrarily (laptop **lid closed**).
- **Context switches:** the OS or **hypervisor** can preempt a thread anywhere; CPU time lost to other VMs is **steal time**; a long run queue lengthens the wait.
- **Synchronous disk I/O**, sometimes unexpected — e.g. the **Java classloader lazily loads class files** on first use. **I/O and GC pauses combine.** If the "disk" is a **network filesystem** or **network block device** (e.g. **Amazon EBS**), I/O inherits **network delay variability**.
- **Swapping (paging):** a memory access can trigger a **page fault** and disk load; under pressure the OS may **thrash**. Hence paging is often **disabled on servers** (better to kill a process).
- **SIGSTOP** (e.g. **Ctrl-Z**) pauses a process until **SIGCONT** — possibly sent **by accident** by an ops engineer.

The core problem: a thread can be **preempted** anywhere and resumed **without noticing**. On one machine, thread-safety tools exist — **mutexes**, **semaphores**, **atomic counters**, **lock-free data structures**, **blocking queues** — but they **don't translate** to distributed systems, which have **no shared memory**. Each node must assume it may be **frozen at any instant**, even mid-function, while others **declare it dead**.

Response time guarantees

These pauses **can be eliminated** with enough effort. **Hard real-time systems** — controlling **aircraft, rockets, robots, cars** — have **deadlines** whose violation may fail the whole system (you don't want the **airbag delayed by a GC pause**).

> **Is real-time really real?** In **embedded systems**, "real-time" means engineered and tested to **always meet timing guarantees**. On the **web** it loosely means pushing data to clients or stream processing, with **no hard constraints** (Chapter 11).

Requirements (every layer): (1) a **real-time operating system (RTOS)** guaranteeing CPU time in specified intervals; (2) **library functions** documenting **worst-case execution times**; (3) **dynamic memory allocation** restricted or **forbidden** (real-time GCs exist, but must not be overloaded); (4) **enormous testing and measurement**.

Costs: much extra work and a narrow choice of languages/tools, so it is **very expensive** and used mainly in **safety-critical embedded devices**. **"Real-time" ≠ "high-performance"** — throughput may be **lower** since deadlines come first. For **server-side data systems** it is **not economical**, so they **live with** pauses and clock instability.

Limiting the impact of garbage collection

Runtimes can choose **when** to collect (they know allocation rate and free memory):

- **GC as a planned outage:** the runtime **warns** the app a pause is coming; the app **stops routing new requests** to that node, **waits** for in-flight ones, then the node collects with nothing in progress. Hides pauses from clients and cuts **high-percentile latency**; used in some **latency-sensitive financial trading systems**.
- **Only collect short-lived objects, restart periodically** before long-lived objects need a **full GC** — **one node at a time**, shifting traffic away first, like a **rolling upgrade**.

These **reduce** but **cannot fully prevent** pauses.

Knowledge, Truth, and Lies

With **no shared memory**, only **message passing over an unreliable network**, plus **partial failures, unreliable clocks and pauses**, no node can be **certain** of anything — it can only **infer** from messages received (or not). If a remote node is silent, its state is **unknowable**: network problems can't be told from node problems.

The practical answer: system models. State assumptions about behaviour — the **system model** — and design to meet them. **Algorithms can be proved correct within a system model**, so reliability is possible even when the model promises little. Chapter 9 gives such algorithms.

The Truth Is Defined by the Majority

1. **Asymmetric fault:** a node **receives** fine but its **outgoing** messages are lost. It is **declared dead** — carried to the graveyard shouting it's alive, unheard.
2. **Aware but powerless:** it notices its messages aren't acked and realises there's a network fault, but **can do nothing**.
3. **Long GC pause:** all threads stop for **one minute**; others declare it dead. It then resumes as if nothing happened — a corpse sitting up in its coffin — unaware a minute passed.

Moral: a node **cannot trust its own judgement**, and the system must not depend on **any single node**.

Quorums (Chapter 5): decisions need a **minimum number of votes**. This includes **declaring nodes dead** — if a quorum says so, the node **must step down** even if it feels alive. Usually a strict **majority**: **3 nodes survive 1 failure**, **5 survive 2**. It is **safe** because there can be **only one majority** at a time. (Quorums for consensus: Chapter 9.)

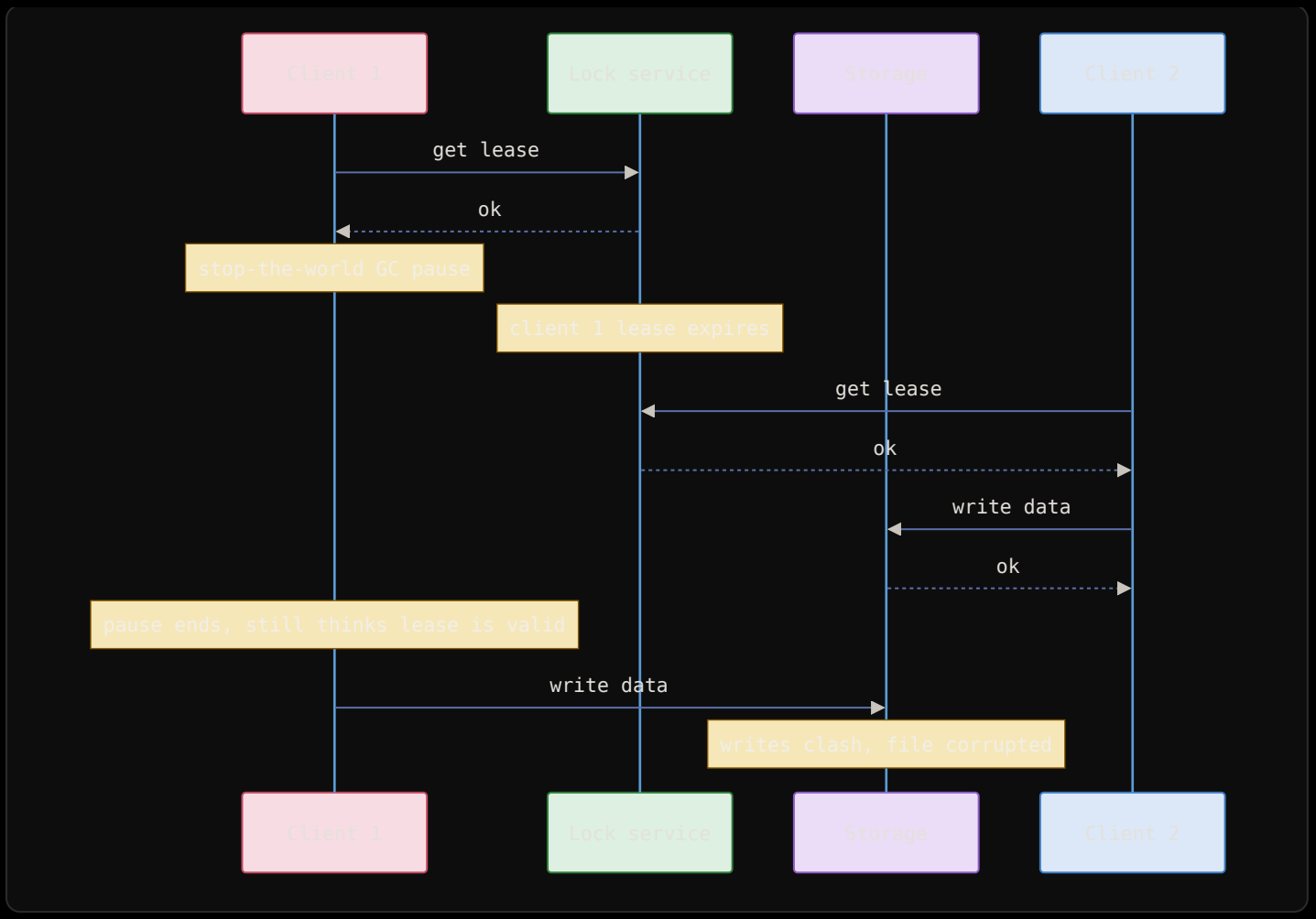
The leader and the lock

Systems often need **exactly one** of something: **one leader** per partition (avoid **split brain**), **one lock holder** per resource, **one owner** per **username**. A node may **believe it is "the chosen one"** while a **quorum** has declared it dead (network interruption, GC pause) and **elected** a replacement; if it keeps acting the part, others may believe it.

*Figure 8-4 — a real bug (**HBase used to have it**).* A storage file must be written by **one client at a time**; clients get a **lease** from a **lock service** first:

1. Client 1 gets the lease, then **pauses** (GC) until it **expires**.
2. Client 2 gets the lease and **writes**.
3. Client 1 resumes, **thinks its lease is valid**, and **also writes** — the file is **corrupted**.

Figure 8-4 (redrawn): client 1's lease expires during a GC pause, but it still writes, corrupting the file



Fencing tokens

A node that **falsely believes** it holds the lock must be **unable to disrupt** the system. **Fencing** (Figure 8-5):

1. Each lock/lease grant includes a **fencing token** that **increases on every grant**.
2. Every write to storage **includes the token**.
3. Storage **remembers the highest token processed** and **rejects lower ones**.

Client 1 gets token **33**, pauses, its lease expires; client 2 gets **34** and writes; client 1 then writes with 33, which storage **rejects** because it has seen 34.

Figure 8-5 (redrawn): storage accepts writes only in increasing token order, so the stale token 33 is rejected



With **ZooKeeper** (coordination service offering locks, leader election, small config data), the transaction ID `zxid` or node version `cversion` serve as tokens — both **monotonically increasing**.

Key requirement: the **resource itself must check tokens**; clients checking their own lock status is **not enough**. For resources without support, work around it (e.g. **token in the filename**). Server-side checking is **good practice** anyway: services should **protect themselves from accidentally abusive clients**, often run by people with different priorities.

Byzantine Faults

Fencing catches nodes wrong **by accident**; a **deliberately** malicious node could send a **fake token**. The book assumes nodes are **unreliable but honest**: slow, silent or outdated, but whatever they say is "true" as far as they know.

A **Byzantine fault** is a node **"lying"** — arbitrary or corrupted responses, e.g. claiming to have received a message it never got. Reaching consensus despite this is the **Byzantine Generals Problem**.

> **The Byzantine Generals Problem (sidebar).** It generalises the **Two Generals Problem** — two generals agreeing on a plan via **messengers** who may be delayed or lost (consensus: Chapter 9). Here there are **n generals**, some **traitors** sending fake messages, and **nobody knows in advance** who. *Name:* **Byzantium** became **Constantinople**, now **Istanbul**; "Byzantine" means **excessively complicated, bureaucratic, devious** — no evidence its generals were worse. **Lamport** wanted a nationality nobody would be offended by; he was warned off "**The Albanian Generals Problem**".

A **Byzantine fault-tolerant** system works correctly even if some nodes **disobey the protocol** or **attackers** interfere. Relevant in:

- **Aerospace: radiation** corrupts memory/registers; failure is catastrophic (a crash, or a **rocket hitting the ISS**), so **flight control must tolerate Byzantine faults**.
- **Multiple organisations** who may **cheat**: **Bitcoin and other blockchains** (peer-to-peer networks sharing a tamper-evident ledger) let mutually distrusting parties agree on transactions **without a central authority**.

Why the book usually ignores them:

- In **your own datacenter** nodes are yours and **radiation** is low.

- BFT protocols are **complicated** and embedded BFT needs **hardware support** — **impractical cost** for most server-side systems.
- **Web apps** must expect **malicious clients**, hence **input validation, sanitisation and output escaping** (against **SQL injection** and **cross-site scripting**) — but the **server is the authority**, so no BFT protocol is needed. BFT matters more in **peer-to-peer** networks.
- **Software bugs** could be Byzantine faults, but BFT needs a **supermajority (> 2/3)** of correct nodes (**4 nodes tolerate 1** bad), so you'd need **four independent implementations**.
- Against **attacks**, an intruder who compromises one node can likely compromise **all** (same software). **Authentication, access control, encryption, firewalls** remain the main defence.

Weak forms of lying

Cheap pragmatic guards against invalid messages from **hardware issues, bugs, misconfiguration** (not full BFT):

- **Corrupted packets: TCP/UDP checksums** usually catch them but sometimes miss — add **application-level checksums**.
- **Input sanitising:** public apps check value **ranges** and **string length limits** (preventing **denial of service** via huge allocations). Internal services may relax this, but keep **basic sanity checks** in protocol parsing.
- **Multiple NTP servers:** the client queries all, **estimates each one's error**, checks that **most agree**, and excludes an **outlier**.

System Model and Reality

Algorithms for problems like **consensus** (nodes agreeing on one value, Chapter 9) must tolerate these faults without depending on specific hardware, so we formalise faults in a **system model** — what an algorithm may assume.

Timing models:

Model	Assumption	Realistic?
Synchronous	**Bounded** delay, pauses and clock error (a known upper bound, not zero)	**No** for most systems
Partially synchronous	Synchronous **most of the time**, but bounds are **sometimes exceeded arbitrarily**	**Yes** — realistic for many systems
Asynchronous	**No timing assumptions** — **no clock**, hence **no timeouts**	Possible but **extremely limiting**

Node failure models:

Model	Assumption
Crash-stop	A node fails only by **crashing** and is **gone forever**.
Crash-recovery	Nodes crash and **maybe resume**; **stable storage** survives, **in-memory state is lost**.
Byzantine (arbitrary)	Nodes may do **anything**, including mislead others.

The most useful combination for real systems is **partial synchrony + crash-recovery**.

Correctness of an algorithm

Define correctness by **properties** (as "sorted" means each element is smaller than those to its right). For **fencing-token generation**:

- **Uniqueness:** no two requests get the **same value**.
- **Monotonic sequence:** if request x **completed before** y began, then **$t_x < t_y$**.
- **Availability:** a non-crashed requester **eventually** gets a response.

An algorithm is **correct within a system model** if its properties hold in **every** scenario the model permits. But if all nodes crash, nothing can work — hence the next distinction.

Safety and liveness

Uniqueness and monotonic sequence are **safety**; availability is **liveness**. Liveness properties often contain **"eventually"** — **eventual consistency is a liveness property**. Informally: safety = **nothing bad happens**; liveness = **something good eventually happens**. Precisely:

	Safety	Liveness
When violated	You can **point to the moment** it broke (the duplicate token)	May be unmet now (no reply yet)
Recoverable?	**No** — damage done	**Yes** — may still be satisfied later
Requirement	Must **always** hold, even if all nodes crash or the network fails — **never a wrong result**	May have **caveats**, e.g. only if a **majority is up** and the network **eventually recovers**

The **partially synchronous** model requires the system to **eventually return to synchrony** — any interruption lasts a **finite** time.

Mapping system models to the real world

Models simplify; reality bites:

- Crash-recovery assumes **stable storage survives** — but disks can be **corrupted or wiped**, or a **firmware bug** may stop a server **detecting its own drives after reboot** (the book's references cite a GitHub incident).
- **Quorum algorithms** assume nodes **retain data they claimed to store**; a node with **amnesia** breaks correctness. Modelling "storage usually survives" is **harder to reason about**.

A real implementation may still need code for the "impossible" case, even just exiting with an error for a **human** to clean up — perhaps the difference between **computer science** and **software engineering**. My own illustration:

```c
if (stored_version > proposed_version && is_leader) {
    fprintf(stderr, "invariant broken: stale leader wrote\n");
    exit(99);   /* stop and let an operator investigate */
}
```

Models are still **incredibly helpful**: they reduce reality to a **manageable set of faults**, and proving properties within a model is how algorithms are **proved correct**. A proof doesn't guarantee a correct implementation, but it exposes flaws that might hide for years. **Theory and real-world testing matter equally.**

Chapter summary

(Book section: Summary)

- **Networks:** any packet or reply may be **lost or arbitrarily delayed**; with no reply you **can't tell** what happened.
- **Clocks:** can be **far out of sync** despite NTP, **jump**, and usually have an **unknown error interval**.
- **Process pauses:** a process can **pause for a long time** (GC), be **declared dead**, and resume **unaware**.
- **Partial failure** is the **defining characteristic** of distributed systems; we build **tolerance** into software.
- **Detecting faults is hard: timeouts** can't distinguish network from node failure and cause **false suspicion**. **Limping nodes** — e.g. a **driver bug** dropping a **Gigabit NIC to 1 Kb/s** — can be harder to handle than dead ones.
- **Tolerating faults is hard:** no **shared memory, global state, or agreed time**; **major decisions can't be made by one node**, so we need **quorum** protocols.
- **If one machine suffices, keep Pandora's box shut** — but distribution is also needed for **fault tolerance** and **low latency**, not only **scalability**.
- **Unreliability isn't a law of nature:** bounded delays and real-time guarantees are **possible but expensive**; most systems choose **cheap but unreliable**.

- **Supercomputers** stop entirely on a fault; **distributed systems can run forever**, handling faults **per node** (though a **bad config rolled to all nodes** still brings them down).
- **Chapter 9** moves to solutions.

Interview angle

1. Why is a timeout the only practical way to detect failures, and how do you choose one? *30-second answer:* In an asynchronous network a lost request, dead node, paused node and lost reply all look like silence. Explicit signals (TCP RST/FIN, HBase-style crash scripts, switch queries, ICMP unreachable) help but can't be relied on. Too short a timeout causes false positives, duplicate actions and cascading failures; too long delays recovery. Measure round-trip distributions and choose experimentally, or adapt with jitter-aware detectors like Phi Accrual (Akka, Cassandra).

- *Follow-up: Why not bounded delays like the phone network?* Ethernet/IP are packet-switched for bursty traffic; circuits reserve fixed bandwidth (ISDN: 16 bits every 250 µs) and waste capacity. Bounded delay via static partitioning or QoS is a cost/utilisation trade-off not enabled in clouds.
- *Follow-up: If TCP acked my packet, did the server process it?* Not necessarily — the app may crash first. You need an application-level response.

2. Time-of-day vs monotonic clocks — which for what? *30-second answer:* Time-of-day (`System.currentTimeMillis()`) gives a point in time since the 1970 epoch; NTP-synced, can jump backward, ignores leap seconds — bad for durations. Monotonic (`System.nanoTime()`) only moves forward, slewed at most 0.05%, meaningless absolute value, not comparable across machines — use it for timeouts and durations.

- *Follow-up: How accurate is NTP?* Limited by network delay: ~35 ms minimum over the internet, spikes around a second; Google's 200 ppm drift is 6 ms per 30 s or 17 s per day.

3. Why is last-write-wins with timestamps dangerous? *30-second answer:* Skew can give a causally later write an earlier timestamp (Figure 8-3: 42.003 vs 42.004 with < 3 ms skew), so it's silently dropped. LWW can't tell sequential from concurrent writes, and tiebreakers can break causality. Cassandra and Riak use LWW. Use logical clocks/version vectors for ordering.

- *Follow-up: How does Spanner use clocks safely?* TrueTime returns [earliest, latest]; Spanner waits out the interval before committing so intervals don't overlap; GPS/atomic clocks per datacenter keep uncertainty ~7 ms.

4. How can a leader or lock holder be wrong about holding its lease? How do you fix it? *30-second answer:* Process pauses — GC (sometimes minutes), VM suspend/live migration, context switches/steal time, disk I/O (lazy classloading, EBS), swapping/thrashing, SIGSTOP. It resumes believing its lease is valid and writes (the HBase bug). Fix: fencing tokens — increasing numbers per lease (ZooKeeper zxid/cversion) checked by storage, rejecting stale ones (33 after 34).

- *Follow-up: Do fencing tokens stop malicious nodes?* No — a liar can fake a token. That needs Byzantine fault tolerance (>2/3 honest nodes), rarely worth it inside one datacenter.

5. What system model do real systems use, and what are safety vs liveness? *30-second answer:* Partially synchronous timing with crash-recovery faults. Safety ("nothing bad happens") must always hold; a violation is permanent and pinpointable. Liveness ("something good eventually happens") can have caveats like "if a majority is up and the network recovers". Eventual consistency is liveness.

- *Follow-up: Why a majority quorum?* Only one majority can exist at a time, so decisions can't conflict; 3 nodes tolerate 1 failure, 5 tolerate 2.

Self-check

1. **List the six things that can happen when you send a request and get no reply.** Request lost; request queued; remote node crashed; remote node paused (e.g. GC); processed but reply lost; processed but reply delayed.
2. **How do supercomputers handle a node failure, and why doesn't that work for internet services?** Checkpoint, stop the whole job, fix, restart from checkpoint — partial becomes total failure.

Internet services must stay online with low latency on failure-prone commodity hardware, and at thousands of nodes something is always broken.

3. **What did the medium-sized datacenter study find?** About 12 network faults per month; half disconnected one machine, half a whole rack.
4. **Scenario:** a very short timeout on an overloaded cluster. What might happen? Slow-but-alive nodes are declared dead, their load shifts to others, which slow down and are declared dead too — cascading failure; actions like sending emails may run twice.
5. **Why can ISDN guarantee delay while Ethernet can't?** ISDN reserves 16 bits per frame (4,000 frames/s) along the whole route, so nothing queues. Ethernet/IP share bandwidth dynamically, so queues form and delay is unbounded.
6. **Trade-off:** why would a VoIP app choose UDP over TCP? Late audio is worthless; UDP avoids retransmission and flow-control delays, and the app fills gaps with silence.
7. **Google's 200 ppm drift — what are the numbers?** 6 ms drift with resync every 30 s, 17 s with resync once a day.
8. **What is a clock confidence interval and which API exposes it?** A range containing the true time (e.g. 95% sure it's 10.3–10.5 s past the minute). Spanner's TrueTime returns [earliest, latest]; `clock_gettime()` doesn't.
9. **Scenario:** storage receives a write with token 41 after processing token 42. What should it do and why must storage do it? Reject it — 41 comes from a holder whose lease expired. The resource must check because a paused client can't know its lease lapsed.
10. **Classify: "no two nodes are ever leader for the same term" and "every request eventually gets a response." Which can have caveats?** The first is safety (always holds); the second is liveness, which may be conditioned on a majority being up and the network eventually recovering.

Chapter 9 — Consistency and Consensus

Book: Chapter 9, pp. 321–383

Overview

Chapter 8 catalogued what breaks: packets dropped, duplicated, reordered or delayed; clocks drift; processes pause (garbage collection) or crash. Chapter 9 asks: **given all that, how do we build systems that still behave correctly?** The cheap answer is to let the service fail; otherwise we need *fault tolerance*.

The strategy mirrors transactions (Chapter 7), which let an app *pretend* crashes don't happen (atomicity), nobody else touches the data (isolation), and disks don't lose data (durability). Chapter 9 looks for **general-purpose abstractions for distributed systems**, implemented once and relied on by many applications.

The most important is **consensus**: getting nodes to agree on something. Classic use: a single-leader database (the *leader* accepts all writes and copies them to *followers*) loses its leader and the rest must agree on *one* new leader. Two self-declared leaders (**split brain**) usually lose data. Doing consensus correctly despite faults is surprisingly hard.

The chapter builds up in three steps:

1. **Linearizability** — among the strongest consistency models (precise promises about what reads may return): replicated data behaves like one copy.
2. **Ordering guarantees** — causality, total orders, Lamport timestamps, and **total order broadcast**, which is deeply linked to linearizability and consensus.
3. **Distributed transactions and consensus** — two-phase commit (2PC), why it is a weak form of consensus, and real fault-tolerant algorithms (Paxos, Raft, Zab, Viewstamped Replication) and services built on them (ZooKeeper, etcd).

It also explains the *limits* of what is possible (CAP theorem, FLP result — see System Design Fundamentals) intuitively, without proofs. The epigraph (Jay Kreps, 2013) asks whether it is better to be available but wrong or correct but down — the tension running through the chapter.

> **Analogy.** Friends picking a restaurant over a flaky group chat. *Linearizability* = everyone reads one shared whiteboard. *Causality* = nobody sees "see you there!" before the message naming the place. *Consensus* = the group settles on one restaurant, nobody changes their mind later, and it still works if a couple of phones die.

Key terms

Term	Plain-English meaning
Fault tolerance	Working correctly although some components have failed.
Consensus	Several nodes irrevocably agree on one value despite failures.
Split brain	Two nodes both act as leader; data diverges.
Eventual consistency (convergence)	If writes stop, replicas *eventually* return the same value — no bound on when.
Consistency model	A promise about what values reads may return under replication and concurrency.
Linearizability	Aka atomic, strong, immediate or external consistency: behaves like one copy with each operation atomic at one instant; a *recency* guarantee.
Register	Theory term for one readable/writable object — a key, row or document.
Regular register	Reads overlapping a write may return old or new (and flip-flop); weaker than linearizable.
Compare-and-set (CAS)	Atomically "set x to *new* only if x equals *old*, else fail".
Serializability	Transactions behave as if run one after another in *some* order (Chapter 7).
Strict serializability (strong-1SR)	Serializability plus linearizability.
Quorum	Subset of nodes (usually a majority) whose agreement is required; Dynamo-style: w + r > n.
CAP theorem	Brewer, 2000: under a network partition, choose linearizability or total availability.
Network partition	Nodes alive but unable to talk (not data partitioning).
CP / AP	"Consistent but unavailable when partitioned" / "available but inconsistent"; the book advises avoiding these labels.
Memory barrier / fence	CPU instruction forcing caches/memory to sync so another core sees a write.
Causality / happens-before	Cause before effect; A happened before B if B could have known about or depended on A.
Concurrent operations	Neither happened before the other.
Causally consistent	Respects the order imposed by causality.
Total vs partial order	Every pair comparable (numbers) vs only some pairs (sets, causality).
Version vector	Per-node counters telling concurrent from causally related operations (Chapter 5).
Sequence number / logical clock	Counter-based number ordering operations without physical clocks.
Lamport timestamp	(counter, node ID), counters bumped to the max seen; total order consistent with causality.
Total order broadcast (atomic broadcast, total order multicast)	Every node receives the same messages in the same order, none lost.
State machine replication	Replicas stay identical by applying the same deterministic writes in the same order.
Fencing token / zxid	Number increasing per lock grant, used to reject stale holders / ZooKeeper's monotonically increasing transaction ID.

Term	Plain-English meaning
Sequential consistency (timeline consistency)	Same order for everyone, but reads may be stale; slightly weaker than linearizability.
Chain replication	Reads go to a replica updated synchronously on writes.
Atomic commit	All nodes of a distributed transaction commit, or all abort.
Two-phase commit (2PC)	*Prepare* (vote) phase, then *commit/abort* phase.
Coordinator (transaction manager) / participant	Runs 2PC and records the decision / a database node in the transaction.
Commit point	When the coordinator durably logs its decision.
In doubt (uncertain)	Voted "yes" but hasn't heard the decision; can't commit or abort alone.
Blocking atomic commit	Can get stuck waiting for a crashed node (2PC).
Three-phase commit (3PC)	Nonblocking alternative that needs bounded delays and response times.
Perfect failure detector	Always correctly tells whether a node crashed; impossible with unbounded delay.
XA (X/Open eXtended Architecture)	1991 C API standard for 2PC across heterogeneous technologies.
JTA / JDBC / JMS	Java Transaction API (Java's XA binding); Java database and messaging driver APIs supporting it.
Heterogeneous vs database-internal distributed transaction	Spans different systems (DB + broker) vs nodes of one database product.
Exactly-once processing	A message's effects happen once in total despite retries.
Heuristic decision	XA escape hatch: a participant decides an in-doubt transaction alone — "probably breaking atomicity".
Orphaned transaction	In-doubt transaction whose outcome can't be determined; needs manual resolution.
FLP result	Fischer, Lynch, Paterson: no deterministic algorithm always reaches consensus if a node may crash, in the asynchronous model.
Uniform agreement / integrity / validity / termination	The four consensus properties (below).
Safety vs liveness	"Nothing bad ever happens" vs "something good eventually happens".
Byzantine fault	A node behaving arbitrarily or maliciously.
Epoch number (ballot / view / term)	Monotonically increasing leadership period; leader unique within an epoch.
Multi-Paxos	Paxos deciding a sequence of values.
Dynamic membership	Consensus extensions allowing the voter set to change.
Lease	Lock with an expiry, released automatically if the holder dies.
Ephemeral node	ZooKeeper entry tied to a session; disappears on session timeout.
Watch	Subscription notifying a client when a ZooKeeper value changes.
Service discovery	Finding the network address for a named service.
Membership service	Decides which nodes are currently live cluster members.

Consistency Guarantees

Two replicas looked at the same instant often differ, because writes reach replicas at different times — with *every* replication style: single-leader, multi-leader (several nodes accept writes) and leaderless (any **replica** — copy of the data on its own node — accepts writes) (Chapter 5, "Problems with Replication Lag").

Eventual consistency: stop writing, wait an unspecified time, and all reads return the same value. The book suggests **convergence** as a better name. It is **weak** because it says nothing about *when*; until then a read may return anything or nothing — e.g. you may not **read your own write** (Chapter 5). It is **hard for developers**: a database looks like a variable, but unlike `x = 5; read x` in single-threaded code, you may get the old value. Such bugs are subtle and appear only under faults or heavy concurrency.

Stronger models are easier to program correctly but may perform worse or tolerate fewer faults.

> **Consistency models vs isolation levels.** Some overlap with Chapter 7's isolation hierarchy, but largely separate: *isolation* avoids race conditions between concurrent transactions; *distributed consistency* coordinates replica state despite delays and faults.

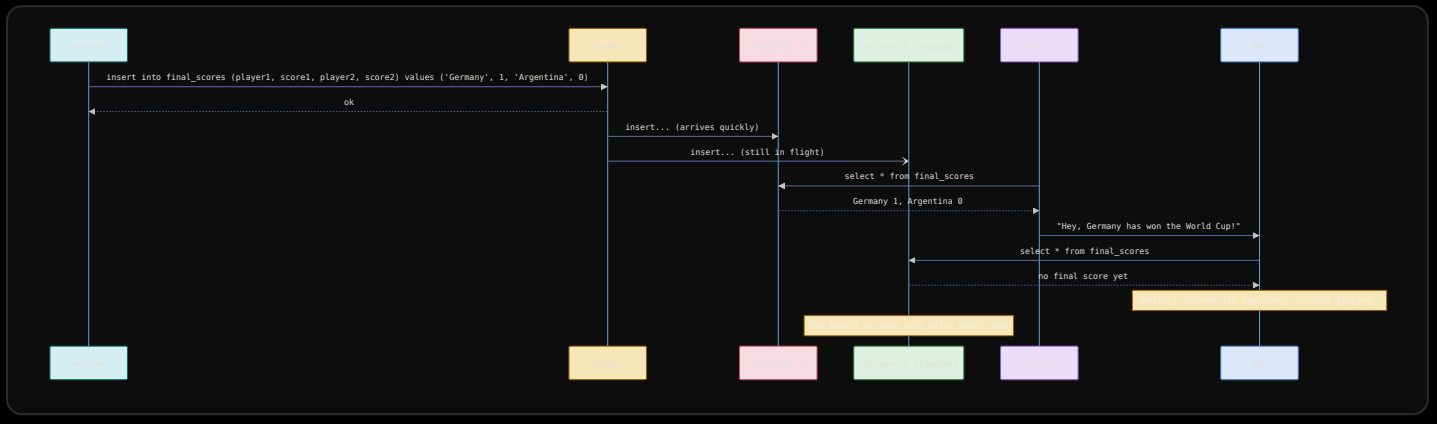
Roadmap: linearizability → ordering (causality, total order) → atomic commit of distributed transactions → consensus. These turn out to be tightly connected.

Linearizability

What if the database could pretend there is **a single copy** of the data? That illusion is **linearizability** (also *atomic*, *strong*, *immediate* or *external consistency*): the system appears to hold **one copy**, and every operation appears to take effect **atomically at one instant**.

Recency guarantee: once a write *completes*, every later read by any client must see it (or newer) — never a stale cache or lagging replica.

Figure 9-1 (redrawn): a non-linearizable sports website. The referee's final score reaches follower 1 quickly but follower 2 late; Alice reads follower 1 and announces the result, then Bob reads the lagging follower 2 and is told the game is still on.



Alice and Bob, in the same room, watch for the 2014 World Cup final result. Alice refreshes, sees the winner and tells Bob; Bob's refresh hits a lagging replica that says the match is still on. Simultaneous refreshes with different results would be unsurprising, but Bob *knows* he refreshed *after* Alice spoke, so an older answer is a **linearizability violation**.

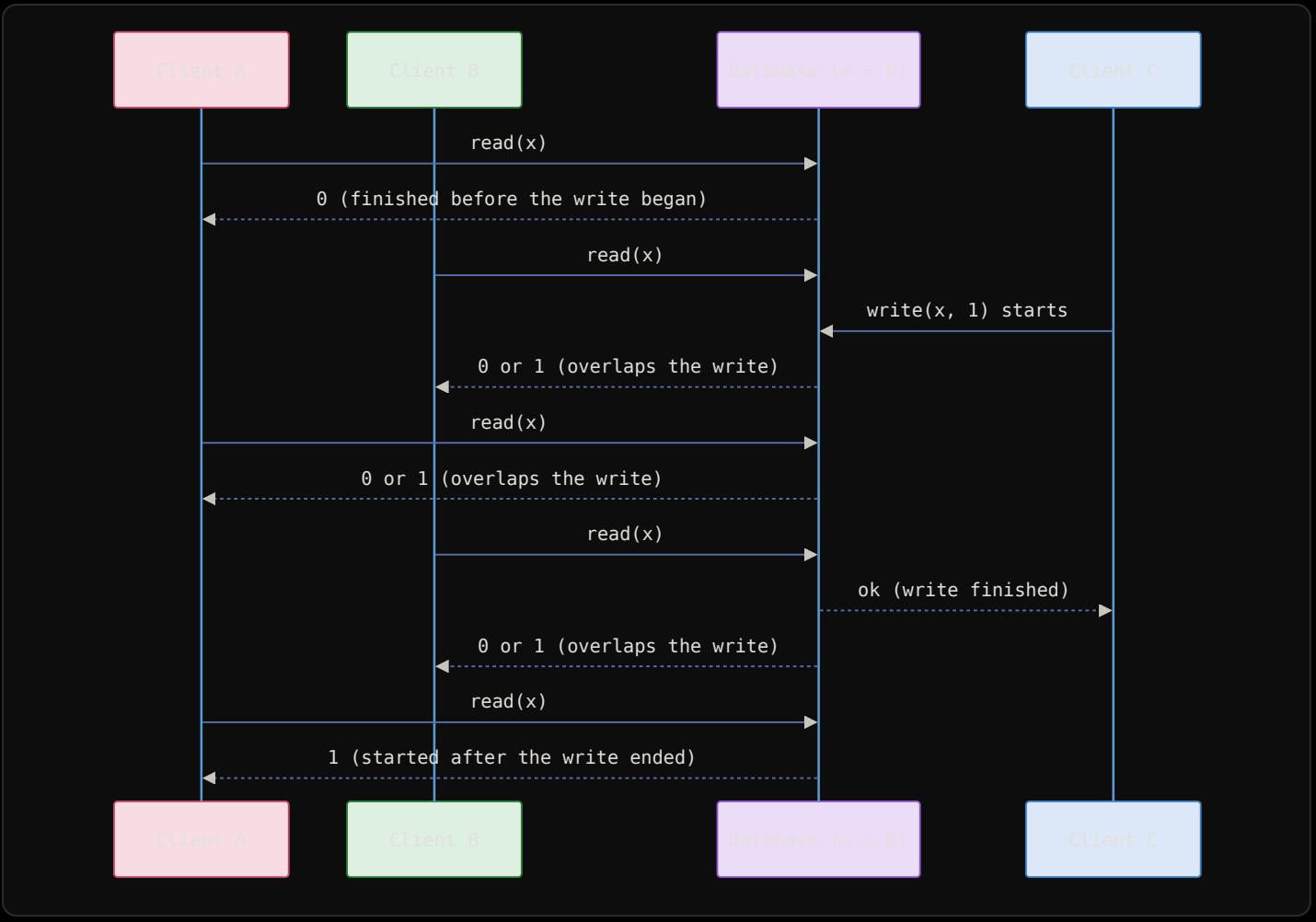
What Makes a System Linearizable?

Setup. Clients read and write a key `x` — a **register** (in practice a key in a key-value store, a row, or a document). Each operation is a **bar** from when the request was *sent* to when the response *arrived*; the database executed it somewhere inside the bar.

> **Aside: the imaginary global clock.** Diagrams assume a perfect global time axis for *analysis*; the algorithm itself can only use imperfect local clocks (quartz + NTP).

Notation: `read(x) ⇒ v` (read returned `v`); `write(x, v) ⇒ r` (`r` is `ok` or `error`).

Figure 9-2 (redrawn): client C writes x = 1 while A and B keep reading. A's first read ends before the write starts (0); A's last read starts after the write ended (1); the three reads that overlap the write may return either 0 or 1.



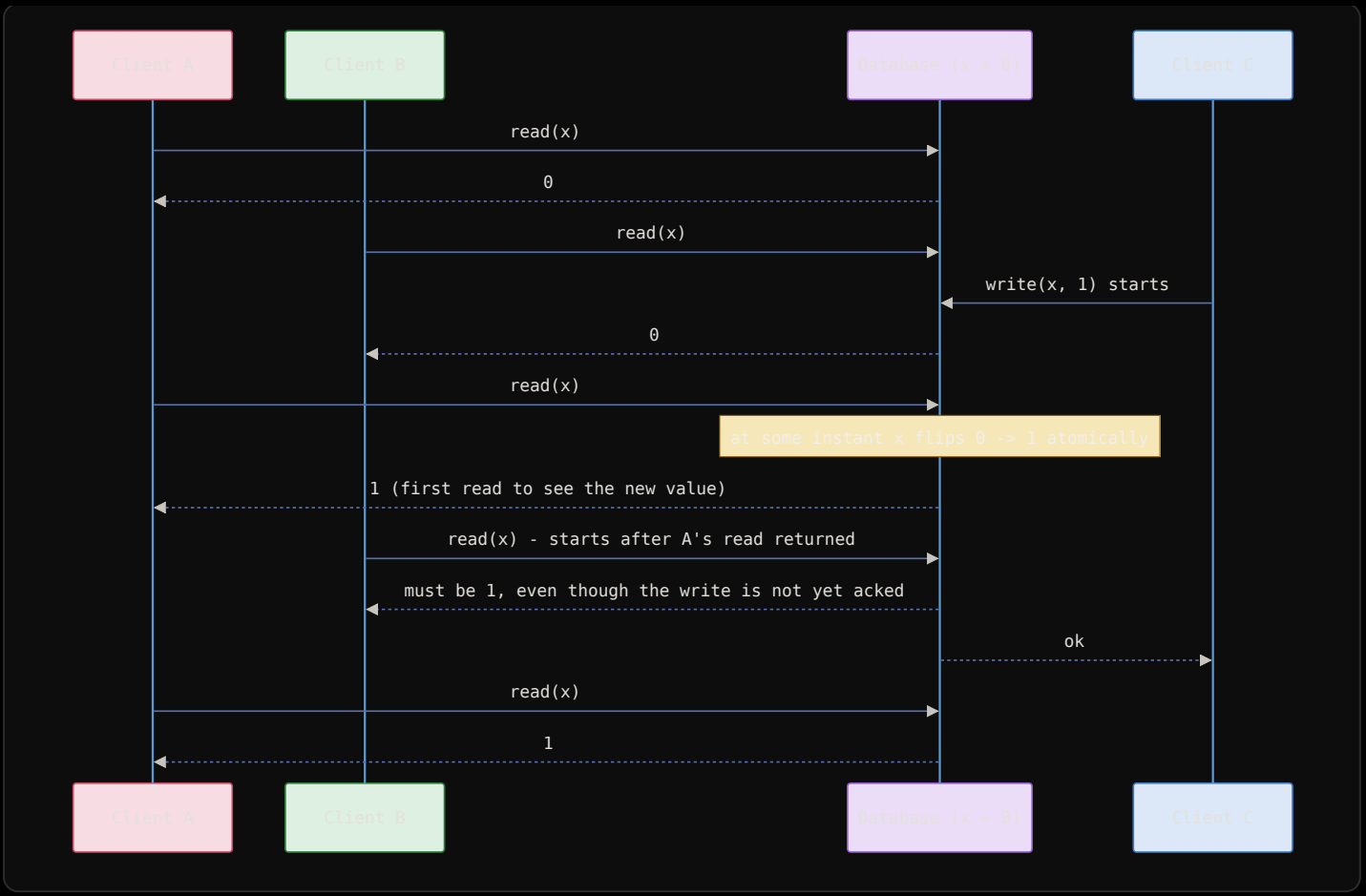
With `x = 0` and C writing 1:

1. A read **finishing before the write starts** returns **0**.
2. A read **starting after the write finishes** returns **1** (both took effect inside their bars, and the bars don't overlap).
3. A read **overlapping** the write (*concurrent* with it) may return **0 or 1**.

That alone would let a reader see 0 → 1 → 0 → 1 during the write — no single copy does that.

> **Regular register.** A register whose overlapping reads may return old or new value. Weaker than linearizable.

Figure 9-3 (redrawn): same clients as Figure 9-2, with the extra linearizability constraint. A's second read is the first to return 1; B's second read starts after that and so must also return 1, although C's write has not been acknowledged yet; all later reads return 1.

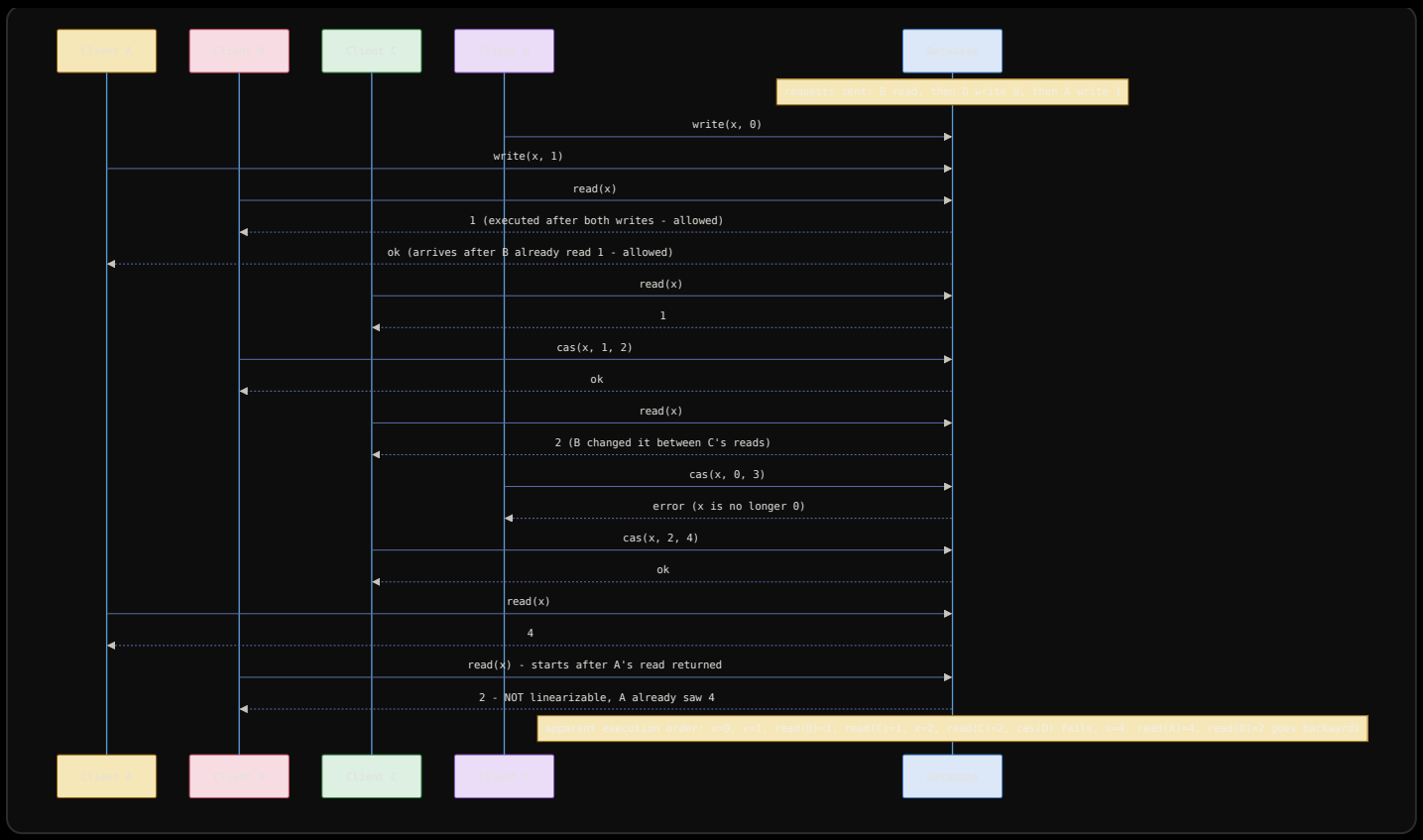


The extra rule (Figure 9-3): `x` **flips atomically** at some instant inside the write's bar, so:

> Once any read has returned the new value, every read beginning *after that read completed* must also see it — even if the write is still unacknowledged.

Adding compare-and-set (Figure 9-4): `cas(x, v_old, v_new) ⇒ r` sets `x` to `v_new` only if it equals `v_old`, else returns `error`. Mark inside each bar the instant the operation *appears* to execute and join the markers. Requirements: (1) the sequence is a **valid sequential history** (each read returns the latest preceding write); (2) the joining line **always moves forward in time**. Rule 2 enforces recency: once a value is written *or read*, later reads see it until overwritten.

Figure 9-4 (redrawn): the order in which the database appears to execute operations; B's last read violates linearizability



Details in Figure 9-4:

- **Processing order ≠ sending order:** B's read was sent before D's and A's writes yet returned 1 — the concurrent requests may be processed in any order (B's may have been delayed).
- **A read can see a write before the writer gets "ok":** A's acknowledgment was just delayed.
- **No transaction isolation assumed:** C read 1 then 2 because B changed it. CAS detects interference: B's and C's succeed, D's fails (x was no longer 0).
- **B's last read breaks linearizability:** it overlaps C's `cas(x, 2, 4)`, so 2 would be fine alone — but A already read 4 before B's read began.

Testing: the formal definition is in Herlihy and Wing (1990). You can check a recorded history by searching for a valid sequential order — possible but **computationally expensive** (the book cites Kingsbury's Knossos checker; such checkers search possible orderings, which grows costly with history size).

Linearizability versus serializability (sidebar):

	Serializability	**Linearizability**
Guarantee	*Isolation* of **transactions**	*Recency* of reads/writes on a **single register**
Scope	Many objects per transaction	One object; no transactions
Promise	Same result as *some* serial order — may differ from real order	Each operation takes effect at one instant between start and end, respecting real time
Prevents write skew?	Yes	No, not by itself (unless e.g. materialising conflicts — creating rows just to lock them)

Both together = *strict serializability* (*strong one-copy serializability*, strong-1SR). **Two-phase locking** or **actual serial execution** are typically linearizable too. **Serializable snapshot isolation (SSI) is not**: it deliberately reads from a snapshot (avoiding reader/writer lock contention), which excludes newer writes.

Relying on Linearizability

A score seen late does no harm. Linearizability matters in three areas.

Locking and leader election

A single-leader system must have **one** leader. A common way is a **lock** every node tries to grab at startup; it **must be linearizable** so all nodes agree who holds it.

- **Coordination services** like **Apache ZooKeeper** and **etcd** provide distributed locks and **leader election**, using consensus internally for fault-tolerant linearizable operations.
- Traps remain (e.g. **fencing**, Chapter 8); **Apache Curator** adds higher-level recipes over ZooKeeper.

> **Footnote: ZooKeeper/etcd reads can be stale.** Their *writes* are linearizable, but by default reads may hit any replica. For a linearizable read: etcd **quorum read**; ZooKeeper `sync()` before reading.

Locks can be very fine-grained: **Oracle Real Application Clusters (RAC)** locks **each disk page** (a fixed-size block, often a few KB), with several nodes sharing one storage system. Since these locks are on every transaction's critical path, RAC deployments usually have a **dedicated cluster interconnect network**.

Constraints and uniqueness guarantees

Enforcing a unique username/email or file path **at write time** (one of two simultaneous attempts gets an error) needs linearizability — it is a **lock** on the name, or a **CAS** "set `username → user_id` only if unclaimed". Likewise: balance never negative, don't oversell stock, don't double-book a seat — all need **one up-to-date value every node agrees on**.

Some constraints can be **loose** (overbooked flights compensate passengers) and avoid linearizability (Chapter 12, "Timeliness and Integrity"). But a **hard uniqueness constraint** requires it. **Foreign keys** and **attribute constraints** (e.g. age ≥ 0) can be enforced *without* it.

Own SQL illustration:

```sql
CREATE TABLE accounts (
  handle  TEXT PRIMARY KEY,    -- hard uniqueness constraint
  user_id BIGINT NOT NULL
);

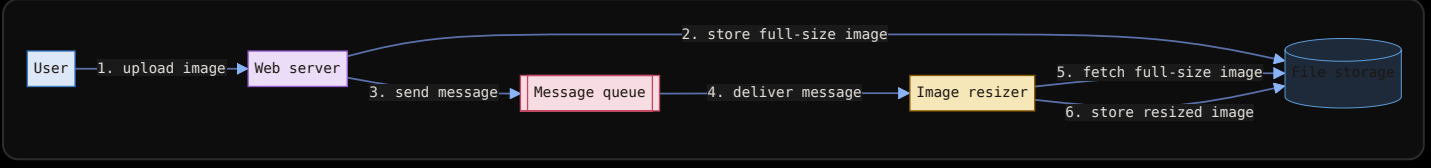
-- Two sign-ups race for the same handle:
INSERT INTO accounts VALUES ('river_otter', 101);  -- session 1
INSERT INTO accounts VALUES ('river_otter', 202);  -- session 2
-- Exactly one succeeds; the other gets a unique-violation error.
```

One node decides the winner locally; in a replicated system, deciding "who was first" *immediately* everywhere is what needs linearizability (effectively CAS on the key).

Cross-channel timing dependencies

Bob only noticed the stale score because of a **second channel** (Alice's voice). Software has the same issue: a photo site with a background thumbnail job.

Figure 9-5 (redrawn): web server and image resizer communicate through two channels, file storage and a message queue, so the message can outrun the storage's replication.



The web server writes the photo to **file storage** (not the queue — brokers are designed for small messages, photos can be megabytes), then enqueues "resize photo X" on a **message queue** (Chapter 11); the resizer fetches the photo. If storage isn't linearizable, the queue may **outrun storage replication**: the resizer gets an old version or nothing, and full-size and thumbnail become **permanently inconsistent**.

Linearizability is the *simplest* fix. If you control the second channel, techniques like **read-your-own-writes** (Chapter 5) also work, with more complexity. (*Own illustration, not the book's:* put the photo's version number in the message and have the resizer wait until it can read at least that version.)

Implementing Linearizable Systems

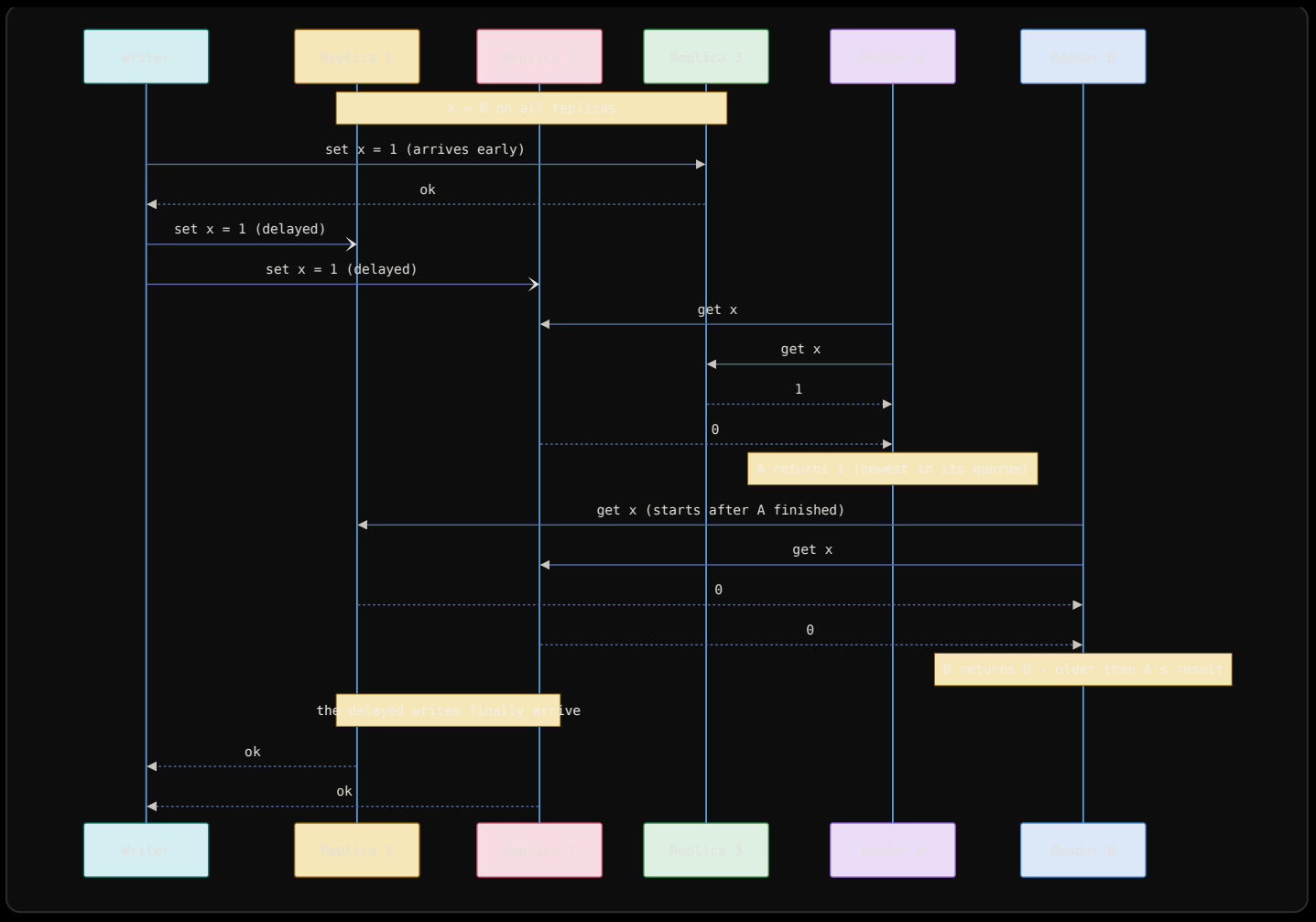
Literally keeping one copy isn't fault-tolerant, so which replication methods (Chapter 5) can be linearizable?

Replication method	Linearizable?	Why
Single-leader	*Potentially*	Reading from the leader or synchronously updated followers can be. Not all are (snapshot isolation by design, concurrency bugs). Relies on knowing the leader: a **delusional leader** (Chapter 8, "The Truth Is Defined by the Majority") breaks it. With **asynchronous replication**, failover can lose committed writes, violating durability and linearizability.
Consensus algorithms	*Yes*	Resemble single-leader but guard against split brain and stale replicas. **ZooKeeper**, **etcd**.
Multi-leader	*No*	Concurrent writes on several nodes, replicated asynchronously, create conflicts — precisely because there is no single copy.
Leaderless (Dynamo-style)	*Probably not*	Quorum "strong consistency" claims aren't quite true. **LWW** with time-of-day clocks (**Cassandra**) is almost certainly non-linearizable (clock skew). **Sloppy quorums** (writes to substitute nodes) destroy it. Even strict quorums can fail (below).

> **Footnote: sharding doesn't break it.** Linearizability is a *single-object* guarantee, so per-partition leaders are fine; cross-partition transactions are another matter.

Linearizability and quorums

Figure 9-6 (redrawn): a non-linearizable execution despite a strict quorum (n = 3, w = 3, r = 2). The write reaches replica 3 early but replicas 1 and 2 late; reader A sees 1 via replica 3, then reader B, starting after A finished, reads replicas 1 and 2 and gets 0 from both.



With n = 3, w = 3, r = 2 (3 + 2 > 3), reader A sees 1 from replica 3 mid-write; reader B starts after A finished, reads two replicas still at 0, and returns the *older* value. Not linearizable.

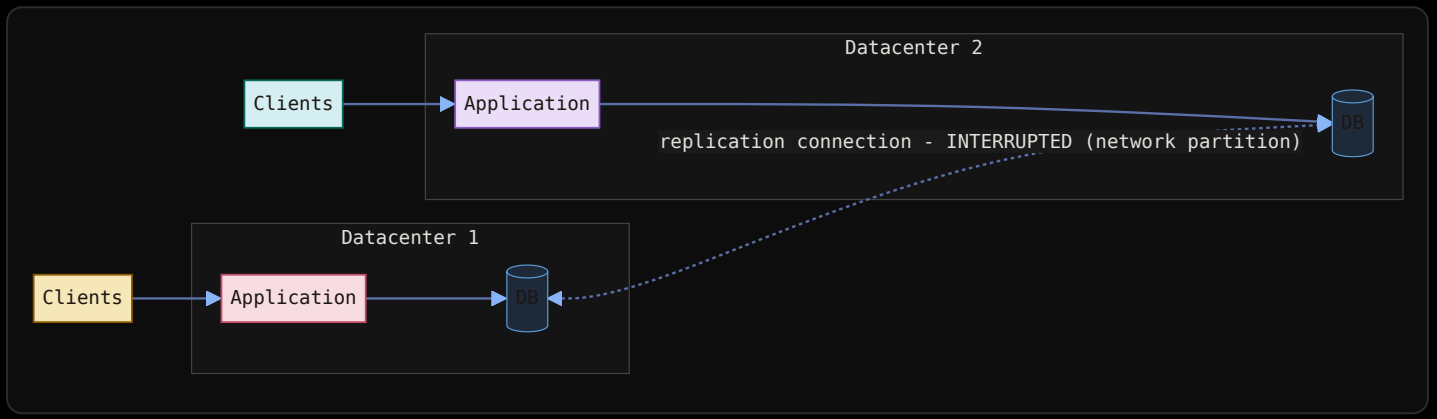
Fix (at a performance cost): readers do **synchronous read repair** (write the newest value back to stale replicas) before returning; writers **read the latest state from a quorum** before writing.

- **Riak** skips synchronous read repair (performance).
- **Cassandra** waits for read repair on quorum reads but still breaks linearizability with concurrent writes to one key, because of **LWW**.
- Only linearizable **reads and writes** can be built this way — **not CAS**, which needs consensus.

Assume leaderless Dynamo-style systems are not linearizable.

The Cost of Linearizability

Figure 9-7 (redrawn): two datacenters, each with an application server and a database replica; the replication connection between the databases is cut by a network partition, while clients can still reach their local datacenter.



If the inter-datacenter link breaks:

- **Multi-leader** (suited to multi-datacenter, Chapter 5): each datacenter carries on; writes queue up and sync later.
- **Single-leader:** clients in the follower datacenter must reach the leader for writes and linearizable reads. Cut off, they **can't write or read linearizably** — stale local reads only — so the app is **unavailable** there until the link is repaired.

The CAP theorem

This applies to **any linearizable database**, and to any unreliable network, even within one datacenter:

- **Require linearizability** → disconnected replicas **can't process requests** (wait or error): *unavailable*.
- **Don't require it** → each replica serves requests independently (e.g. multi-leader): *available*, not linearizable.

> **Footnote: CP and AP.** The first is sometimes called **CP**, the second **AP**; the book says this classification is flawed and best avoided.

This is the **CAP theorem**, named by **Eric Brewer in 2000**, though known to designers **since the 1970s**. It began as a deliberately imprecise *rule of thumb* to start discussion. When many distributed databases targeted linearizability on **shared-storage** clusters (the book cites an Oracle RAC reference), CAP nudged engineers toward **shared-nothing** designs (each machine its own CPU, memory, disk) for web-scale services — credit for the culture shift behind mid-2000s **NoSQL** databases.

> **The unhelpful CAP theorem** (sidebar). "Pick 2 of 3" misleads: a partition is a *fault*, not a choice. When the network works you can have both; only during a fault must you choose — **"either Consistent or Available when Partitioned"**. Discussions also use contradictory definitions of "availability", and the formal one doesn't match common usage; many "highly available" systems don't meet it. The book recommends **avoiding CAP**.

Limited practical value: formally (Gilbert and Lynch, 2002) it covers only **one consistency model** (linearizability) and **one fault** (network partitions) — nothing about delays, crashed nodes or other trade-offs. It has been **superseded by more precise results** and is mostly of historical interest.

> **Footnote: two meanings of "partition".** The book's *partitioning* means sharding (Chapter 6); a *network partition* is a fault. The clash is unavoidable here because it is CAP's "P".

Linearizability and network delays

Few systems are linearizable in practice. Even **RAM on a multi-core CPU isn't**: one core's write may not be visible to another core's subsequent read without a **memory barrier (fence)**, because each core has its own **cache** and **store buffer** (pending writes) that sync to main memory asynchronously (keeping them in sync is *cache coherence* — general background). That trade is made for **performance**; CAP is irrelevant, since we don't expect a core to keep running when cut off from the machine.

Likewise many distributed databases drop linearizability mainly for **speed**. **Linearizability is slow — all the time, not only during faults. Attiya and Welch** proved linearizable read/write **latency is at least proportional to network delay uncertainty**, so on real networks (highly variable delays, Chapter 8) it is inevitably slow; weaker models can be much faster. Chapter 12 discusses avoiding linearizability without losing correctness.

Ordering Guarantees

A linearizable register implies a **well-defined order** of operations (the joined markers of Figure 9-4). Ordering recurs throughout the book:

- **Chapter 5:** the single leader fixes the **order of writes** in the replication log; without one, concurrent writes conflict.
- **Chapter 7:** serializability means behaving *as if* in some sequential order (literally serial, or concurrent with conflicts prevented by locks/aborts).
- **Chapter 8:** timestamps and clocks try to impose order (which write came later?).

Ordering, linearizability and consensus are **deeply connected**.

Ordering and Causality

Ordering matters because it **preserves causality** — causes before effects. Earlier examples:

- **Consistent prefix reads (Figure 5-5):** an observer saw an answer before its question — a **causal dependency** was violated.
- **Multi-leader replication (Figure 5-9):** an update arrived before the row's creation.
- **Detecting concurrent writes (Chapter 5):** A before B, B before A, or concurrent — the **happens-before** relation *is* causality.
- **Snapshot isolation (Chapter 7):** a "consistent" snapshot **respects causality** (an answer implies its question is visible). **Read skew** (Figure 7-6) is reading a causality-violating state.
- **Write skew (Figure 7-8, doctors on call):** going off call was *causally dependent* on who was on call; **SSI** detects this by tracking causal dependencies between transactions.
- **Alice and Bob (Figure 9-1)** and the image resizer: **causality violations**.

Chains of dependent operations define the system's **causal order**. A system obeying it is **causally consistent** — e.g. **snapshot isolation provides causal consistency**: reading some data means you also see everything causally before it (unless deleted).

The causal order is not a total order

- **Total order:** any two elements comparable (13 > 5).
- **Partial order:** some pairs **incomparable** — is {a, b} bigger than {b, c}? Neither contains the other.

	Linearizability	**Causality**
Kind of order	**Total**	**Partial**
Why	One copy + atomic ops ⇒ any two operations are ordered	Ordered only if one happened before the other; **concurrent** = **incomparable**
Concurrency?	None — one timeline	Yes — timeline **branches and merges**

So **a linearizable datastore has no concurrent operations**: requests may wait, but each is handled atomically on one timeline. Concurrency means branches whose operations can't be compared (Figure 5-14).

> **Git analogy.** Version history in **Git** is a causal dependency graph: mostly linear, but **branches** when people work concurrently and **merge** commits rejoin them.

Linearizability is stronger than causal consistency

Linearizability implies causality, even across multiple channels (Figure 9-5) with no extra machinery — which makes it easy to reason about, but it costs performance and availability, especially with large delays.

The middle ground: a system can be **causally consistent without linearizability's performance cost**, and **CAP does not apply** to it. **No consistency model stronger than causal consistency can avoid being slowed by network delays while staying available through network failures.** Many systems that *seem* to need linearizability only need causality. Research on causality-preserving databases with eventual-consistency-like performance is recent, little is in production, but it's promising.

Capturing causal dependencies

Key ideas (only sketched in the book):

1. Know **what happened before what** (a partial order): concurrent operations may go in any order, but causally ordered ones must be applied in that order on every replica.
2. **Wait for predecessors** before applying an operation.
3. **Track each node's knowledge**: if a node had seen X when it wrote Y, Y may depend on X ("did the CEO know about X when deciding Y?").
4. **Generalise version vectors** from one key (Chapter 5, "Detecting Concurrent Writes") to **the whole database**.
5. **Know which version the application read** — hence passing the previously read version number back with writes (Figure 5-13); SSI similarly tracks what each transaction read and checks it at commit.

Sequence Number Ordering

Tracking *every* causal dependency is impractical (apps read lots before writing). Instead, give operations **sequence numbers or timestamps** from a **logical clock** (usually counters incremented per operation), not a physical time-of-day clock. They are **compact** (a few bytes) and **totally ordered**. We want a total order **consistent with causality**: if A happened before B, A's number is lower; concurrent operations are ordered arbitrarily (more ordering than strictly needed).

> **Footnote:** a total order ignoring causality is easy but useless — e.g. sorting random **UUIDs** (128-bit random IDs).

Single-leader case: the leader increments a counter per write, giving **monotonically increasing** numbers; followers applying the log in order are **causally consistent** even while lagging.

Noncausal sequence number generators

Without a single leader (multi-leader, leaderless, partitioned):

1. **Per-node sequences** — e.g. one node odd, one even; generally reserve bits for a **unique node ID**.
2. **Physical-clock timestamps** — high enough resolution may totally order operations; **LWW** relies on this.
3. **Pre-allocated blocks** — node A takes **1–1,000**, B takes **1,001–2,000**, each grabbing a new block when low.

All **perform and scale better** than one leader's counter, but **aren't consistent with causality**:

Method	Why causality breaks
Odd/even	Nodes process at different rates, so one counter lags; odd vs even tells you nothing.
Physical timestamps	**Clock skew** can give a later operation a lower timestamp (Figure 8-3).
Blocks	A later operation may get a number from 1–1,000 while the earlier got 1,001–2,000.

> **Footnote: Spanner's exception. Spanner** estimates clock skew and **waits out the uncertainty interval** before committing, so causally later transactions get larger timestamps (Chapter 8). Most clocks can't supply the needed uncertainty metric.

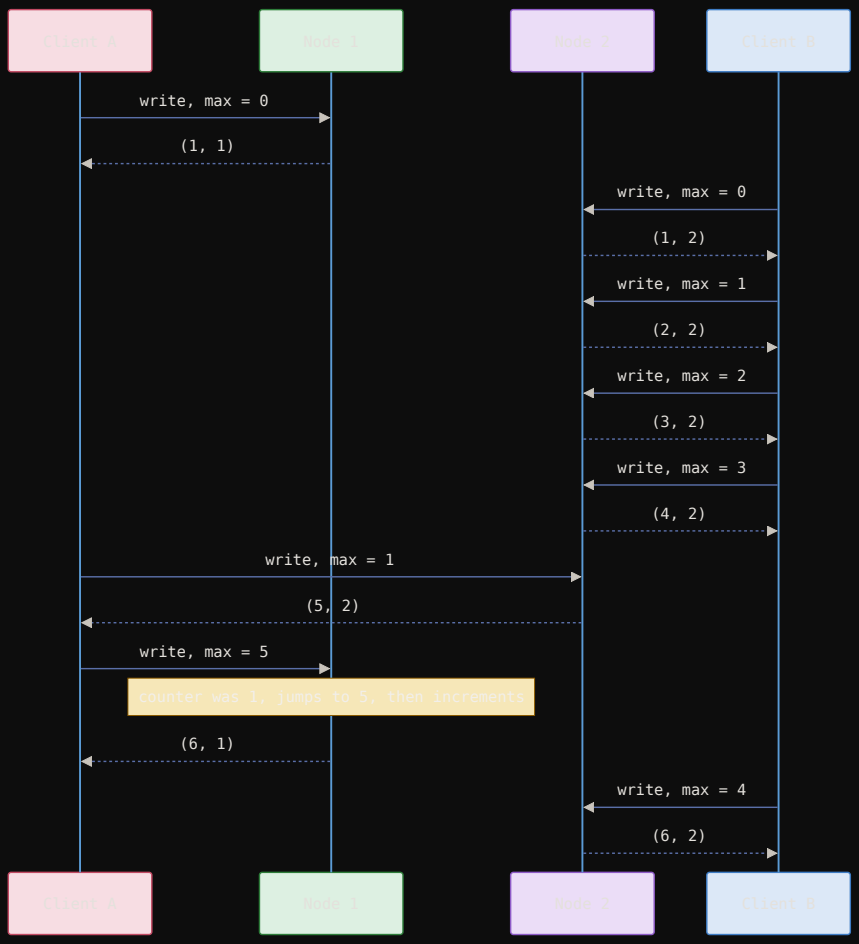
Lamport timestamps

Proposed by **Leslie Lamport in 1978** in one of the field's most-cited papers:

1. Each node has a **unique ID** and a **counter** of operations processed.
2. Timestamp = **(counter, node ID)** — unique even if counters tie.
3. Compare: larger counter wins; tie → larger node ID. A **total order**, unrelated to physical time.
4. **Key trick:** every node *and client* tracks the **largest counter seen** and sends it with each request; a node receiving a larger maximum **raises its counter to it immediately**.

Steps 1–3 alone equal the odd/even scheme; step 4 makes **every causal dependency yield a larger timestamp**.

Figure 9-8 (redrawn): Lamport timestamps (counter, node ID). Each client sends the maximum counter it has seen; a node first raises its counter to that maximum, then increments it. The result is a total order consistent with causality.



Node 2 gives client A counter **5**; A passes 5 to node 1, whose counter was **1** — it jumps to 5, so its next operation gets **6**.

Own code:

```python
class LamportNode:
    def __init__(self, node_id):
        self.node_id = node_id
        self.counter = 0

    def handle(self, request_max_seen):
        # 1. catch up to anything the caller has already seen
        self.counter = max(self.counter, request_max_seen)
        # 2. this operation gets the next value
        self.counter += 1
        # 3. timestamp = (counter, node_id); caller remembers counter as its new max
        return (self.counter, self.node_id)

n1, n2 = LamportNode(1), LamportNode(2)
print(n1.handle(0))   # (1, 1)
n2.counter = 4        # pretend node 2 has already handled 4 operations (as in the diagram)
print(n2.handle(1))   # (5, 2)  client A only knew max 1; node 2's own counter wins
print(n1.handle(5))   # (6, 1)  A passes max 5; node 1 jumps from 1 to 5, then +1
```

`max(...)` is the catch-up step; tuples compare counter then node ID — the Lamport rule.

	Version vectors	Lamport timestamps
Purpose	Tell **concurrent** from **dependent**	Always impose a **total order**
Detect concurrency?	Yes	**No**
Size	One counter per node	**More compact** — one pair

Timestamp ordering is not sufficient

Unique usernames: "lowest timestamp wins" works only **after the fact**, once all creation operations are collected. A node receiving a sign-up must decide **now**, without knowing whether another node is concurrently claiming the name with a lower timestamp. Asking every node means **any failure halts the system**.

The total order only *emerges* after gathering all operations; unknown operations might slot in anywhere. For uniqueness you also need to know **when the order is finalised** — what **total order broadcast** provides.

Total Order Broadcast

On one core, the order is just execution order. Single-leader replication gets a total order by sequencing on **one CPU core** of the leader; the hard parts are **scaling** past one leader's throughput and **failover**. This problem is **total order broadcast** or **atomic broadcast**.

> **Footnote on naming.** "Atomic broadcast" is unrelated to ACID's A and only loosely to atomic operations or atomic registers. **Total order multicast** is a synonym.

> **Scope of ordering guarantee** (sidebar). Partitioned databases with per-partition leaders usually order **only within a partition**, so no cross-partition guarantees (consistent snapshots, foreign keys); a cross-partition total order needs extra coordination.

Definition — two **safety** properties that must hold despite faults:

1. **Reliable delivery:** if any node gets a message, all do.
2. **Totally ordered delivery:** all nodes get messages in the **same order**.

During a network outage nothing is delivered, but the algorithm retries until messages get through, still in order.

Using total order broadcast

- **ZooKeeper and etcd implement it** — a hint of its link to consensus.
- **Database replication:** all replicas apply the same writes in the same order — **state machine replication** (Chapter 11).
- **Serializable transactions:** each message is a **deterministic stored-procedure transaction** processed in the same order everywhere (Chapter 7, "Actual Serial Execution").
- **Order is fixed at delivery:** a node may **not insert** a message earlier once later ones were delivered — *why it is stronger than timestamp ordering*.
- **It's a log** (like replication, transaction or write-ahead logs): delivering = appending; every node reads the same sequence.
- **Fencing tokens:** append each lock request; its sequence number is a monotonically increasing token — ZooKeeper's **zxid**.

Implementing linearizable storage using total order broadcast

Not the same thing, but closely linked: total order broadcast is **asynchronous** (order guaranteed, **not *when*** — nodes may lag); linearizability guarantees **recency**.

> **Footnote: formal relationship.** Total order broadcast ≡ consensus, which **can't be solved deterministically in the asynchronous crash-stop model** — yet a plain linearizable read-write

> register *can* be. A register with **CAS** or **increment-and-get** is equivalent to consensus.

Linearizable CAS for usernames via a log. Each username has a register, initially `null`; claiming = CAS from `null` to your account ID. Using total order broadcast as an **append-only log**:

1. **Append** a message tentatively claiming the name.
2. **Read** the log and **wait until your message is delivered back**.
3. Check claims for that name: if the **first** is yours, **commit** (e.g. append a commit message) and acknowledge; otherwise **abort**.

All nodes see the same order, so all agree which claim won. A log can similarly support **serializable multi-object transactions**.

> **Footnote: why wait in step 2.** Acknowledging as soon as it's queued gives something like the **multi-core x86** memory model — **neither linearizable nor sequentially consistent**.

Own pseudocode:

```
def claim_username(log, name, account_id):
    my_msg = log.append({"claim": name, "by": account_id})   # step 1
    log.wait_until_delivered(my_msg)                          # step 2
    first = next(m for m in log.read_all()                    # step 3
                 if m.get("claim") == name)
    if first is my_msg:
        log.append({"commit": name, "by": account_id})
        return "ok"
    return "error: name taken"
```

Reads are still a problem: this makes **writes** linearizable, but reads from an asynchronously updated store may be stale — the result is **sequential consistency** (**timeline consistency**). Options for linearizable reads:

Option	How	Example
Sequence reads through the log	Append a message; do the read when it comes back (its log position = read time)	**etcd quorum reads** (somewhat)
Fetch latest log position	Linearizably get the latest position, wait until delivered up to it, then read	**ZooKeeper** `sync()`
Read from a synchronously updated replica	That replica must be current	**Chain replication**

Implementing total order broadcast using linearizable storage

Given a linearizable register with atomic **increment-and-get** (or CAS):

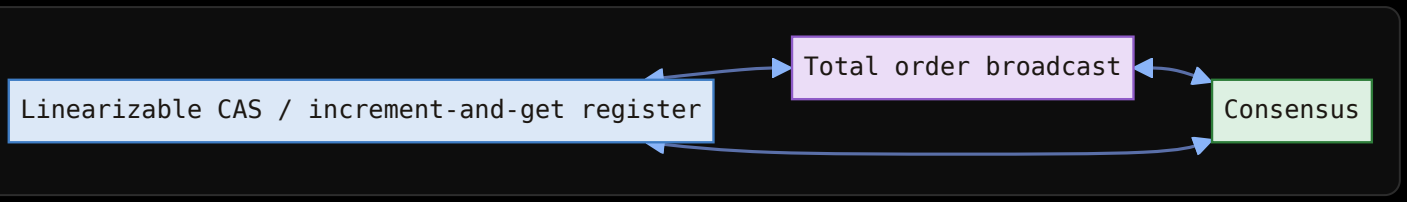
1. For each message, **increment-and-get** the register.
2. Attach the value as the **sequence number**.
3. **Send** to all nodes, retransmitting losses.
4. Recipients **deliver consecutively by sequence number**.

No gaps — unlike Lamport timestamps. A node that delivered 4 and receives 6 knows to **wait for 5**. *That gap-awareness is what separates total order broadcast from timestamp ordering.*

```python
def broadcast(counter, nodes, payload):
    seq = counter.increment_and_get()          # linearizable, gap-free
    for n in nodes:
        n.send_with_retry(seq, payload)

class Receiver:
    def __init__(self): self.next_seq, self.buffer = 1, {}
    def on_message(self, seq, payload):
        self.buffer[seq] = payload
        while self.next_seq in self.buffer:     # deliver strictly in order
            deliver(self.buffer.pop(self.next_seq))
            self.next_seq += 1                  # a gap means "wait"
```

A linearizable increment-and-get is trivial with no failures (a variable on one machine); handling network breaks and **recovering the value when the node fails** inevitably **leads to a consensus algorithm**. Indeed, it is proven that **consensus, total order broadcast, and a linearizable CAS (or increment-and-get) register are equivalent** — a solution to one converts to the others.



(Diagram, not a book figure: the three problems proven equivalent.)

Distributed Transactions and Consensus

Consensus — getting nodes to **agree on something** — sounds easy, and many broken systems assumed so. It is among the most fundamental problems in the field; its subtleties need replication (Ch. 5), transactions (Ch. 7), system models (Ch. 8), linearizability and total order broadcast first. Researchers took **decades** to understand it fully.

Where nodes must agree:

- **Leader election:** a bad failover under a network fault causes **split brain** — two leaders accept writes, data diverges and is lost.
- **Atomic commit:** a transaction spanning nodes/partitions may succeed on some and fail on others; to keep atomicity, **all commit or all abort** — the **atomic commit problem**.

> **Footnote: atomic commit vs consensus.** Atomic commit may commit **only if all vote yes** and must abort if any needs to; consensus may decide **any proposed value**. They are **reducible to each other**; *nonblocking* atomic commit is harder than consensus.

> **The Impossibility of Consensus** (sidebar). The **FLP result** (**Fischer, Lynch, Paterson**) proves **no algorithm always reaches consensus if any node may crash** — but in the **asynchronous model** (deterministic, **no clocks or timeouts**). Allow **timeouts** or any way to *suspect* crashes (even occasionally wrong), or just **random numbers**, and consensus is solvable. Important in theory; in practice systems reach consensus.

Plan: atomic commit and **2PC** (common in databases, messaging systems, app servers) — *a consensus algorithm of sorts, but a poor one* — then better algorithms like **Zab (ZooKeeper)** and **Raft (etcd)**.

Atomic Commit and Two-Phase Commit (2PC)

Atomicity (Chapter 7): a transaction either **commits** (all writes durable) or **aborts** (all rolled back), preventing half-finished state. It matters for **multi-object transactions** and **secondary indexes** (separate structures that must stay **consistent with the primary data**).

From single-node to distributed atomic commit

Single node (storage engine):

1. Client requests commit.
2. Node makes writes durable (usually a **write-ahead log**).
3. Appends a **commit record** to the log.
4. On crash recovery: commit record on disk → **committed**; else **rolled back**.

The decisive moment is the **commit record reaching disk**; atomicity rests on **one device** (one disk controller).

Multiple nodes — e.g. a multi-object transaction in a sharded database, or a **term-partitioned** secondary index (split by indexed value, so entries live on other nodes). Most NoSQL stores **don't support** this; some clustered relational systems do. Sending "commit" to each node independently fails because:

- some nodes hit a **constraint violation or conflict** and abort;
- some commit requests are **lost** and time out;
- some nodes **crash** before writing the commit record.

Nodes become inconsistent (cf. Figure 7-3), and a commit **can't be retracted**. So **a node may commit only once sure all others will too.**

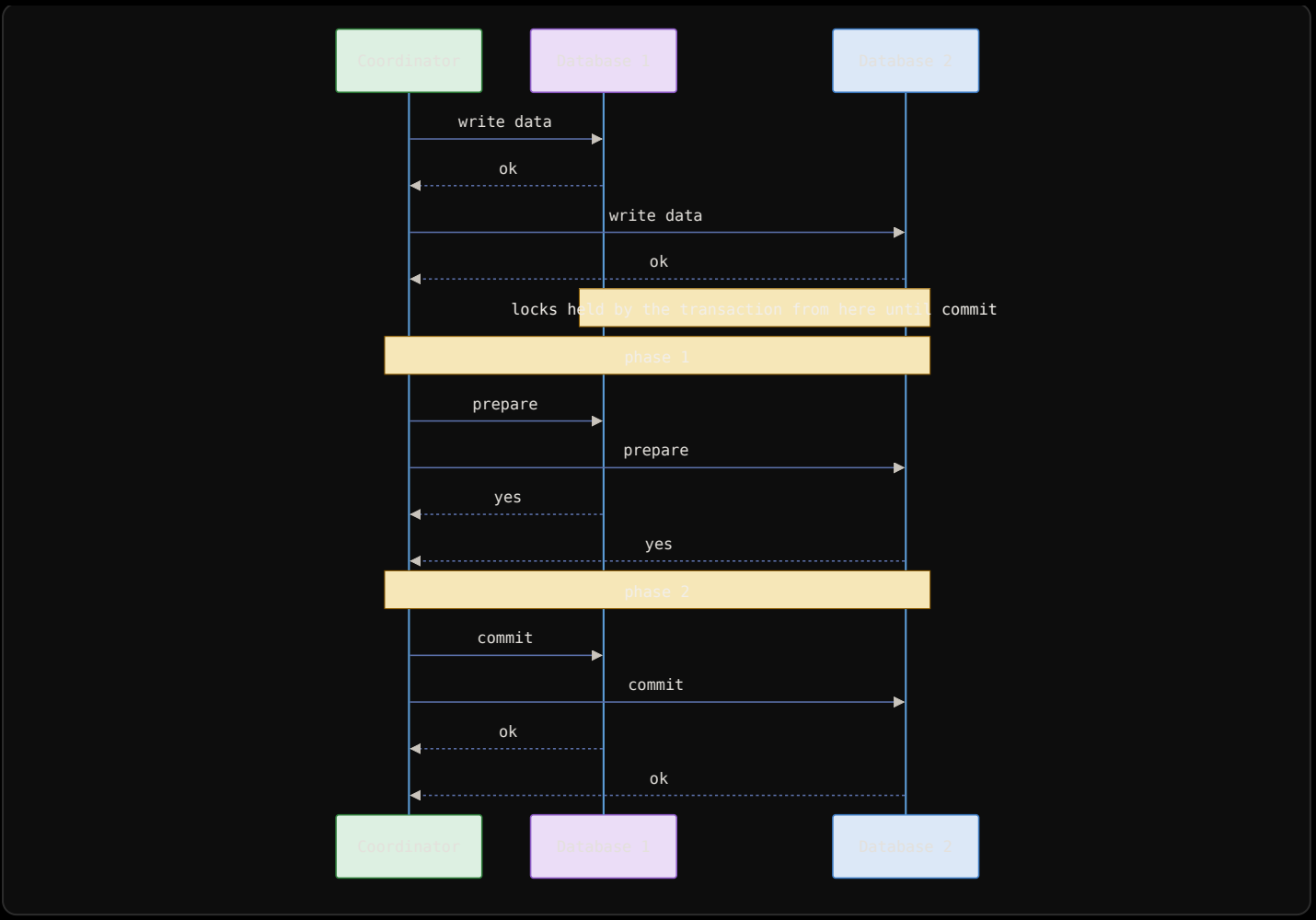
Why commits are irrevocable: committed data is **visible** and others build on it — the basis of **read committed**. Retracting would require reverting everything that read it.

> **Compensating transactions** can undo a committed transaction's *effects*, but the database sees a separate transaction; cross-transaction correctness is the **application's** problem.

Introduction to two-phase commit

2PC achieves atomic commit across nodes — used **internally** by some databases and exposed as **XA transactions** (e.g. **Java Transaction API**) or **WS-AtomicTransaction** for **SOAP** (an older XML-based standard for services calling each other). Commit happens in **two phases**.

Figure 9-9 (redrawn): a successful two-phase commit. Data is written on both databases, then phase 1 asks each to prepare (both vote yes) and phase 2 tells each to commit. Each database holds the transaction's locks from its write until its commit (the book's shaded bars).



> **Don't confuse 2PC and 2PL** (sidebar). Two-phase **commit** = atomic commit across nodes; two-phase **locking** (Chapter 7) = serializable isolation. Unrelated.

The **coordinator** (**transaction manager**) is new. Often a **library in the application process** (e.g. a **Java EE container**), or a separate service. Examples: **Narayana, JOTM, BTM, MSDTC**.

Flow: the app reads/writes on several nodes (**participants**); at commit, the coordinator sends **prepare** (phase 1); if **all** say **yes**, it sends **commit** (phase 2), else **abort** to all.

> **Wedding analogy.** The minister asks each partner "do you?"; with both "I do"s, declares them married (commit) and announces it; if either says no, the ceremony aborts.

A system of promises

Messages can be lost in 2PC too; atomicity comes from the details:

1. **Global transaction ID** from the coordinator.
2. **Local transactions** on each participant, tagged with it. If anything fails now, the coordinator **or any participant** may abort.
3. **Prepare**, tagged with the ID, sent to all; if any **fails or times out**, the coordinator sends **abort** to all.
4. **Participant's promise:** it writes all data to disk and checks constraints/conflicts so it can **commit under all circumstances** (crash, power loss or full disk are not excuses later). Voting **yes surrenders the right to abort** — without committing yet.
5. **Commit point:** the coordinator decides (commit only if **all** voted yes) and **writes the decision durably to its log**.
6. **Enforce forever:** it then sends commit/abort, **retrying forever** on failure. A participant that crashed commits **on recovery** — it voted yes.

Two points of no return: a participant's **yes vote**, and the coordinator's **decision**. (Single-node commit merges both into writing the commit record.)

Wedding, continued: after "I do" you can't take it back; if you faint before the pronouncement you're still married, and on waking can ask the minister about your global transaction ID or wait for the retried commit.

Own coordinator pseudocode:

```
def two_phase_commit(coord_log, participants, txid):
    # Phase 1: collect votes
    votes = []
    for p in participants:
        try:
            votes.append(p.prepare(txid))       # participant fsyncs, then votes
        except Timeout:
            votes.append("no")
    decision = "commit" if all(v == "yes" for v in votes) else "abort"

    coord_log.write_and_fsync(txid, decision)   # COMMIT POINT

    # Phase 2: enforce the decision, forever if necessary
    for p in participants:
        while True:
            try:
                p.finish(txid, decision); break
            except Timeout:
                continue                          # retry until success
```

A phase-1 timeout counts as "no" (safe — nobody committed); `write_and_fsync` is the commit point; the phase-2 loop keeps the promise.

Own example, not from the book — PostgreSQL exposes the participant side as SQL (the basis of its XA support in drivers such as JDBC); it requires `max_prepared_transactions` > 0 (default 0):

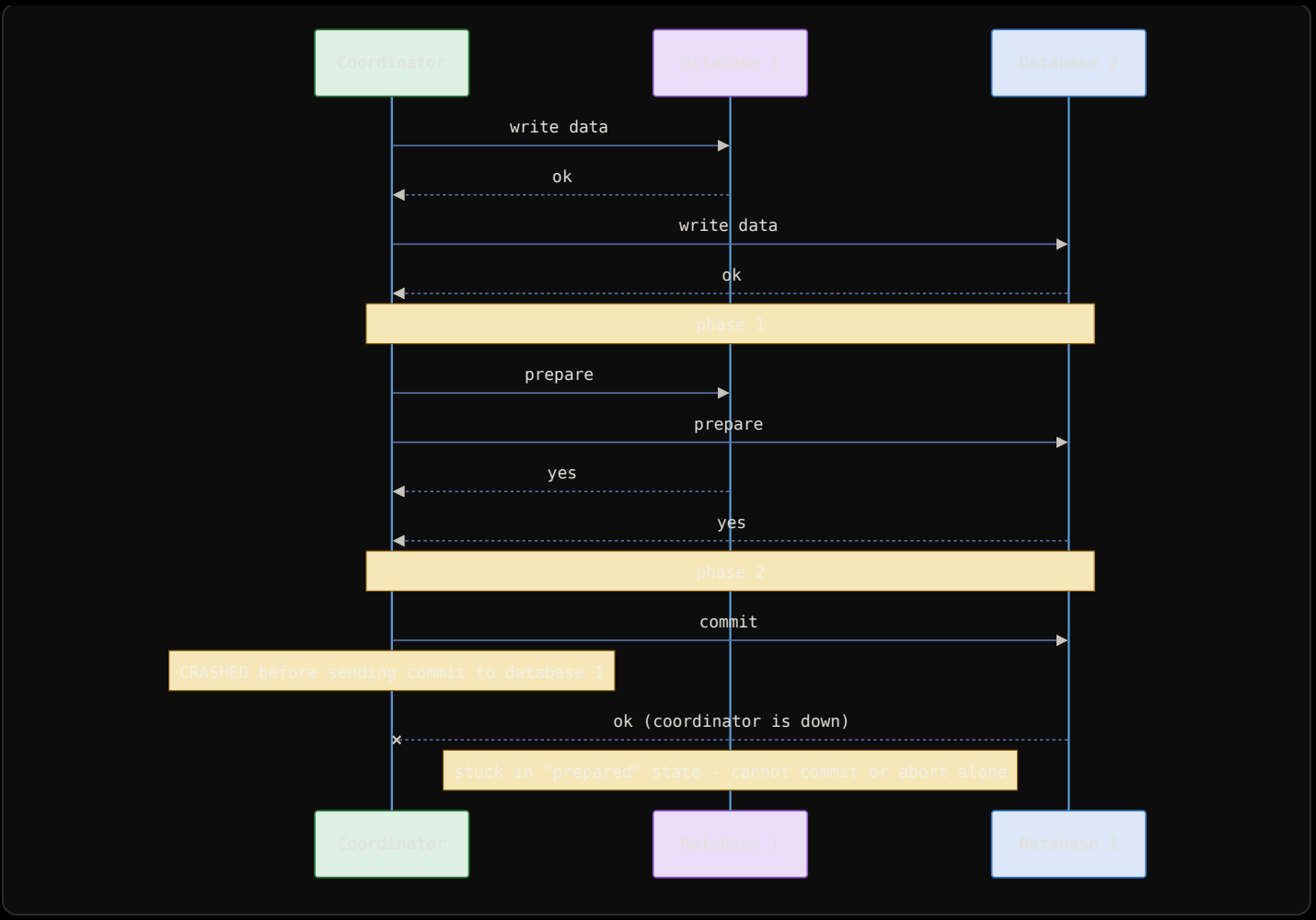
```
BEGIN;
UPDATE stock SET qty = qty - 1 WHERE sku = 'lamp-42';
PREPARE TRANSACTION 'order-7781';   -- phase 1: durable, votes "yes"
-- ... later, when the coordinator has decided:
COMMIT PREPARED 'order-7781';       -- phase 2 (or ROLLBACK PREPARED)
```

- `PREPARE TRANSACTION` persists state and keeps locks without committing (success = "yes"; failure rolls back = "no"). The prepared transaction detaches from the session and survives a restart.
- `COMMIT PREPARED` / `ROLLBACK PREPARED` (any session) applies the decision; until then it is *in doubt*, holding locks, listed in `pg_prepared_xacts`.

Coordinator failure

Participant/network failures are handled (prepare failure → abort; commit/abort failure → retry). If the **coordinator** crashes: before prepare, participants can abort; **after voting yes**, a participant **can't abort alone** and must **wait** — it is **in doubt** (**uncertain**).

Figure 9-10 (redrawn): the coordinator crashes after both participants voted yes. Database 2 gets the commit and commits; the commit for database 1 is never sent, so database 1 is stuck in the "prepared" state, unable to decide on its own.



Database 2 committed; database 1 never got the commit. A **timeout doesn't help**: aborting would contradict DB2, committing is unsafe because another participant might have aborted. Participants could in principle ask each other, but **that's not part of 2PC**.

2PC completes only when the coordinator recovers, reading its log (hence logging the decision *before* sending it). **No commit record in the log ⇒ abort.** So 2PC's commit point is ultimately a **single-node atomic commit on the coordinator**.

Three-phase commit

2PC is **blocking**. A **nonblocking** protocol is possible in theory; **3PC** was proposed, but it assumes **bounded network delay and response times**; with unbounded delays and pauses it **can't guarantee atomicity**. Nonblocking atomic commit needs a **perfect failure detector**, and timeouts can't reliably detect crashes. So **2PC is still used**.

Distributed Transactions in Practice

Distributed transactions have a **mixed reputation**: an important safety guarantee, but operational pain, poor performance and over-promising; **many cloud services don't offer them**. **MySQL** distributed transactions are reported **more than 10× slower** than single-node ones; 2PC's inherent cost is **extra `fsync`s** (forcing writes to disk for crash recovery) and **extra network round-trips**.

	Database-internal	**Heterogeneous**
Participants	Nodes of **one** distributed database, **same software**	**Different technologies** — different vendors' databases, **message brokers**
Examples	**VoltDB**, **MySQL Cluster's NDB**	Database + message queue in one transaction
Protocol freedom	Any protocol, technology-specific optimisations	Must work across very different systems
How well	Often **quite well**	**Much harder**

Exactly-once message processing

A queue message can be acknowledged **exactly when** the database transaction processing it **commits**, by atomically committing the **ack and the DB writes** together. If either fails, **both abort** and the broker redelivers; the message is processed **exactly once in effect**, however many retries.

Catch: every affected system must support the **same atomic commit protocol**. If processing also **sends an email** via a non-2PC server, a retry may send it **twice or more**. (Exactly-once returns in Chapter 11.)

XA transactions

X/Open XA (1991) — 2PC across heterogeneous technologies, widely implemented: **PostgreSQL, MySQL, DB2, SQL Server, Oracle**; brokers **ActiveMQ, HornetQ, MSMQ, IBM MQ**.

- **Not a network protocol** — a **C API** to the coordinator. In **Java EE** it is the **Java Transaction API (JTA)**, supported by many **JDBC** and **JMS** drivers.
- XA-aware **drivers/client libraries** tell the server when an operation belongs to a distributed transaction and expose **callbacks** for the coordinator to **prepare, commit or abort**.
- The **coordinator** implements the API — usually a **library in the application process**, tracking participants and logging decisions on the **local disk**.

If the application crashes, the coordinator dies with it; prepared participants are **in doubt**. Since the log is on that server's disk, **the same server must restart** and the library read its log before telling participants the outcome via callbacks. **The database can't contact the coordinator directly** — only through the client library.

Holding locks while in doubt

The problem is **locks**: **row-level exclusive locks** on modified rows (against dirty writes — overwriting uncommitted data, Chapter 7) and, under **two-phase locking**, **shared locks** on read rows. They are held **until commit/abort** (Figure 9-9), i.e. **for as long as the transaction is in doubt — 20 minutes** if the coordinator takes 20 minutes to restart; **forever** if its log is **lost** (until manual repair). Others **can't modify** (and sometimes can't **read**) those rows, so **large parts of the application** may block.

Recovering from coordinator failure

Orphaned in-doubt transactions occur when the outcome can't be determined (e.g. log **lost or corrupted by a bug**). They **sit forever**, holding locks. **Rebooting databases won't help**: correct 2PC **keeps in-doubt locks across restarts**. An **administrator** must examine participants and apply a consistent outcome — likely **under stress** mid-outage.

Heuristic decisions let a participant **unilaterally** commit or abort an in-doubt transaction. **"Heuristic" is a euphemism for "probably breaking atomicity"** — for **catastrophes** only.

Limitations of distributed transactions

The coordinator is itself a database (it stores outcomes) and needs the same care:

1. **Single point of failure:** unreplicated coordinators block other app servers behind in-doubt locks; **many implementations lack high availability** out of the box, or replicate only basically.
2. **App servers stop being stateless:** the coordinator's **logs are crucial durable state**, so servers can no longer be added/removed freely (as the stateless, HTTP-friendly model assumes).
3. **Lowest common denominator:** XA **can't detect deadlocks across systems** (no standard lock-wait exchange) and **doesn't work with SSI** (no cross-system conflict protocol).
4. **Failure amplification:** even database-internal transactions (where **distributed SSI is possible**) need **all participants to respond**, so any broken component fails the transaction — the opposite of fault tolerance.

Alternatives that keep systems consistent without heterogeneous transactions come in Chapters 11 and 12.

Fault-Tolerant Consensus

Informally: nodes agree on something — e.g. who gets the **last flight seat**, a **theatre seat**, or a **username**. **Formally:** nodes **propose** values and the algorithm **decides** one.

Property	Meaning	Type
Uniform agreement	No two nodes decide differently.	Safety
Integrity	No node decides twice.	Safety
Validity	A decided value was proposed by some node.	Safety
Termination	Every non-crashed node eventually decides.	**Liveness**

> **Footnote:** this is **uniform consensus**, equivalent to regular consensus in asynchronous systems with unreliable failure detectors. Papers say *processes* rather than *nodes*.

- **Agreement + integrity** = everyone decides the same and nobody changes their mind.
- **Validity** rules out trivial answers (always deciding `null`).
- **Termination** = **fault tolerance**. Without it, a hard-coded **"dictator"** node suffices — until it fails, which is **2PC's** situation when the coordinator dies. The algorithm must **make progress** despite failures.

Crash model: a crashed node **never returns** — picture it under **30 feet of mud** after a landslide. An algorithm that waits for recovery can't terminate; **2PC does not satisfy termination**.

Tolerance: termination provably needs **a majority functioning** (a **quorum**), i.e. **fewer than half** crashed or unreachable. But **safety** (agreement, integrity, validity) is kept **always** — a big outage stops processing but **can't cause invalid decisions**.

Byzantine faults: most algorithms assume none; a protocol-violating node may break safety. Byzantine-robust consensus is possible if **under one-third** of nodes are Byzantine (not covered in detail).

Consensus algorithms and total order broadcast

Best known: **Viewstamped Replication (VSR)**, **Paxos**, **Raft**, **Zab** — similar, not identical. High-level ideas suffice unless you build one, which the book **doesn't recommend**.

Most decide a **sequence of values** — they are **total order broadcast algorithms**, equivalent to **repeated consensus** (each round decides the next message):

Consensus property	Total order broadcast consequence
Agreement	All nodes deliver **the same messages in the same order**
Integrity	Messages **not duplicated**
Validity	Messages **not corrupted or invented**
Termination	Messages **not lost**

VSR, Raft and Zab do total order broadcast **natively** (more efficient than repeated single-value consensus); for **Paxos** this is **Multi-Paxos**.

Single-leader replication and consensus

Single-leader replication sends writes in one order — isn't that total order broadcast? It depends on leader selection:

- **Manual** (operators): a **dictatorial** consensus; works in practice but **fails termination** (needs humans).
- **Automatic failover:** closer to fault-tolerant total order broadcast, hence consensus.

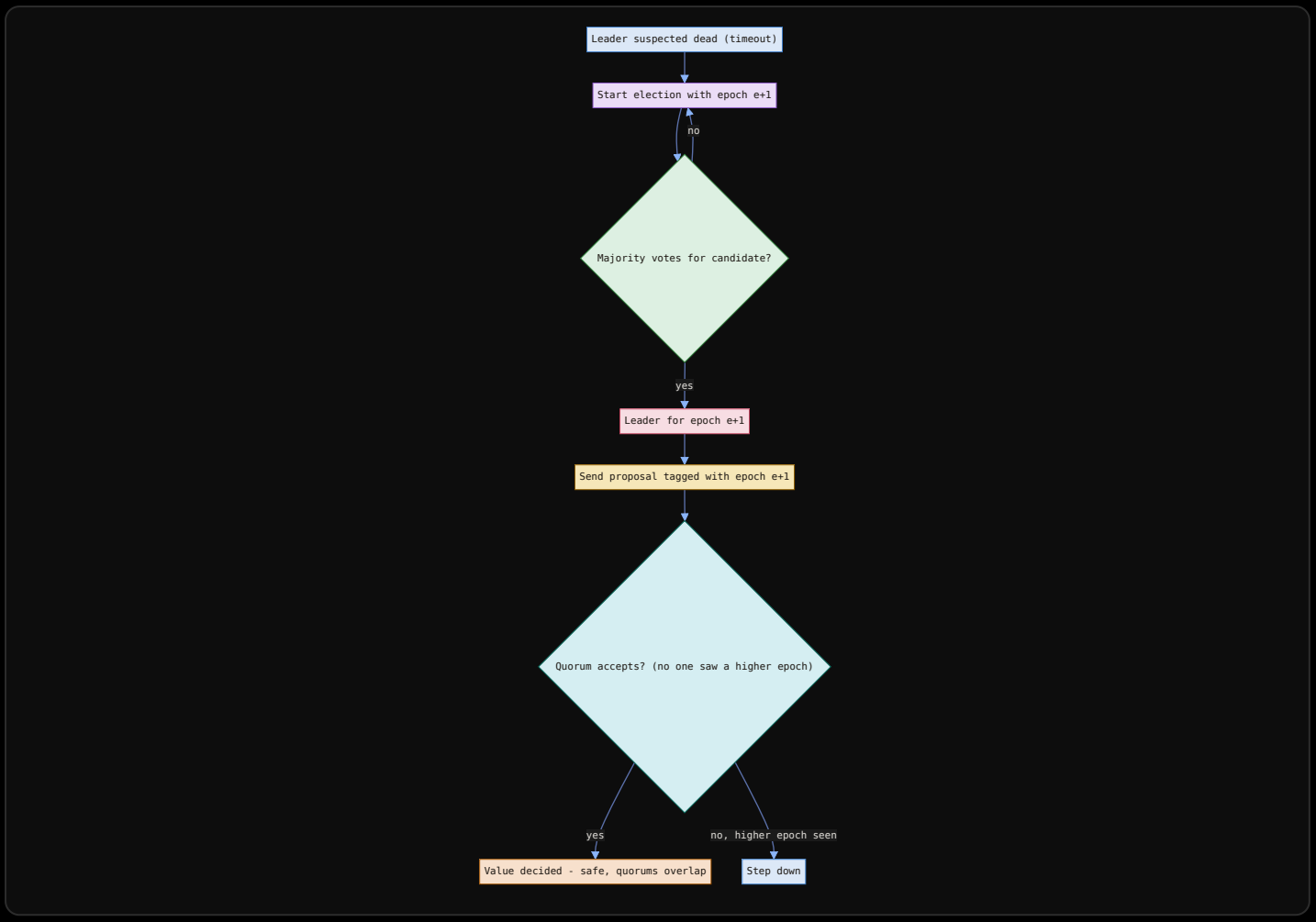
Chicken-and-egg: avoiding split brain requires consensus on the leader; consensus is total order broadcast, which looks like single-leader replication, which needs a leader…

Epoch numbering and quorums

These protocols use a leader but only promise **at most one leader per epoch**:

Algorithm	Epoch number name
Paxos	**ballot number**
Viewstamped Replication	**view number**
Raft	**term number**

1. **New election ⇒ new epoch:** when the leader is *suspected* dead, a vote is held with an **incremented epoch** — totally ordered, monotonically increasing.
2. **Higher epoch wins** a conflict between leaders.
3. **A leader can't trust itself:** before deciding, it must check no higher-epoch leader exists (truth is decided by the majority, Chapter 8).
4. **Quorum vote per proposal:** the leader waits for a quorum (typically **but not always** a majority) in favour; a node votes yes **only if it knows of no higher epoch**.
5. **Two overlapping votes:** one **elects** the leader, one **approves** a proposal. Because the **quorums overlap**, at least one approving node took part in the **latest election**; if none reports a higher epoch, the leader knows **no newer election happened** and can safely **decide**.



(Diagram, not a book figure: the epoch-and-quorum idea in flowchart form.)

	Two-phase commit	Fault-tolerant consensus
Who leads	Coordinator **not elected**	Leader **elected** per epoch
Votes needed	**"Yes" from every** participant	A **majority** (quorum)
After failure	Wait for coordinator	Defined **recovery procedure** under a new leader, preserving safety

These differences make consensus correct and fault-tolerant.

Limitations of consensus

Consensus is a **huge breakthrough** — safety amid uncertainty, progress with a majority, and **fault-tolerant linearizable atomic operations** via total order broadcast — but it has costs:

1. **Synchronous replication cost:** voting before deciding is synchronous; many databases choose **asynchronous** replication for performance, accepting some loss on failover.
2. **Strict majority: 1 failure needs ≥ 3 nodes**, **2 failures ≥ 5**. On a network split, **only the majority side progresses**.
3. **Fixed membership:** most assume a fixed voter set; **dynamic membership** extensions are less well understood.
4. **Timeouts and false elections:** with variable delays (especially **geo-distributed**), transient glitches trigger elections; safety holds, but **frequent elections ruin performance**.
5. **Network sensitivity: Raft** has edge cases — with **one consistently unreliable link**, leadership can **bounce between two nodes** or the leader is repeatedly forced out, so it **never makes progress**. Robust algorithms are **an open research question**.

Membership and Coordination Services

ZooKeeper and **etcd** look like key-value stores (read/write values, iterate keys). Why consensus, and how do they differ?

You rarely use ZooKeeper directly (it's not a general-purpose database); you rely on it **indirectly** via **HBase**, **Hadoop YARN** (cluster resource manager), **OpenStack Nova** (cloud VM manager) and **Kafka**.

Design: small data fitting entirely in memory (also written to disk), replicated with **fault-tolerant total order broadcast**. ZooKeeper is modelled on **Google's Chubby lock service** and adds:

Feature	What it gives you
Linearizable atomic operations	Atomic CAS builds a **lock** — only one concurrent attempt succeeds, guaranteed by consensus despite faults. Locks are usually **leases** that expire if the client fails.
Total ordering of operations	**Fencing tokens** against clashes after process pauses: every operation gets an increasing **zxid** and **cversion**.
Failure detection	Clients hold **long-lived sessions** with **heartbeats**; brief drops or a server failure don't kill the session, but silence past the **session timeout** does. Session locks can auto-release — **ephemeral nodes**.
Change notifications	Clients **watch** values and learn when others join (write a value) or fail (session expires, ephemeral nodes vanish) — no polling.

Only the linearizable atomic operations truly need consensus, but the **combination** is what makes it useful.

Allocating work to nodes

1. **Leader/primary selection** among instances of a service (single-leader databases, **job schedulers**).
2. **Partition assignment** for databases, message streams, file storage, **actor systems** (programs of independent message-passing actors): rebalance when nodes **join** (Chapter 6), take over when they **leave or fail**.

Done with **atomic operations, ephemeral nodes and notifications**, the app **recovers automatically**. Not easy even with **Apache Curator**, but far better than writing consensus yourself (a **poor track record**).

Own sketch of leader election with ZooKeeper-style primitives:

```
# Every candidate creates an EPHEMERAL + SEQUENTIAL node under /election
me = zk.create("/election/candidate-", data=my_address,
               ephemeral=True, sequential=True)    # e.g. candidate-0000000017

def check_leadership():
    children = sorted(zk.get_children("/election"))
    if children[0] == me.name:
        become_leader()                            # lowest number wins
    else:
        # watch the candidate just ahead of me; if its session dies,
        # its ephemeral node disappears and I get notified
        prev = children[children.index(me.name) - 1]
        zk.exists("/election/" + prev, watch=lambda _: check_leadership())

check_leadership()
```

Ephemeral = failure detection; **sequential** numbers come from the total order so all agree who's lowest; the **watch** avoids polling.

Scale: majority votes across thousands of nodes would be hopeless, so ZooKeeper runs on a **small fixed set (typically three or five)** serving many clients — **outsourcing** consensus, ordering and failure detection.

Slow-changing data: facts like "**node 10.1.1.23 leads partition 7**", changing over **minutes or hours** — **not** runtime state changing **thousands or millions of times per second**; use e.g. **Apache BookKeeper** (replicated log storage) for that.

Service discovery

ZooKeeper, etcd and **Consul** also do **service discovery**: services **register their endpoints** at startup (VMs come and go, so IPs aren't known in advance). **Does it need consensus? Less clear. DNS** (System Design Fundamentals) uses heavy **caching**; its **reads are absolutely not linearizable** and slight staleness is fine — **availability and robustness** matter more. But **leader election needs consensus**, so a consensus system can also tell others **who the leader is**. Some offer **read-only caching replicas** that receive the decision log asynchronously, **don't vote**, and serve non-linearizable reads.

Membership services

ZooKeeper continues research on **membership services** dating to the **1980s**, important for systems like **air traffic control**. A membership service decides **which nodes are live members**. You can't be certain a node failed, but **coupling failure detection with consensus** lets nodes **agree** on it. A live node may still be wrongly declared dead, but **agreed membership** is valuable — e.g. leader = **lowest-numbered current member** works only if all agree on membership.

Chapter summary

- **Eventual consistency** only promises convergence "some time"; stronger models are easier to use but cost performance/availability.
- **Linearizability** = one copy + atomic operations + real-time recency: once any read sees a new value, all later reads must. Easy to reason about (like a variable), but **slow**. Needed for **locks/leader election**, **hard uniqueness constraints**, and avoiding **cross-channel races**.
- It's **not** serializability; both = **strict serializability**. SSI isn't linearizable.
- **Linearizable replication:** single-leader (potentially), consensus (yes), multi-leader (no), leaderless (probably not — even strict quorums fail under variable delay).
- **CAP:** under a partition, linearizability *or* availability; too narrow and confusing to be useful. The deeper cost is **latency** (Attiya and Welch); even multi-core RAM gives it up for **speed**.
- **Causality** is a **partial** order (timeline branches and merges); linearizability a **total** order. Linearizable ⇒ causal. **Causal consistency** is the strongest model that stays fast and available under faults.
- **Lamport timestamps** give a causally consistent total order but can't tell when it is **final** → can't enforce uniqueness on the spot. That leads to consensus.

- **Total order broadcast** (reliable, same-order delivery fixed at delivery) = a replicated log: state machine replication, fencing tokens (zxid), linearizable CAS.
- **2PC:** prepare (promises) → coordinator logs decision (commit point) → retried forever. Coordinator crash leaves participants **in doubt** holding locks; 3PC needs bounded delays; XA coordinators are single points of failure; heuristic decisions break atomicity.
- **Consensus:** agreement, integrity, validity (safety) + termination (liveness, needs a majority). VSR, Paxos (Multi-Paxos), Raft, Zab use **epochs** and **overlapping quorums**.

Problems equivalent to consensus (a solution to one converts to the others):

Problem	What must be decided
Linearizable CAS registers	Whether to set the value, atomically, given the current value
Atomic transaction commit	Commit or abort
Total order broadcast	Message delivery order
Locks and leases	Which racing client got the lock
Membership/coordination service	Which nodes are alive (via a failure detector such as timeouts)
Uniqueness constraint	Which of several concurrent conflicting writes succeeds

All are easy on **one node** or by giving one node all the power — a **single-leader database**. If the leader fails, the options are:

1. **Wait for recovery** (many **XA/JTA coordinators**) — violates **termination**; may block forever.
2. **Manual failover** (many **relational databases**) — consensus by **"act of God"**, at human speed.
3. **Automatic election** — needs a **proven consensus algorithm** that handles adverse networks.

A single-leader database avoids a consensus round per write but **still needs consensus to keep and hand over leadership** — the leader **"kicks the can down the road"**. **ZooKeeper** provides **outsourced consensus, failure detection and membership**; use something like it for anything **reducible to consensus** that must be **fault-tolerant**. Not every system needs it: **leaderless and multi-leader** systems avoid global consensus and live with **branching and merging histories**. Theory papers, though hard and sometimes unrealistic, show what is and isn't possible. This ends **Part II**; **Part III** builds applications from heterogeneous building blocks.

Interview angle

1. "What is linearizability?" *30-second answer:* A replicated system behaving like one copy, with each operation taking effect atomically between request and response. Once a write completes — or *anyone* has read the new value — every later read returns it or newer. A recency guarantee on single objects.

- *Follow-up: vs serializability?* Serializability: multi-object transactions as if in some serial order (not necessarily real-time). Linearizability: single-register real-time recency. Both = strict serializability; SSI is serializable but not linearizable (reads snapshots).
- *Follow-up: do quorums ($w + r > n$) give it?* Not necessarily: one reader sees the new value on one replica while a later reader hits two stale ones. Needs synchronous read repair and read-before-write; Cassandra still breaks with LWW, and CAS can't be done this way.

2. "Explain the CAP theorem." *30-second answer:* During a network partition, refuse some requests to stay linearizable, or keep serving and give it up. "Pick 2 of 3" misleads — partitions aren't optional — so it's "consistent or available *when partitioned*".

- *Follow-up: why do many databases avoid linearizability?* Mainly performance: it is slow all the time, with latency at least proportional to delay uncertainty. Multi-core CPUs drop it for the same reason.
- *Follow-up: strongest model that stays available?* Causal consistency.

3. "How does two-phase commit work, and what's its main weakness?" *30-second answer:* Coordinator assigns a global ID; participants work; on *prepare* each durably writes data, checks constraints and

votes yes (a promise). If all vote yes, the coordinator logs "commit" (commit point) and sends commit, retrying forever. Weakness: if it crashes after yes votes, participants are *in doubt*, holding locks until it recovers.

- *Follow-up: why not time out?* Aborting might contradict a committed participant; committing might contradict an aborted one.
- *Follow-up: does 3PC fix it?* Only with bounded delays and response times; nonblocking commit needs a perfect failure detector.
- *Follow-up: XA's practical problems?* Coordinator often an unreplicated library in the app server (SPOF, app no longer stateless), lowest common denominator (no cross-system deadlock detection, no SSI), failure amplification, orphaned transactions needing manual/heuristic resolution; MySQL distributed transactions reported >10× slower.

4. "What is consensus and what properties must it have?" *30-second answer:* Nodes propose values and all decide one. Uniform agreement, integrity (decide once), validity (value was proposed) are safety; termination (every non-crashed node eventually decides) is liveness and needs a majority up.

- *Follow-up: isn't it impossible (FLP)?* Only in the asynchronous model without clocks, timeouts or randomness.
- *Follow-up: how do Raft/Paxos avoid split brain?* One leader per epoch (term/ballot/view); each decision needs a quorum overlapping the election quorum, so a stale leader learns of a higher epoch.
- *Follow-up: nodes to tolerate 2 failures?* Five.

5. "Relationship between total order broadcast, linearizability and consensus?" *30-second answer:* Total order broadcast is a replicated log with order fixed at delivery. Linearizable CAS can be built from it (append claim, wait, first claim wins), and it can be built from a linearizable increment-and-get counter (gap-free numbers). Linearizable CAS, total order broadcast and consensus are equivalent; Raft and Zab are really total order broadcast.

- *Follow-up: why aren't Lamport timestamps enough?* The order is never known to be final — another node may later produce a lower timestamp, and gaps can't be detected.

6. "What is ZooKeeper used for?" *30-second answer:* Outsourced coordination running consensus (Zab) on 3 or 5 nodes: linearizable atomic operations for locks/leases, zxid ordering for fencing tokens, session heartbeats with ephemeral nodes for failure detection, watches for notifications. HBase, Hadoop YARN, OpenStack Nova and Kafka use it for leader election, partition assignment and membership.

- *Follow-up: store application data in it?* No — small, slow-changing data (minutes/hours); use e.g. Apache BookKeeper for high-rate replicated state.

Self-check

1. **Why is eventual consistency hard to program against?** No bound on convergence; you may not even read your own write. Bugs appear only under faults or high concurrency.
2. **In Figure 9-4, why is B's final read not linearizable though it overlaps C's `cas(x, 2, 4)`?** A already read 4 before B's read began; after any read returns a new value, later reads must too.
3. **Thumbnails occasionally show an old photo. What's happening; two fixes?** A cross-channel race: the queue outran file-storage replication. Fix with linearizable storage, or a read-your-own-writes-style technique on the queue you control (own illustration: include the photo's version in the message and have the resizer wait for at least that version).
4. **Does a Dynamo-style store with n = 3, w = 3, r = 2 give linearizability?** No — a later reader can hit two replicas lacking the in-flight write. Synchronous read repair plus read-before-write fixes reads/writes at a cost, not CAS.
5. **Why don't odd/even counters give causal order, and how do Lamport timestamps fix it?** Nodes run at different rates, so numbers ignore cross-node causality. Lamport timestamps carry the max counter seen and bump local counters to it, so causal successors get larger timestamps.
6. **Why can't Lamport timestamps enforce unique usernames immediately?** A node can't know whether another is concurrently assigning a lower timestamp to the same name; checking with every node

halts on any failure.

7. **The two points of no return in 2PC?** A participant's "yes" vote, and the coordinator logging its decision.
8. **The coordinator's log is lost. What happens?** In-doubt transactions become orphaned, holding locks indefinitely (even across database restarts); an admin must resolve them manually, or use a heuristic decision (probably breaking atomicity).
9. **How does a Raft leader know it's still leader when deciding?** Its proposal needs a quorum, and nodes accept only if they know no higher term; the proposal quorum overlaps the latest election quorum, so no higher term means no newer election.
10. **Does a 4-node consensus cluster tolerate more failures than 3 nodes?** No. Majority of 4 is 3, so it tolerates 1 — same as 3 nodes. Five nodes tolerate 2.

Chapter 10 — Batch Processing

Book: Chapter 10, pp. 389–438 (with the Part III introduction just before it)

Overview

Part III opener: Derived Data (summary of the Part III introduction)

Parts I–II covered *one* distributed database. Real applications need to access data in many ways no single database handles well, so they combine an OLTP database (online transaction processing — the main everyday read/write store), a cache, a full-text search index, an analytics warehouse, and so on, plus plumbing to move data between them. Part III is about that **integration problem**, which "our product does everything" vendors tend to ignore but which is one of the most important jobs in any non-trivial application.

Kind	What it means	Examples
System of record (*source of truth*)	Authoritative copy; new data (e.g. user input) is written here first; each fact stored once, usually **normalized** (no duplication). On disagreement it wins by definition.	The app's primary OLTP database
Derived data system	Data produced by transforming data from another system; if lost, it can be **rebuilt** from the source.	Caches, indexes, denormalized values, materialized views (stored precomputed query results), recommendation models

- Derived data is **redundant** but often essential for fast reads; typically denormalized, and one source can feed several derived "views".
- Making the distinction explicit clarifies the system's **dataflow**: which component consumes and produces what.
- A database product is *not* inherently one or the other — it depends on **how your application uses it**.
- Part III plan: Chapter 10 batch dataflow (e.g. MapReduce); Chapter 11 the same ideas on streams (lower delay); Chapter 12 ties it together for future applications.

What this chapter answers

How do you process a huge, fixed pile of data (terabytes of logs, a database snapshot) into useful output, reliably, across many machines?

Style	Triggered by	Main metric	Example
Services (online)	Client request, waits for reply	Response time; availability	Web server, database, cache
Batch processing (offline)	A job (often scheduled, e.g. daily) over a large fixed input; runs minutes to days, nobody waits	**Throughput** — time to process an input of a given size	MapReduce, Spark
Stream processing (near-real-time / nearline)	Consumes events shortly after they occur; inputs and outputs, not request/response	Lower latency than batch	Chapter 11

Batch jobs build search indexes, recommendation models, analytics tables and derived databases; their ideas (immutable inputs, derived outputs, rerunnable jobs) underpin stream processing and Part III's derived-data architecture.

History:

- **MapReduce** was published by Google in **2004** (once over-enthusiastically called the algorithm behind Google's scalability) and implemented in **Hadoop**, **CouchDB** and **MongoDB**.

- Compared with older data-warehouse parallel query engines it is *low-level*, but it enabled processing at unprecedented scale on cheap **commodity machines** (see System Design Fundamentals). Its importance is declining, but it is still the clearest way to understand batch processing.
- Batch processing predates computers: **Hollerith** punch-card tabulating machines computed statistics for the **1890 US Census**, and MapReduce eerily resembles the **IBM card sorters** used for **business data processing** in the 1940s–50s.

> **Analogy:** a bank's end-of-day reconciliation — tellers serve customers one at a time (online); at night a clerk sorts the day's slips by account and totals each (batch). What matters is finishing the stack by morning.

Key terms

Term	Plain-English meaning
System of record / source of truth	Authoritative copy; written first; wins disagreements
Derived data	Computed from other data; rebuildable (caches, indexes, views)
Online system / batch / stream	Request-response judged by latency / bounded job judged by throughput / events processed soon after they happen
Throughput	Data processed per unit of time
Bounded input	Known, fixed-size input, so the job can finish
Working set	Data a job needs random access to in memory
Unix pipe	Connects stdout of one program to stdin of the next via a small memory buffer
Uniform interface	Every program reads/writes the same kind of thing (Unix: a file descriptor = byte sequence)
stdin / stdout	A program's default input and output streams
Loose coupling / late binding / inversion of control	The program doesn't choose its input/output; the caller wires it
HDFS / NameNode	Hadoop Distributed File System (open-source GFS) / its central server tracking which machine holds which block
Shared-nothing	Each machine has its own CPU, RAM, disks; only a normal network connects them
Erasure coding (e.g. Reed–Solomon)	Redundancy codes that rebuild lost data using less space than full copies
MapReduce / mapper / reducer	Model where a stateless *mapper* emits key-value pairs per record, the framework sorts by key, and a *reducer* processes each key's values via an iterator
Shuffle	Partitioning mapper output by reducer, sorting, copying to reducers (no randomness)
Data locality	Running a task on a machine that already holds its input
Workflow / workflow scheduler	Chain of jobs linked by output→input directories / tool that starts jobs when inputs are ready (Oozie, Azkaban, Luigi, Airflow, Pinball)
Join / equi-join	Combining records that refer to each other / matching on equal field values
Full table scan	Reading every input record
Activity events / clickstream	Log of actions logged-in users took on a site
Sort-merge join	Reduce-side join: both inputs keyed by join key, sorted, merged in the reducer
Secondary sort	Ordering values *within* a key (e.g. user record first, then events by time)
Sessionization	Grouping one user session's events to see the action sequence
Skew / hot spot; hot key / linchpin object	One partition gets far more work; a key with disproportionate data (e.g. a celebrity)
Reduce-side vs map-side join	Join in reducers after shuffle (no input assumptions) vs in mappers only, exploiting known input properties
Broadcast hash join	Small input loaded into a hash table in every mapper of the large input

Term	Plain-English meaning
Partitioned hash join (bucketed map join)	Both inputs partitioned identically; each mapper joins one matching pair of partitions
Map-side merge join	Inputs partitioned *and sorted* identically; mapper merges them
Immutable	Never modified once written
Human fault tolerance	Recovering from buggy code by rolling back and rerunning
MPP database	Massively parallel processing database running analytic SQL over a cluster
Data lake / enterprise data hub	Raw organizational data dumped in one place, schema decided later
Schema-on-read	The reader interprets structure, not enforced at write time
Sushi principle	"Raw data is better" — keep it unprocessed so many views can be derived
Preemption	Killing a low-priority task to free resources for a higher-priority one
Materialization	Eagerly computing an intermediate result and writing it out
Straggler	A task much slower than its peers
Dataflow engine / operator	Engine (Spark, Tez, Flink) running a whole multi-stage workflow as one job / its generalized processing function
RDD	Spark's Resilient Distributed Dataset, recording how each piece of data was computed
Checkpoint	Saving state durably so computation can resume
Deterministic operator	Same input → same output
Pipelined execution	Passing output to the next operator incrementally
DAG	Directed acyclic graph — how dataflow engines arrange operators (not graph data!)
Transitive closure	Repeatedly following edges until no new vertices are reached
BSP / Pregel model	Graph processing in rounds; vertices message each other between rounds
"Thinking like a vertex"	Writing graph code from one vertex's viewpoint
Declarative query language / cost-based optimizer	State *what*, optimizer decides *how* / picks join algorithm and order by estimated cost
Vectorized execution	Tight CPU-cache-friendly loops without per-record function calls
k-nearest neighbors	Finding items closest to a given item in multi-dimensional space

Batch Processing with Unix Tools

Unix design ideas carry straight over to huge distributed systems.

Running example: a web server log, one line per request. nginx's default format records client IP, a literal hyphen, remote user (hyphen if not logged in), local time, request line (method, path, protocol), HTTP status (200 OK, 404 not found), response bytes, referrer URL and user-agent (browser/OS) string. The book's sample decodes as: on 27 Feb 2015 at 17:55:11 UTC, IP 216.58.210.78 requested `/css/typography.css`, status 200, 3,377 bytes, Chrome 40, referred by `http://martin.kleppmann.com/`.

A made-up line in the same shape:

```
10.1.2.3 - - [03/Mar/2021:08:00:01 +0000] "GET /blog/intro.html HTTP/1.1" 200 5120 "https://example.org/"
"Mozilla/5.0 (X11; Linux x86_64) Firefox/86.0"
```

Fields: `$remote_addr` (`10.1.2.3`), fixed `-`, `$remote_user` (`-`), `$time_local`, `$request` (split on spaces, the URL is the **7th field**), `$status` (`200`), `$body_bytes_sent` (`5120`), `$http_referer`, `$http_user_agent`.

Simple Log Analysis

Goal: the 5 most requested pages (own example, same idea):

```
cat /srv/logs/web-access.log |   # 1. stream the log file
  awk '{print $7}'           |   # 2. keep only field 7: the requested URL
  sort                       |   # 3. sort URLs so identical ones sit next to each other
  uniq -c                    |   # 4. collapse runs of identical lines, prefixing a count
  sort -r -n                 |   # 5. sort numerically (-n) by that count, biggest first (-r)
  head -n 5                      # 6. keep the first 5 lines only
```

`sort` makes a URL requested *n* times appear as *n* adjacent lines; `uniq -c` collapses *adjacent* duplicates with a count; the second sort ranks by count. (A footnote: `cat` is technically unnecessary, but makes the linear pipeline obvious.) Output looks like `4189 /favicon.ico`, `3631 /2013/05/24/...`. This one-liner processes **gigabytes in seconds** and is easy to tweak — skip CSS with `'$7 !~ /\.css$/ {print $7}'`, or count client IPs with `'{print $1}'`.

Learn `awk`, `sed`, `grep`, `sort`, `uniq` and `xargs`: analyses take minutes to write and run fast (a book reference claims command-line tools beating a Hadoop cluster by 235x).

Chain of commands versus custom program

Own Python version (the book uses Ruby):

```
from collections import Counter

hits = Counter()                                # dict of URL -> count, default 0
with open('/srv/logs/web-access.log') as f:
    for line in f:
        url = line.split()[6]                   # 7th whitespace field (index 6, zero-based)
        hits[url] += 1                          # bump that URL's counter
for url, n in hits.most_common(5):              # sort by count descending, take top 5
    print(n, url)                               # print "count url"
```

`Counter` is an in-memory **hash table** (finds a value by key in roughly constant time) from URL to count. Readability is partly taste; the *real* difference is execution on large inputs.

Sorting versus in-memory aggregation

Approach	How it counts	Memory	Wins when
Hash table (script)	One counter per distinct URL	Proportional to **distinct** URLs (a million hits to one URL = one entry)	Working set fits in RAM — for most small/mid sites about 1 GB, even on a laptop
Sorting (pipeline)	Sort so duplicates are adjacent, count runs	Can spill to disk	Working set exceeds memory

Sorting scales via the SSTable/LSM-tree trick (Chapter 3): sort chunks in memory, write sorted segments to disk, merge them. Mergesort's **sequential** I/O suits disks (a recurring theme). GNU Coreutils `sort` **spills to disk** automatically and **parallelizes across cores**, so the bottleneck becomes disk read speed.

The Unix Philosophy

> **Historical note.** Doug McIlroy, inventor of Unix pipes, wrote in **1964** that programs should connect like segments of a garden hose.

The **1978** Unix philosophy, paraphrased:

1. Each program does one thing well; for new jobs write new programs rather than adding features.

2. Expect output to feed another, unknown program: no clutter, no rigid columnar/binary input, no interactive input.
3. Ship software (even an OS) for trial early, ideally in weeks; rebuild clumsy parts without fear.
4. Use tools rather than unskilled help to lighten programming, even if tools are thrown away later.

This (automation, rapid prototyping, incremental iteration, experimentation, splitting big projects) resembles today's **Agile** and **DevOps** movements. `sort` is the model tool: arguably better than most languages' standard-library sorts (which neither spill to disk nor use multiple threads), yet useful mainly combined with others like `uniq`. A shell like `bash` composes programs by different authors. What enables that?

A uniform interface

- If any output can feed any input, **all programs need one I/O interface** — in Unix a **file** (*file descriptor*): an ordered sequence of bytes.
- Many things fit behind it: a real file, a pipe (stdin, stdout, Unix sockets), a device driver (`/dev/audio`, `/dev/lp0`), a TCP socket.
- By convention many tools treat bytes as **ASCII text** with records separated by `\n` (0x0A); `awk`, `sort`, `uniq`, `head` assume this. The ASCII **record separator 0x1E** would arguably have been better — what matters is agreement.
- Parsing *within* a record is less standard (whitespace, tabs, CSV, pipes; `xargs` has about half a dozen options). `{print $7}` is cryptic — `{print $request_url}` would be nicer; structured formats like **Avro/Parquet** on Hadoop address this later.

> **Aside: the web as a uniform interface.** URLs and HTTP let any site link to any resource. Before that, each **bulletin board system (BBS)** had its own phone number and modem settings; following a reference meant hanging up, dialing another BBS and searching manually.

Few systems compose as well today: you can't pipe your email and shopping history through a tool into a spreadsheet. Even same-model databases make data movement hard — the **Balkanization of data**.

Separation of logic and wiring

- Tools read **stdin**, write **stdout** (default keyboard/screen), redirectable to files or piped; a pipe uses a small in-memory buffer, **not** disk.
- A program that ignores file paths and uses only stdin/stdout lets the shell user decide the wiring — **loose coupling**, **late binding**, **inversion of control**. You can drop your own tool (user-agent → browser, IP → country) into the pipeline.

Limits:

- **Several inputs or outputs** are awkward.
- You can't pipe output into a **network connection** (except via `netcat` or `curl`). BSD sockets broke "everything is a file"; research OSes **Plan 9** and **Inferno** expose TCP connections as files under `/net/tcp`.
- A program that opens files, subprocesses or connections itself owns that wiring (configurable via command-line options), losing shell flexibility.

Transparency and experimentation

- **Inputs are immutable** — rerun with different options without damaging input.
- **Stop anywhere**, pipe into `less`, inspect output — great for debugging.
- **Save a stage's output to a file** and restart later stages from it.

Blunt compared with a relational query optimizer, but superb for experimentation. **Biggest limitation: one machine** — hence Hadoop.

MapReduce and Distributed Filesystems

MapReduce is roughly "Unix tools across up to thousands of machines" — blunt and brute-force but effective.

- One **job** ≈ one Unix **process**: one or more inputs → one or more outputs.
- It **doesn't modify input** and has **no side effects** besides output.
- Output files are written **once, sequentially**, never modified.
- Instead of stdin/stdout it uses a **distributed filesystem** — in Hadoop, **HDFS**, an open-source reimplementation of Google's **GFS**.

Others: **GlusterFS**, **Quantcast File System (QFS)**; object stores (**Amazon S3, Azure Blob Storage, OpenStack Swift**) are similar in many ways.

> **Footnote: HDFS vs object stores.** HDFS lets the scheduler run a task where a copy of its input lives (helps when network bandwidth is the bottleneck); object stores usually separate compute and storage. With **erasure coding**, the locality benefit disappears, since pieces from several machines must be combined.

How HDFS works:

1. **Shared-nothing**, unlike **shared-disk** storage — **NAS** (Network Attached Storage), **SAN** (Storage Area Network) — which needs a central appliance and special networks like **Fibre Channel**.
2. A **daemon** (background process) on every machine serves its local files over the network.
3. The **NameNode** records which blocks live where.
4. Together: one huge filesystem over all machines' disks.
5. Blocks survive failures via **replication** (copies, Chapter 5) or **erasure coding** like **Reed–Solomon codes** (less overhead) — like **RAID** but over an ordinary network.

Scale at writing: largest clusters had **tens of thousands of machines** and **hundreds of petabytes** (1 PB = 1,000 TB). Viable because commodity hardware and open-source software cost far less than a dedicated storage appliance.

MapReduce Job Execution

The log example maps almost exactly:

1. **Read input files, split into records** (log: one line per record) — *input format parser*.
2. **Mapper** extracts key and value per record (log: `awk '{print $7}'`, key = URL, empty value) — *you write this*.
3. **Sort pairs by key** (log: first `sort`) — *implicit, always happens*.
4. **Reducer** walks sorted pairs; equal keys are adjacent, so little state is needed (log: `uniq -c`) — *you write this*.

- **Mapper:** called once per record; emits any number of pairs (including zero); **no state** between records.
- **Reducer:** called once per key with an **iterator** over its values (streamed, not all in memory); emits output records.

The second sort (ranking) needs a **second job** taking the first job's output. In short: mappers prepare data for sorting; reducers process sorted data.

Own pseudocode:

```
def mapper(line):                # called once per input record (one log line)
    fields = line.split()
    yield (fields[6], 1)         # key = URL, value = 1 (one hit)

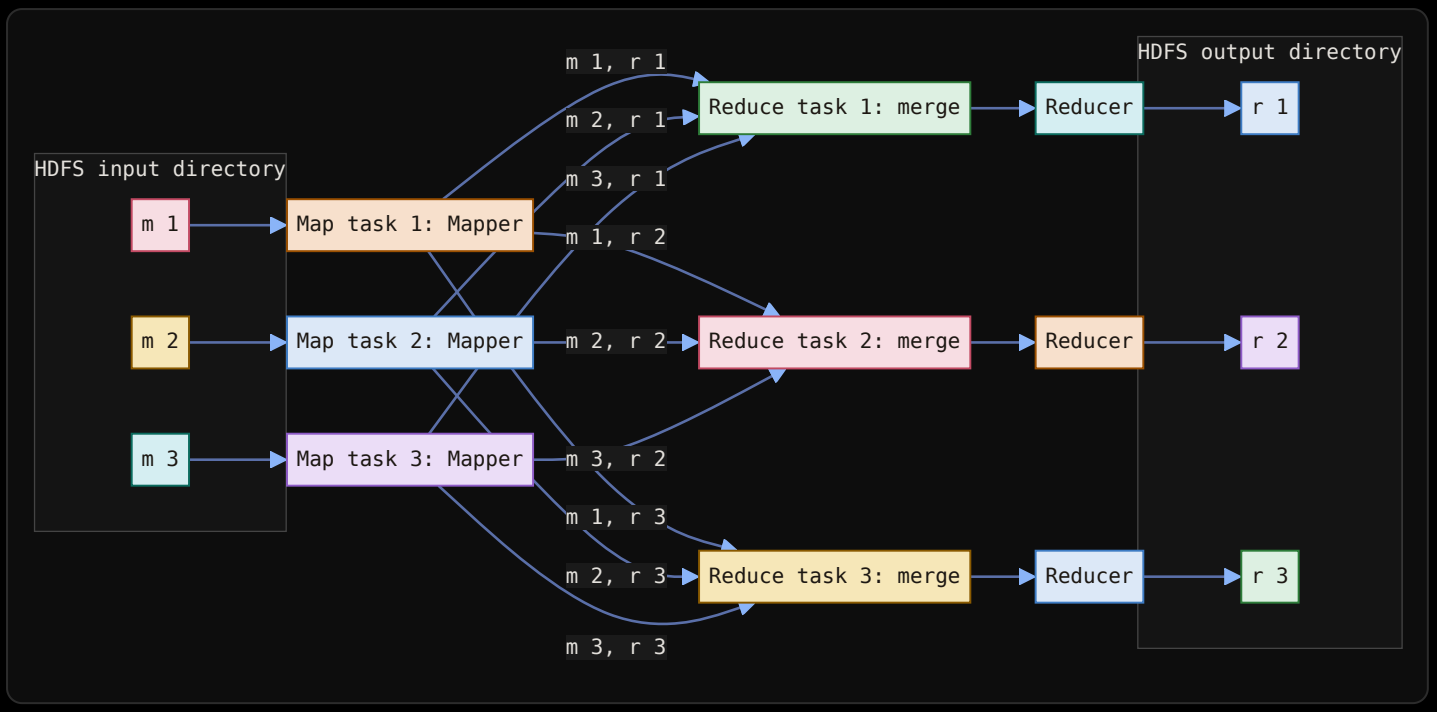
def reducer(url, counts):        # called once per distinct key, after sort
    yield (url, sum(counts))     # counts is an iterator of all the 1s for this URL
```

Distributed execution of MapReduce

MapReduce **parallelizes across machines without parallel code**: mappers and reducers see one record/key at a time and don't know data locations, so the framework moves data.

- Mappers/reducers can be Unix tools but are usually functions — **Java classes implementing an interface** in **Hadoop MapReduce**, **JavaScript functions** in **MongoDB and CouchDB**.
- Parallelism comes from **partitioning** (Chapter 6): input is usually an HDFS directory, and **each file or block is a partition** handled by one **map task**.

Figure 10-1 (redrawn): a job with three map tasks and three reduce tasks. Map task i reads input block m i from HDFS, and its mapper writes three sorted files "m i, r 1", "m i, r 2", "m i, r 3" on local disk, one per reducer. Reduce task j fetches file "m i, r j" from every map task, merges them, runs the reducer, and writes output r j to HDFS.



1. **Map tasks near the data:** inputs are typically hundreds of MB; the scheduler runs each mapper on a machine holding a replica of its block if it has spare RAM/CPU — **putting the computation near the data** (less network copying, better locality).
2. **Ship code:** the framework copies the mapper code (e.g. **JAR files** for Java) to that machine and starts the task, which feeds its block to the mapper record by record.
3. **Reducers:** map task count = input block count; reduce task count is **chosen by the job author**. A key's reducer is picked by **hash of the key** (Chapter 6), so equal keys meet.
4. **Sort in stages:** each map task **partitions its output by reducer** and writes each partition as a **sorted file on local disk** (SSTable-like).
5. **Fetch:** when a mapper finishes, reducers download their partition's file from every mapper.
6. Steps 4–5 are the **shuffle** — despite the name, **no randomness**.
7. **Merge:** reducers merge files preserving order, so equal keys from all mappers are adjacent.
8. **Reduce:** per key, an iterator streams its records; arbitrary logic, any number of outputs.
9. **Output** goes to the distributed filesystem (one copy local, replicas elsewhere).

MapReduce workflows

- One job can count views per URL but not also rank them (needs a second sort), so jobs are chained into **workflows**.
- Hadoop has **no built-in workflow support**: chaining is implicit via **directory names** — more like commands writing **temporary files** than a pipe (see "Materialization of Intermediate State").
- Output is valid **only if the job fully succeeds** (partial output of failed jobs is discarded), so a job starts only after all its input-producing jobs succeed.

- **Workflow schedulers: Oozie, Azkaban, Luigi, Airflow, Pinball** manage dependencies and large job collections.
- Recommendation systems commonly run **50 to 100 MapReduce jobs**; many teams read each other's outputs, so tooling matters.
- Higher-level tools (**Pig, Hive, Cascading, Crunch, FlumeJava**) build and wire multi-stage workflows automatically.

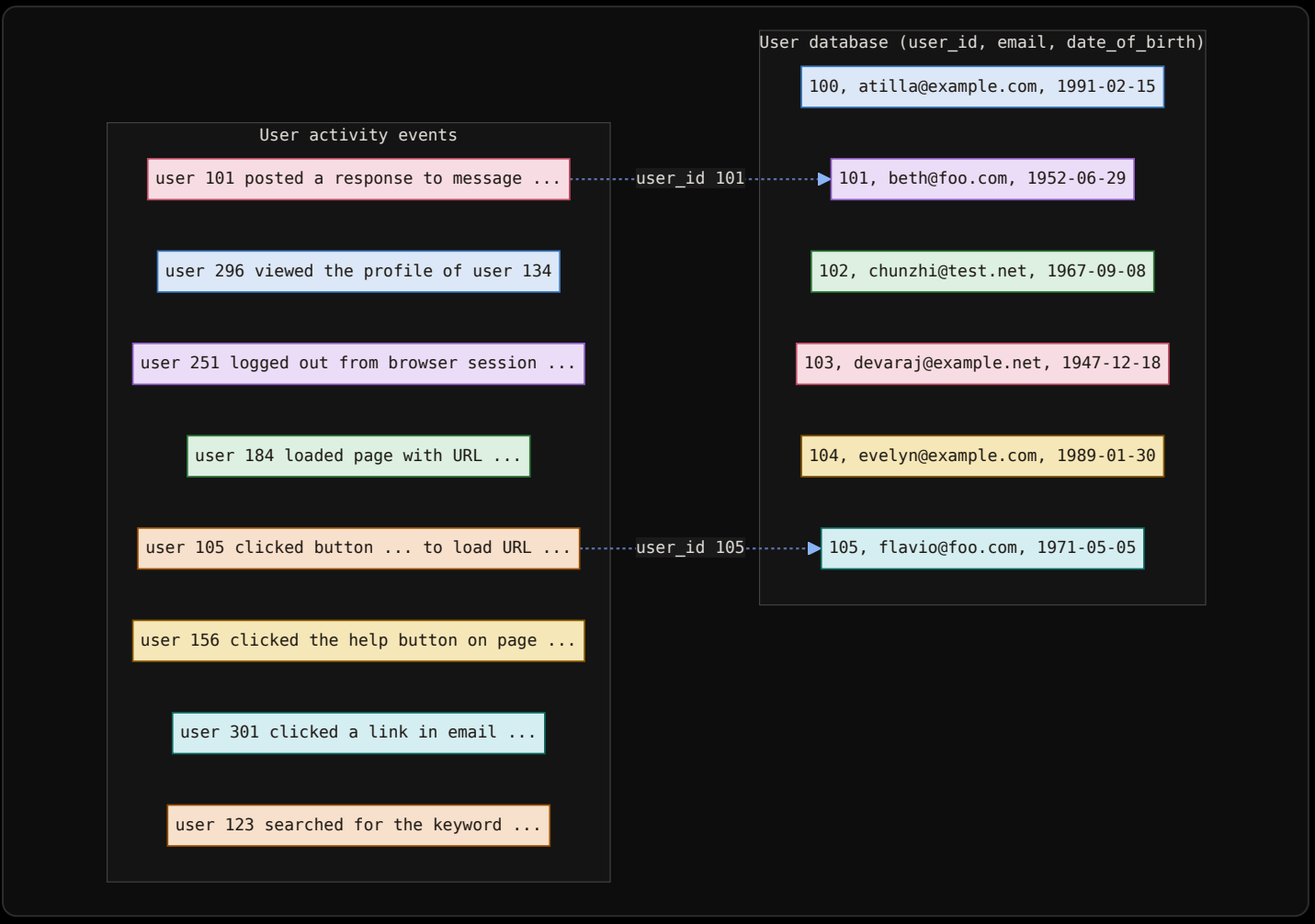
Reduce-Side Joins and Grouping

- Records reference each other via a **foreign key** (relational), **document reference**, or graph **edge**; code needing both sides needs a **join**. Denormalization reduces but rarely removes joins.
- Databases use an **index** for queries touching few records. MapReduce has **no indexes**: it reads **all** input — a **full table scan**. Absurd for a few records, reasonable for analytics, especially in parallel.
- So a batch "join" resolves **every** occurrence of an association across the dataset (all users at once), not one lookup.

> **Footnote: equi-joins.** The book's joins are **equi-joins** (matching identical values); more general joins (e.g. less-than) aren't covered.

Example: analysis of user activity events

Figure 10-2 (redrawn): on the left, a log of user activity events (each mentions only a user ID); on the right, a user database table with user_id, email and date_of_birth. The arrows show two events being matched to their user rows (values as in the book's figure).



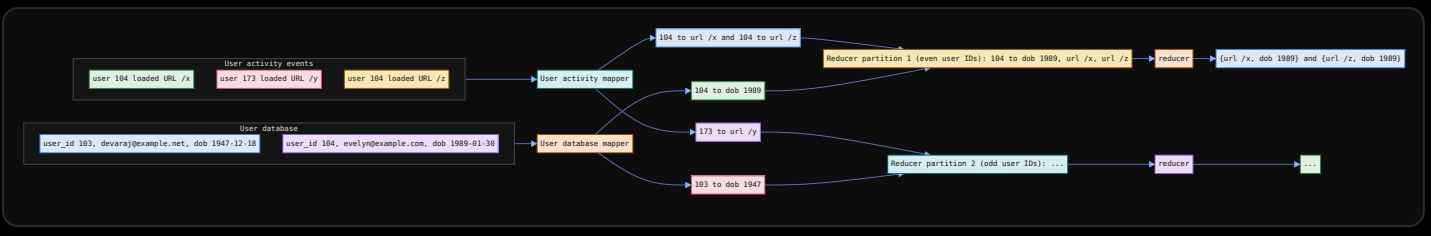
The **activity events** (**clickstream data**) are the star schema's **fact table**, the user DB a **dimension** (Chapter 3). Goal: correlate activity with profiles, e.g. which pages are popular with which age groups. Events carry only user IDs (copying profiles in would be wasteful), so we must join.

Naive approach — query a remote user DB per event — fails: throughput capped by **round-trip time**; a cache helps only with favorable distributions; parallel queries can **overwhelm the database**; and the job becomes **nondeterministic** because the DB changes during the run.

Better: keep computation local. Copy the user DB into HDFS (e.g. from a backup via **ETL** — Extract, Transform, Load) next to the events, so MapReduce can bring matching records together.

Sort-merge joins

Figure 10-3 (redrawn): joining on user ID in the reducers via sort-merge, using the book's example values. The activity mapper turns each event into (user ID, URL); the user-database mapper turns each profile into (user ID, date of birth). Partitioning sends even-numbered user IDs to reducer partition 1 and odd-numbered ones to partition 2. Partition 1's reducer sees user 104's date of birth first, then both URLs, and outputs one joined record per URL. (If an input is split across several files, several mappers can process it in parallel.)



1. One set of mappers emits (user ID, event); another emits (user ID, date of birth). Multiple files → parallel mappers.
2. Partitioning and sorting make each user's record and events **adjacent** in one reducer.
3. A **secondary sort** delivers the **user record first**, then events in **timestamp order**.
4. The reducer, called once per user ID, stores the date of birth and emits (viewed URL, viewer age) per event. Later jobs can compute age distributions per URL.

Efficient: **one user record in memory** at a time, **no network requests**. Named **sort-merge join** because mapper output is sorted by key and the reducer merges both sorted inputs.

Own pseudocode:

```
def event_mapper(event):                    # input 1: activity events
    yield (event['uid'], ('E', event['ts'], event['url']))

def user_mapper(user):                      # input 2: user profiles
    yield (user['uid'], ('U', user['birth_year']))

def join_reducer(uid, values):              # values arrive user-record-first (secondary sort)
    birth_year = None
    for v in values:
        if v[0] == 'U':
            birth_year = v[1]               # remember the one profile field we need
        elif birth_year is not None:
            _, ts, url = v
            yield (url, 2021 - birth_year)  # emit (page, approximate viewer age)
```

The `'U'`/`'E'` tag tells the reducer which input a value came from.

Bringing related data together in the same place

- Everything for one key lands in **one reducer call**, so reducers are simple, single-threaded, high-throughput, low-memory code.
- Mental model: mappers **"send messages"** to reducers; the key is the **destination address**.
- MapReduce **separates network communication from application logic** (unlike databases, where fetches hide inside app code), and the framework shields your code from **partial failures** by transparently retrying failed tasks.

GROUP BY

Grouping (SQL `GROUP BY`) uses the same pattern: mappers emit the **grouping key**, then aggregate per group — count records (`COUNT(*)`), sum a field (`SUM(fieldname)`), take the top k. So **grouping and joining look**

very similar on MapReduce.

```
SELECT url, COUNT(*) AS views      -- one output row per distinct URL
FROM page_views                    -- the input "table" (log records)
GROUP BY url;                      -- in MapReduce: mapper emits url as the key
```

Sessionization groups a user session's events to see their sequence — e.g. for **A/B testing** (show versions to random groups and compare purchases) or judging a marketing campaign. One user's events are scattered over many web servers' logs, so group by **session cookie** or user ID.

Handling skew

When **one key has huge data** — e.g. **celebrities** with millions of followers (**linchpin objects** / **hot keys**) — its reducer gets **skew** (a **hot spot**). A job finishes only when **all** tasks finish, so everything waits for the slowest reducer.

Technique	How it works	Cost / requirement
Pig skewed join	A **sampling job** finds hot keys; hot-key records go to **a random one of several reducers**, and the other input's matching records are **replicated to all** of them	Replicating the other input
Crunch sharded join	Same idea	Hot keys **specified explicitly** (no sampling)
Hive skewed join optimization	Hot keys declared in **table metadata**, stored in **separate files**, joined with a **map-side join**	Explicit declaration
Two-stage aggregation (grouping)	Stage 1: records go to **random reducers** that emit compact partial aggregates; stage 2 combines them per key	Extra job

The random-reducer trick mirrors hot-spot relief in partitioned databases (Chapter 6, "Skewed Workloads and Relieving Hot Spots").

Map-Side Joins

	Reduce-side join	Map-side join
Input assumptions	**None**	Known properties (size, partitioning, sorting)
Cost	Sorting, copying and merging are **expensive**; data may hit disk several times	Faster: no reducers, no sorting
Structure	Mappers + shuffle + reducers	Each mapper reads one input block, writes one output file

Broadcast hash joins

When: a **large** input joined with one **small enough to fit in memory**. Each mapper (one per block of the large input) loads the **entire** small input from HDFS into an **in-memory hash table**, then scans its block and probes per record. **Broadcast:** the small input goes to every partition of the large one. Supported by **Pig** ("replicated join"), **Hive** ("MapJoin"), **Cascading**, **Crunch**, and **Impala**.

> **Footnote.** One entry per key is assumed (one user per ID); in general the table may hold several and the join outputs every match.

Variant: store the small input as a **read-only index on local disk**; hot parts stay in the OS **page cache**, nearly as fast as memory without needing to fit in RAM.

Partitioned hash joins

When: both inputs are **partitioned the same way** — e.g. by the **last decimal digit of user ID** (10 partitions each); **mapper 3** loads users ending in 3 into a hash table and scans events for those users. Each mapper reads one partition of each input, with a smaller hash table. Requires the **same number of partitions**, **same key** and **same hash function** — plausible if earlier jobs grouped them so. Hive calls these **bucketed map joins**.

Map-side merge joins

When: inputs are partitioned **and sorted by the same key** identically. Neither needs to fit in memory: the mapper merges both files incrementally in key order, like a reducer. The join could have been done in the earlier job's reduce stage, but a separate map-only join makes sense if other jobs also want those datasets.

MapReduce workflows with map-side joins

- **Reduce-side** join output is **partitioned and sorted by join key**; **map-side** output follows the **large input's** layout.
- Map-side joins require knowing datasets' **physical layout** — encoding, directory, **number of partitions**, and partitioning/sorting **keys** — often kept in **HCatalog** and the **Hive metastore**.

The Output of Batch Workflows

OLTP queries look up a few records by key via indexes; **analytic** queries scan, group and aggregate into **reports** (Chapter 3). Batch is closer to analytics but often produces **some other kind of structure rather than a report**.

Building search indexes

- Google's **original** MapReduce use was building search indexes — **5 to 10 chained jobs**. Google later moved on, but **Hadoop MapReduce remains a good way to build indexes for Lucene/Solr** (Lucene: open-source search library; Solr: server built on it).
- A **Lucene** full-text index (Chapter 3) maps each keyword (**term dictionary**) to the document IDs containing it (**postings list**), plus ranking, spelling and synonym data.
- Batch build: mappers **partition documents**, each reducer **indexes its partition**, files go to HDFS — **document-partitioned** indexes (Chapter 6) parallelize well. Index files are **immutable**.

Option on document changes	How	Pros	Cons
Periodic full rebuild	Rerun everything, swap index files wholesale	Simple: documents in, indexes out	Expensive for few changes
Incremental	Lucene writes new **segment files** and merges/compacts them in the background (Chapter 3)	Efficient for small changes	More complex (Chapter 11)

Key-value stores as batch process output

Machine learning systems — **classifiers** (spam filtering, anomaly detection, image recognition) and **recommendation systems** (people you may know, related products) — usually output a **database** queried by user or product ID, served by a web app **separate** from the Hadoop cluster.

Bad: write records one by one from mappers/reducers into the production DB (if firewalls allow):

1. A network request **per record** is orders of magnitude slower than batch throughput, even with client-side batching.
2. Many parallel tasks can **overwhelm the database**, hurting serving.
3. It breaks MapReduce's **all-or-nothing** guarantee (a successful job's output equals each task running once; a failed job produces none): external writes are **visible side effects**, raising partial-result issues with Hadoop **task attempts** and **speculative execution** (running a duplicate of a slow task and keeping whichever finishes first).

Good: build a fresh database as files in the job's HDFS output, then **bulk-load** them into read-only servers.

- Stores supporting this: **Voldemort**, **Terrapin**, **ElephantDB**, **HBase bulk loading**.
- Fits MapReduce: extracting and sorting by key is much of building an index.
- Read-only stores are simple — e.g. **no WAL** (write-ahead log) needed.
- **Voldemort loading:** keep serving **old** files; download new files from HDFS; **atomically switch** to them (no request sees a half-switched state); **switch back** if anything goes wrong, since old files are immutable and still there.

Philosophy of batch process outputs

Like Unix (read input, write output, leave input alone, fully replace output, no other side effects), MapReduce's immutability and side-effect-free outputs give performance **and** maintainability:

- **Easy bug recovery:** roll back code and rerun, or switch back to the old output directory. With read-write databases, rolling back code **doesn't undo** bad writes. This is **human fault tolerance**.
- **Faster development: minimizing irreversibility** aids Agile.
- **Safe automatic retries:** a crashing bug fails the job after a few tries; transient faults are tolerated — safe only because inputs are immutable and failed outputs discarded.
- **Input reuse,** including **monitoring jobs** that compute metrics and compare output with previous runs.
- **Separation of logic and wiring:** one team writes a job; others decide where and when it runs.

Unlike Unix's untyped text and parsing (`{print $7}`), Hadoop often uses structured formats: **Avro** (Chapter 4; e.g. object container files) and **Parquet** (columnar, Chapter 3) — efficient schema-based encoding with schema evolution.

Comparing Hadoop to Distributed Databases

Hadoop is a distributed Unix: **HDFS** is the filesystem and **MapReduce** an oddly specific process that always runs `sort` between map and reduce. Its algorithms were **not new**: **massively parallel processing (MPP) databases** — **Teradata**, **Tandem NonStop SQL**, the **Gamma database machine** — had them over ten years earlier. **Biggest difference:** MPP databases run **analytic SQL in parallel**; MapReduce + a distributed filesystem is more like a **general-purpose operating system** running **arbitrary programs**.

Diversity of storage

- Databases impose a data model; HDFS files are **just bytes** — DB records, text, images, video, genome sequences, sensor readings, feature vectors, sparse matrices.
- Hadoop let you **dump data first and figure out processing later**; MPP databases need careful up-front modeling into proprietary formats. Purists prefer modeling, but **making data available quickly** often matters more.
- Like a **data warehouse**, collecting data enables joins across previously separate datasets; collecting raw data and designing schemas later is a **data lake** or **enterprise data hub**.
- Interpretation shifts to the **consumer** (**schema-on-read**, Chapter 2) — useful when producer and consumer teams differ; there may be no single ideal model. This is the **sushi principle: "raw data is better."**
- Hence Hadoop for **ETL**: dump raw OLTP data into HDFS, clean/transform with MapReduce into relational form, load into an MPP warehouse — modeling **decoupled from collection**.

Diversity of processing models

	MPP database	Hadoop
Architecture	**Monolithic**: storage, planning, scheduling, execution tuned together	Open platform: many models over shared files
Strength	Excellent on designed-for queries; SQL usable from **BI** (business intelligence) tools like **Tableau** without code	**Arbitrary code** over large datasets
Weakness	Not everything fits SQL	Raw MapReduce limited/slow for some tasks

- ML, recommendations, full-text search with ranking, image analysis — feature engineering, language models, fraud risk — need **code, not just queries**.
- You can build SQL on HDFS + MapReduce (**Hive** did) plus jobs SQL can't express; when MapReduce proved too slow/limiting, Hadoop's openness allowed new models ("Beyond MapReduce").
- All run on **one shared cluster** over the **same files** — no importing into several specialized systems.
- The ecosystem includes **HBase** (LSM-based random-access OLTP) and **Impala** (MPP-style analytics): **neither uses MapReduce, both store data in HDFS**.

Designing for frequent faults

	MPP databases	MapReduce
Node crash mid-query	**Abort the whole query** (fine for seconds-to-minutes queries)	**Retry only the failed task**
Memory vs disk	Keep data **in memory** (e.g. hash joins)	**Eager to write to disk** — for fault tolerance and because data won't fit anyway
Best for	Short queries	Huge, long jobs likely to hit a task failure

Batch jobs are less fault-sensitive (no immediate user impact; rerunnable). Task-level recovery slows the fault-free case, and machine failures are rare — so why?

Google's environment:

1. **Mixed-use datacenters**: online services and batch share machines.
2. Each task gets a fixed **resource share** (cores, memory, disk) enforced by **containers**.
3. Tasks have **priorities**; lower-priority tasks are **preempted** when higher ones need resources.
4. Priority sets **price** — higher costs more.
5. Low-priority resources can be **overcommitted**, improving utilization versus segregating production and batch.
6. MapReduce runs at low priority — batch jobs "pick up the scraps under the table".

At Google, a **one-hour** task has about a **5% risk** of preemption — **more than 10x** the rate from hardware failures, reboots etc. A job of **100 tasks of 10 minutes** has a **greater than 50% risk** that at least one is killed. So MapReduce tolerates frequent task kills **not because hardware is unreliable** but because killing processes enables better utilization.

Open source: **YARN's CapacityScheduler** preempts to balance queues, but **general priority preemption** was missing from **YARN, Mesos and Kubernetes** at the time; where tasks are rarely killed, MapReduce's choices make less sense.

Beyond MapReduce

- MapReduce is one programming model among several. The book dwells on it because it is a **clear, simple abstraction** — simple to *understand*, not to *use* (e.g. you implement joins yourself with raw APIs).
- **Pig, Hive, Cascading, Crunch** make common tasks easier and are easy to learn if you know MapReduce.
- Some problems lie in the **execution model** itself and show as poor performance. MapReduce is **very robust** (finishes almost any job on unreliable, **multi-tenant** clusters with frequent kills, just slowly); other tools can be **orders of magnitude faster**. Chapter 11 treats streaming as another speedup.

Materialization of Intermediate State

- Jobs are **independent**, linked via input/output directories and an external scheduler. That's good when output is a dataset **published widely** (others reuse it by name — **loose coupling**).
- Often, though, output feeds **one other job** by the **same team** — just **intermediate state** (plenty of it in 50–100-job workflows).
- Writing it out is **materialization** (eagerly computing and storing a result, as in materialized views). Unix pipes **stream** instead through a small buffer.

Downsides of full materialization:

1. **Waiting for stragglers:** a job starts only when **all** tasks of its predecessors finish; skew and load create **straggler** tasks that slow the workflow.
2. **Redundant mappers:** mappers often just re-read a reducer's output to repartition/sort; reducers could be chained directly.
3. **Over-replication:** intermediate files are **replicated across nodes** — overkill for temporary data.

Dataflow engines

Spark, **Tez** and **Flink** handle **the entire workflow as one job** — **dataflow engines**. Like MapReduce they call a user function **one record at a time, single-threaded**, parallelize by **partitioning**, and ship output over the network. Unlike it, functions are general **operators** connected by:

1. **Repartition and sort by key** (like the shuffle) — for sort-merge joins and grouping.
2. **Partition identically without sorting** — for partitioned hash joins (hashing scrambles order anyway).
3. **Broadcast** the same output to all partitions — for broadcast hash joins.

Based on research systems **Dryad** and **Nephele**. Advantages:

- **Sort only where needed.**
- **No unnecessary map tasks** — mapper work folds into the previous operator (it doesn't change partitioning).
- **Locality:** declared dependencies let the scheduler co-locate producer and consumer, exchanging data via a **shared memory buffer**.
- **Intermediate state in memory or local disk**, far less I/O than replicated HDFS (generalizing what MapReduce did for mapper output).
- **Operators start as soon as input is ready.**
- **Reuse JVM processes** instead of a **new JVM per task** (slow startup).

Same computations, usually **significantly faster**. **Pig, Hive or Cascading workflows can switch from MapReduce to Tez or Spark by configuration**, without code changes. **Tez** is a **thin library** using the **YARN shuffle service** to move data; **Spark and Flink** are **big frameworks** with their own network layer, scheduler and APIs.

Fault tolerance

- MapReduce: durable materialized data → failed task restarts elsewhere on the same input.
- Spark, Flink, Tez **keep intermediate state out of HDFS**, so lost state is **recomputed** from an earlier stage or the original input. The framework must know each piece's **recipe** (source partitions and operators): **Spark's resilient distributed dataset (RDD)** tracks **ancestry**; **Flink checkpoints operator state**.

Determinism matters: if recomputed data differs from what downstream already consumed, downstream operators must be **killed and rerun** too. Make operators **deterministic**; watch for **unordered hash-table iteration**, **random numbers** (use pseudorandom numbers with a **fixed seed**), the **system clock** and **external data sources**.

Recomputation isn't always best: if an intermediate result is **much smaller** than its source or **CPU-expensive**, materializing is cheaper.

Discussion of materialization

- MapReduce ≈ writing each command's output to a temp file; dataflow engines ≈ **Unix pipes** — **Flink** especially uses **pipelined execution**.
- **Sorting** must consume all input before emitting (the last record may have the smallest key), so sort operators accumulate state; most other stages pipeline.
- Inputs and final outputs still usually live on HDFS (immutable inputs, fully replaced outputs) — the gain is not materializing **intermediate** results.

	MapReduce workflow	Dataflow engine (Spark/Tez/Flink)
Unit of scheduling	Independent jobs chained by directories	Whole workflow as one job (DAG of operators)
Intermediate state	HDFS, replicated	Memory or local disk
Sorting	Always between map and reduce	Only where needed
Next stage starts	After all preceding tasks finish	As soon as input is ready
Fault recovery	Rerun failed task on durable input	Recompute from lineage (RDD) or checkpoints (Flink)
Process startup	New JVM per task	Reuse JVMs

Graphs and Iterative Processing

Chapter 2 covered graph **OLTP** queries; here, **batch** analysis over **entire graphs** for ML (recommendations) and ranking — famously **PageRank**, estimating a page's popularity from incoming links for search ranking.

> **Naming confusion warning.** Dataflow engines arrange operators as a **directed acyclic graph (DAG)** — that is *not* graph processing: there the *flow* is a graph while records are relational-style tuples; in graph processing the *data* is the graph.

Many graph algorithms follow **one edge at a time**, passing information between neighbors, **until done** (no more edges, or a metric **converges**) — e.g. Figure 2-6's places within North America is a **transitive closure**. MapReduce does a **single pass**, so iteration needs an **external scheduler**: run one step; check the **completion condition**; repeat. Very **inefficient**: each iteration reads the **whole input** and writes a **whole new output** even if little changed.

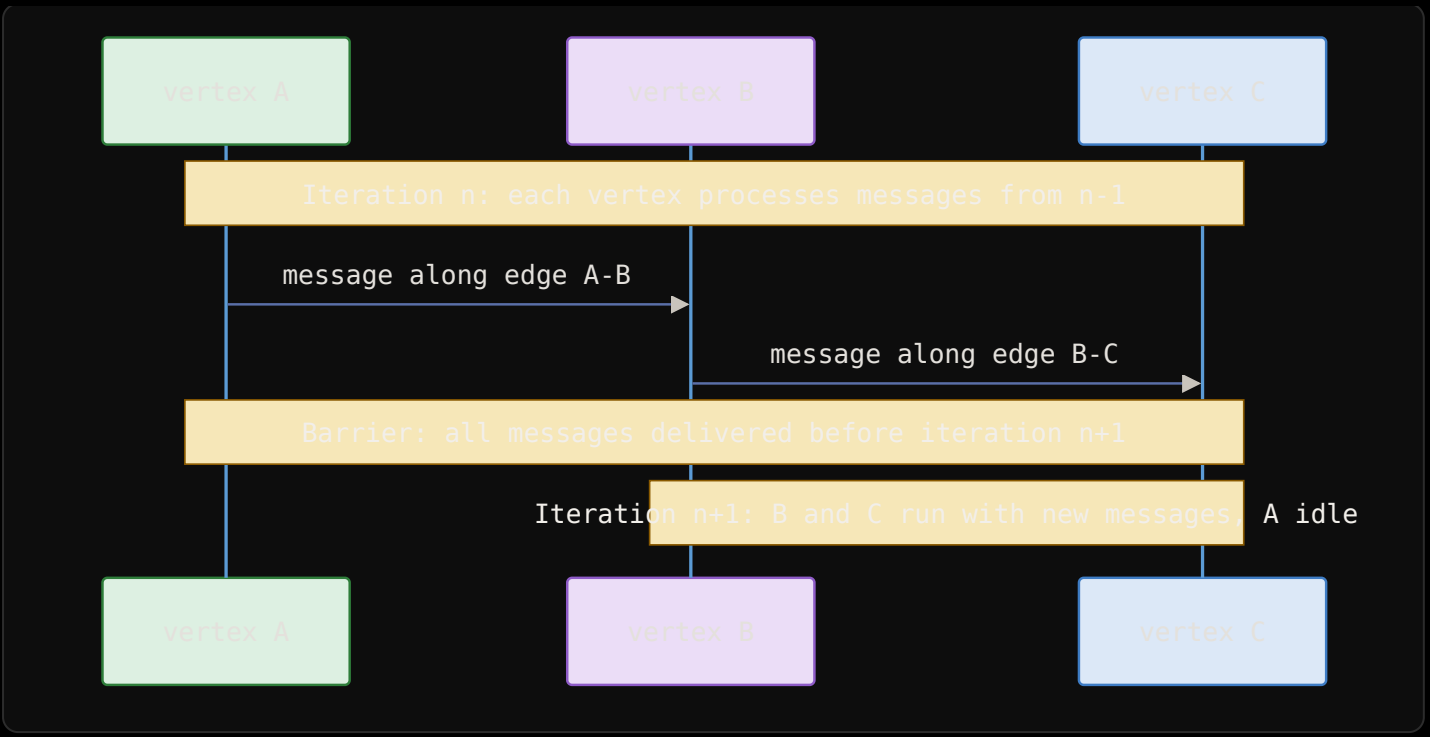
The Pregel processing model

The **bulk synchronous parallel (BSP)** model — implemented by **Apache Giraph**, Spark's **GraphX**, Flink's **Gelly**; also the **Pregel model** after Google's paper. As mappers "send messages" to reducers, in Pregel **a vertex sends messages to other vertices**, usually along edges:

1. Computation runs in **iterations**.
2. Each iteration calls a function **once per vertex** with its incoming messages (like a reducer call).
3. A vertex **keeps its state in memory** between iterations, so only **new** messages are processed.
4. Quiet regions of the graph need **no work**.
5. Between rounds, the framework delivers all messages from the previous round.

Like the **actor model** (Chapter 4: actors with private state communicating by messages), but vertex state and messages are **durable and fault-tolerant**, and communication is in **fixed rounds**.

Illustrative sketch (no numbered figure in the book): one Pregel superstep.



Own vertex-centric pseudocode (shortest distance from a source):

```
def compute(vertex, messages):                # called once per vertex per iteration
    best = min(messages, default=vertex.value)
    if best < vertex.value or vertex.first_round:
        vertex.value = best                   # state is kept in memory across rounds
        for edge in vertex.out_edges:
            send(edge.target, best + edge.weight)   # message delivered next round
    vertex.vote_to_halt()                     # inactive until a new message arrives
```

Each call sees one vertex and its messages; `vertex.value` persists between rounds; `send` addresses a vertex ID.

Fault tolerance (Pregel)

- Vertices communicate **only by messages** (no direct queries), so messages can be **batched** and waiting is reduced — the only wait is **between iterations**.
- Despite lossy, duplicating, delaying networks (Chapter 8), each message is processed **exactly once** by its target in the next iteration; recovery is invisible to the algorithm.
- Mechanism: **checkpoint all vertex state at the end of an iteration**; on node loss, **roll the whole computation back** to the latest checkpoint. With **deterministic** algorithms and **logged messages**, only **the lost partition** need be recovered.

Parallel execution

- Vertices message a **vertex ID**; the framework partitions the graph and routes messages. Thanks to **"thinking like a vertex"**, any partitioning works — ideally co-locating chatty vertices, but that is hard, so graphs are often split by **arbitrarily assigned vertex ID**.
- Result: heavy **cross-machine communication**; messages can be **bigger than the graph**, slowing distributed graph algorithms.

Graph size	Best approach
Fits in one machine's memory	A **single-machine** (even single-threaded) algorithm likely beats a distributed job
Exceeds memory but fits on one machine's disks	Single-machine frameworks like **GraphChi**
Too big for one machine	Distributed **Pregel**-style is unavoidable; efficient parallel graph algorithms are ongoing research

High-Level APIs and Languages

- Infrastructure now handles **many petabytes** on **over 10,000 machines**; with scale more or less solved, focus moved to a better **programming model**, more **efficient processing**, and a **wider range of problems**.
- **Hive, Pig, Cascading, Crunch** spared hand-written MapReduce and moved to **Tez without rewriting job code**; **Spark and Flink** have their own high-level dataflow APIs, often inspired by **FlumeJava**.
- They use **relational-style building blocks** (join on a field, group by key, filter, aggregate) implemented with this chapter's algorithms. Benefits: **less code**, **interactive use** in a shell (Unix-like exploration), and better **machine-level efficiency**.

Own PySpark-style example:

```
views = spark.read.parquet("hdfs:///logs/views")            # large input
users = spark.read.parquet("hdfs:///snapshots/users")       # small input
by_age = (views.join(users, "uid")                          # declarative join: engine picks algorithm
               .groupBy("url", "age_band")                  # grouping by key
               .count())                                    # aggregation
by_age.write.parquet("hdfs:///derived/views_by_age")        # final output, materialized
```

The optimizer may pick a broadcast hash join if `users` is small; columnar Parquet reads only needed columns.

The move toward declarative query languages

- Joins as **relational operators** let the framework **choose the join algorithm**; **Hive, Spark and Flink** have **cost-based query optimizers** that can also **reorder joins** to minimize intermediate state. This is the **declarative** idea from Chapter 2 ("Query Languages for Data").
- Unlike fully declarative SQL, these systems use **function callbacks** running **arbitrary code**, giving access to huge library ecosystems (parsing, NLP, image analysis, numerics) — long a distinction from **MPP databases**, whose **user-defined functions** are cumbersome and poorly integrated with package managers (**Maven**, **npm**, **Rubygems**).
- Declarative features beyond joins also pay off: a per-record callback for simple filtering/projection costs **CPU overhead**, whereas declarative filters let the optimizer use **column-oriented storage** and read only needed columns. **Hive, Spark DataFrames and Impala** use **vectorized execution**; **Spark generates JVM bytecode**, **Impala uses LLVM** to emit native code.
- Net: batch frameworks **look more like MPP databases** in performance while keeping the **flexibility** of arbitrary code and formats.

Specialization for different domains

Common patterns deserve reusable building blocks; MPP databases traditionally serve **business intelligence** and reporting, just one domain:

- **Statistical/numerical ML algorithms** (classification, recommendation): **Mahout** (on MapReduce, Spark, Flink) and **MADlib** (inside **Apache HAWQ**, a relational MPP database).
- **Spatial algorithms** like **k-nearest neighbors** (similarity search).
- **Approximate search** for **genome analysis** (similar but not identical strings).

Batch systems gain declarative operators while MPP databases become more programmable — the two **converge**; both are just systems for storing and processing data.

Chapter summary

- **Unix roots:** `awk`, `grep`, `sort` embody **immutable inputs**, outputs for an **unknown next program**, and **composing small tools**.
- **Uniform interface:** Unix files and pipes; MapReduce the **distributed filesystem**; dataflow engines add pipe-like transport, but initial input and final output usually stay on HDFS.
- **Two core problems:**
  - **Partitioning:** mappers per input block; output **repartitioned, sorted and merged** into the configured reducer partitions to bring equal keys together. Dataflow engines sort only when needed.
  - **Fault tolerance:** MapReduce writes to disk often (easy task recovery, slower fault-free runs); dataflow engines keep more in memory and recompute more; **deterministic operators** limit recomputation.
- **Joins: sort-merge** (partition, sort, merge into one reducer call); **broadcast hash** (small input in each mapper's hash table); **partitioned hash** (identical partitioning, per-partition hash join).
- **Restricted programming model hides distributed-systems problems:** callbacks are **stateless** with **no external side effects**, so tasks retry safely, failed outputs are discarded, and only one successful attempt per partition is visible — output equals a fault-free run, far stronger than online services updating databases as side effects.
- **Defining feature:** batch reads **bounded** input (fixed size, e.g. logs at a point in time or a DB snapshot) and writes **derived** output without modifying input, so it knows when it's done and **finishes**.
- **Next:** stream processing — **unbounded** input, so a job never completes.

Interview angle

1. Explain MapReduce in 30 seconds. A batch framework over a distributed filesystem. A *mapper* turns each record into key-value pairs; a *reducer* processes all values of one key. The framework runs mappers near the data (one per block), partitions output by key hash, sorts, ships partitions to reducers (the shuffle), merges and calls the reducer per key; output goes to HDFS. Inputs are never modified, so failed tasks retry safely.

- *Follow-up: Why always a sort?* Equal keys become adjacent so reducers need little memory; external merge sort with sequential I/O scales past RAM.
- *Follow-up: What is the shuffle?* Partitioning mapper output by reducer, sorting on mapper local disk, reducers fetching and merging their partition from every mapper. No randomness.

2. How do you join two large datasets in a batch job? Default: **reduce-side sort-merge join** — map both to (join key, record), shuffle, optionally secondary-sort, join in the reducer. Small side: **broadcast hash join**. Identically partitioned: **partitioned hash join**; also sorted: **map-side merge join**. Never query a remote DB per record (slow, overloads it, nondeterministic).

- *Follow-up: Hot key (celebrity)?* Detect hot keys (Pig samples; Crunch/Hive need declaration), spread them over random reducers and replicate the other side; for aggregation, two-stage aggregation.
- *Follow-up: Does join choice affect downstream jobs?* Yes: reduce-side output is partitioned/sorted by join key; map-side follows the large input's layout.

3. How should a batch job publish results to a serving database? Not record-by-record into production (slow, overloads it, breaks all-or-nothing with visible partial results and duplicate attempts). Build immutable DB files in the job, write to HDFS, bulk-load into a read-only store (Voldemort, Terrapin, ElephantDB, HBase bulk load). Voldemort serves old files while copying, atomically switches, can switch back.

- *Follow-up: Operational benefit?* Human fault tolerance: roll back code and rerun, or point back to the previous output.

4. MapReduce vs Spark/Flink (dataflow engines)? MapReduce materializes every intermediate result to replicated HDFS, always sorts, has redundant map stages, waits for whole jobs, starts a JVM per task.

Dataflow engines run a DAG of operators, sort only when needed, keep intermediates in memory/local disk, pipeline, reuse JVMs and optimize locality; they recover by recomputing from lineage (RDD) or checkpoints (Flink).

- *Follow-up: Why does determinism matter?* If recomputed data differs from what downstream consumed, downstream must be rerun too. Avoid unordered hash iteration, unseeded randomness, clocks, external lookups.
- *Follow-up: When is recomputation worse?* When the intermediate result is tiny relative to its source or CPU-expensive.

5. Why was MapReduce so eager to write to disk and retry per task? At Google batch runs at low priority and is preempted: an hour-long task has ~5% kill risk, over 10x hardware failure rates; 100 ten-minute tasks have >50% chance one is killed. Task-level retry avoids rerunning jobs. YARN, Mesos and Kubernetes mostly lacked general priority preemption then, so the trade-off fits less there.

6. Hadoop vs MPP databases? MPP: monolithic, upfront schemas, excellent analytic SQL, aborts whole query on failure, in-memory. Hadoop: general-purpose OS-like — any format (raw dumps, schema-on-read, data lake, sushi principle) and any model (MapReduce, Hive SQL, Spark, HBase, Impala) on one cluster; retries tasks, spills to disk. They are converging.

Self-check

1. **System of record vs derived data — can one product be either?** System of record: authoritative, normalized, written first. Derived: computed, rebuildable. Yes — it depends on how the application uses the tool.
2. **Why can `sort | uniq -c` handle more distinct URLs than a hash table?** `sort` spills sorted chunks to disk and merges them (sequential I/O); a hash table needs every distinct key in RAM.
3. **Three things that make Unix tools composable and easy to experiment with?** A uniform interface (byte-sequence files, newline-separated text), separation of logic and wiring (stdin/stdout), and transparency (immutable inputs, inspect any stage, save stage output).
4. **What decides the number of map and reduce tasks?** Map: number of input blocks. Reduce: the job author. A key's reducer is chosen by hashing the key.
5. **Scenario: join 10 TB of clicks with a 200 MB product table. Which join?** Broadcast hash join — the small table fits in memory in each mapper; no shuffle or sort.
6. **Three reasons not to write from reducers into a production DB?** Per-record calls are orders of magnitude slower; parallel tasks overwhelm the DB; external side effects break all-or-nothing output (partial results, retried and speculative attempts).
7. **What is "human fault tolerance" and how do immutable outputs provide it?** Recovering from buggy code: inputs unchanged and outputs fully replaced, so roll back and rerun or switch to the previous output. A read-write DB keeps the bad writes.
8. **Three drawbacks of materializing intermediate state, and the dataflow-engine fixes?** Waiting for stragglers, redundant mappers, replicating temporary files. Dataflow engines run one job, fold map work into previous operators, keep intermediates in memory/local disk, start operators when input is ready.
9. **Why is iterative graph processing slow in MapReduce; how does Pregel help?** Each iteration rereads and rewrites the whole graph. Pregel keeps vertex state in memory and processes only vertices with new messages, checkpointing at iteration boundaries.
10. **When might a single machine beat a distributed graph framework?** When the graph fits in memory (or on disk, with GraphChi): arbitrary vertex-ID partitioning causes heavy cross-machine messaging that can exceed the graph's size.
11. **What does "bounded input" mean and why is it batch's defining property?** Known, fixed-size input, so the job knows when it has read everything and finishes. Stream processing drops this.

Chapter 11 — Stream Processing

Book: Chapter 11, pp. 439–487

Overview

Question: what if the input never ends? How do you move, store and process continuously arriving data (clicks, sensor readings, database writes) with low delay while keeping batch processing's good properties (derived outputs, rerunnability, fault tolerance)?

Where it fits: Chapter 10's batch jobs read files and write **derived data** (rebuildable by rerunning). Batch assumes **bounded** input — sorting, the heart of MapReduce, can't emit anything until it has seen all input, since the last record might have the smallest key. Real data is **unbounded**: users keep producing it. Batch fakes completeness with time slices (a day's data each day); a daily job shows changes a day late, too slow for impatient users. Shrink the slices, or drop them and **process each event as it happens** — **stream processing**.

A "stream" is data made available **incrementally over time** — as in Unix stdin/stdout, lazy lists, file APIs like Java's `FileInputStream`, TCP connections, and internet audio/video. This chapter treats **event streams as a way of managing data**:

1. Representing, storing and transmitting streams ("Transmitting Event Streams").
2. How streams relate to databases ("Databases and Streams").
3. Processing streams continuously ("Processing Streams").

Why it matters: fraud detection, monitoring, live dashboards, syncing caches and search indexes with a database, real-time analytics. Log-based messaging and change data capture underpin modern data integration (Kafka — a widely used open-source event-stream platform — and friends).

> **Analogy:** batch is yesterday's newspaper (finished bundle); stream processing is a live news ticker with no last page.

Key terms

Term	Plain-English meaning
Bounded / unbounded data	Finite, known-size input vs input that keeps arriving forever
Stream	Data made available bit by bit over time
Event	Small, self-contained, immutable record of something that happened, usually timestamped
Producer (publisher, sender) / consumer (subscriber, recipient)	Creates events / processes events
Topic / stream	Named group of related events (like a filename for records)
Polling	Consumer repeatedly asking "anything new?"
Trigger	Database feature reacting to a change (e.g. row insert); limited
Messaging system	Pushes messages containing events from producers to consumers
Publish/subscribe	Many producers publish to a topic; many consumers subscribe
Backpressure (flow control)	Blocking the producer when the consumer can't keep up
UDP multicast	Unreliable one-to-many sending; low-latency feeds
Brokerless messaging	Libraries (ZeroMQ, nanomsg) doing pub/sub directly between nodes
Webhook	Callback URL a service calls when an event occurs
Message broker (message queue)	Server — a database specialized for message streams — between producers and consumers
JMS / AMQP	Standards for traditional message brokers
Load balancing vs fan-out	Each message to one consumer (share work) vs to all consumers
Acknowledgment / redelivery	Consumer confirming completion so the broker deletes the message / resending an unacked message, possibly to another consumer
Log / partitioned log	Append-only records on disk / topic split into independent logs on possibly different machines
Offset	Monotonically increasing message number within a partition
Log-based message broker	Broker storing messages in a durable partitioned log (Kafka, Kinesis Streams, DistributedLog)
Consumer group	Consumer nodes sharing a topic's partitions
Head-of-line blocking	One slow message delays everything behind it
Consumer offset	Position up to which a consumer has processed a partition
Log segment	Chunk of a log; old segments are deleted or archived
Circular / ring buffer	Fixed-size buffer discarding oldest data when full
Replay	Re-reading old messages from an earlier offset
Dual writes	Application writing each change separately to every system
Race condition	Outcome depends on unlucky timing of concurrent operations
Change data capture (CDC)	Extracting every change written to a database as a stream
Initial snapshot	Consistent full database copy matched to a known log offset

Term	Plain-English meaning
Log compaction / tombstone	Keeping only the latest value per key / null value marking a key deleted
Change stream / change feed	A database's first-class API for subscribing to its changes
Event sourcing	Storing all application changes as immutable application-level events
Domain-driven design (DDD)	Design community where event sourcing originated
Command	User request that may still fail validation
Changelog / ledger	Log of all changes / accountants' append-only transaction log
Compensating transaction	New entry correcting an earlier mistaken one instead of erasing it
CQRS	Command query responsibility segregation: separate write form from read forms
Excision / shunning	Datomic's / Fossil's features for truly deleting data from immutable history
Operator / job	Code that processes streams
Complex event processing (CEP)	Searching streams for event patterns with stored queries
Stream analytics	Aggregations and statistics over streams
Window	Time interval over which an aggregation is computed
Probabilistic algorithm	Approximate answers with far less memory (Bloom filter, HyperLogLog)
Materialized view	Derived, query-optimized view kept up to date as data changes
Percolator	Elasticsearch feature matching documents against stored queries
Distributed RPC	Storm feature interleaving user queries with stream events
Event time / processing time	When the event happened (its timestamp) / when the processor handles it (its clock)
Straggler event	Arrives after its window was declared complete
Tumbling / hopping / sliding / session window	Window types (defined below)
Stream-stream join (window join)	Joining related events from two streams within a time window
Stream-table join (enrichment)	Adding database info to each event via a local CDC-updated copy
Table-table join	Joining two changelog streams to maintain a materialized join view
Slowly changing dimension (SCD)	Joining with a value that changes over time (warehouse term)
Exactly-once (effectively-once)	Output looks as if each input was processed exactly once despite retries
Microbatching	Splitting a stream into ~1-second batches processed like batch jobs
Checkpointing	Periodically saving operator state durably
Idempotent	Doing it many times = doing it once
Fencing	Stopping a node wrongly believed dead from interfering

Transmitting Event Streams

Batch inputs/outputs are **files**. The streaming equivalents:

- **Events instead of records:** a small, self-contained, **immutable** object describing something at a moment, usually with a **timestamp** from a time-of-day clock (Chapter 8). Sources: user actions (page view, purchase) or machines (temperature sensor, CPU metric); each line of Chapter 10's web log is an event.
- **Encoding:** text, JSON, or binary (Chapter 4), so events can be stored (file, relational table, document DB) or sent over the network.
- **Producers and consumers:** an event is created once by a **producer** (**publisher**, **sender**) and processed by possibly many **consumers** (**subscribers**, **recipients**). Related events form a **topic** or **stream** (like a filename groups records).

Why not just a database or file? Producers write events; consumers **poll** for new ones — effectively a daily batch job. For low delay, frequent polling gets expensive: the more often you poll, the fewer requests find anything new. Better to **notify** consumers. Databases have been weak at this: relational **triggers** react to changes (e.g. an inserted row) but can do little and are an afterthought. Hence specialized tools.

Messaging Systems

A **messaging system pushes** messages carrying events from producers to consumers (cf. Chapter 4, "Message-Passing Dataflow"). A Unix pipe or TCP connection links **one sender to one receiver**; messaging systems let **many producers** and **many consumers** share a topic — **publish/subscribe**. Two questions distinguish them:

1. Producers faster than consumers?

Option	What happens	Example / caveat
Drop	Excess discarded	Only if loss is tolerable
Buffer	Messages queue	As the queue grows, does it crash when memory runs out or spill to disk, and how does disk hurt performance?
Backpressure (flow control)	Producer blocked	Unix pipes and TCP: small fixed buffer; sender blocks until it drains

2. Nodes crash or go offline — are messages lost? Durability needs disk writes and/or replication, at a cost; tolerating some loss buys more **throughput** and lower **latency** on the same hardware (System Design Fundamentals). It depends on the app: occasional missing **sensor readings and metrics** may be fine (a fresh value comes soon) — but **beware** that heavy dropping may silently make metrics wrong; **counting events** needs reliability, since every loss skews counts.

Batch systems guarantee output equal to a failure-free run (retry tasks, discard partial output); similar guarantees for streams come later.

Direct messaging from producers to consumers

- **UDP multicast** is popular in **finance** for low-latency **stock market feeds**. UDP doesn't guarantee delivery, but application-level protocols can recover lost packets if the producer remembers and retransmits on request.
- **Brokerless libraries ZeroMQ** and **nanomsg** do pub/sub over TCP or IP multicast.
- **StatsD** and **Brubeck** collect metrics over unreliable **UDP**; StatsD **counters** are correct only if every message arrives, so they're approximate at best (Chapter 8, "TCP Versus UDP").
- If the consumer exposes a service, producers can push via **HTTP or RPC** (remote procedure call) — the idea behind **webhooks**: register a callback URL that the other service calls on each event.

Limits: application code must cope with **message loss**; fault tolerance is limited — protocols usually assume producers and consumers are **always online**, so an offline consumer misses messages, and producer retries fail if the **producer crashes** and loses its retry buffer.

Message brokers

A **message broker** (**message queue**) is a **database optimized for message streams**: a server that producers write to and consumers read from.

- **Centralizing data** handles clients connecting, disconnecting and crashing; **durability becomes the broker's job**. Some keep messages **in memory**, others persist **to disk**.
- Slow consumers usually get **unbounded queueing** (configurable).
- Consumers are **asynchronous**: producers wait only for the broker to **buffer** the message; delivery is usually well under a second, but can be much later with a backlog.

Message brokers compared to databases

Some brokers can join **two-phase commit** via **XA** or **JTA** (Chapter 9), resembling databases, but:

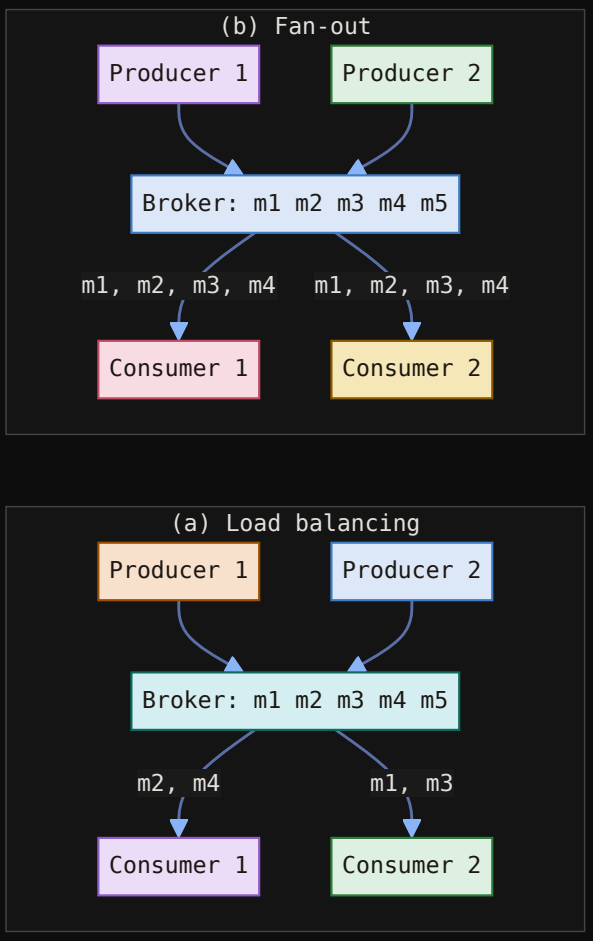
Aspect	Database	Traditional message broker
Retention	Until explicitly deleted	Most **delete a message once delivered** — not for long-term storage
Working set	Can be large	Assumes **small** (short queues); buffering many messages (spilling to disk) **degrades throughput**
Selecting data	**Secondary indexes** and search	**Subscribing to topics matching a pattern** — same purpose, different mechanism
Change notification	Results are a **point-in-time snapshot**; you don't learn of later changes unless you re-query	**No arbitrary queries**, but clients **are notified** of new messages

This model is standardized in **JMS** and **AMQP** and implemented by **RabbitMQ**, **ActiveMQ**, **HornetQ**, **Qpid**, **TIBCO Enterprise Message Service**, **IBM MQ**, **Azure Service Bus** and **Google Cloud Pub/Sub**.

Multiple consumers

- **Load balancing:** each message goes to **one** consumer — for **expensive** messages and adding parallel consumers. (**AMQP**: several clients on one queue; **JMS**: **shared subscription**.)
- **Fan-out:** each message goes to **all** consumers, independently — like several batch jobs reading one file. (**JMS**: **topic subscriptions**; **AMQP**: **exchange bindings**.)

Figure 11-1 (redrawn): producers 1 and 2 publish m1 to m5 into the broker. (a) Load balancing: consumer 1 processes m2 and m4, consumer 2 processes m1 and m3, so each message goes to one consumer. (b) Fan-out: consumer 1 and consumer 2 each process m1, m2, m3, m4 (m5 has not been delivered yet when the figure ends).



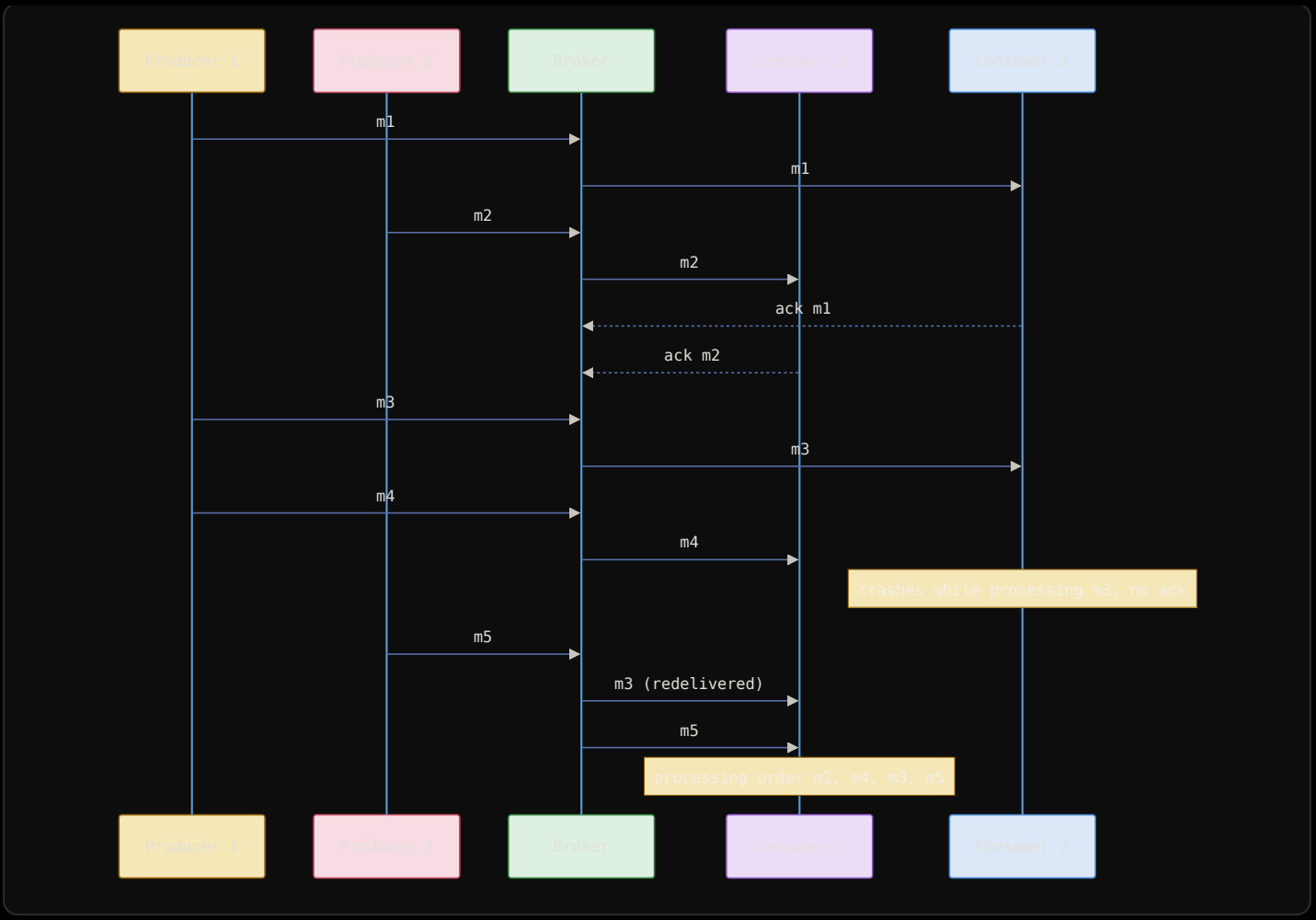
They **combine**: two consumer groups each get every message, but within a group only one node gets each message.

Acknowledgments and redelivery

Consumers may crash mid-message, so they must **explicitly acknowledge** completion; the broker then removes the message. If the connection closes or times out without an ack, the broker **redelivers** it to another consumer. (The message may actually have been processed with only the ack lost — handling that needs an **atomic commit protocol**, Chapter 9.)

Reordering (Figure 11-2): with load balancing, consumer 2 crashes processing **m3** while consumer 1 handles **m4**; m3 is redelivered to consumer 1, which processes **m4, m3, m5** — out of producer 1's order.

Figure 11-2 (redrawn): producer 1 sends m1, m3, m4 and producer 2 sends m2, m5. Consumer 2 handles m1, consumer 1 handles m2, then consumer 2 dies midway through m3 while consumer 1 is busy with m4. The broker later redelivers m3 to consumer 1, which therefore processes m4, m3, m5.



Though **JMS and AMQP require** order preservation, load balancing + redelivery **inevitably reorders**. Use a **separate queue per consumer** to avoid it. Harmless for independent messages; serious with **causal dependencies**.

Partitioned Logs

Network communication is normally **transient** (no permanent trace unless you capture packets), and even disk-persisting brokers delete messages soon after delivery. Databases and filesystems keep data until deleted. This matters for **derived data**:

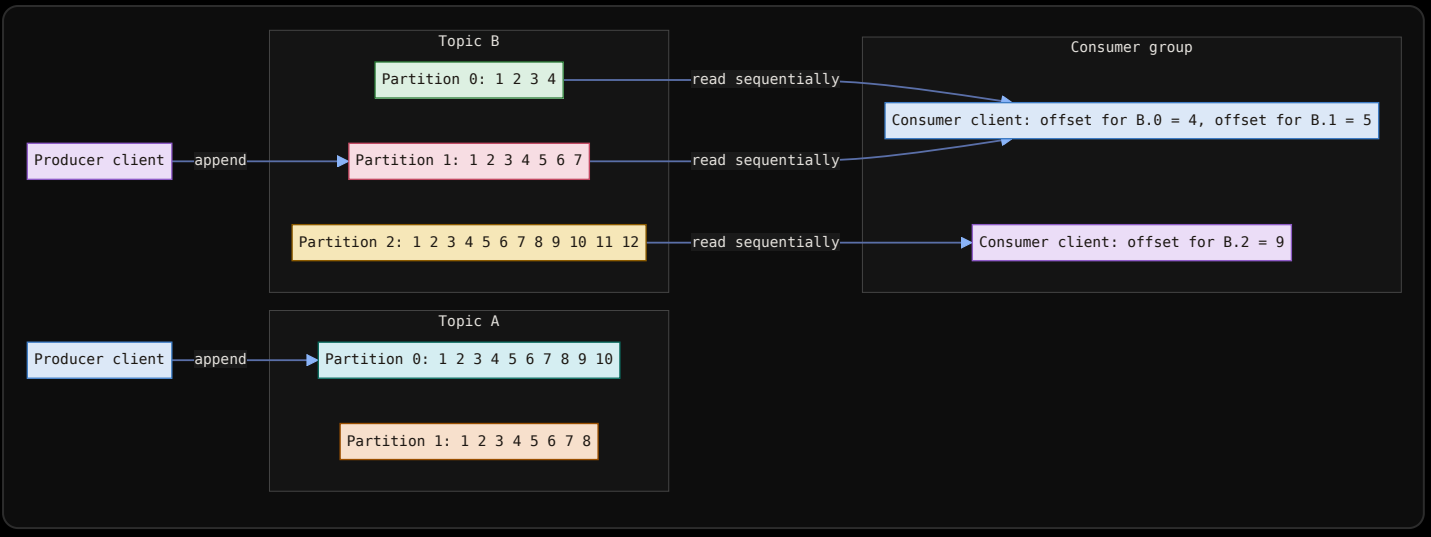
- Batch jobs can rerun on read-only input harmlessly.
- With **AMQP/JMS**, receiving is **destructive** (the ack deletes), so rerunning a consumer gives different results.
- A **new consumer** only gets messages sent **after it registered**; with files/databases, a new client can read old data.

Idea: combine databases' **durable storage** with messaging's **low-latency notifications** — the **log-based message broker**.

Using logs for message storage

- A **log** is an append-only sequence of records on disk (Chapter 3 log-structured storage and WALs; Chapter 5 replication).
- A **producer appends** messages; a **consumer reads sequentially**, then waits for notification of new appends — like `tail -f`.
- One disk limits throughput, so the log is **partitioned** (Chapter 6) across machines; a **topic** is a **group of partitions** carrying one type of message.
- Within a partition each message gets a **monotonically increasing offset**; a partition is append-only, so its messages are **totally ordered** — but there is **no ordering across partitions**.

Figure 11-3 (redrawn): Topic A has partitions 0 (messages 1 to 10) and 1 (1 to 8); topic B has partitions 0 (1 to 4), 1 (1 to 7) and 2 (1 to 12). The numbers in the boxes are offsets. Producer clients append new messages to the end of a partition (one to topic A partition 0, one to topic B partition 1). A consumer group of two clients reads topic B sequentially: one client owns B.0 and B.1 (recorded offsets 4 and 5), the other owns B.2 (offset 9).



Examples: **Apache Kafka**, **Amazon Kinesis Streams**, **Twitter's DistributedLog**; **Google Cloud Pub/Sub** is architecturally similar but exposes a **JMS-style API**. Despite writing everything to disk they handle **millions of messages per second** via partitioning, and **replicate** for fault tolerance.

Logs compared to traditional messaging

- **Fan-out is trivial:** consumers read independently; reading deletes nothing.
- **Load balancing** assigns **whole partitions** to nodes in a **consumer group**; each consumes its partitions **sequentially, single-threaded**. Downsides: consumers ≤ **number of partitions**, and one **slow message holds up** its partition (**head-of-line blocking**, Chapter 1).

> **Footnote: sharing one partition.** Two consumers could split one partition by even/odd offsets, or use a **thread pool**, but that complicates offset management. Prefer single-threaded per partition and add partitions for parallelism.

Situation	Better choice
Messages **expensive**, want **per-message parallelism**, order **unimportant**	**JMS/AMQP** broker
High throughput, messages **fast** to process, order **important**	**Log-based** broker

Consumer offsets

- Sequential reading means all messages **below** the consumer's offset are done, all **above** unseen — the broker only **periodically records consumer offsets**, not per-message acks. Less bookkeeping plus **batching and pipelining** raise throughput.
- The offset is like the **log sequence number** in single-leader replication (Chapter 5, "Setting Up New Followers"): **the broker acts like a leader database, the consumer like a follower**.
- **Consumer failure:** another group member takes its partitions from the **last recorded offset**; messages processed after that offset are **processed a second time** (addressed later).

Disk space usage

The log is split into **segments**; old ones are **deleted or archived** (log compaction comes later). A **slow consumer** whose offset points at a **deleted segment misses messages**. The log is a **circular buffer / ring buffer** on disk — so it can be **very large**.

Book's estimate: a typical large drive held **6 TB** with **150 MB/s** sequential write; filling it takes about **11 hours** (6,000,000 MB ÷ 150 MB/s ≈ 40,000 s). So a disk buffers **~11 hours** at full speed (the ratio

holds across many disks); real deployments rarely use full bandwidth, so logs keep **several days or even weeks**.

Throughput stays **roughly constant regardless of retention**, since every message hits disk anyway — unlike memory-first systems that are fast with short queues but **much slower** once they spill to disk.

When consumers cannot keep up with producers

- A log is **buffering** with a **big fixed-size buffer** (disk space). A consumer lagging beyond retention can't read old messages — effectively **dropped**.
- **Monitor consumer lag** and **alert**; the big buffer gives humans time to fix it.
- Only the lagging consumer suffers — others aren't disrupted, so you can safely read a **production log** for development, testing or debugging.
- A stopped/crashed consumer **uses no resources** except its offset — unlike traditional brokers, where abandoned **queues must be deleted** or they keep accumulating messages and stealing memory.

Replaying old messages

- In AMQP/JMS, processing+ack is **destructive**; in a log, consuming is **read-only**, like reading a file.
- The only side effect is the **consumer offset moving forward**, which the consumer controls — e.g. run a **copy of a consumer with yesterday's offsets**, write output elsewhere, **reprocess the last day**, as often as you like with different code.
- This makes log-based messaging like **batch**: derived data cleanly separated from input by a repeatable transformation — good for experimentation, recovering from bugs, and **integrating dataflows across an organization**.

Own Kafka-style pseudocode:

```
consumer = Consumer(group="search-indexer", topic="page-edits")
consumer.seek(partition=0, offset=41_200)   # rewind to replay from an older position
for msg in consumer.poll_forever():         # sequential read; nothing is deleted
    index.update(msg.key, msg.value)        # derived output goes somewhere else
    consumer.commit(msg.offset + 1)         # record progress periodically, not per-message ack
```

`seek` works because the log is retained; after a crash, processing resumes from the last `commit` (later work is redone).

Databases and Streams

Log-based brokers borrowed from databases; the reverse works too.

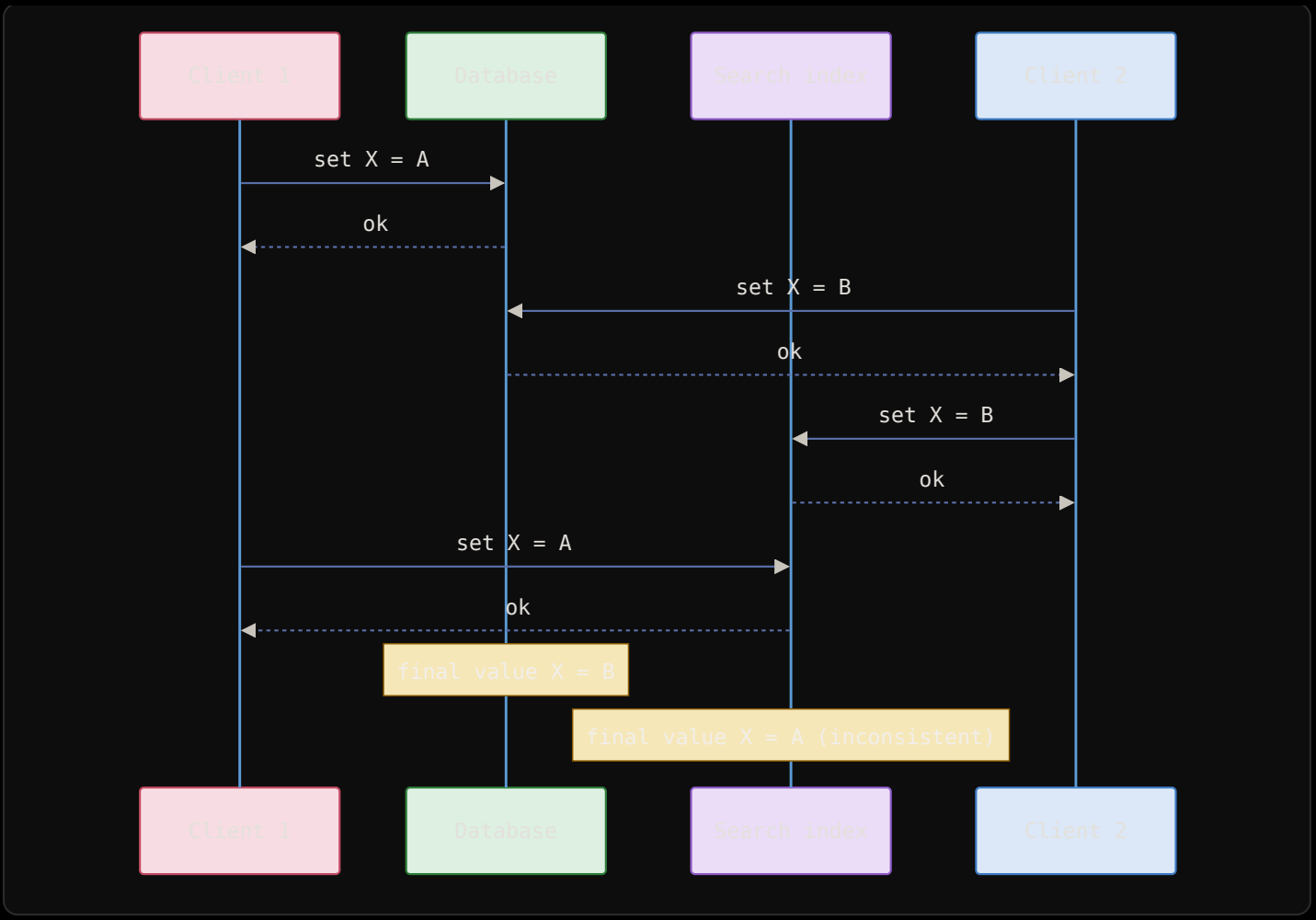
- A **database write** is itself an event that can be captured, stored and processed — the link is **fundamental**.
- A **replication log** (Chapter 5) *is* a stream of write events the leader emits; followers apply it.
- **State machine replication** (Chapter 9, "Total Order Broadcast"): replicas applying identical events in identical order reach the same state (given **deterministic** processing).

Keeping Systems in Sync

- Nontrivial apps combine an **OLTP database**, a **cache**, a **full-text index**, a **data warehouse** — each with its own optimized copy, which must stay **in sync**.
- Warehouses typically use **ETL** (Extract, Transform, Load) — a **batch** full copy; search indexes and recommenders can also be built in batch (Chapter 10).
- If full dumps are too slow, apps sometimes use **dual writes**: the application writes each change to every system (DB, then search index, then **invalidate** cache entries — or concurrently).

Problem 1: race condition (Figure 11-4). Clients 1 and 2 concurrently set **X** to **A** and **B**, each writing the database then the search index. The **database** gets A then B (ends **B**); the **index** gets B then A (ends **A**) — **permanently inconsistent**, with no error.

Figure 11-4 (redrawn): each write is acknowledged with "ok". The database gets A then B (final X = B), but the search index gets B then A (final X = A).



Without concurrency detection like **version vectors** (Chapter 5, "Detecting Concurrent Writes"), you wouldn't even notice.

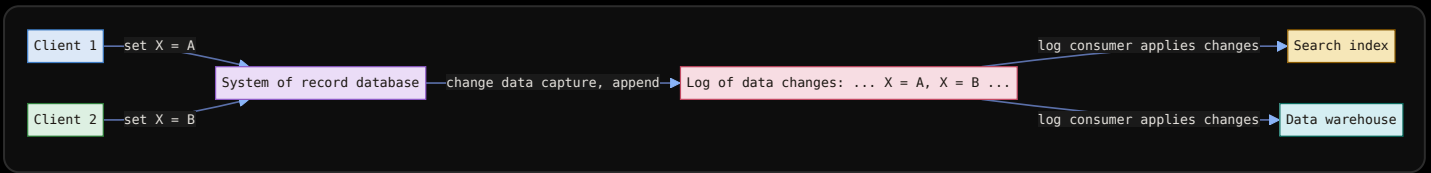
Problem 2: partial failure. One write may fail and the other succeed — a **fault-tolerance** issue; ensuring all-or-nothing is the costly **atomic commit** problem (Chapter 9, 2PC).

Root cause: there's **no single leader** — DB and index each act independently, as in multi-leader replication (Chapter 5). Better: make the database the **one leader** and the index its **follower**. Enter CDC.

Change Data Capture

- Replication logs were long an **internal implementation detail**; for **decades** many databases had **no documented way** to get their change log, making replication to other technologies hard.
- **Change data capture (CDC) observes every change written to a database** and extracts it in a form **other systems can replicate**, ideally **as a stream the moment it's written**.
- Applying changes to a search index **in the same order** keeps it matching the database; the index is just **another consumer of the change stream**.

Figure 11-5 (redrawn): client 1 sets X = A and client 2 sets X = B in the system-of-record database. Change data capture appends the changes, in the order the database applied them, to a log of data changes (... X = A, X = B ...). A log consumer applies them in that same order to the derived data systems: a search index and a data warehouse.



Implementing change data capture

Log consumers are **derived data systems** — other **views** of the **system of record**. **CDC makes one database the leader and the others followers.** A **log-based broker** suits transport because it **preserves order** (avoiding Figure 11-2's reordering).

Approach	How	Trade-off
Database triggers (Chapter 5, "Trigger-based replication")	Triggers record all table changes in a **changelog table**	**Fragile**, **significant performance overhead**
Parsing the replication log	Read the DB's own log	**More robust**, but must handle **schema changes**

Systems: **LinkedIn's Databus**, **Facebook's Wormhole**, **Yahoo!'s Sherpa** at large scale; **Bottled Water** (PostgreSQL, via an API decoding its **write-ahead log**); **Maxwell** and **Debezium** (MySQL, parsing the **binlog**); **Mongoriver** (MongoDB **oplog**); **GoldenGate** (Oracle).

CDC is usually **asynchronous**: the system of record doesn't wait for consumers. A **slow consumer** barely affects it, but all **replication lag** issues apply (Chapter 5).

Initial snapshot

Replaying a log of **all** changes would rebuild the database, but keeping everything needs **too much disk** and replay takes **too long**, so logs are **truncated**. A new full-text index needs a **full copy**, so start from a **consistent snapshot** (Chapter 5, "Setting Up New Followers") tied to a **known log offset**, then apply later changes. Some CDC tools do this; others leave it **manual**.

Log compaction

An alternative to repeated snapshots:

- As in Chapter 3 ("Hash Indexes", Figure 3-2), the engine periodically, in the **background**, finds records with the same **key**, discards duplicates and keeps **only the newest per key**.
- A **tombstone** (special **null** value) marks a **deletion** and removes the key during compaction; otherwise a key stays **forever**.
- So a compacted log's size depends on **current contents**, **not on how many writes** occurred; overwritten values are **garbage-collected**.
- For brokers and CDC: if each change has a **primary key** and **replaces** the previous value, the latest write per key suffices.
- Rebuild a derived system by reading the compacted topic from **offset 0** — it holds the **latest value for every key**, a **full copy without another snapshot**.
- **Apache Kafka supports log compaction**, letting the broker serve as **durable storage**, not just transient messaging.

Own example:

```
before compaction (offset: key=value)      after compaction
0: user:7 = {name: "Ana"}                  3: user:9 = {name: "Raj"}
1: user:9 = {name: "Raj"}                  4: user:7 = {name: "Ana M."}
2: user:5 = {name: "Li"}                   5: user:5 = null (tombstone, removed later)
3: user:9 = {name: "Raj"}
4: user:7 = {name: "Ana M."}
5: user:5 = null
```

API support for change streams

Databases increasingly offer change streams as a **first-class interface**, not reverse-engineered CDC:

- **RethinkDB**: queries **subscribe to notifications** when results change.
- **Firebase** and **CouchDB**: sync based on a **change feed** also exposed to apps.
- **Meteor**: follows the **MongoDB oplog** to refresh the UI.
- **VoltDB**: transactions **export data as a stream** — a **table that transactions insert tuples into but that cannot be queried**; its log of committed tuples, **in commit order**, is read asynchronously by consumers.

- **Kafka Connect**: integrates CDC tools for many databases with **Kafka**, from which search indexes and stream processors are updated.

Event Sourcing

Event sourcing, from the **domain-driven design (DDD)** community, also keeps **every change as an event in a log**, but at a different **level of abstraction**:

	Change data capture	Event sourcing
How the app uses the DB	**Mutably** (updates, deletes)	Logic **built on immutable events** in an **event log**
Log origin	Extracted **at a low level** (replication log), guaranteeing true write order (avoiding Figure 11-4's race)	**Append-only event store**; updates/deletes **discouraged or prohibited**
App awareness	**Need not know**	Designed around events
Event meaning	**Low-level state changes**	Things that happened **at the application level**

Why powerful: recording the **user's actions** as immutable events is more meaningful than recording their **effects** on mutable state. It eases **evolution**, aids **debugging**, and guards against **application bugs**.

Book example: "**student cancelled their course enrollment**" states intent neutrally; the side-effect version (row deleted from **enrollments**, reason added to **student feedback**) bakes in assumptions about future data use. A later feature — **offering the place to the next person on the waiting list** — simply chains off the existing event.

Event sourcing resembles the **chronicle data model**; an event log resembles a star schema's **fact table** (Chapter 3). **Event Store** is a specialized database for it, but any **conventional database** or **log-based broker** works.

Own JSON example:

```
{"type": "SeatHoldRequested", "show": "S12", "seat": "C7", "user": "u42", "ts": "2021-03-03T19:00:05Z"}
{"type": "SeatHoldConfirmed", "show": "S12", "seat": "C7", "user": "u42", "ts": "2021-03-03T19:00:06Z"}
{"type": "SeatReleased",      "show": "S12", "seat": "C7", "user": "u42", "ts": "2021-03-04T08:12:40Z"}
```

Each line is an immutable fact; releasing the seat is a new event, not a deletion.

Deriving current state from the event log

Users want **current state** (a **shopping cart's** current contents), not history. So apps **transform the event log** (how data is **written**) into display state (how it is **read**) with **deterministic** logic, so reruns give the same state.

	CDC	Event sourcing
Event content	Usually the **entire new record version**	**User intent**, not state-update mechanics
Later events override earlier?	Yes — **latest** event per key determines state	Usually **no**
Log compaction	Possible	**Not possible the same way** — need **full history**

Apps store **snapshots** of derived state to avoid full replays, but that is **only a performance optimization**; the intent is to **keep every raw event forever** and be able to **reprocess the whole log** (questioned under "Limitations of immutability").

Commands and events

1. A user request arrives as a **command**, which **may still fail** (e.g. an integrity violation).
2. The app **validates** it.
3. Once accepted it becomes an **event**: **durable** and **immutable**.

Example: **registering a username** or **booking a seat on a plane or in a theater** (Chapter 9, "Fault-Tolerant Consensus") → "username registered by user ID X" / "seat reserved for customer Y".

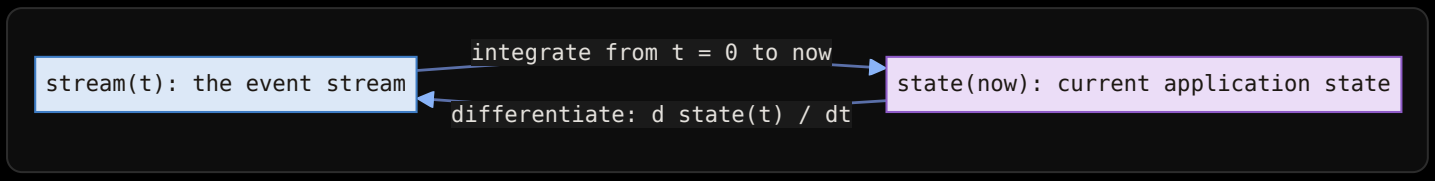
- An event is a **fact**: a later change or cancellation is a **separate event**; it remains true they **once held** the reservation.
- Consumers **may not reject** an event (it's already in the log, possibly read by others), so **validation must be synchronous, before the event is written** — e.g. a **serializable transaction** that validates and emits atomically.
- **Alternative:** a **tentative reservation** event then a separate **confirmation** (Chapter 9, "Implementing linearizable storage using total order broadcast"), allowing **asynchronous** validation.

State, Streams, and Immutability

Immutability — what makes batch inputs safe to experiment on — powers event sourcing and CDC too. Databases store **current state** and support updates/deletes, but changing state is always **produced by events**: **available seats** from **reservations**, an **account balance** from **credits and debits**, a **response-time graph** from individual request times. **Mutable state and an append-only event log are two sides of the same coin**; the **changelog** shows how state evolved.

Calculus view (Figure 11-6): integrating the event stream over time gives the state; **differentiating state over time gives the change stream**. The analogy has limits (a second derivative of state seems meaningless).

Figure 11-6 (redrawn): the book shows two formulas. state(now) = integral from t = 0 to now of stream(t) dt, and stream(t) = d state(t) / dt.



Storing the changelog **durably** makes state **reproducible**; treating the **event log as the system of record** and mutable state as **derived** simplifies reasoning about dataflow.

> **Pat Helland's view (paraphrased).** The transaction log records every change; the database is just a **cache of the latest values** in the log. **The log is the truth.**

Log compaction bridges the two: it keeps only the newest version of each record.

Advantages of immutable events

- **Accounting** has used immutability for **centuries**: an **append-only ledger** of transactions; the **profit and loss** statement and **balance sheet** are **derived** by summing it.
- **Mistakes aren't erased:** add a **compensating transaction** (e.g. a refund); the wrong entry stays for **auditing**, and published figures get a **correction** next period.
- **Auditability** helps beyond finance: like batch outputs (Chapter 10), recovering from buggy code is much easier when it can't **destructively overwrite** data.
- **More information than current state:** a customer adds then removes a cart item — irrelevant for fulfillment, but valuable for **analytics** (they considered it; maybe they'll buy later or chose a substitute). Deleting the item loses this.

Deriving several views from the same event log

- Several **read-oriented representations** can come from one log, like multiple consumers (Figure 11-5): **Druid** ingests directly from **Kafka**; **Pistachio** (a distributed key-value store; the book's index files it under Yahoo!) uses **Kafka as its commit log**; **Kafka Connect sinks** export to many databases and indexes; **search servers** could do likewise.
- **Easier evolution:** a new feature builds a **separate read-optimized view** from the log, running **alongside** existing systems — often easier than an in-place **schema migration**; retire the old view when done.

- **Flexibility:** much of schema/index/storage-engine complexity comes from supporting particular query patterns (Chapter 3). Separating **the form data is written in** from **the forms it is read in** is **command query responsibility segregation (CQRS)**.
- **Normalization debate dissolves:** traditional design assumes the **fallacy** that data must be stored as it will be queried. With a **write-optimized event log** translated into **read-optimized state**, it's fine to **denormalize** read views (Chapter 2) — the translation keeps them **consistent**.
- **Twitter home timelines** (Chapter 1, "Describing Load"): a highly **denormalized** per-user cache of followed users' tweets, kept consistent by the **fan-out service** that copies each tweet to followers and reacts to follow changes.

Concurrency control

Biggest downside: consumers are **asynchronous**, so a user may write to the log, read a derived view, and **not see their write** (Chapter 5, "Reading Your Own Writes"). Fixes: (1) update the read view **synchronously** with the append, via a **transaction** — both in **one storage system**, or a **distributed transaction**; (2) Chapter 9's "Implementing linearizable storage using total order broadcast".

But event logs also **simplify** concurrency:

- **Multi-object transactions** (Chapter 7) are often needed because **one user action touches several places**; an event can **describe the whole action**, needing **one atomic append**.
- If log and state are **partitioned the same way** (an event in log **partition 3** only changes state **partition 3**), a **single-threaded consumer** needs **no write concurrency control** (cf. "Actual Serial Execution", Chapter 7): the log fixes a **serial order**, **removing nondeterminism**.
- Events touching **multiple state partitions** need more work (Chapter 12).

Limitations of immutability

- Immutability is common elsewhere: databases use **immutable structures or multi-version data** for snapshots (Chapter 7); **Git, Mercurial and Fossil** keep immutable history.
- **Keeping all history forever** depends on **churn**: mostly-append workloads are easy; workloads with **heavy updates/deletes on a small dataset** make history **prohibitively large**, cause **fragmentation**, and make **compaction and garbage collection** performance crucial.
- **Administrative deletion: privacy regulations** (erase personal data after account closure), **data protection legislation** (remove erroneous data), and **accidental leaks**. A "treat as deleted" event isn't enough — you must **rewrite history**: **Datomic's excision**, **Fossil's shunning**.
- **True deletion is hard:** storage engines, filesystems and **SSDs** often write new data elsewhere rather than overwrite, and **backups** are deliberately immutable. Deletion is more "harder to retrieve" than "impossible" — but sometimes you must try (Chapter 12, "Legislation and self-regulation").

Processing Streams

Having covered stream sources (user activity, sensors, database writes) and transport (direct messaging, brokers, logs), what can you **do** with a stream?

1. **Write events to a storage system** (database, cache, search index) — the streaming equivalent of batch outputs; works best if the **stream consumer is the only writer** (Figure 11-5).
2. **Push events to users** — **email alerts**, **push notifications**, a **real-time dashboard**; a **human** is the consumer.
3. **Process input streams into output streams**, possibly via a **pipeline** of stages ending in 1 or 2.

Code doing 3 is an **operator** or **job**: like Unix processes and MapReduce jobs, it **reads inputs read-only** and **writes output append-only elsewhere**. Partitioning, parallelization and **mapping** (transform, filter) work as in Chapter 10. **The crucial difference: a stream never ends** — so **sorting makes no sense** (no **sort-merge joins**), and **fault tolerance changes**: a job running **for years** can't restart from the beginning.

Uses of Stream Processing

Long used for **monitoring**: **fraud detection** (unusual credit-card patterns → block), **trading systems** (rule-based trades on price movements), **manufacturing** (identify malfunctions quickly), **military and**

intelligence (alarm on signs of attack). These need **pattern matching and correlations**.

Complex event processing

- **CEP** (from the **1990s**) **searches for patterns of events**, as a **regular expression** finds character patterns.
- Patterns are written in a **declarative, SQL-like language** or a **GUI**; an engine runs an internal **state machine** and emits a **complex event** when a pattern matches.
- **Reversed relationship:** databases store data **permanently** and queries are **short-lived**; CEP stores **queries long-term** while events **flow past** them.
- Implementations: **Esper**, **IBM InfoSphere Streams**, **Apama**, **TIBCO StreamBase**, **SQLstream**; **Samza** and others are adding **SQL support**.

Stream analytics

Less about specific sequences, more about **aggregations and statistics**: event **rates**, **rolling averages** over a recent period, **comparing current statistics with previous intervals** (trends, alerts vs last week).

- Usually over **fixed time intervals** — e.g. mean **queries per second** over the **past 5 minutes** and the **99th percentile** response time (99% of requests are faster) for that span (smoothing noise yet reacting promptly). The interval is a **window**.
- **Probabilistic algorithms** — **Bloom filters** (set membership: "definitely not" or "probably present", Chapter 3), **HyperLogLog** (cardinality), **percentile estimation** (Chapter 1) — use **much less memory** for approximate results.
- **Common confusion:** stream processing is **not inherently lossy or inexact** — probabilistic algorithms are **merely an optimization**.
- Frameworks: **Apache Storm, Spark Streaming, Flink, Concord, Samza, Kafka Streams**; hosted **Google Cloud Dataflow**, **Azure Stream Analytics**.

Maintaining materialized views

- Keeping caches, indexes and warehouses current from a change stream is maintaining **materialized views** (Chapter 3); in **event sourcing**, application state is **also a materialized view**.
- Unlike analytics, a limited window **isn't enough**: you need **all events over an arbitrary period** (minus what compaction discards) — a **window back to the beginning of time**.
- This **clashes with** analytics frameworks built for short windows; **Samza and Kafka Streams** support it via **Kafka's log compaction**.

Search on streams

- Searching **individual events** with complex criteria like **full-text queries**: **media monitoring** (news feeds matched against stored queries for companies/products/topics), **real estate websites** alerting users to new matching listings. **Elasticsearch's percolator** implements this.
- **Inverted processing:** normal search **indexes documents, then runs queries**; stream search **stores queries** and runs documents past them (like CEP). Testing every document against every query is slow, so **index the queries** too, checking only candidates.

Message passing and RPC

Message-passing systems like the **actor model** (Chapter 4) aren't usually considered stream processors:

Actor frameworks	Stream processing
Manage **concurrency** and **distributed execution** of modules	A **data management** technique
Communication often **ephemeral and one-to-one**	Event logs **durable and multi-subscriber**
Arbitrary, including **cyclic request/response**	Usually **acyclic pipelines**: each stream produced by one job from defined inputs

Crossover: **Apache Storm's distributed RPC** interleaves user queries with stream events and aggregates results (Chapter 12, "Multi-partition data processing"). Actor frameworks can process streams, but many **don't guarantee delivery on crashes**, so they aren't fault-tolerant without extra retries.

Reasoning About Time

Windows like "**the average over the last five minutes**" are surprisingly tricky.

- **Batch** processes **historical** events quickly — a **year's** events in **a few minutes** — so it must use the **timestamp in each event**, not the machine clock. This also makes processing **deterministic** (Chapter 10, "Fault tolerance").
- **Many stream frameworks** window by the processor's **local clock** (**processing time**) — simple, and fine when lag is negligible, but it **breaks down** with significant **processing lag**.

Event time versus processing time

Delays come from **queueing**, **network faults** (Chapter 8), **contention** in broker or processor, consumer **restarts**, and **replaying history** after faults or bug fixes. Delays also cause **reordering**: a user's first request (server **A**) and second (server **B**) emit events, but **B's reaches the broker first**.

> **Star Wars analogy (credited to Kostas Kloudas of the Flink community).** Episodes IV, V, VI came out in 1977, 1980, 1983; I, II, III in 1999, 2002, 2005; VII in 2015. Watching in release order differs from story order: **episode number = event timestamp; viewing date = processing time.** Humans cope; algorithms must handle it explicitly.

Confusing them gives bad data: a processor measuring **request rate** is **redeployed** and down for **a minute**; draining the backlog afterwards looks like a **spike** by processing time, while the real rate was **steady**.

Figure 11-7 (described): the top of the figure shows a web server emitting request events at a steady pace, and a stream processor that consumes them but has a "restarting" gap, after which it works through the backlog at a faster pace. Below are two charts. "Rate as measured by processing time" starts at 10, dips to 0 while the processor restarts, then climbs above 10 (to about 15) while the backlog drains, before settling back down. "Actual request rate" (by event time) is a flat line at 10 the whole time. The dip and the spike are artifacts purely of variations in processing rate.

Knowing when you're ready

With event-time windows you can **never be sure** all events have arrived. Counting per **one-minute window**: events for the **37th minute** are counted and new events are now in the **38th and 39th** — when do you close the 37th? A **timeout** helps, but events may still be **buffered on another machine** behind a **network interruption**. For such **straggler events**:

1. **Ignore them** (usually few); **count dropped events** and **alert** if significant.
2. **Publish a correction** — re-emit the window's value, possibly **retracting** the earlier result.

Producers can emit a marker "**nothing with a timestamp before t will follow**", but with many producers consumers must **track each one**, and **adding/removing producers** gets harder.

Whose clock are you using, anyway?

Events may be **buffered at several points**: a **mobile app** used **offline** sends events when reconnected, perhaps **hours or days later** — extreme stragglers. The meaningful timestamp is the **device's local clock**, which **can't be trusted** (accidentally or deliberately wrong, Chapter 8); the **server's receive time** is **accurate** but **less meaningful**.

Fix: log three timestamps — (1) **occurred** (device clock), (2) **sent** (device clock), (3) **received** (server clock). **Offset** = (3) − (2), assuming **negligible network delay**; **true event time** ≈ (1) + offset, assuming the device offset **didn't change** between event and send.

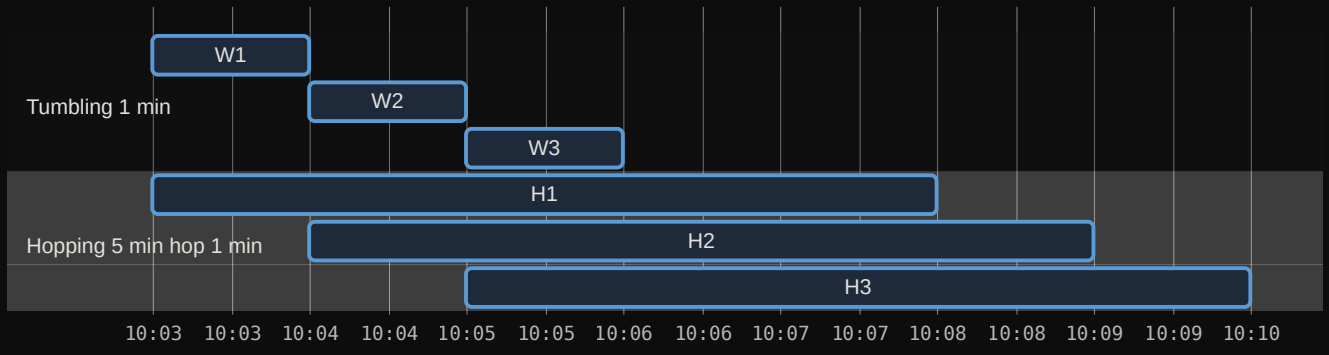
```
t1 (occurred, device clock)  = 09:00:00
t2 (sent,     device clock)  = 12:00:00   # phone came back online 3 h later
t3 (received, server clock)  = 12:05:00
offset       = t3 - t2 = +5 min          # device clock runs 5 min slow
true time est = t1 + offset = 09:05:00
```

This problem **isn't unique to streaming** — batch has it too; streaming just makes time more visible.

Types of windows

Window	Definition	Book's example	Implementation
Tumbling	**Fixed length**; each event in **exactly one** window	1-minute: **10:03:00–10:03:59**, then **10:04:00–10:04:59**	**Round timestamp down** to the minute
Hopping	**Fixed length**, **overlapping** for smoothing	**5-minute** window, **1-minute hop**: **10:03:00–10:07:59**, **10:04:00–10:08:59**, …	**1-minute tumbling** windows aggregated over adjacent ones
Sliding	All events **within some interval of each other**	5-minute sliding includes **10:03:39** and **10:08:12** (under 5 min apart); fixed **boundaries** of tumbling/hopping would split them	**Time-sorted buffer**, evicting expired events
Session	**No fixed duration**; a **single user's** events close together, ending after **inactivity**	Closes after **30 minutes** without events	Sessionization, common in **website analytics** (Chapter 10, "GROUP BY")

Illustrative diagram (not a numbered book figure): tumbling vs hopping boundaries for the same minute-by-minute timeline.



Own tumbling-window count by event time:

```
from collections import defaultdict
counts = defaultdict(int)
def on_event(ev):
    window_start = ev.ts - (ev.ts % 60)   # round event time down to the minute
    counts[window_start] += 1             # tumbling: each event lands in exactly one window
```

Stream Joins

Joins are as essential to streaming pipelines as to batch (Chapter 10), but events may arrive **anytime**. Three types: **stream-stream**, **stream-table**, **table-table**.

Stream-stream join (window join)

Book's example: spotting URL search trends. Each search logs the **query and results**; each click logs a **click** event. **Click-through rate** per URL requires joining searches and clicks by **session ID** (ad systems need the same).

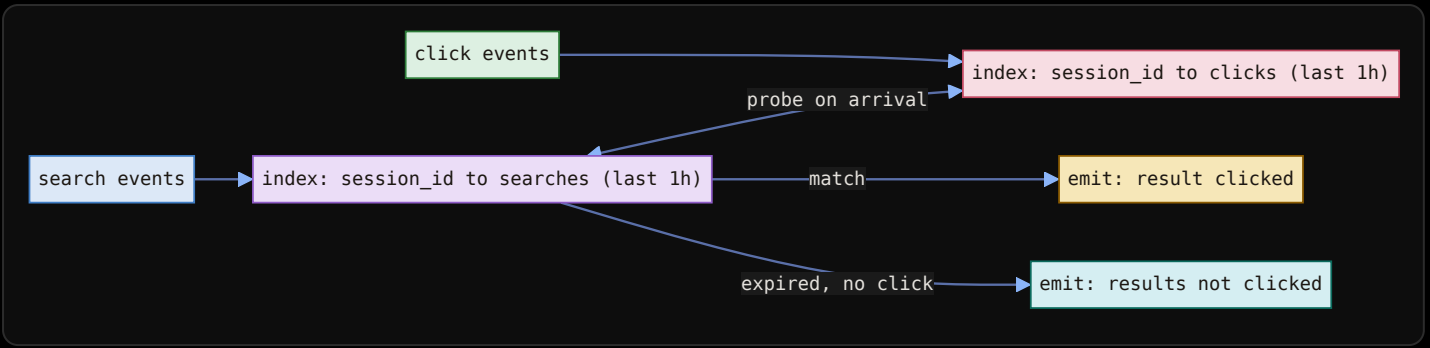
- An abandoned search produces **no click**.
- Click delay varies from seconds to **days or weeks** (forgotten tab).
- Network delay can make the **click arrive before the search**.

- So choose a **join window**, e.g. **no more than one hour** apart.

> **Common confusion:** embedding search details in the click event is **not** a join — it misses searches with **no clicks**, which accurate click-through rates need.

How: keep **state** — events from the **last hour indexed by session ID**. On each event, add it to its index and probe the **other** index; on a match emit **which result was clicked**; when a search **expires** unmatched, emit **which results were not clicked**.

Illustrative diagram (not a numbered book figure): state kept for a stream-stream window join.



Stream-table join (stream enrichment)

Book's example: Figure 10-2's join of **user activity events** with **user profiles**, done continuously: events with a user ID come in; events **augmented with profile information** go out — **enriching** the events.

- **Querying a remote database** per event is **slow** and risks **overloading** it.
- Instead keep a **local copy** in the processor — like Chapter 10's **map-side hash joins**: an **in-memory hash table** if small, or an **index on local disk**.

Difference from batch: batch uses a **snapshot**; a long-running stream processor must keep its copy **up to date** — via **CDC**, subscribing to the **profile changelog** and updating the copy. So it is really a **join of two streams**. Unlike a stream-stream join, the **table changelog** side uses a window back to the **"beginning of time"** with newer versions **overwriting** older ones, while the **event** side may keep **no window**.

```
profiles = {}                                  # local copy of the users table
def on_profile_change(chg):                    # from the CDC changelog stream
    profiles[chg.user_id] = chg.new_row        # newer version overwrites older (infinite window)
def on_activity(ev):                           # from the activity event stream
    emit({**ev, "profile": profiles.get(ev.user_id)})   # enriched event, no window kept
```

Table-table join (materialized view maintenance)

Book's example: Twitter timelines (Chapter 1). Instead of merging followed users' tweets at read time, keep a per-user **timeline cache** (**"inbox"**) so reads are a **single lookup**. Handlers:

- User **u** tweets → add to the timeline of **every follower of u**.
- A tweet is **deleted** → remove it from **all** timelines.
- **u1 follows u2** → add u2's **recent tweets** to u1's timeline.
- **u1 unfollows u2** → remove u2's tweets from u1's timeline.

The processor consumes streams of **tweets** (send, delete) and **follows** (follow, unfollow) and keeps a database of each user's **followers** — i.e. it maintains a **materialized view** of a join of two tables. Own SQL (different names than the book's):

```
SELECT f.reader_id AS timeline_owner,                        -- one timeline per follower
       array_agg(p.* ORDER BY p.posted_at DESC) AS timeline  -- that follower's posts, newest first
FROM   posts p
JOIN   subscriptions f ON f.author_id = p.author_id          -- follower sees posts by authors they follow
GROUP  BY f.reader_id;
```

The timelines are a **cache of this query's result**, updated whenever either table changes.

> **Footnote: product rule.** If a stream is a table's derivative (Figure 11-6) and a join is a product **u·v**, changes obey **(u·v)' = u'v + uv'**: tweet changes join with follower lists as they are **now**, and follower changes join with tweets as they are **now**.

Time-dependence of joins

- All three build **state** from **one** input and **look it up** for messages on the **other**.
- **Order matters** (follow then unfollow ≠ the reverse). Partitioned logs order **within** a partition but **not across streams or partitions**.
- So which activity events see the **old** profile and which the **new**? **What point in time do you join against?**
- **Tax rates** depend on **country/state**, **product type** and **date of sale**; reprocessing history, you want the rate **at the time of sale**, not today's.
- Undetermined cross-stream order makes the join **nondeterministic** — reruns may differ.
- Warehouses call this the **slowly changing dimension (SCD)** problem, solved by giving each **version** a **unique ID** (each tax-rate change gets a new ID recorded on the invoice): **deterministic**, but **log compaction becomes impossible** (all versions retained).

Join type	Inputs	State kept	Window	Book example
Stream-stream	Two activity streams (or one self-joined)	Recent events of both, indexed by key	Limited (e.g. 1 hour)	Searches + clicks by session ID
Stream-table	Activity stream + DB changelog	Local table copy	Infinite for table; none for stream	Activity enriched with profiles
Table-table	Two DB changelogs	Latest state of both	Infinite for both	Twitter timelines from tweets + follows

Fault Tolerance

Batch recap: a failed MapReduce task restarts elsewhere and its output is discarded — possible because **inputs never change**, each task writes **its own HDFS file**, and output becomes visible **only on success**. The result equals a failure-free run, as if each record was processed **exactly once** (restarts may process records repeatedly, but the **visible effect** is once) — **exactly-once semantics**, better called **effectively-once**. **Streaming problem:** a stream is infinite, so you **can't wait for a task to finish** before exposing output.

Microbatching and checkpointing

Microbatching (**Spark Streaming**): split the stream into **small blocks** (typically **about one second**), each a **miniature batch**. Smaller batches raise **scheduling/coordination** cost; bigger ones **delay results**. It implicitly creates a **tumbling window of one batch** by **processing time**; larger windows must **carry state** across microbatches.

Checkpointing (**Apache Flink**): periodically take **rolling checkpoints of operator state** to **durable storage**; on a crash, **restart from the latest checkpoint** and **discard output** produced since. Triggered by **barriers** in the stream — like microbatch boundaries but **without forcing a window size**.

Limit: both give exactly-once **inside** the framework, but output that **leaves the processor** (DB writes, external messages, emails) **can't be discarded**, so a restart repeats external **side effects**.

Atomic commit revisited

For exactly-once despite faults, **every output and side effect** of processing an event must happen **if and only if** processing succeeds: messages to **downstream operators** or **external systems** (emails, **push notifications**), **database writes**, **operator state** changes, and **acknowledging input** (in a log, **advancing the offset**) — **all atomically or none** (Chapter 9, "Exactly-once message processing").

XA has problems (Chapter 9), but **restricted environments** allow efficient atomic commit: **Google Cloud Dataflow**, **VoltDB**, and planned for **Apache Kafka**. Rather than spanning **heterogeneous technologies**, they

keep transactions **internal** to the framework (managing both state and messaging), and **amortize** cost by grouping many input messages per transaction.

Idempotence

Alternative to distributed transactions: **idempotent** operations — repeating has the **same effect as once**.

- **Setting a key to a fixed value** is idempotent; **incrementing a counter** is **not**.
- **Make it idempotent with metadata:** Kafka messages carry a durable, **monotonically increasing offset**; store the **offset of the message that caused the latest write** with the value, and skip updates already applied.
- **Storm's Trident** (higher-level API on Apache Storm) uses a similar idea.

```
def apply(msg):                                  # msg.offset is monotonically increasing
    row = db.get(msg.key) or {"total": 0, "last_offset": -1}
    if msg.offset <= row["last_offset"]:
        return                                   # already applied before the crash: skip
    row["total"] += msg.amount                   # non-idempotent increment...
    row["last_offset"] = msg.offset              # ...made safe by storing the offset with it
    db.put(msg.key, row)                         # single write stores both atomically
```

Assumptions: (1) replay gives **identical messages in identical order** (log-based brokers do); (2) processing is **deterministic**; (3) **no other node concurrently updates** the value. On failover, **fencing** (Chapter 8, "The leader and the lock") blocks a node thought dead. With these, idempotence gives exactly-once at **small overhead**.

Rebuilding state after a failure

Stateful operators — **windowed aggregations** (counters, averages, histograms) and **join tables/indexes** — must **recover state**:

Option	How	Trade-off / example
Remote replicated datastore	State in a remote DB	Querying it **per message can be slow**
Local state, replicated periodically	New task reads replicated state and resumes **without data loss**	**Flink** snapshots operator state to durable storage such as **HDFS**; **Samza and Kafka Streams** write state changes to a **dedicated log-compacted Kafka topic** (like CDC); **VoltDB processes each input on several nodes redundantly** (statement-based replication — sending commands, not results; Chapter 7, "Actual Serial Execution")
Rebuild from input streams	Recompute	**Short window**: **replay** its input; **CDC-maintained local replica**: rebuild from the **log-compacted change stream**

Choices depend on **infrastructure performance**: sometimes the **network beats disk on latency** and **network throughput rivals disk**. Nothing is **best everywhere**, and local vs remote trade-offs may **shift as technology evolves**.

Chapter summary

- **Stream processing = batch done continuously on unbounded input**; brokers and event logs are the streaming **filesystem**.

	AMQP/JMS-style broker	Log-based broker
Delivery	**Individual messages** to consumers	**Whole partition** to **one consumer node**, in **fixed order**
Progress	**Ack per message**; **deleted** once acked	**Checkpoint the offset**
Parallelism	Per message	Via **partitioning**
Rereading	Not possible	**Retained on disk**; jump back and reread
Good for	**Asynchronous RPC** (fire a request, don't wait), e.g. a **task queue**, where order doesn't matter and old messages needn't be reread	Producing **derived state or streams**

- Logs resemble **database replication logs** (Chapter 5) and **log-structured storage engines** (Chapter 3).
- **Stream sources:** user activity, periodic sensor readings, **financial market data** feeds, **database writes** — captured **implicitly via CDC** or **explicitly via event sourcing**; **log compaction** retains a **full database copy**.
- **Databases as streams** keep search indexes, caches and analytics continuously updated, and let you build **fresh views** by consuming the log from the start.
- **State-as-streams and replay** underpin **joins** and **fault tolerance**.
- **Purposes: CEP** (patterns), **stream analytics** (windowed aggregations), **materialized views**.
- **Time is hard: processing vs event time**; **stragglers**.
- **Joins: stream-stream** (related events within a window, e.g. same user within **30 minutes**; can be a **self-join** of one stream); **stream-table** (events **enriched** from a CDC-maintained local copy); **table-table** (both **changelogs**, each change combined with the other side's **current state** to update the **materialized join view**).
- **Exactly-once:** discard failed partial output — but streams run forever, so use finer-grained **microbatching, checkpointing, transactions, or idempotent writes**.

Interview angle

1. Kafka-style log vs RabbitMQ-style queue: when to use which? Traditional (AMQP/JMS) brokers deliver individual messages, delete on ack, load-balance per message — good for task queues with expensive, order-insensitive messages. Log-based brokers append to durable partitioned logs consumed in order per partition, track offsets, and allow replay — good for high-throughput, order-sensitive streams and derived data.

- *Follow-up: Why does a queue reorder?* Load balancing + redelivery: a crashed consumer's message goes to one that has moved on (m4, m3, m5). Fix: one queue per consumer.
- *Follow-up: What limits log parallelism?* Consumers per group ≤ partitions; a slow message blocks its partition (head-of-line blocking).

2. How do you keep a search index / cache in sync with the database? Avoid dual writes (race: DB ends B, index A; partial failure). Use CDC: database as single leader, capture its log (Debezium/Maxwell for MySQL binlog, Bottled Water for PostgreSQL WAL, Mongoriver for the oplog), publish in order through a log-based broker, consumers as followers. Bootstrap from a snapshot tied to an offset, or a compacted topic from offset 0.

- *Follow-up: Downside?* Asynchronous, so replication-lag anomalies apply.

3. What is event sourcing and how does it differ from CDC? Immutable application-level events ("student cancelled enrollment") are the source of truth and state is derived; CDC extracts low-level changes from a mutable database. Event sourcing separates commands (may fail) from events (facts consumers can't reject), enables CQRS read views, auditability and evolution — but logs can't be compacted per key, snapshots are needed for speed, and deletion needs excision-like features.

- *Follow-up: Validating a seat booking?* Synchronously before emitting (serializable transaction), or tentative-reservation + confirmation events.

4. Event time vs processing time; late data? Event time = when it happened; processing time = when processed. Processing-time windows create artifacts (restart backlog = spike). Event-time windows are never known complete: ignore stragglers (monitor drops) or publish corrections; optionally per-producer "nothing before t" markers. For device clocks, log occurred/sent (device) and received (server) timestamps and correct by the offset.

- *Follow-up: Window types?* Tumbling, hopping, sliding, session.

5. Stream joins? Stream-stream (searches + clicks within 1 hour by session ID), stream-table (enrichment from a local CDC-maintained table), table-table (materialized view like Twitter timelines). All keep state from one input and probe it with the other; cross-stream order isn't guaranteed, so joins can be nondeterministic — SCD fix: version joined records (tax-rate IDs), losing log compaction.

6. How do stream processors achieve exactly-once? Inside the framework: microbatching (Spark Streaming, ~1 s) or barrier checkpoints (Flink). For external effects: framework-internal atomic commit (Google Cloud Dataflow, VoltDB, planned for Kafka) or idempotent writes (store the Kafka offset with the value; needs same-order deterministic replay, no concurrent writers, fencing on failover). State recovery: snapshots (Flink → HDFS), compacted changelog topics (Samza, Kafka Streams), redundant processing (VoltDB), or replaying input.

- *Follow-up: Literally exactly once?* No — records may be processed more than once; the visible effect is once ("effectively-once").

Self-check

1. **Why does polling become expensive at low latency, and what do messaging systems do instead?** More frequent polls find new data less often, so overhead grows. Messaging systems notify consumers of new events.
2. **Three options when producers outpace consumers; which do pipes/TCP and log brokers use?** Drop, buffer, backpressure. Pipes/TCP: backpressure. Log brokers: buffering in a large fixed-size on-disk buffer.
3. **Book's numbers: how long can one disk buffer messages, and why longer in practice?** 6 TB at 150 MB/s fills in ~11 hours; deployments rarely use full bandwidth, so logs hold days or weeks.
4. **Scenario: dual writes of X = A and X = B to a DB and search index. What goes wrong; how does CDC fix it?** Different interleavings (DB ends B, index A) cause silent permanent inconsistency; one write may also fail. CDC makes the DB the single leader and applies its changes to the index in commit order.
5. **Why can a CDC topic be compacted but an event-sourced log generally not?** CDC updates carry the full record, so the latest per key determines state; event-sourced events express intent and don't override earlier ones.
6. **Command vs event; where must validation happen?** A command may fail validation; once accepted it becomes an immutable event. Consumers can't reject events, so validate synchronously before writing (or tentative + confirmation events).
7. **Two non-performance problems with immutability?** Legal/administrative deletion (privacy, erroneous data, leaks) needs history rewriting (Datomic excision, Fossil shunning); and true deletion is hard because copies persist in SSDs, filesystems and immutable backups.
8. **Dashboard spike right after redeploying the stream processor — why?** Processing-time windowing: the backlog from the one-minute downtime was processed at once, though the event-time rate was steady.
9. **Which window puts 10:03:39 and 10:08:12 in one 5-minute window; why not others?** Sliding (under 5 minutes apart); tumbling and hopping have fixed boundaries that separate them.
10. **Assumptions behind idempotent-write exactly-once?** Same messages in the same order on replay, deterministic processing, no concurrent updater, plus fencing on failover.
11. **When rebuild operator state from input instead of replicating it?** When state aggregates a short window (fast replay) or is a CDC-maintained local copy rebuildable from a compacted change stream.

Chapter 12 — The Future of Data Systems

Book: Chapter 12, pp. 489–552

Overview

Earlier chapters describe how data systems work **today**; this one argues how they **ought** to be built. Being opinion, Kleppmann writes in the first person and invites disagreement.

The epigraph (Thomas Aquinas, usually paraphrased as *a captain whose only goal was to keep the ship safe would never leave port*) makes the point that a system exists for a purpose beyond survival. Reliability, scalability and maintainability (Chapter 1) are means; the goal is applications that are **robust, correct, evolvable, and ultimately good for people**.

Four big questions:

1. **Data integration** — real apps glue together a database, search index, cache, warehouse…; how do you keep all copies consistent?
2. **Unbundling databases** — can a database's internal mechanisms (indexes, views, replicas kept in sync) be pulled "out" to wire a whole organisation together?
3. **Aiming for correctness** — with asynchronous logs (append-only event sequences read by each consumer at its own pace) and stream processors instead of distributed transactions, how do we avoid lost, duplicated or corrupted data?
4. **Doing the right thing** — ethics around prediction, bias, surveillance and privacy.

Why it matters: most sizeable companies have a "zoo" of data stores fed from each other. The ideas here — one ordered log as the source of truth, everything else derived, idempotent consumers, end-to-end request IDs, auditing — are how modern **event-driven architectures** (Kafka + stream processors + downstream stores) are built. New to cache, replication, partitioning or queue? See System Design Fundamentals.

> **Analogy:** a spreadsheet — change a cell and every dependent formula updates. The dream is company-wide data infrastructure that behaves the same, correctly, even when machines crash.

Key terms

Term	Plain-English meaning
Data integration	Getting the same data, in the right shape, into every system that needs it
System of record	Authoritative source-of-truth copy; new writes land here first
Derived data / derived dataset	Computed from other data by a repeatable process (index, cache, view, ML model); can be rebuilt
Dataflow	The path data takes as it moves and transforms between systems
Change data capture (CDC)	Reading a database's change stream so others can apply the same changes
Event sourcing	Storing every user action as an immutable event and deriving state from the log
Total order / total order broadcast	Every consumer sees all events in the same sequence; equivalent to consensus
Causal dependency	B happened because of (or after seeing) A, so B mustn't be processed as if A hadn't happened
Logical timestamp	Counter-based timestamp (e.g. Lamport clock) ordering events without physical clocks
Idempotent	Same effect applied once or many times
Deterministic	Same input always gives the same output (no randomness, clock reads, network calls)
Reprocessing	Rerunning a (new) transformation over all old data to build a new view
Lambda architecture	Batch and stream systems side by side over the same events, outputs merged
Unified batch and stream processing	Historical reprocessing and live processing in one engine
Exactly-once (effectively-once)	End result as if each message were processed once despite retries
Event time vs processing time	When an event happened vs when it's processed
Unbundling databases	Treating a database's features (indexes, views, replication) as separate components joined by logs
Federated database / polystore	One query interface over many stores ("unifying reads")
Database inside-out	Another name for the unbundling/dataflow approach (author's 2014 talk)
Derivation function	Code turning base data into a derived dataset
Stateless service	App server that forgets each request after answering; state lives in a database
Observer pattern	Subscribing to a value to be notified of changes instead of polling
Stream-table join (enrichment)	Joining each event with a locally kept, up-to-date table
Write path / read path	Eager work when data arrives / lazy work when someone asks
Offline-first application	Works against a local on-device database, syncing in the background
Server-sent events / WebSockets	Protocols letting a server push messages to a browser over a long-lived connection
End-to-end argument	Some guarantees (dedup, integrity, security) need the endpoints, not just lower layers

Term	Plain-English meaning
Operation identifier / request ID	Client-generated unique ID for one logical action, used to drop duplicates downstream
Uniqueness constraint	No two records may share a value (username, seat, request ID)
Compensating transaction	A later corrective action (refund, apology, rebooking) instead of prevention
Timeliness	Users see up-to-date state; violations are temporary ("eventual consistency")
Integrity	No lost, duplicated, contradictory or false data; violations are permanent ("perpetual inconsistency")
Coordination-avoiding data system	Keeps integrity without synchronous cross-node coordination
System model	Assumptions about what can and can't go wrong
Auditing	Actively checking that data is still correct and complete
Rowhammer	Repeated memory-row access flipping bits in neighbouring rows
Merkle tree	Tree of hashes cheaply proving a record belongs to a dataset
Certificate transparency	Public Merkle-tree log for checking TLS/SSL certificates
Predictive analytics	Forecasting, including about individuals (credit risk, reoffending)
Algorithmic prison	Systematic exclusion from jobs, loans, travel etc. because an algorithm labelled you risky
Feedback loop	A prediction changes the world so as to reinforce itself
Systems thinking	Considering the whole system, humans included, to foresee consequences
Surveillance	Tracking people mainly to serve someone else's interests
Data exhaust	Behavioural data produced as a by-product of using a service
Data broker	Company that buys, combines, infers and resells personal data

---

Data Integration

Every problem in the book had several trade-off-laden solutions — log-structured storage vs B-trees vs column storage (Chapter 3), single-leader vs multi-leader vs leaderless replication (Chapter 5). One product must commit to one approach (covering every use case guarantees mediocrity), so even "general-purpose" databases suit particular usage patterns. Two challenges follow:

1. **Matching tools to situations** — vendors don't advertise weaknesses; the earlier chapters give you the questions to ask.
2. **Using data in many ways at once** — no single tool fits all of an application's access patterns, so you combine several.

Combining Specialized Tools by Deriving Data

Classic example: an OLTP database plus a full-text search index. PostgreSQL's built-in full-text search suffices for simple apps, but serious search needs a dedicated tool (e.g. Elasticsearch), and search indexes make poor durable systems of record — so you need both. Often also: analytics copies (warehouses, batch and stream processors), caches or **denormalized** objects, ML/classification/ranking/recommendation systems, and change-triggered notifications.

> **Beware "99% of people only need X".** Such claims reveal more about the speaker's experience than the technology; what's useless to one team is central to another. Data integration needs become

> obvious only across the **whole organisation**.

Reasoning about dataflows

With copies in several stores, be clear where data is **written first**, which representation is **derived from which**, and how and in what format each copy is populated. A clean design:

1. The app writes **only** to the system-of-record database.
2. **CDC** (Chapter 11) reads its change stream.
3. The index applies changes **in exactly the order they happened**.

If CDC is the *only* path, the index agrees with the database (barring bugs).

Dual writes go wrong: if the app writes to both, concurrent conflicting writes may be applied in opposite orders (Figure 11-4); neither store controls ordering, so they diverge **forever**.

Fix — a single ordering authority: route all input through one system that orders writes, and derive everything else in that order — state machine replication (Chapter 9, "Total Order Broadcast"). CDC or event-sourcing log, the principle is the same: **decide on a total order once**. Updates from a log can usually be **deterministic and idempotent**, so recovery is just replay.

Derived data versus distributed transactions

The traditional alternative is a distributed transaction (atomic commit / 2PC, Chapter 9: prepare all, then commit or abort all).

Aspect	Distributed transactions	Log-based derived data
Write ordering	Locks for mutual exclusion (e.g. two-phase locking)	Position in a log
Exactly-once	Atomic commit	Deterministic retry + idempotence
Timing	Usually **linearizable** — read your own writes	Usually **asynchronous** — no such guarantee by default
Faults	One failing participant aborts all, spreading failure	Faults stay local; slow consumers just lag
Cross-vendor practicality	**XA** has poor fault tolerance and performance	Simpler; works across heterogeneous systems

Author's view: distributed transactions work in constrained environments willing to pay; XA is severely limited; a better protocol would be hard to get adopted. So **log-driven derived data is the best bet** for integrating systems today. But he rejects "eventual consistency is inevitable, deal with it" without guidance — guarantees like read-your-writes matter; "Aiming for Correctness" seeks a middle ground.

The limits of total ordering

A single-leader database already totally orders writes, but at scale:

1. **Throughput:** one leader may not keep up; **partitioning** the log (Chapter 11) leaves order **across partitions** undefined.
2. **Geography:** multiple **datacenters** typically have a leader each (cross-datacenter sync is too slow; multi-leader, Chapter 5), so events from different datacenters are unordered.
3. **Microservices:** each service has its own state; no order across services.
4. **Client-side / offline state:** clients and servers likely see events in different orders.

Total order broadcast is **equivalent to consensus** (Chapter 9); most algorithms assume one node handles the whole stream. Scaling consensus beyond one node and across geographies is **an open research problem**.

Ordering events to capture causality

Unrelated events can go in any order, and same-object updates can be ordered by routing by object ID. But causality can hide. **The book's example:** A and B break up; A **unfriends** B, then messages friends complaining about B, expecting B not to see it. Friendships and messages live in different systems, so

the notification service may process the message **before** the unfriend and notify B. The notification is effectively a **join** of messages with the friend list — Chapter 11's time-dependence-of-joins problem.

Technique	Helps because	Limitation
Logical timestamps (Chapter 9, "Sequence Number Ordering")	Total order without coordination	Recipients still handle out-of-order delivery; extra metadata travels with events
Log an event recording **what the user saw** before deciding, with a unique ID later events reference	Records the causal dependency explicitly	Revisited in "Reads are events too" — logging reads costs storage and I/O
Conflict resolution algorithms (Chapter 5)	Handle unexpected order when maintaining state	Useless for external side effects like notifications

The author hopes for patterns that capture causality without the total-order bottleneck.

Batch and Stream Processing

Data integration's goal: **every place that needs data gets it in the shape it needs** — reading, transforming, joining, filtering, aggregating, training models and writing outputs (search indexes, materialized views, recommendations, metrics). Batch and stream share principles; the key difference is **unbounded** streams vs **finite** batch inputs. Implementations are blurring: **Spark** streams via **microbatches** (e.g. one second each) on a batch engine; **Apache Flink** does batch on a streaming engine. Either can emulate the other, with performance differences — microbatching can be slow for **hopping or sliding windows** (overlapping fixed-length windows).

Maintaining derived state

Batch has a **functional** flavour: **deterministic, pure functions**; **no side effects** beyond explicit output; **immutable inputs**, **append-only outputs**. Stream processing adds **managed, fault-tolerant operator state** (Chapter 11). Clear deterministic functions aid fault tolerance (idempotence) and make dataflows easier to reason about — caches, models and indexes are all **pipelines** deriving one thing from another.

Why asynchronous? You could update derived systems synchronously like a database's secondary indexes, but asynchrony keeps **faults local**, whereas distributed transactions abort if **any** participant fails, **amplifying failures**. Cross-partition indexes (Chapter 6) — **term-partitioned** (global; writes go to several partitions) or **document-partitioned** (local; reads go to all partitions) — also scale and behave best when updated **asynchronously**.

Reprocessing data for application evolution

Stream processing	Batch processing
Reflects input changes in views with low delay	Reprocesses accumulated history into **new** views

Reprocessing powers **evolution** (Chapter 4). Without it, schema evolution — under **schema-on-write** or **schema-on-read** — is limited to small steps (optional field, new record type); with it you can restructure into an **entirely different model**.

> **Schema migrations on railways (book sidebar).** 19th-century English railways used competing **gauges**. After a standard was chosen in **1846**, lines were converted by adding a **third rail** ("dual/mixed gauge") so both gauges could run, removing it once all trains converted. Expensive — which is why nonstandard gauges survive, e.g. **BART** in the San Francisco Bay Area differs from most of the US.

Derived views enable gradual migration: keep old and new schemas as **two independently derived views**; route a small share of users to the new one to test; increase gradually; drop the old view. Every step is **reversible**, lowering risk and letting you move faster.

The lambda architecture

1. Record input as immutable events in a growing dataset (like event sourcing).

2. Derive read-optimised views.
3. Run **batch** (e.g. **Hadoop MapReduce**) and **stream** (e.g. **Storm**) systems **in parallel**: streaming gives a quick **approximate** view; batch later produces a **corrected** one.

Rationale: batch was simpler and less buggy, stream processors less reliable, and streaming could use fast approximate algorithms. Influential — it popularised immutable events and reprocessing — but:

- **Double implementation:** logic in two frameworks; **Summingbird** (Twitter) abstracts over both, but you still operate and debug two systems.
- **Merging outputs** per request: easy for **tumbling-window** aggregations, much harder for joins, sessionization or non-time-series output.
- **Full reprocessing is expensive**, so batch runs incrementally (e.g. hourly), reintroducing **stragglers** and cross-boundary windows — defeating its simplicity.

Unifying batch and stream processing

Newer systems do **both** in **one system**, requiring:

Feature	Why	Example
Replay history through the live engine	Same code path for reprocessing	Log-based brokers replay; some stream processors read from **HDFS**
Exactly-once in stream processors	Output equals a fault-free run (discard failed partial output)	Chapter 11 fault tolerance
Windowing by **event time**	Processing time is meaningless during replay	**Apache Beam** API, running on **Apache Flink** or **Google Cloud Dataflow**

Unbundling Databases

Abstractly, databases, Hadoop and operating systems all **store data and let you process and query it** — records in a data model vs files. (Chapter 10: Hadoop is like a distributed Unix.) Differences exist — many filesystems struggle with **10 million small files** in a directory, while 10 million small records is normal for a database.

	Unix	Relational databases
Goal	Logical but **low-level** hardware abstraction	**High-level** abstraction hiding disk structures, concurrency, crash recovery
Building blocks	Pipes and byte-sequence files	SQL and transactions
"Simple" because	Thin wrapper over hardware	A short declarative query taps huge machinery (optimizer, indexes, joins, concurrency control, replication)

Both from the early 1970s; the tension is unresolved. The author sees **NoSQL** (from about 2009) as applying Unix-style low-level thinking to distributed OLTP. This section tries to combine the best of both.

Composing Data Storage Technologies

Database features — **secondary indexes**, **materialized views** (Chapter 3), **replication logs** (Chapter 5), **full-text search indexes** — have parallels built outside databases in Chapters 10–11: batch-built full-text indexes, stream-maintained materialized views, and CDC replication to derived systems.

Creating an index

`CREATE INDEX`:

1. Takes a **consistent snapshot** of the table and scans it.
2. Extracts indexed values, **sorts** them, writes the index.

3. Processes the **backlog** of writes since the snapshot (if the table wasn't locked).
4. Keeps the index updated on every later write.

Nearly identical to **setting up a new follower** (Chapter 5) and **bootstrapping CDC** with a snapshot (Chapter 11): it **reprocesses the existing dataset** into a new view (state being the result of applying the change stream).

The meta-database of everything

All of an organisation's dataflow looks like one giant database. Batch, stream and ETL jobs ≈ **triggers, stored procedures and view-maintenance routines**; derived systems ≈ **index types** (B-tree, hash, spatial…). The difference: separate software, machines and teams. Two ways to compose tools into a whole:

	Federated databases: unifying reads	**Unbundled databases: unifying writes**
Idea	One query interface over many engines (a **polystore**)	Reliably propagate every change to every store, despite faults
Example	PostgreSQL's **foreign data wrapper**	CDC + event logs feeding many stores
Specialised access	Apps can still use engines directly	Each store keeps its own API
Tradition	Relational: integrated, high-level, elegant, complicated	Unix: small tools, uniform low-level API (pipes), composed by a high-level language (shell)
Weakness	Doesn't solve synchronising **writes**	Still missing the "shell"

"Unbundling" means taking index maintenance apart to sync writes across **different technologies**.

Making unbundling work

Federated reads need data-model mapping (manageable); **keeping writes in sync is harder**. Heterogeneous distributed transactions are, in the author's view, the **wrong** solution. Transactions **within** one store or stream processor are fine (some use them internally for exactly-once), but across systems built by **different groups** (e.g. a stream processor writing to a key-value store or search index) there's no standard protocol. An **ordered event log with idempotent consumers** is far simpler.

The win is **loose coupling**:

1. **System level:** if a consumer is slow or down, the log **buffers**; others continue and the broken consumer catches up later without loss. Synchronous transactions instead turn local faults into large failures.
2. **Human level:** teams build and run components independently behind clean interfaces. Event logs are strong enough for fairly strong consistency (durable, ordered) yet general enough for any data.

Unbundled versus integrated systems

Unbundling won't **replace** databases: stream processors need state stores, outputs need serving databases, and specialised engines (e.g. **MPP data warehouse** query engines for exploratory analytics) still matter. Costs: each component has its own learning curve and quirks, so **deploy as few moving parts as possible**; an integrated product may perform **better and more predictably**; building for unneeded scale is premature optimisation. Unbundling aims for **breadth, not depth**. **If one product does everything you need, use it.**

What's missing?

The unbundled equivalent of the **Unix shell** — a high-level declarative language for wiring systems. The author's wish: `mysql | elasticsearch`, an unbundled `CREATE INDEX` that indexes every MySQL document in Elasticsearch and then continuously applies later changes, with no custom glue. An imaginary sketch:

```
# hypothetical "data shell" - not a real product
postgres://shop/products  |  opensearch://catalog-index
#  left side : source of truth; take a snapshot, then follow its change stream
#  "|"       : keep the right side continuously in sync with the left
#  right side: derived view that can be dropped and rebuilt at any time
```

Likewise, **declaring caches** as materialized views over complex (even recursive graph) queries, maintained automatically. Research like **differential dataflow** (efficiently updating results when a small part of the input changes) points this way.

Designing Applications Around Dataflow

Composing specialised stores and processors with app code is the **"database inside-out"** approach (author's 2014 talk) — a design pattern, not a grand new architecture. Influences: dataflow languages **Oz** and **Juttle**, **Elm** (functional reactive programming, FRP), logic language **Bloom**; the term *unbundling* comes from **Jay Kreps**.

Spreadsheets beat most programming languages at dataflow — change an input and dependent formulas recalculate. Data systems still have lessons from **VisiCalc (1979)**, but must also be **fault-tolerant, scalable and durable**, and integrate technologies written by different people over time.

Application code as a derivation function

Derived dataset	Derivation function
Secondary index	Pick indexed values per row/document and sort (B-tree or SSTable, Chapter 3)
Full-text search index	NLP — language detection, word segmentation, stemming or lemmatization, spelling correction, synonyms — then an **inverted index** (word → documents)
ML model	**Feature extraction** and statistical analysis over training data; predictions derive from input + model
Cache	Data aggregated into the UI's shape — UI changes may require redefining and rebuilding it

Secondary indexes are so common databases build them in (`CREATE INDEX`); basic full-text search may be built in, advanced needs tuning; ML **feature engineering** is notoriously app-specific. Non-standard derivations need **custom code**, where databases struggle: **triggers, stored procedures and UDFs** (user-defined functions) are an afterthought.

Separation of application code and state

Databases make poor hosts for modern development needs: dependency/package management, version control, **rolling upgrades**, evolvability, monitoring, network calls, external integration. **Mesos, YARN, Docker and Kubernetes** focus on running code and do it far better. So some parts specialise in **durable storage**, others in **running code**.

Most web apps are **stateless services** (any server handles any request and forgets it) with state in a database — logic out of the database, state out of the app.

> **The joke.** Functional programmers favour keeping "Church and state" apart — *Church* being Alonzo Church, creator of the **lambda calculus**, which has no mutable variables.

The database acts as a **mutable shared variable** accessed synchronously over the network. But most languages **can't subscribe** to a variable's changes (you must hand-code the **observer pattern**), and databases inherited this: you **poll** to detect changes. Change subscriptions are only emerging (Chapter 11, "API support for change streams").

Dataflow: Interplay between state changes and application code

Think of **state, state changes, and code that processes them** collaborating: code reacts to changes in one place by producing changes elsewhere. Precedents: subscribable database change logs (Chapter 11); **actors** and message passing (Chapter 4); 1980s **tuple spaces**; triggers and index updates inside

databases. Unbundling applies this to derived datasets **outside** the database (caches, full-text indexes, ML, analytics).

Maintaining derived data ≠ running background jobs asynchronously (messaging's traditional job):

Requirement	Why	What breaks it
Order of state changes matters	Views from the same log must stay mutually consistent	Brokers that reorder when redelivering unacknowledged messages; dual writes
Fault tolerance is essential	One lost message leaves a view permanently wrong	Actor systems that keep state and messages in memory by default

Stable ordering and fault-tolerant processing are demanding but **much cheaper and more robust than distributed transactions**; modern stream processors provide them at scale and run arbitrary app code as operators. Like Unix pipes, **stream operators compose**: state-change streams in, state-change streams out.

Stream processors and services

Today's **services** communicate via **synchronous** requests (e.g. REST APIs); their advantage over a **monolith** is **organisational scalability via loose coupling** (teams deploy independently). Stream operators resemble microservices but use **one-way, asynchronous message streams**.

Dataflow can also be **faster**. **Book's example:** a purchase priced in one currency, paid in another, needs the exchange rate.

1. **Microservices: query** an exchange-rate service at purchase time.
2. **Dataflow: subscribe in advance** to rate updates, store them in a **local database**, and read locally at purchase time.

The second is **faster** and **more robust** to the other service failing — the fastest, most reliable network request is the one you don't make. The RPC becomes a **stream-table join** of purchases and rate updates.

> **Footnote.** A microservice could cache the rate, but keeping it fresh needs polling or subscribing — i.e. the dataflow approach.

The join is **time-dependent**: reprocessing later needs the **historical** rate at purchase time (Chapter 11, "Time-dependence of joins"). Subscribing to changes moves us toward the spreadsheet model; questions remain, but the author finds it very promising.

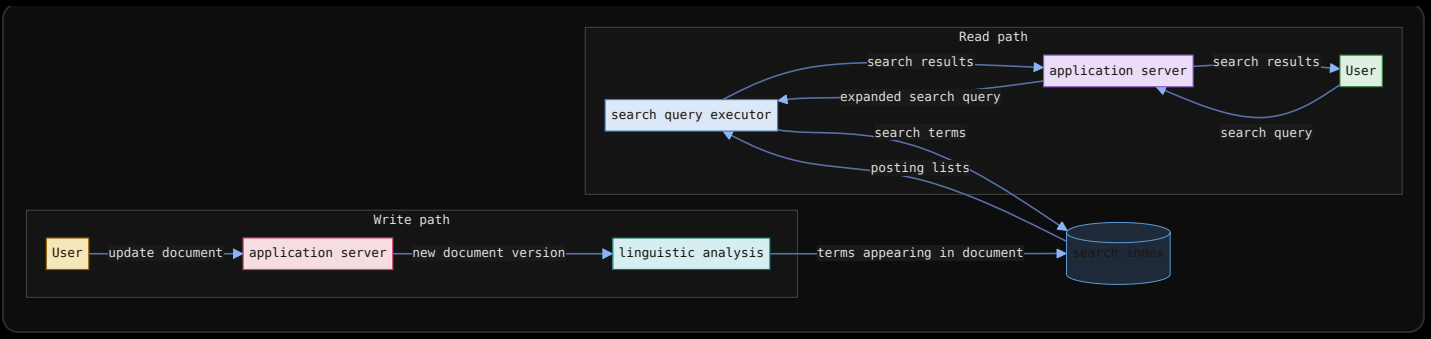
Observing Derived State

The **write path** is the eager process by which written information flows through batch/stream stages into every derived dataset. The **read path** serves queries from those datasets, perhaps with further processing. Together they cover data from collection to consumption.

Write path	Read path
Precomputed **eagerly** on arrival, whether or not anyone asked	Only when someone asks
Like **eager evaluation**	Like **lazy evaluation**

The derived dataset is where they meet, embodying a write-time vs read-time trade-off.

Figure 12-1 (redrawn): in a search index, writes (document updates) and reads (queries) meet at the index. On the write path a new document version goes through linguistic analysis and its terms go into the index; on the read path a query is expanded, its terms are looked up in the index, and the posting lists (for each term, the list of documents containing it) come back as results.



Materialized views and caching

In a full-text index, **writes** update entries per term; **reads** look up query words and apply Boolean AND/OR (e.g. synonyms). Moving the boundary:

Option	Write-path work	Read-path work	Verdict
No index (scan like `grep`)	None	Huge	Only for small collections
Normal index	Update entries per term	Lookups + Boolean logic	Usual balance
Precompute every possible query	Infinite	Trivial	Impossible
Precompute popular queries, index for the rest	Update cached results as matching documents arrive	Fast for common queries	A **materialized view** ("cache of common queries")

> **Footnote.** For a finite corpus, queries with **non-empty** results are finite but grow exponentially with term count — still hopeless.

Caches, indexes and materialized views all shift the boundary: more write-path work to save read-path work. This is Chapter 1's **Twitter** home-timeline example, with the boundary drawn differently for celebrities — after some 500 pages, full circle.

Stateful, offline-capable clients

We assume the **client/server model** (stateless clients, servers own data) too readily. Browsers were stateless and useless offline, but **single-page JavaScript apps** hold UI state and persistent local storage, and **mobile apps** keep much state on-device. This revives **offline-first** apps: use a **local database**, work offline, **sync in the background** — valuable on slow, unreliable mobile networks (Chapter 5, "Clients with offline operation"). Mental model: **on-device state is a cache of server state** — screen pixels are a materialized view of client model objects, which replicate state in a remote datacenter.

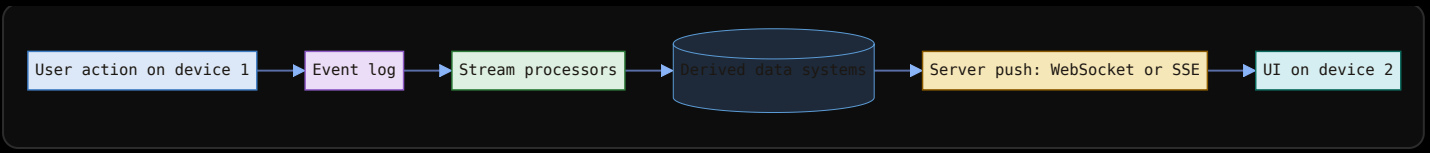
Pushing state changes to clients

A loaded web page doesn't learn of server changes until reload — device state is a **stale cache** unless you poll (RSS feeds are polling too). **Server-sent events** (the **EventSource API**) and **WebSockets** keep a TCP connection open so the server can **push** changes. This **extends the write path to the end user's device**: read initial state once via the read path, then follow a **stream of state changes**. **Offline devices** reconnect and resume from their **offset**, like log consumers (Chapter 11, "Consumer offsets") — each device is a small subscriber to a small stream.

End-to-end event streams

UI tools — **Elm**, Facebook's **React, Flux and Redux** — already manage client state by consuming event streams (user input, server responses), much like event sourcing. Let the **server feed its state-change events into that pipeline** too, giving an **end-to-end write path**:

Illustration (not a book figure): the end-to-end write path from one user's device to another's screen.



Changes could propagate in **under one second**, as in instant messaging and online games ("real-time" meaning low delay, not Chapter 8's hard guarantees). Why not everywhere? Request/response is baked into databases, libraries, frameworks and protocols; few datastores offer "one request, a **stream** of responses" (subscriptions). Moving from **request/response** to **publish/subscribe** is worth it for responsive UIs and offline support — design for **subscribing to changes**, not just querying.

Reads are events too

The derived-data store is where the write path hands over to the read path, allowing random-access lookups instead of reading the whole log. Stream processors already hold state for aggregations and joins; some let outside clients **query that state**. Going further: represent **read requests as events**, send reads and writes through the **same stream processor**, and emit results on an **output stream**. That is a **stream-table join** (queries vs database); reads must be routed to the partition holding the data (Chapter 6, "Request Routing"), just as joins co-partition inputs by key (Chapter 10). A **one-off read** passes through the join and is forgotten; a **subscription** is a **long-lived join** with past and future events.

Benefit: causality and provenance. Logging reads records **what the user saw before deciding** — e.g. the shipping date and stock status shown in a shop influence purchases. It costs storage and I/O (optimising that is open research); if you already log reads, making the log the **source** of requests is a small step.

Multi-partition data processing

Overkill for single-partition queries, but it enables **distributed execution of complex queries** using the stream processor's routing, partitioning and joins:

- **Storm's distributed RPC** computed **how many people have seen a URL on Twitter** — the union of follower sets of everyone who tweeted it, across partitions.
- **Fraud prevention:** scoring a purchase needs reputation lookups for IP, email, billing and shipping addresses — a **chain of joins** over differently partitioned datasets.

MPP databases' internal query graphs look similar; if you need multi-partition joins, such a database is probably **simpler**, but query-as-stream is an option at very large scale.

Aiming for Correctness

A buggy **stateless** service is fixed by a restart; **stateful** systems remember forever, so mistakes may **last forever**. For about **four decades**, **atomicity, isolation and durability** (Chapter 7) have been the tools for correctness, but:

- they're **weaker than they look** (confusing weak isolation levels);
- some areas **abandon transactions** for faster, messier models (leaderless replication, Chapter 5);
- **consistency** is poorly defined (Chapters 7, 9), and "embrace weak consistency" often lacks clarity.

It's hard to know if an app is safe at a given isolation level or replication configuration; simple solutions work at low concurrency and fail under stress. **Kyle Kingsbury's Jepsen** tests (injecting network faults and crashes) exposed gaps between **claimed** and **actual** guarantees; and apps must still **use** infrastructure correctly, which is error-prone with confusing configuration (isolation levels, quorums).

If your app…	Then…
Tolerates occasional unpredictable loss/corruption	Life is easy
Needs stronger correctness	**Serializability** and **atomic commit** work, but usually only **within one datacenter** and **limit scale and fault tolerance**

Transactions aren't going away, but aren't the **final word**; this section explores correctness in **dataflow architectures**.

The End-to-End Argument for Databases

Strong safety properties (e.g. serializable transactions) don't protect against **application bugs** that write wrong data or delete data — hence **immutable, append-only data** (Chapter 11, "State, Streams, and Immutability"), which helps but isn't a cure-all.

Exactly-once execution of an operation

When processing fails, you either **give up** (data loss) or **retry** (risking processing **twice** if the first try actually succeeded). **Processing twice is corruption too** — double charging, double counting. **Exactly-once** (**effectively-once**, Chapter 11) means the **final effect equals a fault-free run**. The best approach is **idempotence**, which may need extra metadata (e.g. the set of operation IDs already applied) and **fencing** (increasing tokens rejecting a node that lost its role) on failover (Chapter 8, "The leader and the lock").

Duplicate suppression

TCP uses packet sequence numbers to reorder and drop duplicates — but **only within one connection**. Databases tie transactions to connections; if the client sends `COMMIT` and the connection times out before the reply, it **doesn't know** whether it committed (Chapter 8, Figure 8-1).

Our own version of Example 12-1 (the book moves $11.00 between accounts 4321 and 1234; values differ here):

```
BEGIN TRANSACTION;                                    -- start one atomic unit of work
UPDATE wallets SET funds = funds + 25.00 WHERE wallet_id = 77;  -- credit the receiving wallet
UPDATE wallets SET funds = funds - 25.00 WHERE wallet_id = 88;  -- debit the paying wallet
COMMIT;                                               -- make both changes permanent together
```

"Add 25" is **not idempotent**. If the `COMMIT` reply is lost and the client retries on a new connection, TCP can't recognise it — in the book's example **$22 could move instead of $11**. So the textbook atomicity example is **not correct**, and **real banks don't work this way**.

Does 2PC help? It decouples transactions from connections (the coordinator reconnects and resolves in-doubt transactions), but **no**: duplicates also arise between the **end user's device** and the **application server**. A user on weak mobile signal submits an HTTP **POST**; the response is lost; they retry and confirm the browser's resubmission warning. (**Post/Redirect/Get** — redirecting after a POST so refresh doesn't resend — avoids the warning normally but not after a timeout.) The retry is a **new request** and a **new transaction**; usual dedup misses it.

Operation identifiers

For idempotence across **several network hops**, consider the request's **whole journey**: generate a **unique operation ID** in the client — a **UUID** (128-bit random ID) in a hidden form field, or a **hash of the form fields** — so resubmissions carry the **same ID**, and pass it to the database, ensuring **one** execution per ID.

Our own version of Example 12-2:

```
ALTER TABLE transfer_log ADD UNIQUE (op_id);   -- the database will reject a second row with the same op_id

BEGIN TRANSACTION;
INSERT INTO transfer_log (op_id, payer, payee, amount)
  VALUES ('9b1f2c4e-5a6d-4e7f-8a90-b1c2d3e4f501', 88, 77, 25.00);  -- record the request; fails if op_id already exists
UPDATE wallets SET funds = funds + 25.00 WHERE wallet_id = 77;     -- credit
UPDATE wallets SET funds = funds - 25.00 WHERE wallet_id = 88;     -- debit
COMMIT;
```

- The `UNIQUE` constraint makes a retry's `INSERT` fail, **aborting** the whole transaction.
- Databases usually enforce uniqueness correctly **even under weak isolation**, whereas app-level "check then insert" can fail (Chapter 7, write skew and phantoms).
- The requests table is effectively an **event log**: balance updates could be **derived** downstream from the request event, processed exactly once via the same ID.

The end-to-end argument

Formulated by **Saltzer, Reed and Clark in 1984**, paraphrased: *a function can be implemented completely and correctly only with the help of the applications at both ends; the communication system can't provide it fully on its own, though a partial version can improve performance.* TCP dedups **packets**, some stream processors give exactly-once **message processing**, but neither stops a **user** resubmitting — you need an **end-to-end** transaction ID from client to database.

Concern	Low-level mechanism	Misses	End-to-end fix
Data integrity	Checksums in **Ethernet, TCP and TLS**	Software bugs at sender/receiver; disk corruption	**End-to-end checksums**
Encryption	Home **WiFi password**	Attackers elsewhere on the internet	**End-to-end encryption and authentication**
Encryption	**TLS/SSL** (the encryption behind HTTPS) client–server	A compromised server	End-to-end encryption and authentication

Low-level mechanisms remain **useful** (HTTP would often be garbled without TCP) — just **not sufficient**.

Applying end-to-end thinking in data systems

The application must take end-to-end measures like duplicate suppression. Ideally that machinery would be hidden in an **abstraction**, but we **haven't found it yet**. **Transactions** reduce many problems (crashes, disk failures, concurrent writes, constraint violations, network interruptions) to **commit or abort** — a huge simplification but not enough, and **expensive**, especially across **heterogeneous** stores (Chapter 9). Avoiding them pushes fault tolerance into **application code**, where the author suspects most mechanisms **don't work correctly**. We need abstractions that make **application-specific end-to-end correctness** easy and perform well at scale.

Enforcing Constraints

Within the unbundled approach, a request ID flowing to the database gives end-to-end dedup. What about other constraints, especially **uniqueness** (Chapter 9): one user per username/email, one file per name, one buyer per seat? Similar: balance never negative, don't oversell stock, no overlapping meeting-room bookings.

Uniqueness constraints require consensus

Concurrent requests for the same value need a winner — **consensus** (Chapter 9).

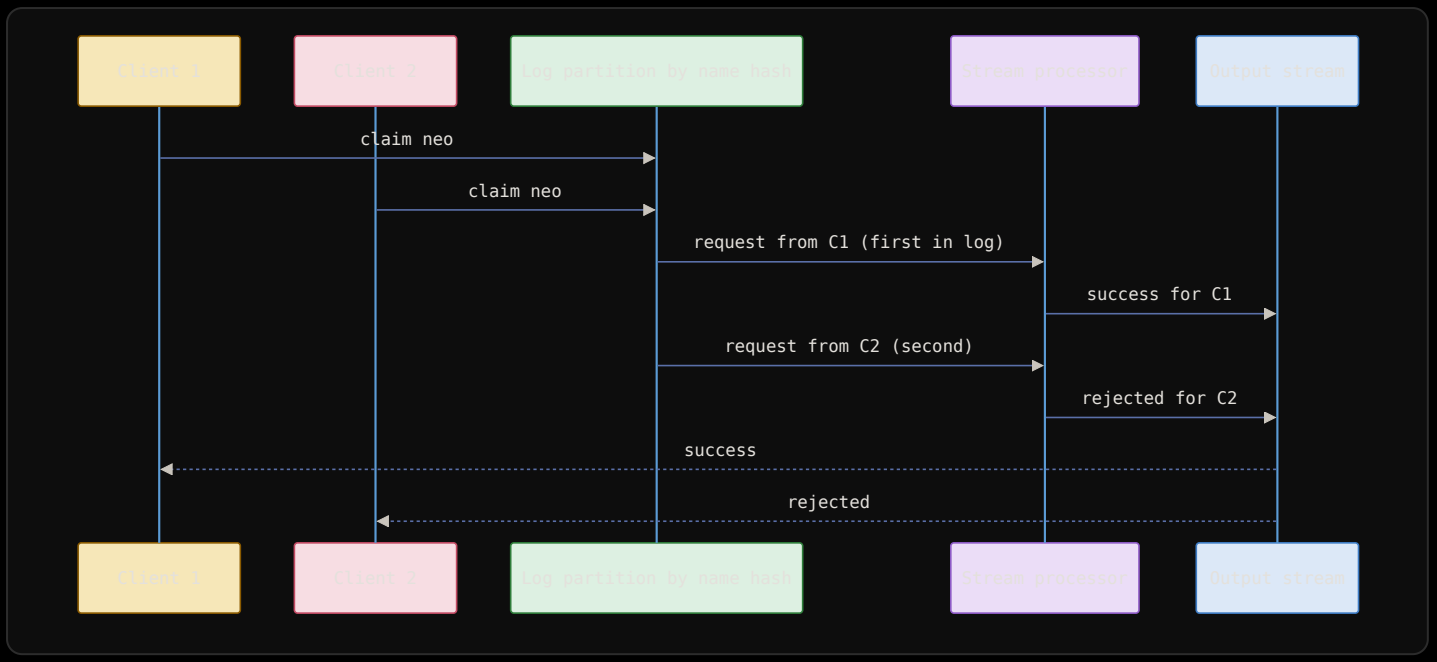
- **Common method:** a single **leader** decides — fine if you accept routing through one node; surviving its failure brings back consensus.
- **Scale out** by partitioning on the unique value: same request ID → same partition; usernames by **hash of username**.
- **Asynchronous multi-master replication** is **ruled out** (masters could accept conflicting writes).
- **Immediately rejecting** violations requires **synchronous coordination**.

Uniqueness in log-based messaging

A log gives all consumers an **identical order** — **total order broadcast**, equivalent to consensus. A processor reads a partition **sequentially on one thread**, so partitioning by the unique value lets it decide which conflicting operation came **first**. **Usernames:**

1. Each request is appended to a partition chosen by **hash of the username**.
2. A processor tracks taken names in a **local database**, emitting **success** or **rejection** to an output stream.
3. The client **watches the output stream** for its result.

Illustration (not a book figure): two clients race for the same username; the log order decides the winner.



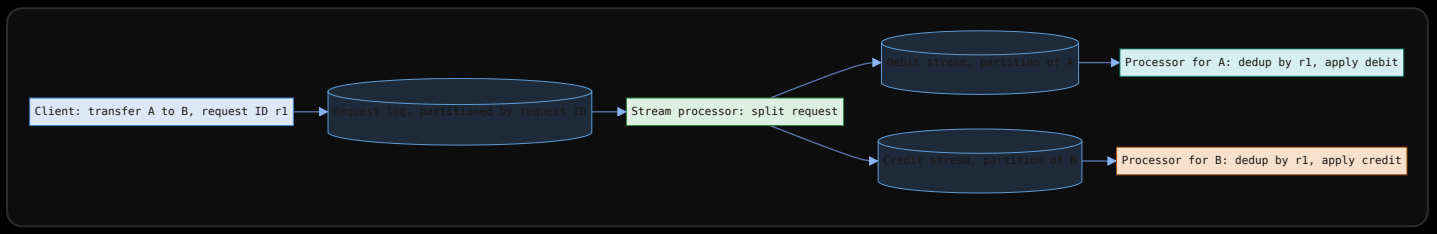
This is Chapter 9's "implementing linearizable storage using total order broadcast", and it **scales** by adding partitions. The principle — **route potentially conflicting writes to the same partition and process them sequentially** — works for many constraints with **arbitrary validation logic** (what counts as a conflict: Chapter 5 "What is a conflict?", Chapter 7 write skew). **Bayou** (Xerox PARC, replicated data on mobile devices) pioneered this in the **1990s**.

Multi-partition request processing

Example 12-2 may touch **three** partitions: the **request ID**'s, the **payee**'s and the **payer**'s. **Traditionally**, an **atomic commit across all three** forces a total order with everything else on them, hurting **throughput**. **Log-based, without atomic commit:**

1. The client gives the transfer (A → B) a **unique request ID** and appends it to a partition chosen **by request ID**.
2. A processor emits a **debit instruction** for A (partitioned by A) and a **credit instruction** for B (by B), both with the request ID.
3. Further processors **dedupe by request ID** and apply the balance changes.

Illustration (not a book figure): a money transfer split into two differently partitioned stages, no atomic commit needed.



Sending debit and credit directly would need a two-partition atomic commit; instead the request is **logged durably as one message** — **single-object writes are atomic** almost everywhere. If the step-2 processor crashes it resumes from its **checkpoint**, possibly re-emitting instructions, but being **deterministic** it emits the **same** ones, which step 3 drops via the request ID. **Overdraft prevention:** a processor partitioned by **payer account** validates transfers before they enter the step-1 log. Result: **two differently partitioned stages** plus an end-to-end request ID give **exactly-once** application to payer and payee **without atomic commit** (cf. event-sourcing concurrency control, Chapter 11).

Timeliness and Integrity

Transactions are usually **linearizable**: once committed, **every reader sees** the writes. Unbundled stages are **asynchronous**, though a client **can** wait on an output stream (as in the username check). There, **correctness doesn't depend on waiting** — waiting only notifies the sender; it is **separable** from the processing.

Consistency conflates two requirements:

	Timeliness	**Integrity**
Meaning	Users see the **latest** state	**No corruption**: no loss, no contradictory or false data; derived data correctly reflects its source
Violation example	Stale replica read (Chapter 5, replication lag)	An index missing records
Nature	**Temporary** — wait and retry	**Permanent** — needs explicit checking and repair
Related	CAP "consistency" = **linearizability** (strong timeliness); weaker **read-after-write** consistency also useful	ACID "consistency" ≈ app-specific integrity; **atomicity and durability** protect it
Slogan	**"Eventual consistency"**	**"Perpetual inconsistency"**

Integrity usually matters much more: timeliness violations annoy, integrity violations can be **catastrophic**. **Credit card example:** a purchase from the **last 24 hours** missing from your statement is normal (banks reconcile **asynchronously**); but a balance not equal to **previous balance plus transactions**, or being **charged while the merchant isn't paid**, are integrity violations.

Correctness of dataflow systems

ACID gives **both** (linearizability for timeliness, atomic commit for integrity). **Event-based dataflow decouples them: no timeliness guarantee** unless consumers wait, but **integrity is central** — exactly-once exists to preserve it, so **reliable delivery** and **duplicate dropping** (idempotence) are essential. Integrity without distributed transactions:

1. Represent the **write as a single message** (atomic; fits event sourcing).
2. **Derive all other updates** with **deterministic functions**, like stored procedures (Chapter 7, "Actual Serial Execution").
3. Pass a **client-generated request ID** through every stage for end-to-end dedup.
4. Keep messages **immutable** and **recompute** derived data periodically to recover from bugs.

Loosely interpreted constraints

A **traditional** uniqueness constraint needs consensus (one node per partition) — unavoidable, even with streams. But many apps accept **weaker** constraints:

Situation	Handled after the fact by
Two people take the same username or seat	Apologise and ask one to choose again — a **compensating transaction**
More items ordered than in stock	Reorder, apologise, discount — the same process needed if a **forklift truck destroyed** stock, so a linearizable stock constraint may be unnecessary
Airlines **overbook** flights; hotels overbook rooms	Deliberately broken; compensation (refunds, upgrades, a room at a neighbouring hotel); apologies needed anyway for **bad weather or strikes**
Withdrawing more than you have	**Overdraft fee** and repayment; a **daily withdrawal limit** bounds risk

Apologies are often cheap: send a correction email; refund a double charge (fees, maybe a complaint); cash from an **ATM** can't be recalled, though debt collectors could pursue an unrepaid overdraft. Whether the cost is acceptable is a **business call**; if so, **write optimistically and check afterwards** (but still validate before **expensive-to-undo** actions). These apps need **integrity** (no lost reservations, no vanishing money) but not **timeliness** of constraint enforcement — like Chapter 5's conflict resolution ("Handling Write Conflicts").

Coordination-avoiding data systems

1. Dataflow systems can guarantee derived-data integrity **without atomic commit, linearizability or synchronous cross-partition coordination**.
2. Many apps accept **loose constraints** temporarily violated and fixed later, **as long as integrity holds**.

Together these give **coordination-avoiding** systems with **strong integrity** that outperform and outlast coordinated ones. E.g. **multi-datacenter**, **multi-leader**, **asynchronous** cross-region replication: weak timeliness (no linearizability without coordination) but strong integrity. **Serializable transactions** still help **within a limited scope**; **XA** isn't required; synchronous coordination can be added **only where needed** (e.g. before irreversible operations).

The apology trade-off: coordination **reduces apologies for inconsistency** but may **increase apologies for outages**; aim for the balance where both are acceptably rare.

Trust, but Verify

Our **system model** (Chapter 8) assumes crashes, power loss and arbitrary network delay/loss can happen, but perhaps not data loss after **fsync**, memory corruption, or a wrong CPU **multiplication instruction**. Reasonable, but models are **binary** while reality is **probabilistic**: do violations happen **often enough to matter**? Data rots on disk (Chapter 7); network corruption can **slip past TCP checksums** (Chapter 8). **Anecdote:** crash reports in an app the author worked on made sense only as **random memory bit-flips** — rare per device, inevitable across many. Besides **radiation** and faulty hardware, the **rowhammer** effect flips bits **in healthy memory** via pathological access patterns, breaking OS security. Such flips remain **very rare** on modern hardware — just not impossible.

Maintaining integrity in the face of software bugs

Checksums can't catch software bugs. The author has seen **MySQL** fail to uphold a **uniqueness constraint** and **PostgreSQL's serializable isolation** exhibit **write skew** — in robust, battle-tested databases; less mature software is likely worse. **Application code** has far more bugs and often misuses integrity features (**foreign keys**, **uniqueness constraints**). **ACID consistency** assumes **bug-free transactions**; misuse (e.g. unsafe weak isolation) voids it.

Don't just blindly trust what they promise

Corruption is **inevitable sooner or later**, so **detect**, fix and trace it — **auditing**. Crucial in finance (Chapter 11, "Advantages of immutable events"), useful everywhere. **HDFS** and **Amazon S3 don't fully trust disks**: background processes **read back files, compare replicas and move files** to fight silent corruption. **Read your data to check it's there**, and **test restoring backups**.

A culture of verification

HDFS and S3 assume disks **usually** work, not **always**. Few systems have this **"trust, but verify"** attitude; the author wants more **self-validating / self-auditing** systems. He fears the **ACID culture** encouraged **blind trust**, neglecting auditability; then **NoSQL** normalised **weaker consistency** and **less mature storage** without audit mechanisms — blind trust became more dangerous.

Designing for auditability

A multi-object transaction's **meaning** is hard to recover later; even CDC logs show inserts/updates/deletes, not **why** — the deciding app invocation is **transient**. **Event-based systems are more auditable:** user input is a **single immutable event** and state is **derived deterministically and repeatably**. Explicit dataflow (Chapter 10) gives clear **provenance**, enabling checks: **hash** the event log; **rerun** derivations and compare, or run a **redundant derivation side by side**. Deterministic dataflow also enables **time-travel debugging** — recreating the exact conditions behind a surprise.

The end-to-end argument again

Without fully trustworthy components, **periodically check integrity**, or corruption surfaces only after downstream damage. Checks work best **end to end**: verifying a whole pipeline **implicitly checks** every disk, network, service and algorithm in it. Like automated tests, continuous auditing gives **confidence** to **move faster** and adopt new technology.

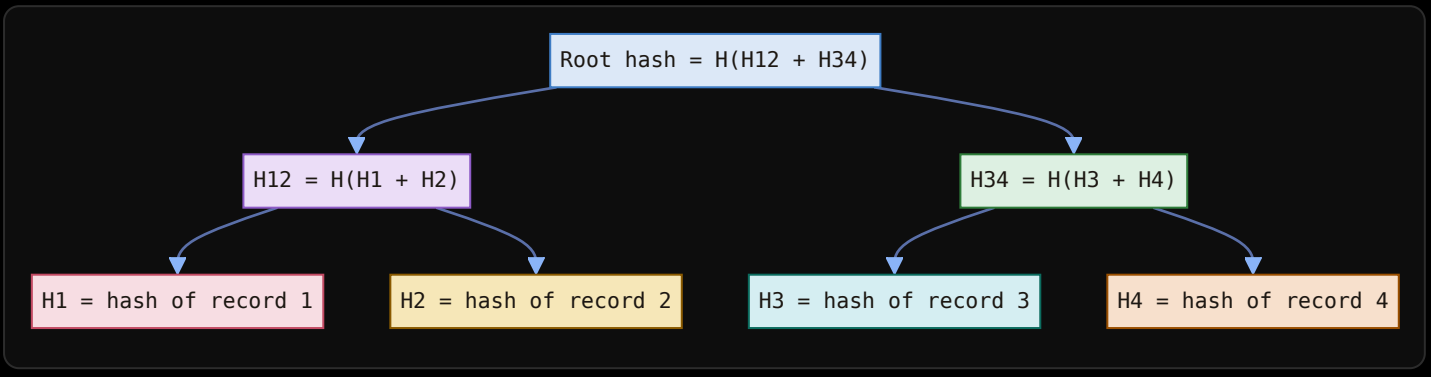
Tools for auditable data systems

Auditability is **first-class** in few systems. Apps write **audit tables**, but can't easily verify those or the database are intact. Signing a log with a **hardware security module** makes it **tamper-proof** but doesn't prove the **right** transactions were written.

Cryptographic tools could prove integrity even against **malicious** actions: **cryptocurrencies, blockchains and distributed ledger technologies** — **Bitcoin, Ethereum, Ripple, Stellar**. The author can't judge them as currencies, but as data systems they're **distributed databases** whose replicas belong to **mutually distrusting organisations**, constantly verifying each other and running **consensus**. He's **somewhat skeptical** of their **Byzantine fault tolerance** aspects and finds **proof of work** (mining puzzles) **hugely wasteful**; Bitcoin's throughput is **rather low** (for political/economic more than technical reasons). The **integrity-checking** is the interesting part.

Merkle trees — **trees of hashes** efficiently proving a record is in a dataset — underpin this. **Certificate transparency** uses them to check **TLS/SSL certificates**.

Illustration (not a book figure): a Merkle tree over four records. To prove record 3 is included you only need H4 and H12 to recompute the root.



The author expects such auditing algorithms to spread to general data systems, once they **scale** comparably and their **performance cost** is small.

Doing the Right Thing

The book's final section addresses what it left out: **ethics**.

- Every system has a **purpose** and **intended and unintended consequences**; engineers are **responsible** for thinking them through.
- Many datasets are **about people**; handle them with **humanity and respect — human dignity comes first**.
- Guidelines exist — the ACM's (Association for Computing Machinery) **Software Engineering Code of Ethics and Professional Practice** — but are rarely discussed or enforced, so engineers are sometimes **very cavalier** about privacy and harm.
- **Technology is neither good nor evil by itself** — what matters is its use and effects (true of a search engine as of a gun). Engineers **share** the responsibility.

Predictive Analytics

A big part of the "Big Data" hype. Forecasting **weather** or **disease spread** is harmless; predicting whether a **convict will reoffend**, a **loan applicant will default**, or an **insurance customer will make expensive claims directly affects lives**. Organisations avoid risk — stopping **fraudulent transactions**, **bad loans**, **hijackings**, **ineffective or untrustworthy** hires — and since a missed opportunity costs little, **when in doubt they say no**. As algorithmic decisions spread, someone labelled risky (**rightly or wrongly**) may get **many** "no"s — exclusion from **jobs, air travel, insurance, property rental, financial services** — called **"algorithmic prison"**. Justice **presumes innocence**; automated systems can exclude **systematically and arbitrarily**, without proof and with little **chance to appeal**.

Bias and discrimination

Humans are biased too, and discrimination can be **culturally institutionalised**; data-driven decisions might be **fairer** and help the **overlooked**. But predictive analytics **infers the rules from data**, yielding **opaque** patterns; **systematically biased** input is **learned and amplified**.

- **Anti-discrimination laws** protect traits like **ethnicity, age, gender, sexuality, disability, beliefs**.
- Other data can **correlate** with them: in **racially segregated neighbourhoods**, race is predictable from **postal code** or even **IP address**.
- Expecting **fair, impartial output** from **biased data** is absurd — machine learning as **"money laundering for bias"**.
- Models **extrapolate from the past** and **codify** past discrimination; a better future needs **moral imagination**, which only humans have. **Data and models should serve us, not rule us.**

Responsibility and accountability

Humans who err can be held accountable and appealed against; algorithms? Who's to blame when a **self-driving car** crashes? Can people disadvantaged by **credit scoring** by **race or religion** get redress? Can you explain an ML decision under **judicial review**?

Credit rating agencies at least use **relevant facts about actual borrowing history**, and errors can be corrected (with difficulty). **ML-based scoring** uses **far wider inputs** and is **far more opaque**.

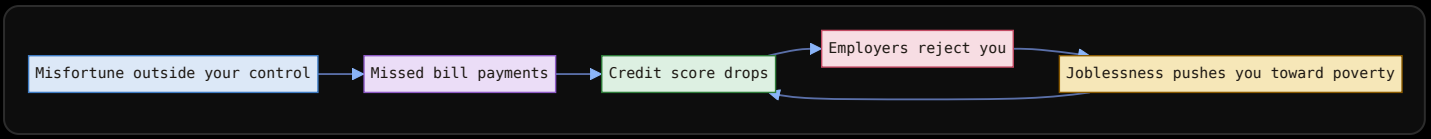
Traditional credit score	Predictive analytics
Your own past behaviour	**Who resembles you** and how they behaved
Your history	**Stereotyping**, e.g. by where you live (proxy for race and class)

People put in the **wrong bucket**, or judged on **wrong data**, have almost no recourse. Data is **statistical**: with average life expectancy **80 years**, you're not expected to die on your **80th birthday** — predictions can be wrong for individuals. **Blind faith in data is delusional and dangerous**: we need algorithms that are **transparent and accountable**, that **don't entrench biases**, and ways to **correct** their errors. Analytics can **target aid** to those in need — or help **predatory businesses** sell vulnerable people **high-cost loans** and **worthless college degrees**.

Feedback loops

Recommendation systems can show people **only what they already agree with** — **echo chambers** breeding **stereotypes, misinformation and polarisation**, already affecting **election campaigns**. **Self-reinforcing feedback loops** are worst — e.g. **employers using credit scores**:

Illustration (not a book figure): the credit-score feedback loop described in the book.



A capable employee hit by misfortune enters a **downward spiral** driven by toxic assumptions **disguised as mathematical rigour**. **Systems thinking** — considering computers **and the humans** interacting with them — helps foresee such effects: does the system **reinforce** differences (**rich get richer, poor get poorer**) or **combat injustice**? Beware **unintended consequences**.

Privacy and Tracking

Collecting data itself raises issues. If a system stores what a user **explicitly entered**, it **serves the user** (the **customer**). If behaviour is **recorded incidentally**, the service develops **its own interests**, which may **clash** with the user's. Tracking powers useful features — click-based ranking, "people who liked X also liked Y", **A/B tests**, **user flow analysis** — that **benefit users**. But with an **advertising** business model, **advertisers are the real customers**: tracking deepens, data is kept longer, **detailed per-person profiles** are built, and users are **nudged to engage** with a **free service**. A more sinister word fits: **surveillance**.

Surveillance

Thought experiment: replace *data* with *surveillance* — "Our surveillance-driven company gathers real-time surveillance streams into a surveillance warehouse…". The author jokingly retitles the book **"Designing Surveillance-Intensive Applications"**:

- Making software **"eat the world"** built the **largest mass surveillance infrastructure** in history.
- With the **Internet of Things**, every inhabited space may hold an **internet-connected microphone** — phones, smart TVs, voice assistants, baby monitors, **children's toys** using cloud speech recognition — many with **terrible security**.
- **Totalitarian regimes** could only dream of a microphone in every room and a tracker in every pocket; we accept it **voluntarily, even enthusiastically** — from **corporations** rather than governments.

Why accept it? You may have **"nothing to hide"** — i.e. you fit existing power structures and needn't fear persecution; not everyone is so lucky. Or it seems **benign** (recommendations, marketing) — but with predictive analytics the line blurs: **car insurance** tied to **in-car trackers**, **health insurance** requiring a **fitness tracker**, decisions on **insurance or employment**; a **smartwatch's movement sensor** can infer what you're **typing (e.g. passwords)**.

Consent and freedom of choice

"Users chose the service and accepted the **privacy policy and terms of service**, so they **consent**, and tracking pays for a valuable service." Problems:

1. **Users don't know** what's collected or how it's processed; policies **obscure more than they explain** — **no meaningful consent**.
2. One user's data reveals **other people** who never agreed.
3. **Derived datasets** combining the whole user base with tracking and external data are beyond users' understanding.
4. Extraction is **one-way**, not a fair exchange — **no negotiation**; the relationship is **asymmetric** and the **service sets the terms**.

Opting out isn't a free choice when a service is **necessary for social life** — **de facto mandatory** (smartphone, **Facebook**, **Google**), with **network effects** making non-use socially costly. Declining is possible only for the **privileged few** with time, expertise and opportunities to spare; for the **less privileged**, **surveillance is inescapable**.

Privacy and use of data

"Privacy is dead" is **false**. Privacy isn't secrecy — it's the **freedom to decide** what to disclose, to whom — a **decision right** on the spectrum from **secrecy** to **transparency**, central to **freedom and autonomy**. Surveillance **transfers** that right **to the data collector**, who asks to be **trusted** and keeps much of what

it learns **secret** (revealing it would seem **creepy** and harm a business built on knowing more than competitors). Intimate facts surface **indirectly**, e.g. ad targeting of **people with a particular illness** — even without **re-identification**, people have **lost agency** over disclosure; the **company exercises the privacy right to maximise profit**.

- Companies manage **perceptions** of creepiness rather than actual intrusiveness — and even then badly: **factually correct** content can **trigger painful memories**.
- Expect data to be **wrong, undesirable or inappropriate**; the latter two need **human judgment** algorithms lack — build **mechanisms for these failures** and be **humble**.
- **Privacy settings** limit what **other users** see, but the **service** keeps **unrestricted access**; even promising not to **sell to third parties**, it reserves **unlimited internal processing rights**.

This transfer of privacy rights is **unprecedented in scale**: surveillance used to be **costly and manual**, not **automated and scalable**; trust relationships (doctor, lawyer) were constrained by **ethics, law and regulation**. Internet services amass **sensitive data without meaningful consent** at scale.

Data as assets and power

Behavioural data is called **"data exhaust"**, implying worthless waste. Rather, if **targeted advertising** pays, **behavioural data is the core asset** and the app merely **lures users** to feed the surveillance machine, **cynically exploiting** their creativity and relationships. Evidence: **data brokers** (a **shady, secretive industry** buying, aggregating, inferring and reselling data, mostly for **marketing**); startups valued by users ("**eyeballs**") — i.e. surveillance capacity.

Valuable data attracts **companies**, **governments** (secret deals, coercion, legal compulsion, theft), buyers in **bankruptcy** sales, and **breaches**. Hence data as a **"toxic asset"** or **"hazardous material"**: weigh benefits against it reaching the wrong people — **criminal or foreign-spy break-ins**, **insider leaks**, **unscrupulous future management**, or a **regime takeover** demanding the data. Consider **all possible future governments**; deploying technology that could enable a **police state** is bad civic hygiene. **"Knowledge is power"**, and watching while unwatched is a major power — why **totalitarian governments want surveillance**. Tech companies' data gives them **considerable, largely hidden power over our lives**, outside public oversight.

Remembering the Industrial Revolution

Data defines the **information age**, reshaping the **global economy and society** like the **Industrial Revolution**:

Industrial Revolution	
Brought	Technological and agricultural advances, sustained growth, much better long-run **living standards**
Also brought	**Air** and **water pollution**; owners in splendour while workers had bad housing and harsh hours; common **child labour**, including in **mines**
Eventually fixed by	**Environmental protection**, **workplace safety**, **outlawing child labour**, **food health inspections**
Cost/benefit	Higher business costs, but huge social benefit — few would go back

Bruce Schneier: *data is the information age's pollution problem, and protecting privacy is its environmental challenge* — how we contain and dispose of lingering data is central; our grandchildren will judge us as we judge those who ignored pollution.

Legislation and self-regulation

The **1995 European Data Protection Directive** requires collection for **specified, explicit and legitimate purposes**, no **incompatible** later processing, and data **adequate, relevant and not excessive**. This clashes with the **Big Data philosophy** of maximal collection, combination and exploration for **unforeseen purposes**; **updated regulations are being developed**. Industry **opposes regulation** as anti-innovation — **partly justified**: sharing **medical data** risks privacy but could save lives through better diagnostics; **over-regulation** could block breakthroughs.

The author calls for a **culture shift**: treat users not as **metrics to optimise** but as **humans deserving respect, dignity and agency**; **self-regulate** to earn **trust**; **teach users** what happens to their data; let people keep their **privacy**. Individual data control is like a **national park** — without protection, a **tragedy of the commons**. **Ubiquitous surveillance is not inevitable.** Starting points:

1. **Don't keep data forever** — **purge** it when unneeded (conflicts with **immutability**, Chapter 11, but solvable).
2. Enforce **access control through cryptographic protocols**, not just policy.
3. **Culture and attitude change** overall.

Chapter summary

- **No single tool serves every use case**, so apps **compose** tools — the **data integration** problem, solved by flowing changes through **batch processing and event streams**.
- Designate **systems of record** and **derive** everything else (indexes, views, ML models, summaries); **asynchronous, loosely coupled** derivation contains faults.
- Dataflows as **transformations** aid **evolution**: rerun new code over all input; **fix the code and reprocess** after bugs; derived views allow **gradual, reversible migrations**; unified batch/stream engines avoid the **lambda architecture**'s duplication.
- **Unbundling** takes a database's components apart and composes them (federation unifies reads; unbundling unifies writes); a high-level composition "shell" is still missing.
- **Derived state can be observed** by subscribing, all the way to **end-user devices** (dynamic, **offline**-capable UIs). Caches, indexes and views **move the write-path/read-path boundary**; reads can be events too.
- **Correctness:** strong **integrity** at scale via asynchronous processing, **end-to-end operation IDs** for idempotence, and **asynchronously checked constraints** — clients **wait** or **proceed and risk apologising**. More scalable and robust than distributed transactions, and closer to how businesses work.
- Separate **timeliness** (temporary) from **integrity** (permanent); integrity matters more. Dataflow lets us **avoid most coordination** while keeping integrity.
- **Audits** detect corruption: trust, but verify, end to end.
- **Ethics:** data can cause **significant harm** — hard-to-appeal life-affecting decisions, **discrimination and exploitation**, normalised **surveillance**, exposed **intimate information**, **breaches**, **unintended consequences**. Engineers must work toward treating people with **humanity and respect**.

Interview angle

1. How do you keep a database, a search index and a cache consistent? *30-second answer:* No dual writes. Make one store the system of record, capture its changes in order (CDC or event-sourcing log), and have each derived system apply them in that order with idempotent, deterministic consumers. A single total order prevents permanent divergence; replay rebuilds any derived store.

- *Follow-up: Why not XA/2PC?* — Poor fault tolerance and performance; one failure aborts all and amplifies failures; no standard protocol across heterogeneous systems by different teams. Logs with idempotent consumers are simpler and contain faults.
- *Follow-up: What do you give up?* — Timeliness: asynchronous updates, no read-your-writes by default; clients can wait for changes if needed.

2. Lambda architecture vs a unified stream/batch engine? *30-second answer:* Lambda runs batch (Hadoop MapReduce) and speed (Storm) layers over the same immutable events: fast approximate results corrected later. Problems: logic maintained twice, merging outputs (hard for joins/sessionization), expensive full reprocessing making batch incremental and complex. Unified systems (Flink, Apache Beam on Flink/Cloud Dataflow) use replayable logs, exactly-once and event-time windows.

- *Follow-up: Why event time?* — During replay, processing time is just "now".

3. How do you make a payment request exactly-once end to end? *30-second answer:* Client-generated operation ID (UUID or form-field hash) passed through every hop, with a uniqueness constraint where the

effect is recorded; retries are rejected. TCP dedup, DB transactions and stream-processor exactly-once each cover only their hop — the end-to-end argument (Saltzer, Reed, Clark, 1984).

- *Follow-up: Does 2PC solve it?* — No; it handles coordinator–database reconnection, not a user resubmitting a timed-out POST.

4. How can you enforce uniqueness (e.g. usernames) in a log-based system? *30-second answer:* Partition the request log by username hash; a single-threaded processor per partition checks a local table and emits success/rejection; the client waits on the output. Total order broadcast per partition — still consensus, but it scales by adding partitions.

- *Follow-up: Multi-partition transfer without atomic commit?* — Log the request as one message, deterministically split it into debit/credit messages with the request ID, dedupe per account.

5. Timeliness vs integrity — which matters more? *30-second answer:* Timeliness violations are temporary ("eventual consistency"); integrity violations (loss, duplication, contradiction) are permanent ("perpetual inconsistency"). Integrity matters far more — a lagging card statement is fine, vanishing money isn't. Dataflow gives up guaranteed timeliness but keeps integrity via exactly-once and idempotence.

- *Follow-up: Compensating transactions?* — Fix-ups after a loosely enforced constraint is violated (apology and rebooking, refund, overdraft fee); acceptability is a business decision.

6. Why audit data if the database is ACID? *30-second answer:* Hardware corrupts data (bit-flips, rowhammer, disk rot, errors past TCP checksums) and software has bugs — even MySQL (uniqueness) and PostgreSQL (write skew under serializable). ACID assumes bug-free transactions. HDFS and S3 re-read and compare replicas. Deterministic event-sourced dataflow enables end-to-end checks (hash the log, rerun derivations).

Self-check

1. **Why is it dangerous for an app to write directly to both a database and a search index?** Concurrent writes may apply in different orders and neither store controls ordering, so they diverge permanently. Write once to the system of record and derive the index from its ordered change log.
2. **Four situations where a single total order becomes hard?** Throughput beyond one leader (partitioned logs), multiple datacenters with separate leaders, microservices with separate state, and clients with local/offline state.
3. **Unfriend-then-message: what went wrong; remedies?** The notification service processed the message before the unfriend (different systems), notifying the ex-partner. Remedies: logical timestamps, logging what the user saw (with an ID later events reference), or conflict resolution — useless for external side effects.
4. **What does `CREATE INDEX` share with adding a follower replica?** Snapshot, process it, catch up on the backlog, then follow ongoing changes — deriving a new view by reprocessing existing data.
5. **When should you NOT unbundle?** When one product meets all needs; more moving parts mean learning curves and operational quirks, and integrated products may perform more predictably. Unbundling is breadth, not depth.
6. **Write path vs read path, and how does a cache move the boundary?** Eager work on arrival vs lazy work at query time, meeting at the derived dataset. A cache precomputes more on the write path to save read-path work; no index does the opposite.
7. **Scenario: a flaky-network user retries a payment. Prevent double charging?** Client-generated operation ID (UUID in a hidden field) passed to the database with a unique constraint, so the second insert fails and the transaction aborts.
8. **Why might an airline deliberately violate "one person per seat"?** Overbooking expects no-shows; compensation handles excess demand, and apology processes exist anyway (weather, strikes).
9. **What is "algorithmic prison", and how does predictive analytics differ from a credit score?** Systematic exclusion from jobs, travel, insurance, rentals and finance due to a risk label, often without proof or appeal. Credit scores use your own history; predictive analytics judges you by similar people — stereotyping and opaque.

10. **What does privacy mean, and why is "privacy is dead" wrong?** The freedom to decide what to reveal to whom — a decision right, not secrecy. Public sharing exercises that right; surveillance transfers it to the collector.

Glossary

Book: Glossary, pp. 553–558

The book's glossary is deliberately brief — the core idea of each term, with details in the main text. The first table covers **every** book glossary entry in plain English; the A–Z section covers every other term in these notes. The "Ch" column links to the chapter notes (section in brackets). The book's Index is not reproduced.

Chapters: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12. "Part II intro" is summarised at the start of Chapter 5, "Part III intro" at the start of Chapter 10.

> **New to a word?** Every term, acronym, product and tool used in these notes — from *API*, *JSON* and *SSD* to *MVCC*, *Raft* and *write skew* — is defined once, below or in Every other term used in these notes. Networking basics (latency, caching, load balancers, queues, TCP, DNS) are covered in depth in System Design Fundamentals.

Term	Plain-English meaning	Ch
asynchronous	Not waiting for something (e.g. a network message) to finish, and assuming nothing about how long it takes.	5 (sync vs async replication), 8 (sync vs async networks; system models)
atomic	(1) Concurrency: an operation appears to happen at a single instant, so no one sees it half-done (cf. *isolation*). (2) Transactions: a group of writes is committed or rolled back as a whole, even under faults.	7 (Atomicity), 9 (atomic commit, 2PC)
backpressure	Forcing a producer to slow down because the receiver can't keep up; also called *flow control*.	11 (Messaging Systems)
batch process	A job that reads a fixed, usually large, input and writes a new dataset, leaving the input unchanged.	10
bounded	Having a known upper limit or size — said of network delays and datasets.	8 (timeouts, unbounded delays), 11 (intro)
Byzantine fault	A node misbehaving arbitrarily — e.g. sending contradictory or malicious messages.	8 (Byzantine Faults)
cache	Keeps recently used data for faster repeat reads; usually partial, so misses fall back to a slower complete store.	General term; e.g. 1, 12
CAP theorem	A widely misunderstood theoretical result that, in the book's view, isn't useful in practice.	9 (The CAP theorem)
causality	The dependency when one event "happens before" another: a later event answers, builds on, or only makes sense given an earlier one.	5 (happens-before), 9 (Ordering and Causality)
consensus	Getting several nodes to agree on something (e.g. which node leads); much harder than it looks.	9 (Fault-Tolerant Consensus)
data warehouse	A database combining data from several OLTP systems, prepared for analytics.	3 (Data Warehousing)
declarative	Stating the properties the result should have, not the steps to produce it; for queries, an optimizer picks how to run it.	2 (Query Languages for Data), 10
denormalize	Deliberately adding redundancy to normalized data — usually a cache or index — to speed reads; effectively a precomputed query result, like a materialized view.	7 (single/multi-object operations), 11 (views from one event log)
derived data	A dataset produced from other data by a repeatable process, usually to speed up particular reads (indexes, caches, materialized views).	Part III intro — 10; 12
deterministic	Same input → same output, so it can't use random numbers, the current time, network calls or other unpredictable inputs.	General; 10, 11, 12
distributed	Running on several networked nodes; hallmark is partial failure — some parts broken, others working, and the software often can't tell what broke.	8 (Faults and Partial Failures)
durable	Stored so that you trust it will survive various faults.	7 (Durability)
ETL	Extract–Transform–Load: pull data from a source database, reshape it for analytics, load it into a warehouse or batch system.	3 (Data Warehousing)
failover	In single-leader systems, moving the leader role to another node.	5 (Handling Node Outages)

Term	Plain-English meaning	Ch
fault-tolerant	Recovers automatically when something breaks (machine crash, network link down).	1 (Reliability)
flow control	Same as *backpressure*.	11
follower	A replica that accepts no client writes, only changes from the leader. Aka secondary, slave, read replica, hot standby.	5 (Leaders and Followers)
full-text search	Searching text for arbitrary keywords, often with similar-spelling or synonym matching; served by a full-text index (a kind of secondary index).	3 (full-text search, fuzzy indexes)
graph	Vertices (referable things; aka nodes, entities) connected by edges (aka relationships, arcs).	2 (Graph-Like Data Models)
hash	A function turning input into a random-looking number; same input → same number; different inputs rarely collide (a *collision*). Used to spread keys evenly.	6 (Partitioning by Hash of Key)
idempotent	Safe to retry: doing it several times has the same effect as once (*idempotence*).	11 (Idempotence), 12
index	A data structure for quickly finding all records with a given value in a given field.	3 (Data Structures That Power Your Database)
isolation	How much concurrent transactions can interfere with each other; serializable is strongest, weaker levels are common.	7 (Isolation)
join	Combining records that share something — usually fetching a record referenced by another (foreign key, document reference, graph edge).	2 (many-to-one/many relationships), 10 (reduce-side joins)
leader	The one replica allowed to make changes; chosen by election or by an administrator. Aka primary, master.	5 (Leaders and Followers)
linearizable	Behaving as if there were one copy of the data, changed only by atomic operations.	9 (Linearizability)
locality	Storing data together that is often needed together, for performance.	2 (data locality for queries)
lock	Ensures only one thread, node or transaction uses something at a time; others wait for release.	7 (Two-Phase Locking), 8 (the leader and the lock)
log	An append-only file for storing data: write-ahead log (crash safety), log-structured storage engine, replication log (leader → followers), event log (a stream).	3 (B-tree reliability; SSTables/LSM-Trees), 5, 11 (Partitioned Logs)
materialize	Compute eagerly and store the result rather than computing on request.	3 (data cubes, materialized views), 10 (intermediate state)
node	A running instance of software on a computer that works with other nodes over a network.	General; Part II (5–9)
normalized	No redundancy: each fact is stored and updated in exactly one place.	2 (many-to-one/many relationships)
OLAP	Online analytic processing: workloads dominated by aggregates (count, sum, average) over many records.	3 (Transaction Processing or Analytics?)
OLTP	Online transaction processing: fast queries reading or writing a few records, usually by key.	3 (Transaction Processing or Analytics?)
partitioning	Splitting a dataset or computation too big for one machine across many; aka sharding.	6

Term	Plain-English meaning	Ch
percentile	Describes a distribution by counting values above/below a threshold: the 95th percentile response time *t* means 95% of requests were faster than *t*, 5% slower.	1 (Describing Performance)
primary key	A value (number or string) uniquely identifying a record, often system-generated (sequential or random). Contrast *secondary index*.	General; see 3
quorum	Minimum number of nodes that must vote for an operation to succeed.	5 (quorums for reading and writing)
rebalance	Moving data or services between nodes to even out load.	6 (Rebalancing Partitions)
replication	Keeping copies (replicas) of data on several nodes so it stays reachable if one isn't.	5
schema	A description of data's structure — fields and types. Data may be checked against it at various points, and it may change over time.	2 (schema flexibility), 4
secondary index	An extra structure beside the main data for efficiently finding records matching a condition.	3 (Other Indexing Structures), 6 (Partitioning and Secondary Indexes)
serializable	Concurrent transactions produce the same result as running one after another in some order.	7 (Serializability)
shared-nothing	Independent nodes, each with its own CPU, memory and disks, linked by an ordinary network (vs shared-memory or shared-disk).	Part II intro — 5
skew	(1) Uneven load across partitions — *hot spots*. (2) A timing anomaly making events appear out of order — read skew, write skew, clock skew.	(1) 6 (hot spots), 10 (handling skew); (2) 7 (read/write skew), 8 (clock skew)
split brain	Two nodes simultaneously believing they are leader, which can break guarantees.	5 (Handling Node Outages), 8 (truth defined by majority)
stored procedure	A transaction's logic packaged to run entirely on the database server, with no client round-trips during the transaction.	7 (Actual Serial Execution)
stream process	A never-ending computation reading an endless event stream and producing output.	11
synchronous	Opposite of *asynchronous*: waiting for completion.	5, 8
system of record	Holds the authoritative ("source of truth") version of data; changes land here first and other datasets derive from it.	Part III intro — 10; 12
timeout	Simplest fault detector: no response within some time. Can't tell a failed node from a network problem.	8 (Timeouts and Unbounded Delays)
total order	An ordering where any two items (e.g. timestamps) can be compared. If some pairs can't, it's a *partial order*.	9 (causal order is not total)
transaction	Grouping reads and writes into one logical unit to simplify error handling and concurrency.	7
two-phase commit (2PC)	Algorithm ensuring several database nodes all commit or all abort a transaction.	9 (Atomic Commit and Two-Phase Commit)
two-phase locking (2PL)	Serializable isolation by locking all data a transaction reads or writes until it finishes.	7 (Two-Phase Locking)
unbounded	Having no known limit or size; opposite of *bounded*.	8, 11

Every other term used in these notes (A–Z)

Every other technical term, acronym, algorithm, product and tool in the Preface and Chapters 1–12, alphabetically, each defined once with links to where it is used most. Terms in the table above (e.g. *quorum*, *two-phase commit*) aren't repeated; synonyms point to the one definition.

0–9

Term	Plain-English meaning	Ch
2PC	See *two-phase commit* (table above).	9
3PC (three-phase commit)	Variant of two-phase commit designed to avoid blocking, but only correct if the network has bounded delays and failures can be detected perfectly — assumptions real networks don't meet.	9

A

Term	Plain-English meaning	Ch
Abbreviated keys	Storing only enough of each key in interior B-tree pages to tell ranges apart, so more keys fit per page.	3
Abort / rollback	Cancelling a transaction so that none of its writes take effect.	7, 9
Abstraction	Simpler interface that hides complicated details underneath, like SQL hiding how data is laid out on disk.	1, 12
A/B test	Showing two versions of a feature to different groups of users and comparing which one performs better.	10, 12
Access path	The route a query takes to reach the data (which indexes, which order of lookups); chosen by hand in old databases, by the query optimizer in relational ones.	2
Accidental complexity	Complexity caused by how something was built rather than by the problem itself; good abstractions remove it.	1
ACID	The classic transaction guarantees: Atomicity, Consistency, Isolation, Durability.	7
Acknowledgment (ack)	Message from a consumer telling the broker "I have processed this", so the broker can delete it instead of redelivering it.	9, 11
ActiveMQ / RabbitMQ / HornetQ	Traditional open-source message brokers (JMS/AMQP style) that delete messages once consumers acknowledge them.	4, 11
Activity events / clickstream	Logs of what users did (pages viewed, buttons clicked), often joined with user data in batch jobs.	10
Actor model	Concurrency model where independent "actors" each keep private state and communicate only by sending each other asynchronous messages.	4
Actual serial execution	Running transactions literally one at a time on a single thread, which is simple and serializable if each transaction is short.	7
Admission control	Limiting how much traffic is let into a network or system so that what is admitted gets guaranteed capacity.	8
Aggregation	Combining many values into one summary, such as a count, sum, average or maximum.	3, 10
Aggregation pipeline	MongoDB's JSON-based query language that passes documents through a series of stages (filter, group, sort...).	2
Agile / DevOps	Agile: working in short iterations and adapting to change. DevOps: developers and operations working together, with heavy automation.	10
Ajax	Technique where browser JavaScript calls the server in the background to update part of a page without reloading it.	4
Akka	Framework for the actor model on the Java virtual machine, which can run actors across several machines.	4, 8
Algorithmic prison	Being systematically shut out of jobs, loans, travel and so on because an algorithm labelled you risky.	12
Amazon EBS	Amazon's network-attached virtual disks for EC2 machines; reads and writes go over the network.	8
Amazon S3	Amazon's cloud object storage service for files of any size.	10, 12
Amazon Web Services (AWS)	Amazon's cloud computing platform (EC2 virtual machines, S3 storage, EBS disks and more).	Preface, 1

Term	Plain-English meaning	Ch
AMQP	Advanced Message Queuing Protocol: an open standard for talking to message brokers such as RabbitMQ.	11
Anti-caching	In-memory database design that moves rarely used records out to disk and back when needed, so the dataset can exceed memory.	3, 7
Anti-entropy	Background process that compares replicas and copies over any data that one of them is missing.	5
Apache Beam / Google Cloud Dataflow	Beam is a programming model for batch and stream jobs that can run on several engines; Cloud Dataflow is Google's hosted engine for it.	12
Apache BookKeeper	Replicated log storage service built on ZooKeeper.	9
Apache Curator	Library of ready-made recipes (leader election, locks) on top of ZooKeeper.	9
Apache Jena	Open-source Java framework for RDF data and SPARQL queries.	2
API	Application programming interface: the operations one piece of software offers for other software to call.	4, 9
API key	Secret token a client sends with each request to identify and authorise itself.	4
API versioning	Running several versions of a service interface side by side (e.g., /v1 and /v2) so old clients keep working.	4
Append-only	Data that is only ever added to the end, never changed in place.	3, 11
Archival storage	Long-term copies of data (backups, warehouse dumps) usually written with the latest schema in one go.	4
ASCII	The basic 7-bit character set for English letters, digits and punctuation.	4, 10
ASN.1 / DER	Old, very complex standard for defining data structures and encoding them in binary; still used for TLS/SSL certificates.	4
Asymmetric fault	Failure where a node can receive messages but its outgoing messages are lost or delayed, so others wrongly think it's dead.	8
Asynchronous packet network	Network (like the internet) that gives no guarantee when, or whether, a packet will arrive.	8
ATM (Asynchronous Transfer Mode)	Older networking technology that reserved bandwidth like telephone circuits.	8
Atomic (caesium) clock	Extremely precise clock based on atomic vibrations; used with GPS in Google's TrueTime.	8
Atomic commit	Ensuring a transaction spread across several nodes either commits on all of them or aborts on all of them.	9, 12
Atomicity (ACID)	All of a transaction's writes happen, or none do; on failure, partial writes are undone.	7, 9
Atomic write operation	Database operation, like UPDATE x = x + 1, that reads and changes a value in one step so concurrent updates aren't lost.	7
Auditing	Actively checking that data is still correct and complete, rather than just trusting the system.	12
Authentication / access control	Checking who someone is (authentication) and what they are allowed to do (access control).	4, 12
Availability	The fraction of time a system is up and answering requests.	5, 9
Avro	Binary encoding format from Hadoop that uses a schema; reader and writer schemas can differ and are matched by field name.	4

Term	Plain-English meaning	Ch
Azure	Microsoft's cloud computing platform.	5, 11

B

Term	Plain-English meaning	Ch
Backup	Copy of the data kept so it can be restored after loss or corruption.	1, 7
Backward compatibility	Newer code can read data written by older code.	4
BASE	Basically Available, Soft state, Eventual consistency: a loose label for "not ACID".	7
Base64	Way of writing binary data using only 64 printable text characters, at about 33% size overhead.	4
Bash / shell	Bash is the most common Unix shell: the command-line program that runs commands and pipes them together.	3, 10
Big ball of mud	System so tangled that it's hard to understand or change.	1
Big Data	Loose label for datasets too large or fast-moving for one ordinary machine.	Preface, 12
Big-O notation (O(n), O(log n))	Way to say how cost grows with data size: O(n) grows in step with n; O(log n) grows very slowly.	3
BigQuery / Dremel	Google's columnar analytic query engine (Dremel) and the cloud service built on it (BigQuery).	3
Bigtable	Google's distributed wide-column storage system, which inspired HBase and Cassandra's data model.	3, 6
Binary encoding	Storing data in compact bytes rather than human-readable text.	4
Binary string	Sequence of raw bytes (not necessarily text).	4
Bitcask	Riak's default storage engine: an append-only log plus an in-memory hash map from each key to its file offset.	3
Bitcoin / blockchain	Cryptocurrency whose replicas keep a shared, append-only chain of blocks of transactions secured with hashes and proof of work.	8, 12
Bitmap encoding	In column storage, representing each distinct value as a string of bits (1 = this row has the value), which compresses very well.	3
Blocking atomic commit	The flaw of 2PC: if the coordinator crashes, participants may have to wait indefinitely for its decision.	9
Bloom filter	Small memory structure that can say "definitely not present" or "maybe present", used to skip pointless disk lookups.	3, 11
Bottled Water / Maxwell / Debezium	Open-source CDC tools that read a database's replication log (PostgreSQL, MySQL) and publish the changes to Kafka.	11
Bounded / unbounded delay	Bounded: every message arrives within a known maximum time (not provided by ordinary networks). Unbounded: no upper limit, so a timeout can never be sure a node is dead.	8
Branching factor	How many child pages each page of a B-tree points to (often several hundred).	3
Branch misprediction	When the CPU guesses wrongly which way an if-statement will go and has to discard work; vectorized code avoids many.	3
Broadcast hash join	Map-side join where the smaller dataset is loaded into an in-memory hash table on every mapper.	10
Brokerless messaging	Producers sending messages straight to consumers with no intermediary server (e.g., ZeroMQ, UDP multicast).	11
BSON	MongoDB's binary encoding of JSON documents.	2, 4

Term	Plain-English meaning	Ch
B-tree	The most common index structure: a tree of fixed-size pages kept sorted by key, updated in place on disk.	3
B+ tree	A B-tree variant that keeps all values in the leaf pages; most "B-trees" in practice are B+ trees.	3
Bulk loading	Building a database's files directly in a batch job and swapping them in at once, instead of writing records one by one.	2, 10
Bulk synchronous parallel (BSP) / Pregel	Model for graph processing in rounds: each vertex processes its incoming messages, then sends messages along edges for the next round.	10
Bursty traffic	Network traffic that comes in sudden spikes rather than at an even rate.	8
Business intelligence (BI)	Using data analysis to support business decisions, typically via reports and dashboards over a data warehouse.	3, 10
Byte offset	The position of a record in a file, counted in bytes from the start.	3
Byzantine fault-tolerant	Still working correctly even if some nodes misbehave arbitrarily or maliciously.	8
Byzantine Generals Problem	The classic puzzle of reaching agreement when some participants may be traitors sending conflicting messages.	8

C

Term	Plain-English meaning	Ch
Cache hit / cache miss	Hit means the requested data was found in the cache; a miss means it had to be fetched from the slower store.	1
Cache invalidation	Removing or updating cached copies when the underlying data changes so stale data isn't served.	1, 7
Capacity planning	Estimating future load and making sure enough hardware is available in time.	1
Cascading	A Java library that builds MapReduce workflows from higher-level operators such as joins and groupings.	10
Cascading failure	One component's failure overloads others, which then fail too, and so on.	6, 8
Cassandra	Open-source, Dynamo-style leaderless database with a wide-column data model, designed for many writes.	3, 6
Catch-up recovery	Restarted follower reading the leader's log from where it stopped until it has caught up.	5
Caught up	Follower that has applied every change the leader has made so far.	5, 11
Causal context	Riak's name for the version vector a client sends back with a write so the database can tell what it already saw.	5
Causal dependency	Event B depends on event A (B happened because of, or after seeing, A), so B must not be processed as if A hadn't happened.	5, 9
Causally consistent	Consistency model guaranteeing everyone sees cause-and-effect events in the right order; concurrent events may appear in any order.	9
CDC (change data capture)	Reading a database's stream of changes so other systems can apply exactly the same changes.	11, 12
CDN (content delivery network)	Worldwide network of cache servers that serves static content from a location near each user.	6
CEP engines (Esper, StreamBase, SQLstream, Apama)	Products that run continuous pattern-matching queries over event streams.	11
Certificate transparency	Public, Merkle-tree-based log used to check that TLS/SSL certificates were issued legitimately.	12
Chain replication	Replication where writes flow along a chain of nodes and reads are served by the tail, giving strong consistency.	5, 9
Changelog	Log of every change made to some state; replaying it rebuilds the state.	11
Change stream / change feed	An API that lets clients subscribe to every change in a database as it happens.	11, 12
Chaos Monkey	A Netflix tool that deliberately kills machines in production to make sure the system tolerates faults.	1, 8
Checkpoint	Saved snapshot of a computation's state and progress so it can resume from there after a crash instead of restarting.	10, 11
Checksum	Short value computed from data; if the data changes or is corrupted, the checksum no longer matches.	3, 12

Term	Plain-English meaning	Ch
Chubby	Google's internal lock and coordination service, which inspired ZooKeeper.	9
Circuit switching	Reserving a fixed amount of network bandwidth for a whole call, as old telephone networks did; gives bounded delay.	8
Client / server	Client sends requests; a server answers them. A server may itself be a client of other services.	4, 5
Clock drift	Clock running slightly faster or slower than real time, so it slowly wanders off.	8
Clock skew	The difference between what two machines' clocks say at the same moment.	5, 8
Clos topology	Network layout with many redundant paths between switches, common in datacenters.	8
Cloud computing	Renting computing resources (machines, storage, managed services) over the internet on demand.	8, 11
Cluster	Group of machines working together as one system.	6, 9
Clustered index	Index that stores the whole row inside the index itself, rather than pointing elsewhere.	3
COBOL	Old business programming language, used with early hierarchical and network databases.	2
CODASYL / network model	Old (1970s) database model where records link to each other in a graph that programs navigate by following pointers.	2
Code generation	Producing source code classes automatically from a schema, as Thrift and Protocol Buffers do.	4
Column-oriented storage	Storing all values of each column together instead of each row together; fast for analytics that read a few columns of many rows.	3
Command	In event sourcing, a user's request that may still be rejected; once validated it becomes an immutable event (a fact).	11
Commit	Finishing a transaction so that all its writes become permanent and visible.	7, 9
Commit point	The moment in 2PC when the coordinator writes its commit decision to disk; after that, the transaction must commit.	9
Commodity machines	Ordinary, inexpensive servers, as opposed to specialised supercomputer hardware.	8, 10
Commutative operation	Operation where order doesn't matter (adding 3 then 5 equals adding 5 then 3), making conflicts easy to merge.	7
Compaction	Throwing away overwritten or deleted values in log files, keeping only the latest value per key.	3, 11
Compare-and-set (CAS)	Atomic operation: "set this value to X only if it is currently Y", else fail.	7, 9
Compensating transaction	Later action that corrects an earlier mistake (refund, apology, rebooking) instead of preventing it up front.	9, 12
Complex event processing (CEP)	Searching a stream for patterns of events described by queries that run continuously.	11
Compound primary key	Key made of several columns, e.g., (user_id, timestamp): the first part picks the partition, the rest sorts within it.	6

Term	Plain-English meaning	Ch
Compute-intensive application	Application whose bottleneck is raw CPU work rather than data.	Preface, 1
Concatenated index	Index on several fields combined in a fixed order, like a phone book sorted by (last name, first name).	3, 6
Concurrency	Several operations being in progress at overlapping times.	5, 7
Concurrency control	Mechanisms (locks, versioning, validation) that stop concurrent transactions from corrupting data.	7, 11
Concurrent operations	Two operations where neither knew about the other when it started — neither "happened before" the other.	5, 9
Confidence interval (clocks)	Range of time a clock reading is known to fall within, such as Spanner's TrueTime [earliest, latest].	8
Conflict avoidance	Routing all writes for a given record to the same leader so write conflicts can't happen.	5
Conflict resolution	Deciding what the final value should be when two concurrent writes changed the same data.	5, 7
Consensus properties (uniform agreement, integrity, validity, termination)	The rules consensus must satisfy: no two nodes decide differently, nobody decides twice, the decision was proposed by someone, and every live node eventually decides.	9
Consistency (ACID)	The application's rules (invariants) about the data hold after every transaction; really a property of the application.	7
Consistency model	Precise statement of which results reads may return when there are concurrent writes and replicas.	9
Consistent hashing	Way of mapping keys to nodes so that adding or removing a node moves only a small share of keys.	6
Consistent prefix reads	Guarantee that if writes happened in some order, readers see them in that same order (never an answer before its question).	5
Consul	HashiCorp's tool for service discovery and configuration, built on Raft.	9, 11
Consumer	Program that reads and processes messages or events from a stream or queue.	11
Consumer group	Set of consumers sharing the work of reading a topic, each taking some partitions.	11
Consumer offset	The position in a log partition up to which a consumer has processed messages.	11
Contention	Many transactions or threads competing for the same data or lock at once.	7
Context switch	The CPU switching from running one thread to another, saving and restoring their state.	1, 8
Controller node	Node responsible for deciding which node is leader (or owns which partition).	5, 9
Convergent conflict resolution	Every replica resolves a conflict the same way, so all end up with the same final value.	5
Coordination-avoiding data system	System that keeps data integrity without synchronous cross-node coordination.	12
Coordinator node (leaderless)	In leaderless replication, the node that receives a client's request and forwards it to the replicas (without enforcing an	5

Term	Plain-English meaning	Ch
	order).	
Coordinator (transaction manager)	The component that runs two-phase commit, asking participants to prepare and then telling them to commit or abort.	9
Copy-on-write	Instead of changing data in place, write a modified copy and switch pointers to it, leaving the old version intact.	3, 7
CORBA / DCOM / RMI / EJB	Older RPC frameworks for calling objects on remote machines; mostly abandoned because they hid network problems.	4
Cost-based query optimizer	Optimizer that estimates the cost of each possible execution plan and picks the cheapest.	10
Couchbase	Distributed document/key-value database descended from Memcached and CouchDB.	6
CouchDB	Open-source document database that stores JSON and supports multi-leader replication.	2, 5
Covering index	Index that stores some extra columns so certain queries can be answered from the index alone.	3
CPU	Central processing unit: the chip that executes program instructions.	1, 8
CQRS	Command Query Responsibility Segregation: keeping the write model separate from (possibly many) read models.	11
Crash-recovery fault	Fault model where nodes may crash and later restart, keeping what was on disk.	8
Crash-stop fault	Fault model where a crashed node is gone forever.	8, 9
CRDT	Conflict-free replicated data type: a data structure (counter, set, list...) that merges concurrent edits automatically and sensibly.	5
Cross-device read-after-write	Read-your-writes across a user's devices: an update made on a laptop must be visible on their phone.	5
Cross-partition transaction	Transaction touching data on several partitions, which needs coordination between them.	7
C-Store	Research column-store database (the ancestor of Vertica) that kept data sorted in several orders.	3
CSV	Comma-separated values: a simple text format for tables, ambiguous about types and escaping.	3, 4
curl	Command-line tool for making HTTP requests.	4, 10
Cursor stability	Locking a row while it is being read and modified, to prevent lost updates.	7
Cypher	The declarative query language of the Neo4j graph database.	2

D

Term	Plain-English meaning	Ch
Daemon	Background program that runs continuously on a server (e.g., the HDFS process on each machine).	6, 10
DAG (directed acyclic graph)	Graph of steps with arrows and no cycles; dataflow engines run jobs as DAGs of operators.	10
Database inside-out	Another name for the unbundling / dataflow approach to building systems from logs and derived views.	12
Database-internal distributed transaction	Distributed transaction where all participants run the same database software, so it can be optimised.	9, 12
Data broker	Company that buys, combines, infers and resells personal data.	12
Databus / Wormhole / Sherpa	CDC systems built at LinkedIn (Databus), Facebook (Wormhole) and Yahoo! (Sherpa).	11
Datacenter	Building full of servers, networking and power equipment.	5, 8
Data cube / OLAP cube	Precomputed grid of aggregates over several dimensions, so totals can be read instantly.	3
Data exhaust	Label for behavioural data produced as a side effect of using a service.	12
Dataflow	The path data takes as it is passed and transformed between processes or systems.	10, 12
Dataflow engine	Batch processor like Spark, Tez or Flink that runs a whole workflow as one job without writing every intermediate step to disk.	10
Data integration	Making sure the same data ends up, in the right shape, in every system that needs it.	12
Data-intensive application	Application whose main challenge is the amount, complexity or speed of change of its data, not CPU power.	Preface, 1
Data lake	Big store (often HDFS or object storage) where raw data of any format is dumped first and interpreted later.	10
Data locality	Running computation on the machine that already stores the input, to avoid copying it over the network.	2, 10
Datalog	Old logic query language (a subset of Prolog) that builds queries from small rules.	2
Data model	How data is organised and described to the programmer: tables, documents, graphs and so on.	2
Data outlives code	Code is replaced in minutes, but stored data may stay years old, so new code must keep reading old data.	4
Data system	The book's umbrella term for databases, caches, queues and other tools that store and process data.	1, 12
Datomic	Database that keeps all history as immutable facts and supports Datalog queries.	2, 7
DB2	IBM's commercial relational database.	2, 7
Deadlock	Two transactions each waiting for a lock the other holds, so neither can proceed; the database aborts one.	7
Declarative query language	A *declarative* (see table above) language such as SQL.	2, 10

Term	Plain-English meaning	Ch
Decoding (deserialization, parsing)	Turning bytes back into in-memory objects.	4
Denial of service	Attack (or accident) that floods a system so it can't serve legitimate users.	8
Derivation function	The code that turns source data into a derived dataset (e.g., "extract the field and sort it" for an index).	12
Dimension table	In a star schema, a table describing the who/what/where of events (products, stores, dates).	3
Dirty read	Seeing another transaction's writes before it has committed.	7
Dirty write	Overwriting another transaction's uncommitted write.	7
Distributed actor framework	System (Akka, Orleans, Erlang/OTP) that runs actors across several machines with the same message-passing model.	4, 9
Distributed filesystem	Filesystem whose files are spread across many machines (e.g., HDFS).	10
Distributed RPC	Storm's feature for sending a query through a stream topology and gathering results from many partitions.	11, 12
Django / ActiveRecord / Hibernate	Popular ORM frameworks for Python (Django), Ruby on Rails (ActiveRecord) and Java (Hibernate).	2, 7
DNS	Domain Name System: translates names like example.com into IP addresses.	6, 9
Docker / Mesos	Tools for packaging programs into containers (Docker) and scheduling them across a cluster (Mesos).	10, 12
Document database	Database storing self-contained records as JSON-like documents (e.g., MongoDB).	2
Document-partitioned (local) index	Each partition keeps a secondary index of only its own records; reads must ask all partitions.	6
Document reference	Field in one document holding the ID of another, the document-model equivalent of a foreign key.	2, 10
DOM	Document Object Model: the browser's tree of page elements that JavaScript can change.	2
Domain-driven design (DDD)	Software design approach centred on modelling the business domain; event sourcing is popular in it.	11
Dotted version vector	Refined version vector used by Riak to track causality precisely.	5
Double-spending	Spending the same money or credit twice due to a concurrency bug.	7
Drill-down / slicing and dicing	Exploring analytic data by zooming into detail or viewing it by different dimensions.	3
Druid	Open-source analytic database designed for fast queries over event data.	11
Dryad / Nephele	Research dataflow engines that inspired Spark, Tez and Flink.	10
Dual writes	Application code writing the same change to two systems itself; it can leave them permanently inconsistent.	11, 12
Duration	Amount of elapsed time (e.g., how long a request took), as opposed to a point in time.	8
Dynamic membership	Adding or removing nodes from a consensus group while it keeps running.	9

Term	Plain-English meaning	Ch
Dynamic partitioning	Splitting a partition when it grows too big and merging small ones, so the number of partitions follows data volume.	6
Dynamo	Amazon's in-house leaderless key-value store, whose paper inspired Riak, Cassandra and Voldemort ("Dynamo-style").	5, 9
DynamoDB	Amazon's hosted cloud key-value/document database (different from the original Dynamo design).	5, 6

E

Term	Plain-English meaning	Ch
Edge	Connection between two vertices in a graph.	2
Edit distance	How many single-letter insertions, deletions or substitutions turn one word into another; used for fuzzy search.	3
Elasticsearch	Popular open-source distributed search engine built on Lucene.	6, 11
Elastic system	System that can automatically add or remove machines as load changes.	1, 6
Elm / React / Redux	Front-end tools that model the user interface as a function of a stream of events and state changes.	12
Encapsulation	Hiding a component's internal details behind a well-defined interface.	4
Encoding (serialization, marshalling)	Turning in-memory objects into bytes for storage or sending over a network.	4
Encryption	Scrambling data so only holders of the right key can read it.	4, 12
End-to-end argument	Some guarantees (dedup, integrity, security) can only be fully provided by the endpoints of a communication, not by lower layers alone.	12
Ephemeral node	A ZooKeeper entry that is deleted automatically when the client session that created it dies.	9
Epoch number (ballot / view / term)	Counter that increases with each leader election, so messages from an old leader can be recognised and ignored.	9
Epoch (time)	The fixed starting point clocks count from, usually midnight UTC on 1 January 1970.	8
Equi-join	Join where records are matched on equal key values.	10
Erasure coding	Storing data with extra computed parity pieces so lost parts can be rebuilt, using less space than full copies.	10
Erlang / OTP	Programming language and framework built around lightweight processes (actors) and fault tolerance.	4
Error-correcting codes	Extra bits stored or sent with data so that small errors can be detected and fixed automatically.	8, 10
Espresso	LinkedIn's distributed document database.	4, 6
Etcd	Open-source coordination service and key-value store using the Raft consensus algorithm (used by Kubernetes).	9
Ethernet	The standard technology for wired local networks.	8
Etherpad / Google Docs	Real-time collaborative editors where several people type into the same document at once.	5
Event	Small, immutable record that something happened at a certain time.	11, 12
Event sourcing	Storing every change as an immutable event in a log and deriving current state from those events.	11, 12
Event time	When an event actually happened, according to the device that produced it.	11, 12
Eventual consistency	If writes stop, all replicas will eventually hold the same data; until then reads may be stale.	5, 9

Term	Plain-English meaning	Ch
Evolvability	How easily a system can be changed to meet new requirements.	1, 4
Exactly-once (effectively-once) semantics	The end result is as if each message were processed once, even though retries may have happened.	11, 12
Excision / shunning	Actually deleting data from an immutable event history, e.g., for legal reasons.	11
Explicit locking	The application locking rows itself (e.g., SELECT ... FOR UPDATE) to prevent lost updates.	7
Exponential backoff	Waiting longer and longer (e.g., doubling) between retries so an overloaded system can recover.	7

F

Term	Plain-English meaning	Ch
Fact table	The huge central table of a star schema, one row per event (such as each sale).	3
Failure	The whole system stops providing its service to users (as opposed to a fault in one part).	1, 8
Fan-out	One input causing many outputs: e.g., one tweet delivered to every follower's timeline, or one message sent to every consumer.	1, 11
Fault	One component deviating from its specification (a disk dying, a bug); a failure is when the whole system stops serving users.	1, 8
Federated database / polystore	One query interface that reads from many different underlying stores ("unifying reads").	12
Feedback loop	Prediction changes the world in a way that reinforces the prediction.	12
Fencing / STONITH	Forcibly cutting off or shutting down a node that may wrongly believe it is still leader ("Shoot The Other Node In The Head").	5
Fencing token	Number that increases each time a lock or lease is granted; storage rejects writes carrying an older number.	8, 9
Fibre Channel / SAN / NAS	Specialised shared-storage hardware: a storage area network (SAN) or network-attached storage (NAS) reached over dedicated links.	10
Field tag	Number that identifies a field in Thrift or Protocol Buffers encodings, instead of writing the field name.	4
Finite state automaton	Small machine of states and transitions that accepts or rejects strings; Lucene uses one for fuzzy search.	3
Firebase / Meteor	Application platforms that let clients subscribe to live database changes.	11
Firewall	Network filter that blocks traffic not explicitly allowed.	4, 8
Fixed number of partitions	Creating many more partitions than nodes up front, then moving whole partitions between nodes when rebalancing.	6
Flink	Apache Flink: an open-source stream processor that also does batch jobs.	10, 11
FLP result	Proof that consensus can't always be reached in a purely asynchronous system if a node may crash; timeouts get round it in practice.	9
FlumeJava / Crunch	Libraries for writing batch pipelines as chains of high-level operations that compile down to MapReduce or Spark jobs.	10
Foreign key	Column that refers to the primary key of a row in another table.	2, 7
Forward compatibility	Older code can read data written by newer code (ignoring fields it doesn't know).	4
Forwarding pointer	Note left in a heap file when a row moves, pointing to the row's new location.	3
FoundationDB	Distributed key-value store offering serializable ACID transactions.	7
Fractal tree	A B-tree variant that buffers writes in internal pages to reduce disk I/O.	3
Fragmentation	Free space scattered in small unusable gaps across disk pages.	3
fsync	The system call that forces data buffered in memory onto disk.	7, 9
Full table scan	Reading every row of a table, without using an index.	10

Term	Plain-English meaning	Ch
Functional vs nonfunctional requirements	Functional: what the system must do. Nonfunctional: qualities like security, reliability, scalability and compliance.	1
Futures / promises	Objects representing a result that will be available later, used for asynchronous calls.	4, 12

G

Term	Plain-English meaning	Ch
Garbage collection (GC)	Automatically freeing memory (or old data versions) that is no longer used; can cause program pauses.	7, 8
GFS (Google File System)	Google's distributed file system, which inspired HDFS.	10
Giraph / GraphChi / GraphX	Graph processing frameworks implementing the Pregel model or single-machine variants.	10
Git	The most widely used version control system; it tracks history as branches and merges.	5, 9
GlusterFS / QFS	Other distributed filesystems, alternatives to HDFS.	10
GoldenGate	Oracle's tool for log-based change data capture and replication.	5, 11
Google Cloud Pub/Sub	Google's hosted publish/subscribe messaging service.	11
Gossip protocol	Nodes randomly exchanging state with each other so information spreads through the whole cluster.	6
GPS	Global Positioning System: satellites whose signals give location and very accurate time.	8
Graph database	Database built around vertices and edges, good for highly connected data (e.g., Neo4j).	2
Gremlin	Graph traversal query language (from Apache TinkerPop).	2
gRPC	Google's open-source RPC framework using Protocol Buffers over HTTP/2.	4

H

Term	Plain-English meaning	Ch
Hadoop	The open-source ecosystem (HDFS, MapReduce, YARN...) for storing and processing huge datasets on clusters.	10
Happens-before	Operation A happened before B if B knew about A, depended on it, or built on it.	5, 9
Hard real-time system	System that must respond within a strict deadline or it has failed (e.g., airbag controllers).	8
Hash function	See *hash* (table above).	6
Hash index	In-memory hash map from each key to the location of its value on disk.	3
Hash mod N	Choosing a partition with hash(key) % N; a bad idea because changing N moves almost every key.	6
Hash partitioning	Assigning records to partitions based on a hash of their key, which spreads load evenly but loses key ordering.	6
HATEOAS	Strict REST principle where responses include links telling the client what it can do next.	4
HBase	Open-source Bigtable-style wide-column database on top of HDFS.	3, 6
HDFS	Hadoop Distributed File System: stores huge files across many machines, replicating blocks for safety.	10
Head-of-line blocking	Slow item at the front of a queue holding up everything behind it.	1, 11
Heap file	File holding table rows in no particular order; indexes point into it.	3
Heartbeat	Small periodic "I'm alive" message; missing several in a row suggests the node has failed.	9
Helix	LinkedIn's cluster-management framework that uses ZooKeeper to track partition assignments.	6
Heterogeneous distributed transaction	Transaction spanning different technologies (e.g., two databases from different vendors plus a message broker).	9, 12
Heuristic decision	Participant in 2PC unilaterally deciding to commit or abort an in-doubt transaction, breaking atomicity.	9
Hierarchical model / IMS	IBM's 1960s database that stored records as a tree, each record with one parent.	2
High-frequency trading	Automated trading that relies on microsecond-precise timing.	8
High-performance computing (HPC)	Supercomputers running large scientific calculations; they usually stop the whole job on any failure.	8
Hinted handoff	With a sloppy quorum, a node temporarily stores writes for an unreachable node and passes them on when it returns.	5
Hive	A Hadoop tool that runs SQL-like queries as batch jobs over files in HDFS.	10
Hive metastore / HCatalog	Catalogs recording which files in HDFS form which tables and how they are partitioned.	10
Hot key / linchpin object	Single key with disproportionate traffic (e.g., a celebrity's account), causing skew.	6, 10
Hot spot	Partition with far more load than others; see *skew* (table above).	6, 10

Term	Plain-English meaning	Ch
HTML / CSS	The markup language (HTML) and style language (CSS) of web pages.	2, 4
HTTP	The request/response protocol of the web.	4, 12
Human fault tolerance	Being able to recover from buggy code or mistakes, e.g., by rerunning a job over immutable input.	10
HyperLogLog	Probabilistic algorithm that estimates how many distinct items a stream contains, using very little memory.	11
Hypervisor	The software that runs virtual machines and shares the physical hardware between them.	8

I

Term	Plain-English meaning	Ch
IaaS	Infrastructure as a service: renting virtual machines, storage and networks from a cloud provider.	Preface
ICMP Destination Unreachable	Network message a router may send when it can't deliver a packet; not always sent or reliable.	8
ID	Identifier: a value (number or string) that uniquely names something.	10, 12
Idempotence	See *idempotent* (table above).	11, 12
IDL (interface definition language)	Small language for describing a schema or service interface, from which code can be generated.	4
Immutable	Never changed after it is written; new information is added instead.	10, 11
Impala / Presto	Open-source MPP SQL query engines that read data stored in Hadoop.	10
Impedance mismatch	The awkward translation needed between objects in application code and rows in relational tables.	2
Imperative language	Code that spells out step by step how to compute something, in a specific order.	2
Increment-and-get	Atomic operation that adds one to a counter and returns the new value.	9
Index-range lock (next-key lock)	Simplified predicate lock that locks a whole range in an index, used to prevent phantoms.	7
In doubt (uncertain)	The state of a 2PC participant that voted "yes" and must wait for the coordinator's decision.	9
InfiniBand	Very fast network technology used in supercomputers.	8
Initial snapshot	Full copy of a database used to start CDC, before following the change stream.	11, 12
In-memory database	Database that keeps all its data in RAM (with disk or replication for durability).	3, 4
InnoDB	MySQL's default storage engine, which uses B-trees and supports transactions.	3, 7
Integrity	No lost, duplicated, contradictory or false data; violations are permanent until repaired.	9, 12
Internet of Things	Everyday devices (sensors, appliances, toys) connected to the internet.	6, 12
Invariant	Rule about the data that must always be true (e.g., credits and debits balance).	7
Inversion of control	Letting a framework call your code and wire inputs to outputs, rather than your code wiring everything itself.	10
Inverted index	Map from each word to the list of documents containing it; the core of full-text search.	12
IP	Internet Protocol: delivers individual packets between machines, with no guarantee of delivery or order.	6, 8
Isolation level	Setting choosing how strongly a database isolates concurrent transactions (read committed, snapshot isolation, serializable...).	7, 12
Iterator	Object that hands out items one at a time, such as the values a reducer receives for one key.	10

J

Term	Plain-English meaning	Ch
Java	Widely used programming language that runs on the Java Virtual Machine (JVM).	4, 9
Java EE	Java Enterprise Edition: a set of standards (JTA, JMS, EJB...) for business applications on Java.	9
JavaScript	The programming language that runs in web browsers (and on servers via Node.js).	2, 4
Jepsen	Kyle Kingsbury's test harness that injects faults into distributed databases to check their claimed guarantees.	12
Jitter	Random variation in network delay.	8
JMS	Java Message Service: the standard Java API for message brokers.	11
Job scheduler	Software that decides which machines run which tasks (e.g., YARN).	9
JSON	JavaScript Object Notation: a popular text format of nested objects, arrays, strings and numbers.	2, 4
JTA / JDBC	Java standards: JTA coordinates distributed (XA) transactions; JDBC is the standard API for connecting to SQL databases.	4, 9
JVM	Java Virtual Machine: the runtime that executes Java (and Scala, Kotlin...) programs; it has garbage-collection pauses.	8, 10

K

Term	Plain-English meaning	Ch
Kafka	Apache Kafka: a durable, partitioned, replicated log used as a message broker and event store.	11
Kafka Connect	Framework for streaming data between Kafka and other databases and systems.	11
Kafka Streams	Stream processing library built into the Kafka project.	11
Key-range partitioning	Giving each partition a continuous range of sorted keys, which makes range scans easy but risks hot spots.	3, 6
Key-value store	Database that stores values looked up by a unique key, like a big persistent hash map.	3, 10
Kinesis	Amazon's hosted log-based streaming service, similar to Kafka.	11
k-nearest neighbors	Finding the k items most similar to a given one, e.g., for recommendations.	10
Kryo / pickle	Language-specific serialization libraries (Kryo for Java, pickle for Python).	4
Kubernetes	Open-source system for deploying and running containerised applications across a cluster.	10, 12

L

Term	Plain-English meaning	Ch
Lambda architecture	Running a batch system and a stream system side by side over the same events, then merging their outputs.	12
Lamport timestamp	Pair (counter, node ID) that gives a total order of events consistent with causality, without synchronised clocks.	9
Language-specific format	Encoding built into one programming language (Java serialization, Python pickle); risky and not portable.	4
Last write wins (LWW)	Resolving conflicts by keeping the write with the latest timestamp and silently discarding the others.	5, 7
Latch	Lightweight lock protecting an in-memory data structure, such as a B-tree page during an update.	3
Latency	Time spent waiting: e.g., the delay before a request is handled, or the network delay of a message.	1, 8
Leader-based replication	One replica (the leader) takes all writes and streams them to the followers.	5
Leader election	Choosing which node is the leader, normally via a consensus algorithm.	9
Leaderless replication	Any replica accepts writes from clients directly; reads and writes use quorums (Dynamo-style).	5, 9
Leaf page	The bottom level of a B-tree, which holds the actual values or row references.	3
Leap second	Extra second occasionally added to keep clocks in step with Earth's rotation; it has crashed systems.	8
Lease	Lock that expires automatically after a time unless renewed.	8
Ledger	Append-only record of financial transactions, a natural fit for event sourcing.	7, 11
LevelDB	Google's open-source LSM-tree key-value storage library.	3
Levenshtein automaton	Data structure that efficiently finds words within a given edit distance of a search term.	3
Limping node	Node that is alive but extremely slow, which can be worse than a dead one.	8
Linux / Unix	Families of operating systems built around small programs, files and pipes; Linux is the most widely used.	10
Live migration	Moving a running virtual machine to another host, briefly pausing it.	8
Liveness property	"Something good eventually happens" (e.g., a request eventually gets a response).	8, 9
LLVM	Compiler toolkit that some query engines use to turn queries into fast machine code.	10
LMDB / RocksDB	Embedded key-value storage libraries: LMDB uses a copy-on-write B-tree, RocksDB an LSM-tree.	3
Load balancer	Component that spreads incoming requests across several servers.	4, 6
Load balancing (messaging)	Each message delivered to one of several consumers, to share the work.	6, 11
Load parameters	Numbers describing load, such as requests per second or read/write ratio.	1
Location transparency	Making a remote call look exactly like a local one; misleading because networks behave differently.	4

Term	Plain-English meaning	Ch
Lock-free data structure	Concurrent data structure that uses atomic CPU instructions instead of locks.	8
Lock (shared / exclusive)	Shared lock lets many readers in at once; an exclusive lock admits only one holder (typically a writer).	7
Log-based message broker	Broker (like Kafka) that stores messages in an append-only log that consumers read by offset.	11, 12
Log compaction	Keeping only the latest record per key in a log, so it doesn't grow forever.	11
Logical clock	Counter-based clock that orders events by causality rather than by physical time.	8, 9
Logical (row-based) log	Replication log describing row changes independently of the storage engine's internals.	5
Logical timestamp	Counter-based timestamp (e.g., Lamport) that orders events without relying on physical clocks.	12
Log segment	One file of a log that has been split into several pieces.	3
Log sequence number (LSN)	The position of an entry in a database's replication log.	5, 11
Loose coupling	Components that know little about each other, so each can change or fail without breaking the others.	10, 12
Lost update	Two read-modify-write cycles run concurrently and one overwrites the other's change.	7
LRU (least recently used)	Eviction rule that drops the item that hasn't been used for the longest time.	3
LSM-tree	Log-Structured Merge-Tree: storage that buffers writes in memory, flushes sorted files, and merges them in the background.	3
Lucene	The open-source full-text indexing library behind Elasticsearch and Solr.	3, 10

M

Term	Plain-English meaning	Ch
Machine learning (ML)	Software that learns patterns from data to make predictions or decisions.	10, 12
Mahout / MADlib	Libraries of machine-learning algorithms that run on Hadoop (Mahout) or inside an MPP database (MADlib).	10
Mainframe	Large, centralised business computer of the kind that dominated before networks of commodity servers.	2
Maintainability	How easily people can operate, understand and change a system over its lifetime.	1
Many-to-one / many-to-many relationship	Many records referring to one shared record (many people living in one city), or many records relating to many others (people and the organisations they worked for).	2
Mapper	The function in MapReduce that runs once per input record and emits key–value pairs.	10
MapReduce	Batch programming model: a map function emits key–value pairs, they are sorted and grouped by key, and a reduce function processes each group.	10
Map-side join	Join done in the mappers, possible when inputs are small or already partitioned and sorted compatibly.	10
Map-side merge join	Map-side join when both inputs are partitioned and sorted the same way, so they can be merged directly.	10
Materialized view	Query result stored on disk and kept updated, rather than recomputed every time.	11, 12
Materializing conflicts	Creating rows that exist only so there is something to lock, turning a phantom into a lock conflict.	7
Maven / npm / RubyGems	Package managers that download libraries (dependencies) for Java, JavaScript and Ruby projects.	10
Median	The middle value: half of the measurements are below it, half above (the 50th percentile).	1
Membership service	Service (often ZooKeeper-based) that tracks which nodes are currently alive and part of the cluster.	9
Memcached	Simple, widely used in-memory key-value cache.	1, 3
Memory barrier / fence	A CPU instruction that forces memory reads and writes to be ordered and visible across cores.	9
MemSQL	In-memory relational database (now called SingleStore).	3, 7
Memtable	The in-memory sorted structure where an LSM-tree collects new writes before flushing them to disk.	3
Mergeable persistent data structure	Data structure that tracks history like Git and merges concurrent changes with a three-way merge.	5
Merge (segments)	Combining several sorted files into one, keeping the newest value per key.	3, 5
Merkle tree	Tree of hashes that lets you cheaply prove a record is part of a dataset.	12
Message broker (message queue)	Server that receives messages from producers and delivers them to consumers, buffering them in between.	4, 11
MessagePack	Binary encoding of JSON-like data.	4

Term	Plain-English meaning	Ch
Messaging system	Software that delivers messages (events) from producers to consumers, directly or via a broker.	11
Microbatching	Processing a stream as a series of tiny batch jobs, each covering a second or so.	11
Microservices	Architecture where an application is split into many small services, each deployable independently.	4, 12
Middleware	Software sitting between applications and the operating system or network, e.g., message brokers.	4
Migrating data	Rewriting existing data into a new schema, which is expensive on large datasets.	4
Minimizing irreversibility	Designing so that mistakes can be undone, e.g., by keeping old outputs and immutable inputs.	10
MongoDB	Popular open-source document database storing JSON-like documents.	2, 6
MongoDB oplog	MongoDB's replication log, which CDC tools can read.	11
Monotonically increasing	Always going up (never decreasing or repeating), like a sequence of fencing tokens.	8, 9
Monotonic clock	Clock that only moves forward; good for measuring durations, meaningless across machines.	8
Monotonic reads	Guarantee that a user never sees data go backward in time across successive reads.	5
MPP (massively parallel processing)	Databases that split each analytic SQL query across many machines.	10
MTTF (mean time to failure)	The average time a component runs before it fails; with thousands of disks, some fail every day.	1
Multi-dimensional index	Index that can search on several columns at once, like latitude and longitude (e.g., R-trees).	3, 10
Multi-leader replication	More than one node accepts writes, each passing changes to the others; conflicts are possible.	5, 9
Multi-object transaction	Transaction that reads or writes several records together.	7, 9
Multi-Paxos	The version of Paxos that agrees on a sequence of values (a log) rather than a single value.	9
Multi-tenant	Many customers sharing the same hardware or service.	8, 10
Mutex / semaphore	Synchronisation tools within one machine: a mutex lets one thread at a time into a section; a semaphore lets up to N.	7, 8
MVCC (multi-version concurrency control)	Keeping several versions of each row so each transaction can read a consistent snapshot without blocking writers.	7
MySQL	Popular open-source relational database.	5, 7

N

Term	Plain-English meaning	Ch
NameNode / DataNode	In HDFS, the NameNode tracks which machine holds each file block; DataNodes store the blocks.	10
Neo4j	Popular graph database that uses the Cypher query language.	2
Network congestion	Too much traffic for a network link, causing queues and packet loss.	8
Network partition / netsplit	Some nodes can't talk to others because of a network fault, though both groups are still running.	8, 9
Noisy neighbor	Another customer's workload on shared hardware slowing your system down.	8
Nondeterministic	Can give different results each time (e.g., reading the clock or a random number).	5, 8
Nondeterministic function	Function such as NOW() or RAND() that gives a different result each time, which breaks statement-based replication.	5
NoSQL	Databases that don't use the classic relational/SQL model (document, key-value, wide-column, graph stores).	Preface, 2
NTP	Network Time Protocol: adjusts a computer's clock using time servers over the network.	8
NUMA	Non-uniform memory access: on big servers, memory near a CPU is faster to reach than memory near another CPU.	5
NVM (non-volatile memory)	Memory that keeps its contents without power while being nearly as fast as RAM.	3

O

Term	Plain-English meaning	Ch
OAuth	Standard that lets a user grant an application limited access to their account on another service without sharing their password.	4
Object container file	An Avro file holding many records, with the writer's schema stored once at the start.	4
Object database / XML database	1980s–2000s alternatives to relational databases that stored objects or XML documents; they faded away.	2
Object storage	Cloud storage of whole files ("objects") by name, such as Amazon S3.	10, 12
Observer pattern	Code subscribes to a value and is notified when it changes, instead of repeatedly checking.	12
Offline-first application	App that works mainly against a local on-device database and syncs with servers in the background.	5, 12
Offset	Position number: the byte position in a file, or a message's sequence number within a log partition.	11
One-to-many relationship	One record owning a list of others, like a résumé with many job entries; a natural fit for a tree or document.	2
Online system (service)	System that waits for requests and must reply quickly; response time is its key metric.	10
OpenAPI (Swagger)	Standard format for describing RESTful APIs.	4
OpenStack	Open-source software for running a private cloud.	9, 10
Operability	How easy it is for operations teams to keep a system running smoothly.	1
Operational transformation	The conflict-resolution technique behind collaborative editors like Google Docs.	5
Operation identifier / request ID	Unique ID generated by the client for one logical action, used to detect and drop duplicates everywhere downstream.	12
Operator	One processing step in a dataflow or stream job (map, filter, join...).	10, 11
Optimistic concurrency control	Let transactions proceed without blocking and check for conflicts at commit time, aborting if necessary.	7
Oracle	Oracle Database: a widely used commercial relational database.	5, 7
Orleans / Finagle / Rest.li	RPC and actor frameworks from Microsoft (Orleans), Twitter (Finagle) and LinkedIn (Rest.li).	4
ORM (object-relational mapping)	Library that maps program objects to database rows to reduce boilerplate.	2, 7
Orphaned transaction	In-doubt transaction whose coordinator lost its decision, so it holds locks until someone intervenes manually.	9
OS (operating system)	The software that manages a computer's hardware and runs programs (Linux, Windows...).	8, 10
Overcommitted resources	Promising more CPU or memory to tasks than physically exists, betting they won't all use it at once.	10

P

Term	Plain-English meaning	Ch
p50 / p95 / p99 / p999	p50, p95, p99, p999 are percentiles: e.g., p99 is the response time 99% of requests beat.	1
Packet	Small chunk of data sent across a network on its own.	8
Packet switching	Networks that send data as independent packets sharing links dynamically; efficient but delays are unbounded.	8
Page	Fixed-size block (often 4 KB) that disks and B-trees read and write as a unit.	3
Page cache	Memory the operating system uses to keep recently read disk pages, making repeated reads fast.	10
Page fault / swapping	The OS moving memory to and from disk because RAM is full, which can pause a program unexpectedly.	8
PageRank	Google's algorithm ranking web pages by how many important pages link to them; a classic graph computation.	2, 10
Parquet	Popular open-source columnar file format for analytics.	4, 10
Partial failure	Some parts of a system broken while others work, often with no way to tell which.	7, 8
Partial order	Ordering where some pairs of items can't be compared (e.g., concurrent events).	9
Participant	Database node taking part in a two-phase commit.	9
Partition-aware client	Client that knows which node holds which partition and connects directly to the right one.	6
Partition boundaries	The key values where one partition's range ends and the next begins.	6, 12
Partitioned hash join	Map-side join when both inputs are partitioned the same way, so each mapper joins one matching pair of partitions.	10
Partitioned log	Log split into several partitions on different machines to scale throughput; order is kept only within each partition.	11, 12
Partitioning proportionally to nodes	Giving each node a fixed number of partitions, so the partition count grows with the cluster.	6
Partition (shard, region, tablet, vnode, vBucket)	One piece of a dataset that has been split across machines; the names differ by product.	6, 9
Paxos	The classic fault-tolerant consensus algorithm.	9
Peer-to-peer network	Network where participants talk directly to each other with no central server.	8
Percolator	Elasticsearch's feature for storing queries and matching each new document against them (stream search).	11
Perfect failure detector	Imaginary detector that always correctly knows whether a node has crashed; it can't exist on real networks.	9
Pessimistic concurrency control	Blocking (locking) whenever a conflict might happen, as two-phase locking does.	7
Phantom	Write in one transaction changing the result of a search query in another (e.g., inserting a row that matches its WHERE clause).	7

Term	Plain-English meaning	Ch
Phi Accrual failure detector	Failure detector that adapts its timeout to observed network delays and reports a suspicion level.	8
Physical clock	Clock that measures real elapsed time (seconds), as opposed to a logical counter.	8, 12
Pig	A Hadoop tool with a scripting language for building MapReduce workflows.	10
Pipelined execution	Passing output from one operator to the next as soon as it's ready, rather than waiting for the whole stage.	10
Point in time	Specific date and time, as opposed to a duration.	8
Polling	Repeatedly asking "anything new?" instead of being notified.	11, 12
Polyglot persistence	Using several different kinds of database in one application, each for what it's good at.	2
PostGIS	A PostgreSQL extension for geographic data, using R-tree-like indexes.	3
PostgreSQL	Popular, feature-rich open-source relational database.	5, 7
Postings list	For one word, the list of IDs of documents that contain it.	1, 3
ppm (parts per million)	Unit for clock drift: 200 ppm means up to 17 seconds off per day.	8
Predicate lock	Lock on all records matching a condition, including ones that don't exist yet.	7
Predictive analytics	Using data to forecast things, including about individual people (credit risk, reoffending).	12
Preemption	Stopping a running task (or thread) to give its resources to a higher-priority one.	8, 10
Premise	Fact a transaction read and relied on when deciding what to write; if it changes before commit, SSI aborts.	7
Pre-splitting	Creating initial partitions in advance to avoid all writes hitting one partition at first.	6
Priority queue	Queue that always hands out the most important item next instead of the oldest.	3, 7
Probabilistic algorithm	Algorithm that gives approximate answers using much less memory (e.g., Bloom filters, HyperLogLog).	11, 12
Procedural SQL (PL/SQL, T-SQL, PL/pgSQL)	Each database vendor's programming language for writing stored procedures.	7
Processing lag	How far a stream processor's work is behind the newest events.	11
Processing time	When an event is processed by the stream processor, which can be much later than when it happened.	11, 12
Producer	Program that writes messages or events into a stream or queue.	11
Property graph	Graph model where vertices and edges each have a label and a set of key–value properties.	2
Protocol Buffers (Protobuf)	Google's binary encoding format, using a schema and numeric field tags.	4
PTP (Precision Time Protocol)	Protocol for very accurate clock synchronisation over a local network.	8

Term	Plain-English meaning	Ch
Publish/subscribe	Producers publish messages to a topic, and everyone subscribed to it receives them.	11, 12
Pure function	Function whose output depends only on its input and that has no side effects.	2, 12
Push notification	Message the server sends to a device without the device asking first.	11

Q

Term	Plain-English meaning	Ch
Quality of service (QoS)	Network features that reserve capacity or prioritise some traffic.	8
Quartz crystal oscillator	The vibrating crystal that keeps time in a computer's hardware clock; it drifts with temperature.	8
Query optimizer	The part of a database that chooses how to execute a query (index use, join order...).	2, 10
Queueing delay	Time a request or packet spends waiting in a queue before being processed.	1, 8
Queue / topic	Named channel in a message broker that producers write to and consumers read from.	4, 11
Quorum read	Reading from a majority of replicas so the result reflects the latest committed write.	5, 9

R

Term	Plain-English meaning	Ch
Race condition	Bug where the result depends on the exact timing of concurrent operations.	7, 11
Raft	Widely used consensus algorithm designed to be easier to understand than Paxos (used by etcd).	9
RAID	Combining several disks so data survives the failure of one of them.	1, 10
RAM	Computer's main memory: fast, but its contents are lost on power loss.	3, 10
Range scan / range query	Fetching all records whose keys lie between two values.	3, 6
RDBMS	Relational database management system, such as PostgreSQL, MySQL or Oracle.	2
RDD	Resilient Distributed Dataset: Spark's abstraction that tracks how each piece of data was computed so it can be recomputed after failure.	10
RDF	Resource Description Framework: a graph model storing data as (subject, predicate, object) triples.	2
RDMA	Remote direct memory access: reading another machine's memory over the network with very low latency.	8
Read-after-write (read-your-writes) consistency	Guarantee that users always see their own submitted updates.	5
Read committed	Isolation level preventing dirty reads and dirty writes.	7
Reader's schema / writer's schema	In Avro, the schema data was written with versus the one the reading code expects; they are reconciled when decoding.	4
Read-modify-write cycle	Reading a value, changing it in application code and writing it back; prone to lost updates.	7
Read path	Work done lazily, only when someone asks a question.	12
Read repair	Client reading from several replicas notices a stale one and writes the newer value back to it.	5, 9
Read-scaling architecture	Adding many followers to serve more read traffic.	5
Read skew / nonrepeatable read	Seeing different parts of the database at different points in time within one transaction.	7
Read uncommitted	Weak isolation level that prevents dirty writes but allows dirty reads.	7
Rebalancing	Moving partitions between nodes so load stays even as the cluster changes.	6
Recency guarantee	Promise that a read returns the most recent value written; the essence of linearizability.	9
Recursive common table expression	SQL's WITH RECURSIVE syntax, which can express graph traversals.	2
Red-black tree / AVL tree	Self-balancing in-memory tree structures that keep keys sorted with O(log n) inserts and lookups; used for memtables.	3
Redelivery	Broker sending a message again because the consumer didn't acknowledge it.	4, 11

Term	Plain-English meaning	Ch
Redis	Popular in-memory key-value store with data structures such as lists and sets.	5, 7
Redshift / Vertica / Teradata	Commercial column-oriented MPP data warehouse products.	3
Reducer	The function in MapReduce that receives all values for one key and produces output.	10
Reduce-side join	Join done in the reducers, after the shuffle brings records with the same key together.	10
Refactoring	Restructuring code to make it cleaner without changing what it does.	1
Register	In consistency theory, a single variable that can be read and written.	9
Regular expression	Compact pattern language for matching text.	2, 4
Regular register	Register where a read concurrent with a write may return either the old or the new value (weaker than linearizable).	9
Relational algebra	The mathematical foundation of SQL: operations like select, project and join on tables.	2
Relational model	Data organised as tables (relations) of rows, queried with SQL.	2
Reliability	System keeps working correctly (right function, acceptable performance) even when hardware, software or people make mistakes.	1
Remote code execution	Security hole that lets an attacker run their own code on your machine (e.g., via unsafe deserialization).	4
Repeatable read	Isolation level name with inconsistent meanings across databases; often it really means snapshot isolation.	7
Replay	Re-reading old messages from a log to reprocess them.	11, 12
Replica	Node holding a copy of the data.	5, 9
Replication lag	The delay between a write on the leader and it showing up on a follower.	5, 9
Replication log	The stream of changes a leader sends to its followers.	5, 11
Replication topology	The pattern of which leaders send writes to which (circular, star, all-to-all).	5, 8
Reprocessing	Re-running a (new) transformation over all old data to build a new derived view.	11, 12
Request routing	Getting each request to the node that holds the relevant partition.	Preface, 6
Response time	What the client sees: total time from sending a request to receiving the answer.	1, 8
REST / RESTful	Style of web API built on HTTP, URLs and standard methods like GET and POST.	4
RethinkDB	Open-source document database that supports subscribing to query changes.	2, 6
Retraction / correction (streams)	Publishing an updated result for a window after late events arrive, replacing the earlier output.	9, 11
Riak	Open-source Dynamo-style leaderless key-value store.	5, 6

Term	Plain-English meaning	Ch
Ring buffer (circular buffer)	Fixed-size buffer where new data overwrites the oldest once it is full.	11
Rolling average	Average over a moving window, such as the last five minutes.	11
Rolling upgrade	Updating servers one or a few at a time so the service stays up throughout.	1, 4
Round-robin load balancer	Load balancer that sends each new request to the next server in turn.	6
Routing tier	Layer of servers that forward each request to the node owning the relevant partition.	6
Rowhammer	Hardware effect where repeatedly accessing memory rows flips bits in neighbouring rows.	12
Row-level lock	Lock on a single row rather than on a whole table.	7
Row-oriented storage	Storing all values of each row together; good for transactional workloads.	3
RPC (remote procedure call)	Calling a function on another machine over the network as if it were local.	4
RST / FIN packet	TCP packets that end a connection; their arrival can signal that a remote process has died.	8
RTOS	Real-time operating system: one that guarantees CPU time to processes within deadlines.	8
R-tree	Index for multi-dimensional data such as geographic locations.	3
Rule (Datalog)	In Datalog, a named definition that derives new facts from existing facts (and from other rules, even recursively).	2
Run-length encoding	Compressing repeated values by storing "value × count" instead.	3

S

Term	Plain-English meaning	Ch
Safety guarantees	Promises a system makes about bad things never happening (e.g., no lost writes).	7, 9
Safety property	"Nothing bad happens" (e.g., no two nodes hold the same lock); once violated it can't be undone.	8, 9
Samza	Apache Samza: a stream processing framework closely tied to Kafka.	11
SAP HANA	SAP's in-memory, column-oriented database.	3
Scalability	System's ability to cope with increased load, usually by adding resources.	1, 12
Scaling up / scaling out	Scaling up (vertical) means a bigger machine; scaling out (horizontal) means more machines.	1
Scatter/gather	Sending a query to all partitions and combining the results.	6
Schema evolution	Changing a schema over time while keeping old and new data readable.	4
Schemaless	Databases that don't enforce a schema on write; really schema-on-read.	2, 4
Schema migration	Changing a database's schema (and possibly rewriting existing data).	4, 12
Schema-on-read / schema-on-write	Schema-on-read: structure is interpreted only when reading ("schemaless"). Schema-on-write: data is checked against a schema when stored.	2, 4
Secondary sort	Sorting the values within each reducer group in a particular order.	10
Self-describing file	File that includes its own schema, so any reader can decode it.	4
Semantic web	The idea of websites publishing machine-readable data (as RDF) alongside human-readable pages.	2
Semi-synchronous replication	One follower is synchronous and the rest asynchronous, so at least two nodes have every committed write.	5
Sequence number	Counter attached to each operation to order them.	5, 9
Sequential consistency	All nodes see operations in the same order, consistent with each client's program order, but not necessarily in real time.	9
Serializable snapshot isolation (SSI)	Optimistic algorithm giving full serializability on top of snapshot isolation by detecting dangerous conflicts at commit.	7, 9
Server-sent events / WebSockets	Protocols letting a server push messages to a browser over a long-lived connection.	12
Service	Program that offers an API over the network for other programs to use.	4, 9
Service discovery	Finding which IP address and port a service is currently running on.	4, 9
Service time	The time actually spent processing a request, excluding waiting in queues.	1
Sessionization	Grouping a user's events into separate visits (sessions).	10, 12
Session timeout	How long ZooKeeper waits without heartbeats before declaring a client's session dead.	9
Sharding	Another name for partitioning (see the table above): splitting data across machines.	Preface, 6
Shared-disk / shared-memory	Scaling architectures where machines share one set of disks or one memory, as opposed to shared-nothing.	4, 5

Term	Plain-English meaning	Ch
Shredding	Splitting a document-like structure into several relational tables.	2
Shuffle	The MapReduce step that sorts mapper output and sends all values for each key to the same reducer.	10
Sibling pointers	Links from each B-tree leaf page to its neighbours, so range scans can move along without going back to the root.	3
Siblings	Several concurrent values kept for one key until the application merges them.	5
SIGSTOP / SIGCONT	Unix signals that pause and resume a process; a paused process doesn't know time has passed.	8
SIMD	Single instruction, multiple data: CPU instructions that process several values at once.	3
Simplicity	Making a system easy to understand by removing accidental complexity.	1
Single-object operation	Atomic change to one record (e.g., increment or compare-and-set).	7, 12
Size-tiered / leveled compaction	Two strategies for scheduling LSM-tree merges: merge similar-sized files, or keep data in levels of fixed size.	3
Slewing / smearing	Adjusting a clock gradually (slightly faster or slower) instead of jumping, e.g., to spread a leap second over a day.	8
Sloppy quorum	Accepting writes on nodes outside a key's usual set when the usual ones are unreachable.	5
SLO / SLA	Service level objective / agreement: targets or contracts for performance and availability (e.g., p99 under 1 s).	1
Slowly changing dimension (SCD)	Keeping old versions of dimension data (e.g., a tax rate) with version IDs so joins stay reproducible.	11
Snapshot	Consistent copy of the data as it was at one point in time.	7
Snapshot isolation	Each transaction reads from a consistent snapshot of the database as of when it started.	7
Snowflake schema	Star schema where dimension tables are further split into sub-dimension tables.	3
SOA	Service-oriented architecture: building an application as a set of services; microservices are its modern form.	4
SOAP	An XML-based protocol for web service requests, described with WSDL.	4
Solr	Open-source search server built on Lucene.	1, 10
Sort key	The column(s) a column store sorts its rows by, which speeds up range queries and compression.	3
Sort-merge join	Joining two datasets by sorting both by key and walking through them together.	10
Space-filling curve	Way to map two-dimensional locations to one-dimensional keys so a normal index can be used.	3
Spanner	Google's globally distributed SQL database, which uses TrueTime clocks to order transactions.	8
Spark	Apache Spark: a popular distributed engine for batch (and microbatch streaming) processing.	10
SPARQL	Query language for RDF triple-stores.	2

Term	Plain-English meaning	Ch
Speculative execution	Starting a duplicate copy of a slow task on another machine and using whichever finishes first.	10
SQL	Structured Query Language: the standard declarative language for relational databases.	2, 10
SQL injection / cross-site scripting	Common web attacks where untrusted input is run as database commands or as script in other users' browsers.	8
SQL Server	Microsoft's commercial relational database.	3, 7
SSD	Solid-state drive: flash-memory storage, much faster than spinning hard disks.	3, 7
SSTable	Sorted String Table: an immutable file of key–value pairs sorted by key.	3
Stable storage	Storage that survives crashes (e.g., disk), assumed by the crash-recovery model.	8
Star schema	Data warehouse layout with one big fact table referencing smaller dimension tables.	3
Stateless / stateful service	Stateless server forgets everything between requests (state lives in a database); a stateful one keeps data itself.	1, 12
State machine replication	Every replica applies the same deterministic commands in the same order, so they all end in the same state.	9, 11
Statement-based replication	Replicating by sending each SQL statement to followers; breaks with nondeterministic functions like NOW().	5
Static vs dynamic type checking	Checking types before the program runs (static) versus while it runs (dynamic); compared to schema-on-write versus schema-on-read.	2, 4
StatsD / Brubeck	Metrics daemons that receive counters over UDP and aggregate them.	11
stdin / stdout	Program's standard input and output streams, which Unix pipes connect together.	10
Steal time	CPU time a virtual machine loses to other VMs on the same physical host.	8
Stop-the-world GC pause	Pause where garbage collection halts all program threads.	8
Storage engine	The part of a database that decides how data is laid out on disk and in memory.	3, 5
Store buffer	Per-CPU-core queue of pending memory writes, one reason multi-core memory isn't linearizable.	9
Storm	Apache Storm: an early open-source stream processing framework.	11, 12
Straggler	Task that runs much slower than the rest, or an event that arrives late.	10, 11
Stream	Data that arrives gradually over time and never really ends.	11, 12
Stream analytics	Computing aggregates and statistics over a stream, such as rates or averages per time window.	11
Stream-stream join (window join)	Matching events from two streams that occur close together in time.	11
Stream-table join (enrichment)	Adding information to each event by looking it up in a table, often a local copy kept up to date via CDC.	11, 12
Strict quorum	Quorum where reads and writes overlap (w + r > n), so a read normally sees the latest write.	5, 9
Strict serializability	Serializability plus linearizability: transactions appear in an order consistent with real time.	9

Term	Plain-English meaning	Ch
Supermajority	More than two-thirds of the nodes; needed by Byzantine fault-tolerant protocols.	8
Surveillance	Tracking people mainly to serve someone else's interests (e.g., advertisers).	12
Sushi principle	"Raw data is better": store data in its original form and interpret it later.	10
Synchronous network	Network with reserved bandwidth and bounded delays, like old telephone circuits.	8, 12
Systematic error	Fault that hits many nodes at once (e.g., a software bug triggered by the same input).	1
System model	The set of assumptions about what can go wrong (crashes, delays, clock errors) that an algorithm is designed for.	8
System R	IBM's 1970s research database that pioneered SQL and many transaction ideas.	7
Systems thinking	Considering the whole system, including the humans in it, to foresee consequences.	12

T

Term	Plain-English meaning	Ch
Tableau	Popular commercial tool for visual data analysis and dashboards.	10
Table-table join	Keeping the join of two changing tables up to date by consuming both change streams.	11
tail -f	A Unix command that prints new lines as they are appended to a file — like a consumer following a log.	11
Tail latency	The response times of the slowest requests (high percentiles like p99).	1, 3
Tail latency amplification	When one user request needs many backend calls, it's slow if any of them is slow.	1, 6
Tandem NonStop SQL	A 1980s fault-tolerant commercial database, an early example of partitioning with transactions.	6, 10
TCP	Transmission Control Protocol: builds a reliable, ordered byte stream on top of IP, with retransmission and flow control.	8, 12
t-digest / forward decay / HdrHistogram	Algorithms and libraries for computing approximate percentiles cheaply over a stream of measurements.	1
Telemetry	Detailed monitoring data (metrics, error rates) sent from running systems.	1
Term dictionary	In a full-text index, the sorted list of all words, each pointing to its postings list.	3, 10
Term-partitioned (global) index	Secondary index split up by the indexed value, so a read hits one partition but a write may touch several.	6
Test-driven development (TDD)	Writing automated tests before the code they test.	1
Textual format	Human-readable encodings such as JSON, XML and CSV.	4
Tez	Dataflow engine for Hadoop that runs workflows without writing every step to HDFS.	10
Thinking like a vertex	The Pregel style of writing graph algorithms as code that runs per vertex and talks to neighbours by messages.	10
Thrashing	Computer spending most of its time swapping memory to disk, doing little real work.	8
Three-way merge	Merging two changed versions by comparing each against their common ancestor, as Git does.	5
Thrift	Binary encoding and RPC framework from Facebook, using schemas and numeric field tags.	4
Throughput	How much work a system handles per unit of time (e.g., records per second).	1, 10
Timeliness	Users see an up-to-date state; violations are temporary.	12
Time-of-day clock	Clock giving the current date and time; it can jump backwards when corrected.	8
TLS/SSL	The encryption protocol that secures HTTPS and many other network connections.	4, 12
Tombstone	Special marker recording that a key was deleted, so the deletion survives merging and replication.	3, 5

Term	Plain-English meaning	Ch
Top-of-rack switch / router	Network devices: a top-of-rack switch connects the servers in one rack; a router forwards packets between networks.	8
Total order broadcast (atomic broadcast)	Delivering every message reliably to all nodes in the same order; equivalent to consensus.	9
Transaction ID (txid)	Number the database gives each transaction, used to tag the row versions it creates.	7, 9
Transaction processing	Handling many small reads and writes as business events happen (sales, orders, payments).	2, 3
Transitive closure	Finding everything reachable from a starting point through any number of steps.	10
Trie	Tree that stores strings letter by letter, so words with the same prefix share a path.	3
Trigger	Code a database runs automatically when data changes.	5, 11
Trigger-based replication	Using database triggers to copy changes into a separate table that an external process replicates; flexible but slower.	5, 11
Triple-store	Graph database storing (subject, predicate, object) triples, such as RDF data.	2
TrueTime	Google's clock API in Spanner that returns a time range guaranteed to contain the true time.	8
Tumbling / hopping / sliding / session window	Ways to group stream events by time: fixed non-overlapping blocks; fixed overlapping blocks; a moving interval; or bursts of activity separated by gaps.	11
Tungsten Replicator / BDR / Bucardo	Add-on tools providing multi-leader replication for MySQL (Tungsten) and PostgreSQL (BDR, Bucardo).	5
Tuple	In the relational model, the formal name for a row.	2, 10
Turtle / N3	Human-readable text formats for writing RDF triples.	2
Twitter Snowflake	Twitter's service that hands out unique, roughly time-ordered 64-bit IDs.	8
Two Generals Problem	The puzzle showing two parties can never be certain they agree over an unreliable channel.	8

U

Term	Plain-English meaning	Ch
UDP	User Datagram Protocol: sends packets with no retransmission or ordering; used where late data is useless (e.g., voice calls).	8, 11
UDP multicast	Sending one UDP packet to many receivers at once; low latency but unreliable.	9, 11
Unbundling databases	Treating the features inside a database (indexes, views, replication) as separate components wired together with logs.	12
Unicode / UTF-8	Unicode is the universal character set; UTF-8 is its most common byte encoding.	4
Unified batch and stream processing	Doing both historical reprocessing and live processing in one engine with the same code.	12
Uniform consensus	Consensus where even nodes that later crash must not have decided differently.	9
Uniform interface	Everything speaking the same format (in Unix, a stream of bytes, often lines of text), so any tool can connect to any other.	10
Union type	Schema type allowing a field to be one of several types (e.g., null or string) in Avro.	4
Uniqueness constraint	Rule that no two records may share a value (a username, a seat, a request ID).	9, 12
Unix pipe	Connecting one program's output to the next program's input (the vertical-bar character in a shell command).	10
URI	Uniform Resource Identifier: a string that names a resource; URLs are a kind of URI.	4
URL	Uniform Resource Locator: a web address.	10
UUID	Universally unique identifier: a 128-bit random ID that is practically guaranteed to be unique.	5, 12

V

Term	Plain-English meaning	Ch
Vacuum	PostgreSQL's background process that removes old row versions no transaction can see any more.	7
Variable-length integer (varint)	Encoding small numbers in fewer bytes than large ones.	4
Vectorized processing	Processing a batch of column values in a tight loop at once, using CPU caches and SIMD efficiently.	3, 10
Version vector	Set of counters, one per replica, used to tell whether two versions are concurrent or one supersedes the other.	5, 9
Vertex	Node (thing) in a graph, joined to others by edges.	2, 10
Viewstamped Replication (VSR)	Consensus algorithm from the 1980s, similar in spirit to Raft.	9
Virtual machine (VM)	Simulated computer running on shared physical hardware.	1, 8
Virtual view	Saved query that is re-run whenever it is read (an ordinary SQL view).	3
VoIP	Voice over IP: phone calls carried over the internet.	8
Voldemort	LinkedIn's open-source Dynamo-style key-value store.	5, 6
VoltDB	In-memory database that runs transactions one at a time on a single thread per partition.	7, 11

W

Term	Plain-English meaning	Ch
Wall-clock time	The current date and time of day, as shown by a time-of-day clock.	8
Watch / change notification	A ZooKeeper feature letting a client be notified when some data changes.	9, 12
Webhook	A URL that one service calls when an event happens, to notify another service.	11
Web service	Service accessed using HTTP.	4, 9
Window	Time interval used to group stream events for aggregation or joining.	11
Workflow	Chain of batch jobs where each one's output becomes the next one's input.	10
Workflow scheduler	Tool (e.g., Oozie, Azkaban, Airflow) that runs batch jobs in dependency order.	10
Working set	The data a computation needs in memory at once.	10, 11
Write-ahead log (WAL)	Append-only file where every change is written before it's applied, so a crash can be recovered from.	3, 5
Write amplification	One logical write causing several physical writes to disk over time.	3
Write conflict	Two concurrent writes changing the same data in incompatible ways.	5, 12
Write path	Work done eagerly when data arrives (updating indexes, caches and views).	12
Write skew	Two transactions read the same data, then update different records, together breaking a rule each one checked.	7
WSDL	Web Services Description Language: an XML format describing a SOAP API.	4

X

Term	Plain-English meaning	Ch
XA	X/Open eXtended Architecture: the standard API for two-phase commit across heterogeneous systems.	9
XML	Extensible Markup Language: a verbose text format with tags, used for documents and data.	2, 4
XPath / XSL	Languages for selecting parts of an XML document (XPath) and transforming them (XSL).	2

Y

Term	Plain-English meaning	Ch
YARN	Hadoop's cluster resource manager, which schedules jobs onto machines.	9, 10

Z

Term	Plain-English meaning	Ch
Zab	ZooKeeper's consensus (total order broadcast) protocol.	9
ZeroMQ / nanomsg	Libraries for sending messages directly between programs without a broker.	11
ZooKeeper	Open-source coordination service for leader election, locks, configuration and service discovery.	6, 9
zxid	ZooKeeper's transaction ID, which increases with each change and can serve as a fencing token.	8, 9

Chapter 14 — Interview Question Bank

How to use this bank

This chapter collects the data-systems questions that come up again and again in senior backend, data-engineering and system-design interviews. They are the questions you see in LeetCode Discuss interview-experience posts, in public system-design and database question lists, and in the "explain X" / "X vs Y" rounds that follow a design exercise.

- **Wording.** Each question is written the way interviewers usually ask it, so you recognise it when you hear it.
- **★ = asked very often.** These showed up across many sources. If you are short on time, drill only the ★ questions first.
- **Answers.** Each answer is short enough to say out loud in 30–60 seconds. Every new term is defined the first time it appears. The answers are written for this book and follow the book's explanations.
- **More.** The *More* link points to the chapter of these notes that explains the topic in full. A few questions (for example the transactional outbox, Kafka's exactly-once settings, Raft's details) go beyond the book; they still get a full answer here, and the link points to the closest chapter or to Chapter 0a — System Design Fundamentals.
- **How to drill.** Cover the answer, read the question, say your answer out loud (or write it in two or three lines), then uncover and compare. Mark the ones you missed and repeat them the next day. For the fast-revision version of the same material, use the Interview Edition.
- **The senior move.** Interviewers at senior level want three things in every answer: the definition, the trade-off ("it costs X to get Y"), and a real system that makes that choice (PostgreSQL, Cassandra, Kafka, ZooKeeper, Spanner…). The answers below try to give all three.

Group	Questions
Reliability, scalability and maintainability	Q1–Q12
Data models and query languages	Q13–Q24
Storage engines and indexes (LSM vs B-tree)	Q25–Q42
OLTP vs OLAP and column storage	Q43–Q50
Encoding and schema evolution	Q51–Q62
Replication	Q63–Q84
Partitioning (sharding)	Q85–Q98
Transactions and isolation anomalies	Q99–Q120
The trouble with distributed systems	Q121–Q134
Consistency and consensus	Q135–Q156
Batch processing	Q157–Q166
Stream processing, CDC and event sourcing	Q167–Q184
Exactly-once, idempotency and data integration	Q185–Q194
Scenario and design questions	Q195–Q206

Reliability, scalability and maintainability

Q1. ★ What do reliability, scalability and maintainability mean for a data system? **Reliability** means users still get the correct result, fast enough, while disks, code or operators are misbehaving.

Scalability means having reasonable ways to cope as load grows — data volume, traffic or complexity. **Maintainability** means many different people can work on the system over years productively: operate it, understand it and change it. A good answer adds that these are not features you bolt on; each is a set of design choices with costs. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q2. ⋆ What is the difference between a fault and a failure? A **fault** is one component deviating from its spec — a disk dies, a process crashes, a network link drops packets. A **failure** is when the system as a whole stops giving the required service to the user. Fault-tolerant design aims to stop faults from turning into failures, for example by replicating data so that one dead disk does not lose anything. You cannot reduce faults to zero, so you design to tolerate them, and you may even inject faults on purpose (chaos engineering, Netflix's Chaos Monkey) to test the tolerance. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q3. What kinds of faults should a data system expect? Three kinds. **Hardware faults** (disks, RAM, power, network) are mostly random and independent, so redundancy handles them; at scale they are constant — a cluster of 10,000 disks loses about one per day. **Software errors** are systematic: one bug or one bad input can crash every node at once (for example a leap-second bug), so they are more dangerous. **Human errors** (bad config pushes) cause the largest share of outages; you reduce them with good APIs, sandboxes, fast rollback, gradual rollouts and monitoring. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q4. ⋆ Why are percentiles better than averages for describing response time? The average (mean) hides how many users had a bad experience. The **median (p50)** tells you what a typical request sees; **p95, p99, p999** (the tail) tell you how slow the slowest 5%, 1% and 0.1% are. Tail latency matters because the slowest requests often belong to the most valuable users (they have the most data), and because of **tail latency amplification**: if one page calls 100 backends in parallel, the page is as slow as the slowest call, so a 1-in-100 slow call hits most pages. Amazon famously set targets at p999 for internal services. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q5. What is the difference between latency and response time? Throughput? Response time is what the client sees: network delay + queueing delay + service time. **Latency** strictly is the time a request waits to be handled (it is "latent"), though people often use the two words loosely. **Throughput** is how many requests (or bytes) the system handles per second. A system can have high throughput and bad latency, for example when deep queues keep the CPU busy while each request waits a long time. Queueing is also why you should measure response time on the client side. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q6. ⋆ Explain the Twitter home-timeline example. Fan-out on write or fan-out on read? Fan-out on read: when a user opens their timeline, query all the people they follow and merge the tweets — cheap writes, expensive reads. **Fan-out on write**: when someone tweets, push it into a cached timeline for each follower — cheap reads, expensive writes. Because reads were about 100× more common than writes, Twitter moved to fan-out on write, but celebrities with tens of millions of followers made one tweet into millions of writes, so they ended with a **hybrid**: fan-out on write for most users, fan-out on read for celebrities, merged at read time. The lesson: the right design depends on the **load parameters** (here, the distribution of followers per user). → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q7. ⋆ Vertical vs horizontal scaling — when do you choose each? Vertical scaling (scale up) means a bigger machine: simple, no distribution problems, but it gets expensive fast and has a hard ceiling and a single point of failure. **Horizontal scaling (scale out)** means more machines sharing the load (**shared-nothing**): almost no ceiling and better fault tolerance, but you take on replication, partitioning, and distributed-systems problems. Stateless services scale out easily; stateful databases are harder. A common senior answer is "scale up until it hurts, then partition", since a single big database is simpler to run and many systems never outgrow it. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q8. What does "elastic" mean, and when is manual scaling better? An **elastic** system adds or removes resources automatically when load changes. It helps when load is unpredictable. Manual scaling (a human decides to add nodes) is simpler and has fewer operational surprises — for example, automatic rebalancing of a database can trigger a cascade of data movement at the worst moment. Many databases deliberately keep rebalancing manual or human-approved. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q9. What are operability, simplicity and evolvability? These are the three parts of maintainability. **Operability**: make it easy for operations teams to keep the system healthy — good monitoring, automation, documentation, predictable behaviour. **Simplicity**: remove **accidental complexity** (complexity that comes from the implementation, not the problem) using good abstractions. **Evolvability**: make it easy to change the system as requirements change, which depends heavily on simplicity and on things like schema evolution and rolling upgrades. → *More:* Ch 1 — Reliable, Scalable, and Maintainable Applications

Q10. How do you calculate availability, and what does "four nines" allow? Availability is the fraction of time the service works: uptime / total time. **99.9%** ("three nines") allows about 8.8 hours of downtime per year (~43 minutes per month); **99.99%** allows about 53 minutes per year (~4.3 minutes per month); **99.999%** about 5.3 minutes per year. Two components in series multiply: two services at 99.9% each give about 99.8%. Redundant components in parallel improve it: two independent 99% replicas give $1 - 0.01^2 = 99.99\%$, if their failures really are independent. → *More:* Ch 0a — System Design Fundamentals

Q11. What are SLI, SLO and SLA? What is an error budget? An **SLI** (service level indicator) is a measured number, such as p99 latency or success rate. An **SLO** (objective) is the internal target for it ("p99 < 200 ms over 30 days"). An **SLA** (agreement) is a contract with customers with penalties if it is missed, usually looser than the SLO. The **error budget** is 100% minus the SLO: the amount of unreliability you are allowed to "spend" on risky launches; when it is used up, you freeze risky changes. → *More:* Ch 0a — System Design Fundamentals

Q12. How would you estimate the storage and QPS for a system? (back-of-the-envelope) Start from users and actions: daily active users × actions per user per day ÷ 86,400 seconds ≈ average QPS (queries per second); multiply by 2–10 for peak. Storage = records per day × bytes per record × retention days × replication factor. Round aggressively (a day ≈ 10^5 seconds) and state assumptions out loud. Example: 100M users × 10 writes/day = 10^9 writes/day ≈ 12,000 writes/s average; at 1 KB each that is 1 TB/day, 365 TB/year before replication. → *More:* Ch 0a — System Design Fundamentals

Data models and query languages

Q13. ⋆ Relational vs document databases — when do you choose each? A **relational** database (PostgreSQL, MySQL) stores rows in tables with a fixed schema and is strong at joins, many-to-many relationships and transactions. A **document** database (MongoDB, Couchbase) stores self-contained JSON-like documents; it fits data that is a tree of one-to-many relationships loaded together (a résumé with its jobs and schools), gives **locality** (one read fetches everything) and a flexible schema. Choose documents when your data has a document shape and few cross-document relationships; choose relational when you have many references between entities. The two have converged: PostgreSQL has JSONB, MongoDB has `$lookup` joins and multi-document transactions. → *More:* Ch 2 — Data Models and Query Languages

Q14. What is the object-relational mismatch (impedance mismatch)? Application code works with objects (nested, with lists inside), while relational tables are flat rows. The translation layer between them is awkward — that is the **impedance mismatch**. ORMs (object-relational mappers such as Hibernate, SQLAlchemy) hide some of it but not all, and can cause problems like the N+1 query pattern. JSON documents reduce the mismatch for tree-shaped data. → *More:* Ch 2 — Data Models and Query Languages

Q15. ⋆ Schema-on-read vs schema-on-write? Schema-on-write (relational): the database enforces the schema when data is written, so all stored data matches it — like static type checking. **Schema-on-read** (document databases): the data has an implicit structure that the code interprets when it reads it — like dynamic type checking. Schema-on-read is handy when items are heterogeneous or the structure is controlled by external systems. Changing structure: with schema-on-write you run a migration (`ALTER TABLE`); with schema-on-read the code handles both old and new shapes. → *More:* Ch 2 — Data Models and Query Languages

Q16. ⋆ What is normalization, and when would you denormalize? Normalization means storing each fact once and referring to it by ID (for example storing a `region_id` instead of the text "Greater Seattle Area" in every profile). It avoids duplicated data getting out of sync and makes updates cheap. **Denormalization** copies data into the places where it is read, so reads need no join — faster reads at the cost of more complex, slower writes and the risk of inconsistency. Denormalize for read-heavy paths

(feeds, search indexes, caches) and treat the copies as derived data you can rebuild. → *More:* Ch 2 — Data Models and Query Languages

Q17. How do one-to-many, many-to-one and many-to-many relationships affect the data-model choice? One-to-many (a user has many positions) fits a nested document naturally. Many-to-one (many people live in one region) and many-to-many (people and organisations) need references and joins. Document databases historically had weak join support, so the application had to do the joins itself. As an application grows, data tends to become more interconnected, which pushes toward relational or graph models. → *More:* Ch 2 — Data Models and Query Languages

Q18. Declarative vs imperative query languages — why does SQL being declarative matter? A **declarative** language (SQL, CSS) says *what* result you want, not *how* to compute it. An **imperative** language spells out each step and order. Declarative queries let the database's **query optimizer** choose indexes and join methods, change them later without changing your query, and run the query in parallel across cores or machines. Imperative code fixes the algorithm and the order, which is hard to parallelise. → *More:* Ch 2 — Data Models and Query Languages

Q19. ⋆ When would you use a graph database? When many-to-many relationships are the main point of the data and queries follow paths of unknown length: social networks ("friends of friends"), fraud rings, recommendation, road networks, knowledge graphs. In the **property graph** model (Neo4j) each vertex and edge has a label and properties; **Cypher** is its declarative query language. In SQL you can do variable-length traversals with recursive common table expressions (`WITH RECURSIVE`), but the query becomes much longer. → *More:* Ch 2 — Data Models and Query Languages

Q20. Property graph vs triple-store? A **property graph** stores vertices and edges, each with a set of key-value properties. A **triple-store** stores everything as (subject, predicate, object) triples, for example (`lucy`, `marriedTo`, `alain`) or (`lucy`, `age`, `33`); it is queried with **SPARQL** and is the basis of RDF and the semantic web. They are largely equivalent ways to model the same graph. **Datalog** is an older, rule-based query language that underlies systems like Datomic. → *More:* Ch 2 — Data Models and Query Languages

Q21. Write a recursive SQL query to find all of a node's descendants.

```
WITH RECURSIVE sub(id) AS (
  SELECT id FROM category WHERE id = 42          -- start node
  UNION
  SELECT c.id FROM category c JOIN sub s ON c.parent_id = s.id
)
SELECT id FROM sub;
```

The non-recursive part seeds the result; the recursive part joins children of rows already found until no new rows appear. `UNION` (not `UNION ALL`) also stops cycles from looping forever. → *More:* Ch 2 — Data Models and Query Languages

Q22. What was MapReduce querying in MongoDB, and why did aggregation pipelines replace it? MapReduce querying let you pass two JavaScript functions (map and reduce) to run over matching documents. It was neither fully declarative nor fully imperative, and writing two coordinated functions is harder than one query, and the database can optimise it less. MongoDB added the declarative **aggregation pipeline** (`$match`, `$group`, `$sort`…), and map-reduce was deprecated in MongoDB 5.0. → *More:* Ch 2 — Data Models and Query Languages

Q23. What does "NoSQL" really mean, and what is polyglot persistence? "NoSQL" started as a Twitter hashtag for a 2009 meetup and is now read as "Not Only SQL". It groups databases built for greater scalability, open-source, specialised queries or schema flexibility. **Polyglot persistence** means using several data stores side by side in one application, each for what it is good at (PostgreSQL for orders, Redis for cache, Elasticsearch for search). → *More:* Ch 2 — Data Models and Query Languages

Q24. How do you store and query JSON in PostgreSQL efficiently? Use the `jsonb` type (binary, parsed once, supports indexing) rather than `json` (stored as text). Query with operators like `->>` and containment `@>`, and add a **GIN index** (generalised inverted index) for containment queries. Keep fields you filter or join on often as real columns; use `jsonb` for the truly variable parts.

```
CREATE INDEX ON events USING gin (payload jsonb_path_ops);
SELECT * FROM events WHERE payload @> '{"type": "signup"}';
```

→ *More:* Ch 2 — Data Models and Query Languages

Storage engines and indexes (LSM vs B-tree)

Q25. ⋆ How does a database store and find data on disk? Explain the simplest possible database. The simplest database appends `key,value` lines to a file (a **log**: an append-only sequence of records) and reads by scanning for the last line with that key. Appending is very fast, but reading is O(n). An **index** is an extra structure derived from the data that speeds up reads. Every index slows writes, because it must be updated on each write — this read/write trade-off is the core of storage-engine design. → *More:* Ch 3 — Storage and Retrieval

Q26. What is a hash index (Bitcask)? Keep an in-memory hash map from each key to the byte offset of its latest value in the append-only data file. A read is one hash lookup plus one disk seek; writes are appends. The log is split into **segments**; old segments are **compacted** (duplicate keys thrown away, keeping only the latest) and merged in the background. Deletes write a **tombstone** (a special deletion marker). Limits: all keys must fit in RAM, and range queries are inefficient. Bitcask (Riak's default engine) works this way. → *More:* Ch 3 — Storage and Retrieval

Q27. ⋆ What is an LSM-tree and how does it work? An **LSM-tree** (log-structured merge-tree) buffers writes in an in-memory sorted structure called the **memtable** (for example a red-black tree or skip list). When it gets big (a few MB), it is written to disk as an **SSTable** (sorted string table: a file of key-value pairs sorted by key). Reads check the memtable, then the newest SSTable, then older ones. Background **compaction** merges SSTables, discarding overwritten and deleted values. A **write-ahead log** protects the memtable from crashes. Used by LevelDB, RocksDB, Cassandra, HBase, ScyllaDB. → *More:* Ch 3 — Storage and Retrieval

Q28. ⋆ How does a B-tree work? A **B-tree** splits the database into fixed-size **pages** (traditionally 4 KB; PostgreSQL uses 8 KB, InnoDB 16 KB). Each page holds keys and references to child pages covering key ranges; leaves hold the values (or pointers to them). A lookup walks from the root to a leaf — with a branching factor of a few hundred, a 4-level tree of 4 KB pages can hold up to 256 TB. Updates overwrite the page in place; if a page is full it **splits** into two and the parent is updated. A **write-ahead log (WAL)** makes this crash-safe. B-trees are the standard index in almost every relational database. → *More:* Ch 3 — Storage and Retrieval

Q29. ⋆ LSM-tree vs B-tree — compare them. Which would you pick for a write-heavy workload?

	LSM-tree	B-tree
Writes	Sequential appends, high throughput	Random in-place page writes
Reads	May check several SSTables (Bloom filters help)	One root-to-leaf path, predictable
Write amplification	Compaction rewrites data several times	Each write = WAL + whole page
Space	Compresses well, less fragmentation	Pages partly empty after splits
Latency risk	Compaction can stall or fall behind	Steadier
Transactions	Key may exist in several places	Each key in one place, easy range locks

Rule of thumb: LSM for write-heavy, append-like ingestion (metrics, logs, time series, messaging); B-tree for read-heavy OLTP with predictable latency. Always benchmark with your real workload. → *More:* Ch 3 — Storage and Retrieval

Q30. ⋆ What is write amplification, and why does it matter? Write amplification is the number of bytes actually written to disk per byte the application writes. B-trees write the WAL and then a whole page (even for a tiny change); LSM-trees rewrite data during each compaction level. On SSDs, which wear out after a limited number of write cycles, and on disks with limited write bandwidth, high write amplification directly reduces throughput and lifetime. Read amplification (disk reads per query) and space amplification are the other two sides of the same trade-off. → *More:* Ch 3 — Storage and Retrieval

Q31. ⋆ What is a Bloom filter, and why do LSM-trees use one? A **Bloom filter** is a small bit array plus several hash functions that answers "is this key in the set?" with either "definitely not" or "probably yes" — no false negatives, some false positives. An LSM-tree keeps one per SSTable, so a read for a

missing key can skip almost every file without touching disk. With about 10 bits per key the false-positive rate is about 1%. You cannot delete from a plain Bloom filter (a counting Bloom filter can). → *More:* Ch 3 — Storage and Retrieval

Q32. Size-tiered vs leveled compaction? **Size-tiered** compaction merges newer, smaller SSTables into older, larger ones of similar size: good write throughput, but more space used temporarily and more files to check on reads (Cassandra's default STCS). **Leveled** compaction keeps levels (L0, L1, L2…), each about 10× bigger than the last, with non-overlapping key ranges within a level: lower read and space amplification, more write amplification (LevelDB, RocksDB default). Pick leveled for read-heavy or update-heavy data, size-tiered for write-heavy insert-mostly data. → *More:* Ch 3 — Storage and Retrieval

Q33. How does a B-tree survive a crash in the middle of a page split? A page split writes several pages; a crash midway could leave an orphan page or a broken tree. The B-tree first records every change in a **write-ahead log** (WAL, also called a redo log), an append-only file that is flushed to disk before the pages change. After a crash, the database replays the WAL to restore a consistent tree. Some engines use **copy-on-write** instead (write modified pages to new locations, then atomically switch the parent pointer), as LMDB does; this also helps snapshot reads. → *More:* Ch 3 — Storage and Retrieval

Q34. ⋆ What are clustered, non-clustered and covering indexes? A **clustered index** stores the full row inside the index's leaf (InnoDB's primary key is clustered; there is one per table). A **non-clustered (secondary) index** stores a reference — in InnoDB the primary key, in PostgreSQL a pointer into the **heap file** (the unordered place rows live). A **covering index** (index with included columns) stores enough extra columns to answer a query from the index alone ("index-only scan"). Trade-off: faster reads, more storage and slower writes. → *More:* Ch 3 — Storage and Retrieval

Q35. How does a composite (multi-column) index work, and why does column order matter? A **concatenated index** on `(last_name, first_name)` sorts by the first column, then the second within ties — like a phone book. It can answer queries on `last_name`, or on `last_name AND first_name`, but not on `first_name` alone (the **leftmost-prefix rule**). Put equality-filtered columns first, then the range or sort column. For true two-dimensional ranges (latitude and longitude), use a spatial index such as an R-tree, not a composite B-tree. → *More:* Ch 3 — Storage and Retrieval

Q36. Why might adding an index make the system slower? Every index must be updated on each insert, update and delete, which adds write I/O, WAL volume, lock contention and storage. Unused or redundant indexes also take memory away from the buffer cache. And the optimizer may choose a poor index plan. So index only for real query patterns and remove unused ones (PostgreSQL's `pg_stat_user_indexes` shows usage). → *More:* Ch 3 — Storage and Retrieval

Q37. How do full-text search and fuzzy indexes work? Full-text search uses an **inverted index**: a map from each term to the list of documents (postings) containing it. Lucene (behind Elasticsearch and Solr) stores the term dictionary in SSTable-like segments that are merged like an LSM-tree, and keeps the term dictionary in a finite-state automaton similar to a trie, which allows searching for words within a given **edit distance** (fuzzy search). Text is first analysed: split into tokens, lower-cased, stemmed, synonyms added. → *More:* Ch 3 — Storage and Retrieval

Q38. Are in-memory databases fast because they avoid disk reads? Mostly not. A disk-based database with enough RAM already serves reads from the OS page cache. In-memory databases (Redis, VoltDB, MemSQL/SingleStore) are faster because they avoid the cost of **encoding in-memory structures into a disk format**, and they can offer data structures (sets, priority queues) that are hard to build on disk. They stay durable through snapshots, logs on disk or replication. → *More:* Ch 3 — Storage and Retrieval

Q39. What is a write-ahead log and why does almost every database have one? A **WAL** is an append-only file where every change is recorded and flushed (`fsync`) to disk *before* the change is applied to the main data files. If the machine crashes, replaying the WAL restores all committed changes — this gives the "D" (durability) in ACID. Sequential appends are fast, so the database can delay writing the real data pages. The same log is often reused for replication (PostgreSQL streaming replication ships WAL) and for change data capture. → *More:* Ch 3 — Storage and Retrieval

Q40. What does `fsync` do, and why do databases care about it? Writing to a file normally only copies data into the OS page cache; a power cut can lose it. `fsync` forces the data to stable storage and waits until the device says it is written. Databases call it on commit (or in a group commit for many transactions at once) to guarantee durability. Some settings trade safety for speed — for example `synchronous_commit = off` in PostgreSQL or `innodb_flush_log_at_trx_commit = 2` in MySQL can lose the most

recent commits (roughly the last second) on a crash — for the MySQL setting, an OS crash or power loss, not just a mysqld crash. → *More:* Ch 3 — Storage and Retrieval

Q41. How are deletes handled in an LSM-tree, and what problems do tombstones cause? A delete writes a **tombstone** (a marker that the key is deleted) because older SSTables cannot be changed. The value disappears only when compaction merges the tombstone with all older copies. Problems: reads that scan many tombstones get slow (Cassandra warns about this), and in a replicated system a tombstone must not be purged before every replica has seen it, or the deleted data can come back ("zombie" data; Cassandra's `gc_grace_seconds` sets this window). → *More:* Ch 3 — Storage and Retrieval

Q42. How would you design a key-value store's storage engine? (Bitcask / LSM design question) Start with an append-only log plus an in-memory hash index (Bitcask) and say its limits: keys must fit in RAM, no range scans. Then move to an LSM-tree: WAL, memtable, flush to SSTables with a sparse index and a Bloom filter each, background compaction, tombstones for deletes, checksums to detect torn writes, and crash recovery by replaying the WAL. Discuss compaction strategy, how compaction can fall behind under sustained writes, and how you would add replication and partitioning on top. → *More:* Ch 3 — Storage and Retrieval

OLTP vs OLAP and column storage

Q43. ⋆ What is the difference between OLTP and OLAP? OLTP (online transaction processing) serves the application: many small reads and writes, each touching a few rows found by key, low latency, current data. **OLAP** (online analytic processing) serves analysts: few but huge queries that scan millions of rows and aggregate a few columns, over historical data. They need different storage layouts, so companies run OLAP in a separate **data warehouse** loaded by **ETL** (extract-transform-load) from the OLTP databases. → *More:* Ch 3 — Storage and Retrieval

Q44. ⋆ Why is column-oriented storage faster for analytics? A **column store** keeps all values of each column together, instead of each row together. An analytic query usually reads 4 or 5 columns of a table that has hundreds, so it reads only those columns from disk. Columns also compress very well (similar values next to each other), and the CPU can process compressed column chunks in tight loops that fit in cache and use SIMD (**vectorized processing**). Examples: Parquet files, ClickHouse, Snowflake, BigQuery, Redshift, DuckDB. → *More:* Ch 3 — Storage and Retrieval

Q45. What compression techniques do column stores use? Bitmap encoding: for a column with few distinct values, store one bitmap per value (bit i = 1 if row i has that value); `WHERE x IN (a, b)` becomes a bitwise OR. Sparse bitmaps are then **run-length encoded** (store "5 zeros, 2 ones, …"). Other common schemes: dictionary encoding, delta encoding for sorted numbers, and general-purpose compression (LZ4, Zstd). Sorting the data first makes all of them work better. → *More:* Ch 3 — Storage and Retrieval

Q46. What are star and snowflake schemas? In a **star schema** a central **fact table** stores one row per event (a sale, a click) with measures (price, quantity) and foreign keys to **dimension tables** (who, what, where, when — product, store, customer, date). Drawn out, the dimensions surround the fact table like star points. A **snowflake schema** further normalises dimensions into sub-dimensions (product → brand → category). Star schemas are simpler for analysts; fact tables can have trillions of rows and are often 100+ columns wide. → *More:* Ch 3 — Storage and Retrieval

Q47. How does sort order help a column store, and why keep several sort orders? Rows are stored in a chosen order (for example by date, then product), applied to all columns together so the nth entry of each column still belongs to the same row. Sorting lets queries skip whole ranges and makes the first sort column compress extremely well with run-length encoding. Vertica's idea: since data is replicated anyway, store each replica in a different sort order and let each query use the best one. → *More:* Ch 3 — Storage and Retrieval

Q48. How do column stores handle writes? Inserting a row in the middle of sorted, compressed columns would mean rewriting every column file. So writes go first to an in-memory store (row-oriented or sorted, like an LSM memtable); when enough accumulate they are merged with the on-disk column files and written as new files in bulk. Queries read both and combine them. This is the LSM idea applied to columns. → *More:* Ch 3 — Storage and Retrieval

Q49. What are materialized views and data cubes? A **materialized view** stores the actual result of a query on disk (unlike a virtual view, which is just a saved query) and must be updated when the

underlying data changes — that makes writes costlier, so they suit read-heavy warehouses. A **data cube (OLAP cube)** is a special materialized view: a grid of aggregates (for example SUM of sales) grouped by several dimensions at once, so "sales per store per day" is a lookup. It is fast but inflexible: you cannot ask about a dimension that is not in the cube. → *More:* Ch 3 — Storage and Retrieval

Q50. Data warehouse vs data lake vs lakehouse? A **data warehouse** stores cleaned, structured data in a schema designed for analytics (Snowflake, BigQuery, Redshift). A **data lake** stores raw files of any format cheaply in object storage (S3) and applies schema when reading. A **lakehouse** adds warehouse features on top of lake files using an open **table format** — Apache Iceberg, Delta Lake or Apache Hudi — which adds ACID commits, schema evolution and time travel to Parquet files. These table formats are newer than the book, but they build on its columnar-storage ideas. → *More:* Ch 3 — Storage and Retrieval

Encoding and schema evolution

Q51. ⋆ What are backward and forward compatibility? Backward compatibility: newer code can read data written by older code. **Forward compatibility**: older code can read data written by newer code. You need both because during a **rolling upgrade** (updating nodes one at a time) and with mobile clients that never update, old and new versions run at the same time and read each other's data. Backward compatibility is usually easy (you know the old format); forward compatibility is harder (old code must ignore fields it does not know). → *More:* Ch 4 — Encoding and Evolution

Q52. ⋆ JSON vs Protocol Buffers vs Avro — which would you choose and why? JSON is human-readable and universal, but verbose, has no schema by default, cannot tell integers from floats (numbers above 2^{53} lose precision in JavaScript) and has no binary strings. **Protocol Buffers** (and Thrift) use a schema with numbered **field tags**; the encoded bytes contain tags, not names, so they are compact and support evolution via tags. **Avro** has no tags: the writer's and reader's schemas are resolved side by side by field name, which suits dynamically generated schemas (for example, dumping a database table). Choose JSON for public APIs and debugging, Protobuf for service-to-service RPC (gRPC), Avro for data pipelines and Kafka with a schema registry. → *More:* Ch 4 — Encoding and Evolution

Q53. What are the rules for evolving a Protocol Buffers schema safely? Never change or reuse a field's **tag number** — it is the field's identity on the wire. You may add new fields with new tags; old code skips unknown tags (forward compatible), and new code sees a default for missing fields (backward compatible). Delete a field only if it is not required, and mark its number and name `reserved` so nobody reuses them. Changing types is risky: widening int32 to int64 works for new readers, but old readers truncate the value. In proto3 every field is optional; `required` (proto2) is a trap because you can never remove it. → *More:* Ch 4 — Encoding and Evolution

Q54. ⋆ How does Avro handle schema evolution? What is a writer's and reader's schema? Data is encoded with the **writer's schema**; the reading application decodes with its own **reader's schema**. The Avro library compares the two by field name: fields only in the writer's schema are ignored; fields only in the reader's schema are filled with the default value. So you may only add or remove fields that have a **default** value. The reader must know the writer's schema: store it once at the start of a large file, store a version number per database record, or (in Kafka) put a schema ID in every message that points to a **schema registry**. → *More:* Ch 4 — Encoding and Evolution

Q55. What is a schema registry and what compatibility modes does it enforce? A **schema registry** (Confluent Schema Registry, AWS Glue Schema Registry) stores every version of every schema and gives it an ID. Producers write the ID in each message; consumers fetch the schema by ID. Before registering a new version, it checks compatibility: **BACKWARD** (new readers can read old data — the Confluent default), **FORWARD** (old readers can read new data), **FULL** (both), and *_TRANSITIVE variants that check against all past versions, not just the latest. → *More:* Ch 4 — Encoding and Evolution

Q56. Why should you avoid language-specific serialization (Java Serializable, Python pickle)? They tie the data to one language, evolve poorly between versions, are often slow and bulky, and — most importantly — decoding can instantiate arbitrary classes, so reading untrusted bytes can lead to **remote code execution**. Use them only for very short-lived, same-process purposes, if at all. → *More:* Ch 4 — Encoding and Evolution

Q57. What are the three main modes of dataflow between processes?

1. **Through databases**: the writer encodes, a reader (maybe a future version of the code) decodes later — the book calls this sending a message to your future self. 2. **Through services**: REST or RPC calls — the client encodes a request, the server decodes it, then the reverse for the response. 3. **Through asynchronous message passing**: via a message broker (RabbitMQ, Kafka); the sender does not wait. Each mode has its own compatibility and failure concerns. → *More:* Ch 4 — Encoding and Evolution

Q58. What is the danger when old code reads and rewrites a record containing a new field? If an old version of the app reads a record that has a field added by a newer version, decodes it into an object that has no place for that field, and writes it back, the new field is silently lost. The fix: the old code (or its library) must preserve unknown fields when it rewrites the record. Protobuf preserves unknown fields when it re-serialises a message; hand-written ORM code often does not. → *More:* Ch 4 — Encoding and Evolution

Q59. What does "data outlives code" mean for database migrations? Code is redeployed in minutes, but the data in a database may have been written five years ago by a version long gone. Rewriting all data on each schema change is expensive, so most databases allow cheap changes like adding a nullable column with no default (old rows read as null). In practice you do **expand/contract** migrations: add the new column, deploy code that writes both, backfill, switch reads, then drop the old column — each step compatible with the version before it. → *More:* Ch 4 — Encoding and Evolution

Q60. REST vs RPC — what are the problems with making a remote call look like a local function call? RPC (remote procedure call) tries to make a network request look like a local call (location transparency), but a network call is fundamentally different: it can time out with no idea whether the action happened; retries can repeat the action unless it is **idempotent** (safe to apply more than once); latency varies wildly; parameters must be encoded; and client and server may be in different languages. **REST** embraces the network: resources, HTTP methods, caching and status codes. Modern RPC frameworks (gRPC) are explicit about the network — futures/promises, streams, deadlines. → *More:* Ch 4 — Encoding and Evolution

Q61. How do you version an API so old and new clients keep working? For services you usually only need backward compatibility on requests and forward compatibility on responses, because servers are upgraded before clients. Common approaches: version in the URL (`/v2/orders`) or in an `Accept` header; additive changes only within a version (new optional fields, never removed or retyped fields); and keep old versions alive for as long as clients use them (forever for public APIs with mobile clients). With gRPC/Protobuf, follow the tag rules in Q53. → *More:* Ch 4 — Encoding and Evolution

Q62. What is the actor model, and how does it relate to message passing? An **actor** is a unit of logic with private state that communicates only by sending asynchronous messages to other actors, processing one message at a time — so no locks are needed. Distributed actor frameworks (Akka, Orleans, Erlang/OTP) spread actors across nodes, so the same message encoding and the same forward/backward compatibility rules apply during rolling upgrades. → *More:* Ch 4 — Encoding and Evolution

Replication

Q63. ⋆ Why replicate data? Three reasons: **high availability** (keep working when a node or datacentre dies), **lower latency** (keep a copy geographically close to users), and **read scalability** (serve more reads from more copies). Replication is easy when data never changes; all the difficulty is in handling changes. The three main approaches: single-leader, multi-leader and leaderless. → *More:* Ch 5 — Replication

Q64. ⋆ How does leader-follower (single-leader) replication work? One replica is the **leader** (primary/master); all writes go to it. It writes the change locally and sends it to the **followers** (replicas/secondaries) through a **replication log**; each follower applies the changes in the same order. Reads can go to the leader or followers. Used by PostgreSQL, MySQL, MongoDB replica sets, SQL Server availability groups, Kafka partitions and RabbitMQ quorum queues. → *More:* Ch 5 — Replication

Q65. ⋆ Synchronous vs asynchronous replication — trade-offs? With **synchronous** replication the leader waits for the follower to confirm before telling the client the write succeeded: the follower is guaranteed up to date, but one slow or dead follower blocks all writes. With **asynchronous** replication the leader does not wait: fast and available, but if the leader dies, recently confirmed writes not yet

copied are lost. The usual compromise is **semi-synchronous**: one follower synchronous, the rest asynchronous, so at least two nodes always have the latest data. → *More:* Ch 5 — Replication

Q66. How do you add a new follower without downtime? Take a consistent **snapshot** of the leader (ideally without locking — most databases support this for backups), copy it to the new node, then have the follower request all changes since the snapshot's exact position in the replication log (PostgreSQL's log sequence number, MySQL's binlog coordinates or GTID). When it has caught up, it streams changes like any other follower. → *More:* Ch 5 — Replication

Q67. ⋆ What happens when the leader fails? What can go wrong during failover? Failover: detect the failure (usually a timeout), choose a new leader (the most up-to-date follower, by election or by a controller), and reconfigure clients and the old leader to use it. Things that go wrong: with asynchronous replication, the new leader may be missing writes, which are usually discarded — dangerous if other systems already saw them (GitHub's incident where reused auto-increment IDs leaked data into Redis); **split brain**, where two nodes both act as leader; and choosing the timeout — too short causes needless failovers under load, too long means long outages. → *More:* Ch 5 — Replication

Q68. ⋆ What is split brain and how do you prevent it? Split brain is when two nodes both believe they are the leader and both accept writes, so data diverges or is corrupted. Prevent it by requiring a **majority quorum** to elect a leader (a minority side cannot elect one), by **fencing** the old leader (STONITH — "shoot the other node in the head" — cutting its power or access), and by **fencing tokens**: every new leader gets a higher epoch number and storage rejects writes carrying an older one. → *More:* Ch 5 — Replication

Q69. Compare statement-based, WAL shipping, logical (row-based) and trigger-based replication. Statement-based sends the SQL statements; it breaks on nondeterminism (`NOW()`, `RAND()`, auto-increments, triggers). **WAL shipping** sends the storage engine's low-level log; exact, but tied to the storage format, so leader and followers usually must run the same version (no zero-downtime upgrades). **Logical (row-based) log** sends row-level changes ("row X: column Y changed to Z"), separate from the storage format — it allows mixed versions and is the basis of change data capture (MySQL binlog in row mode, PostgreSQL logical replication). **Trigger-based** uses database triggers to copy changes to another table, read by an external process — flexible but slower and more fragile. → *More:* Ch 5 — Replication

Q70. ⋆ What is replication lag, and what problems does it cause? Replication lag is the delay between a write on the leader and its appearance on a follower — usually a fraction of a second, but seconds or minutes under load or failures. Reads from a lagging follower give stale data: the system is only **eventually consistent** (replicas converge if writes stop, with no bound on how long). Three classic anomalies follow: not reading your own write, time going backwards (monotonic reads), and seeing an answer before its question (consistent prefix). → *More:* Ch 5 — Replication

Q71. ⋆ What is read-your-writes consistency and how do you implement it? Read-after-write (read-your-writes) consistency guarantees a user always sees the writes they made themselves (it says nothing about other users). Implementations: read things the user may have modified (their own profile) from the leader; for a minute after a user's last write, read from the leader; or remember the timestamp/log position of the user's last write and only read from a replica that has caught up to it. Cross-device versions are harder: the "last write" metadata must be centralised, and devices may be routed to different datacentres. → *More:* Ch 5 — Replication

Q72. What are monotonic reads? Monotonic reads guarantee a user never sees time go backwards: after reading newer data, they will not later read older data. The anomaly happens when one refresh hits an up-to-date replica and the next hits a lagging one (a comment appears, then disappears). Fix: route each user to the same replica, for example by hashing the user ID (and reroute only if that replica fails). → *More:* Ch 5 — Replication

Q73. What is consistent prefix reads? Consistent prefix reads guarantee that if writes happen in a certain order, anyone reading them sees them in that order. It breaks in partitioned databases, where different partitions replicate at different speeds — an observer might see an answer before the question it answers. Fix: write causally related data to the same partition, or track causal dependencies explicitly. → *More:* Ch 5 — Replication

Q74. ⋆ When would you use multi-leader replication? What are its drawbacks? Use **multi-leader** (master-master) replication for **multi-datacentre** deployments (each region has a leader, writes are local, regions sync asynchronously), for **offline clients** (each device is effectively a leader, like a calendar

app), and for **collaborative editing** (Google Docs-style). Gains: lower write latency, tolerance of datacentre and network outages. The big drawback: the same data can be changed concurrently in two places, so you must resolve **write conflicts**, and features like auto-increment keys, triggers and integrity constraints become hazardous. → *More:* Ch 5 — Replication

Q75. ⋆ How do you resolve write conflicts in multi-leader or leaderless systems? Best: **avoid** them by routing all writes for a given record to the same leader (for example a user's "home" datacentre). Otherwise converge: **last write wins (LWW)** — keep the write with the highest timestamp, which silently loses data; merge the values (for example union of sets, or concatenation); record the conflict and let application code or the user resolve it later (on write or on read, as CouchDB does); or use **CRDTs** (Q77). Whatever the method, every replica must reach the same final value. → *More:* Ch 5 — Replication

Q76. Why is "last write wins" dangerous? LWW picks the write with the largest timestamp and throws the others away, even though the client was told they succeeded — silent data loss. Worse, the timestamps come from unsynchronised clocks: a node whose clock is ahead can make its writes "win" over writes that actually happened later. LWW is safe only when each key is written once and never updated (for example, using a UUID as the key for each new write). Cassandra uses LWW by default. → *More:* Ch 5 — Replication

Q77. What is a CRDT? A **CRDT** (conflict-free replicated data type) is a data structure designed so that concurrent updates on different replicas can always be merged automatically into the same result, in any order. Examples: G-Counter (each replica counts its own increments; the value is the sum), OR-Set (add/remove with unique tags), sequence CRDTs for collaborative text (as in Automerge and Yjs). Riak has built-in CRDT types; Redis Enterprise uses them for active-active geo-replication. The cost is extra metadata and limited operations. → *More:* Ch 5 — Replication

Q78. ⋆ How does leaderless (Dynamo-style) replication work? There is no leader: the client (or a coordinator node) sends each write to all n replicas and reads from several replicas in parallel. Nodes that were down catch up via **read repair** (a reader that sees a stale value writes the newer one back) and **anti-entropy** (a background process compares replicas, often using Merkle trees, and copies missing data). The design comes from Amazon's Dynamo paper and is used by Cassandra, ScyllaDB and Riak. (Amazon DynamoDB, the cloud service, actually uses leader-based replication per partition.) → *More:* Ch 5 — Replication

Q79. ⋆ Explain quorums: why does w + r > n give up-to-date reads? With n replicas, a write must be acknowledged by w nodes and a read must query r nodes. If **w + r > n**, the set of nodes written and the set read must overlap in at least one node, so at least one of the values read is the latest. A typical choice is n = 3, w = 2, r = 2, which tolerates one unavailable node. Smaller w or r means lower latency and higher availability but more stale reads; w = n, r = 1 makes reads fast but any one dead node blocks writes. → *More:* Ch 5 — Replication

Q80. ⋆ Why can quorum reads still return stale data even with w + r > n? Several edge cases: a **sloppy quorum** may write to nodes outside the usual n; two concurrent writes may be resolved with LWW and one lost; a write concurrent with a read may be seen by some replicas only; a write that failed on most nodes is not rolled back on the ones where it succeeded; a node holding a new value fails and is restored from a replica with an old one; and timing edge cases with clocks. So Dynamo-style quorums do not give **linearizability** (Q135); treat w and r as tuning knobs for the probability of staleness, not hard guarantees. → *More:* Ch 5 — Replication

Q81. What are sloppy quorums and hinted handoff? During a network problem, a client may be unable to reach enough of a key's n "home" nodes. A **sloppy quorum** accepts the write on any w reachable nodes, even ones that do not normally hold that key. When the home nodes return, the stand-in nodes send them the writes — **hinted handoff**. It raises write availability, but a read of r home nodes may miss the latest value, so it is not a real quorum. Riak enables it by default; Cassandra has hinted handoff but does not count hints toward the consistency level (the only exception is consistency level `ANY`). → *More:* Ch 5 — Replication

Q82. How do you detect concurrent writes? What is a version vector? Two operations are **concurrent** if neither knew about the other (not about wall-clock overlap). A server keeps a **version number** per key, incremented on each write; a client must read before writing and send back the version it read. A write that includes version v overwrites everything up to v but keeps later values as concurrent **siblings** for the application to merge. With several replicas you need one counter per replica — a **version vector** —

and comparing two vectors tells you whether one supersedes the other or they are concurrent. (Vector clocks are a closely related idea.) → *More:* Ch 5 — Replication

Q83. How do you deal with deletions when merging siblings (e.g. a shopping cart)? If you merge siblings by taking the union of their items, an item removed in one sibling comes back from the other. So a deletion must leave a **tombstone** — a marker with a version saying "this item was removed" — so the merge knows to drop it. This is exactly the problem CRDTs (such as the OR-Set) solve automatically. The Dynamo paper reported deleted items reappearing in shopping carts for this reason. → *More:* Ch 5 — Replication

Q84. In MongoDB, what do write concern and read concern control? Write concern says how many replica-set members must acknowledge a write before it succeeds: `w: 1` (primary only), `w: "majority"` (the implicit default since MongoDB 5.0, except in some replica sets with arbiters, where it falls back to `w: 1`), plus `j: true` to wait for the journal. **Read concern** sets what data a read may see: `local` (whatever this node has, possibly rolled back later), `majority` (only majority-committed data, never rolled back), `linearizable`. Together with **read preference** (primary vs secondaries) they are MongoDB's version of the quorum and replication-lag trade-offs in this chapter. → *More:* Ch 5 — Replication

Partitioning (sharding)

Q85. ⋆ What is partitioning (sharding) and why do we need it? Partitioning splits a large dataset into smaller pieces called partitions (shards in MongoDB/Elasticsearch, regions in HBase, tablets in Bigtable, vnodes in Cassandra), each stored on a different node. The main reason is **scalability**: data volume and query load are spread across many machines, beyond what one machine can hold. Each record belongs to exactly one partition, and partitioning is usually combined with replication, so each partition is copied on several nodes. → *More:* Ch 6 — Partitioning

Q86. ⋆ Range partitioning vs hash partitioning? Key-range partitioning assigns a contiguous range of keys to each partition (like encyclopedia volumes A–B, C–D…). Range scans are efficient, but some access patterns create **hot spots** — for example, timestamp keys send all of today's writes to one partition. **Hash partitioning** applies a hash function to the key and assigns ranges of hashes; load spreads evenly, but range queries must hit every partition. Cassandra combines both with a **compound primary key**: the first part is hashed to pick the partition, and the rest sorts the rows within it. → *More:* Ch 6 — Partitioning

Q87. ⋆ What is a hot spot / skew and how do you fix a hot key? Skew means some partitions get more data or queries than others; a partition with very high load is a **hot spot**. Even hashing cannot fix one hot key (a celebrity's posts all share one user ID). Fixes: add a random suffix (say 00–99) to the hot key so its writes spread over 100 partitions — reads must then query all 100 and merge, so only do this for the few keys that need it and track which keys are split; or cache hot reads in front of the database. Also prefix timestamp keys with something like a sensor name to spread time-ordered writes. → *More:* Ch 6 — Partitioning

Q88. ⋆ What is consistent hashing? Consistent hashing maps both nodes and keys onto a hash ring; each key belongs to the next node clockwise. Adding or removing a node only moves the keys between it and its neighbour, instead of reshuffling nearly everything as `hash(key) mod N` does. **Virtual nodes** (each physical node owns many small points on the ring) smooth out the load and let a new node take a little data from every other node. Used by Dynamo-style stores (Cassandra, Riak), memcached client libraries and CDNs. The book notes the term is used loosely, and many databases actually use fixed partitions or ranges of hashes. → *More:* Ch 6 — Partitioning

Q89. Why is "hash mod N" a bad way to assign partitions to nodes? When N changes (you add a node), almost every key maps to a different node — with mod 10 → mod 11, about 90% of keys move. That is a massive, unnecessary data transfer. Better strategies move only the minimum data: a fixed number of partitions, dynamic partitioning, or consistent hashing. → *More:* Ch 6 — Partitioning

Q90. ⋆ Compare the rebalancing strategies: fixed partitions, dynamic partitioning, partitioning proportional to nodes. Fixed number of partitions: create many more partitions than nodes (say 1,000 for 10 nodes); a new node steals whole partitions from others; key-to-partition mapping never changes (Elasticsearch, Riak, Couchbase). Hard part: choosing the count up front. **Dynamic partitioning**: split a partition when it grows past a size (HBase 10 GB default) and merge small ones; adapts to data volume,

works with ranges (HBase, MongoDB). **Proportional to nodes**: a fixed number of partitions per node (Cassandra's tokens per node), so partition size grows with data and shrinks when nodes are added. → *More:* Ch 6 — Partitioning

Q91. ⋆ How do secondary indexes work with partitioned data? Local vs global indexes? A **local (document-partitioned) index** — each partition indexes only its own documents. Writes touch one partition, but a query by the indexed field must ask *every* partition and combine the results (**scatter/gather**), which suffers from tail latency (MongoDB, Cassandra, Elasticsearch). A **global (term-partitioned) index** — the index itself is partitioned by the indexed term (colour=red lives on one partition). Reads go to one index partition, but a single write may update several index partitions, so global indexes are usually updated **asynchronously** (DynamoDB's global secondary indexes). → *More:* Ch 6 — Partitioning

Q92. ⋆ How does a client know which node holds a given key (request routing)? Three approaches: (1) the client contacts any node, which forwards the request if needed (Cassandra and Riak, using a **gossip protocol** to share cluster state); (2) a **routing tier** that knows the partition map forwards requests (MongoDB's mongos); (3) clients know the assignment themselves. Someone must hold the authoritative map — often a coordination service like **ZooKeeper** (HBase, SolrCloud, older Kafka) or a Raft-based store (Kafka's built-in KRaft controllers; etcd, which Kubernetes uses) that notifies routers when partitions move. → *More:* Ch 6 — Partitioning

Q93. Should rebalancing be automatic or manual? Fully automatic rebalancing is convenient but risky: rebalancing moves lots of data, and if an overloaded node is wrongly judged dead, the automatic system may move load onto other nodes and cause a **cascading failure**. Many systems (Couchbase, Riak, Voldemort) generate a suggested plan automatically but require an administrator to commit it. A human in the loop is slower but avoids surprises. → *More:* Ch 6 — Partitioning

Q94. ⋆ How would you choose a shard key? A good shard key has high **cardinality** (many distinct values), spreads writes evenly (no monotonic timestamps or auto-increment IDs as a range key), and matches your main query pattern so most queries hit one shard (shard a multi-tenant SaaS by `tenant_id`, chat by `conversation_id`). Watch for very large tenants (split them further), cross-shard queries and transactions (keep data that is transacted together in one shard), and the cost of ever changing the key — resharding is a large migration. → *More:* Ch 6 — Partitioning

Q95. How do you run a query or join that spans many shards? For simple key lookups, route to one shard. For queries by a non-shard field, use scatter/gather over local indexes or a global index. Big analytic queries use **massively parallel processing (MPP)**: the query optimizer splits the query into stages that run in parallel on each partition, then combines results (Teradata, Redshift, Spark SQL, Trino). For joins, either co-partition both tables on the join key or broadcast the small table. If you need this often for OLTP, reconsider the shard key or copy data into an analytics store. → *More:* Ch 6 — Partitioning

Q96. Partitioning vs replication — how do they combine? They are independent: partitioning decides *which node holds which part* of the data, replication decides *how many copies* of each part exist. Usually each partition is replicated to several nodes, and each node holds several partitions — being the leader for some and a follower for others — so the load and the failure impact are spread out. → *More:* Ch 6 — Partitioning

Q97. How do you generate unique IDs across shards? Options: **UUIDs** (random 128-bit, no coordination, but random v4 UUIDs scatter inserts across a B-tree; time-ordered **UUIDv7** fixes that — PostgreSQL 18 added a built-in `uuidv7()` function); **Snowflake-style IDs** (64 bits = timestamp + machine ID + per-machine sequence; roughly time-sorted, compact, but depend on clocks); a central ticket server or database sequence (simple but a bottleneck/single point of failure); or per-shard ranges handed out in blocks. Choose based on whether you need ordering, size and coordination-free generation. → *More:* Ch 0a — System Design Fundamentals

Q98. How would you reshard a live database with no downtime? Typical plan: create the new shards; start **dual writes** or (better) stream changes from the old shards to the new ones using change data capture; **backfill** existing data with a snapshot plus the change stream from the snapshot's position; verify with checksums or comparison reads; move reads to the new shards gradually, then writes (often per tenant or key range, with a short write pause or a routing switch); keep the old shards for rollback. Tools like Vitess and Citus automate parts of this. Call out the hard parts: ordering between the backfill and live changes, and the cutover moment. → *More:* Ch 6 — Partitioning

Transactions and isolation anomalies

Q99. ⋆ What does ACID stand for, and what does each letter really mean? Atomicity: all writes in a transaction happen, or none do — if it fails midway, the database undoes the partial writes (it is really about *abortability*, not concurrency). **Consistency**: the application's invariants (for example, debits equal credits) hold — mostly the application's job, the database only enforces declared constraints. **Isolation**: concurrent transactions do not interfere; ideally the result is as if they ran one after another (serializability), though most databases default to weaker levels. **Durability**: once committed, data is not lost — written to disk (WAL + fsync) and/or replicated. The book notes "ACID" has become partly a marketing term, so ask what exactly a system guarantees. → *More:* Ch 7 — Transactions

Q100. What is BASE? BASE — Basically Available, Soft state, Eventual consistency — was coined as the opposite of ACID for systems that relax guarantees for availability and scale. In practice it only says "not ACID", so it is even vaguer than ACID. In an interview, say what each specific guarantee is rather than relying on either acronym. → *More:* Ch 7 — Transactions

Q101. Why would you need a multi-object transaction? Isn't single-row atomicity enough? Single-object operations (an atomic increment, a compare-and-set on one row) are enough for some tasks, and most stores provide them. But you need multi-object transactions when several pieces of data must stay in sync: a foreign key between two rows, a denormalized counter that must match the rows it counts, or secondary indexes that must match the table. Without them, a crash or a concurrent reader can see half-done changes. Many NoSQL systems dropped multi-object transactions for scale and then added them back (MongoDB 4.0+, DynamoDB TransactWriteItems). → *More:* Ch 7 — Transactions

Q102. Is it safe to just retry an aborted transaction? Usually, but not always. If the transaction actually committed and only the acknowledgment was lost, a retry runs it twice (unless you add idempotency/deduplication). If the error was overload, retrying makes it worse (use exponential backoff and a retry limit). Retry only transient errors (deadlock, serialization failure, timeout), not permanent ones (constraint violation). And side effects outside the database — sending an email — happen again on retry unless they are made part of an atomic commit or deduplicated. → *More:* Ch 7 — Transactions

Q103. ⋆ Name the isolation levels and the anomalies each one prevents.

Level	Dirty read	Dirty write	Non-repeatable read / read skew	Lost update	Phantom / write skew
Read uncommitted	possible	prevented	possible	possible	possible
Read committed	prevented	prevented	possible	possible	possible
Snapshot isolation (often called repeatable read)	prevented	prevented	prevented	depends on DB	write skew possible
Serializable	prevented	prevented	prevented	prevented	prevented

Defaults: PostgreSQL, Oracle and SQL Server use read committed; MySQL InnoDB uses repeatable read. Names differ between databases, so describe the anomaly, not just the level name. → *More:* Ch 7 — Transactions

Q104. ⋆ What is a dirty read and a dirty write? A **dirty read** is seeing another transaction's uncommitted data — which may then be rolled back, so you acted on data that never existed. A **dirty write** is overwriting another transaction's uncommitted value — for example two people buying the same car, where the listing goes to one buyer and the invoice to the other. Read committed prevents both: dirty writes with row-level locks held until commit; dirty reads by giving readers the last committed value (not by read locks, which would make readers wait behind long writes). → *More:* Ch 7 — Transactions

Q105. ⋆ What is read skew (non-repeatable read)? Give an example. Read skew is reading different parts of the database at different points in time. Example: Alice has $500 in each of two accounts; a transfer of $100 commits between her two reads, so she sees $500 in one and $400 in the other — $900 total, as if $100 vanished. Refreshing fixes it for her, but it is fatal for **backups** (a backup mixing

old and new parts) and **long analytic queries or integrity checks**. Snapshot isolation prevents it. → *More:* Ch 7 — Transactions

Q106. ⋆ What is snapshot isolation and how does MVCC implement it? Snapshot isolation lets each transaction read from a consistent **snapshot** of the database as it was when the transaction started, even while others change the data. **MVCC** (multi-version concurrency control) implements it by keeping several committed versions of each row, each tagged with the transaction ID that created it (and the one that deleted it). A transaction sees a version only if its creator committed before the transaction started and it was not deleted by such a transaction. Key property: **readers never block writers and writers never block readers**. Used by PostgreSQL, MySQL InnoDB, Oracle, SQL Server (optional). → *More:* Ch 7 — Transactions

Q107. Why does PostgreSQL need VACUUM? Under MVCC, an `UPDATE` writes a new row version and a `DELETE` just marks the old one as deleted; old versions stay in the table. **VACUUM** (normally autovacuum) removes versions no running transaction can see any more, so the space can be reused, and it also prevents **transaction-ID wraparound** (IDs are 32-bit). A long-running or idle-in-transaction session holds back the cleanup horizon and causes **table bloat**. MySQL InnoDB keeps old versions in its undo log instead and purges them in the background. → *More:* Ch 7 — Transactions

Q108. ⋆ What is a lost update? How do you prevent it? A **lost update** happens with a **read-modify-write cycle**: two transactions read the same value (counter = 42), each computes a new value (43), and the second write overwrites the first — one increment is lost. Prevention: **atomic operations** (`UPDATE counters SET value = value + 1`); **explicit locking** (`SELECT … FOR UPDATE`, then update); **automatic detection** — PostgreSQL's repeatable read and Oracle's serializable abort one of the transactions (MySQL InnoDB's repeatable read does not); **compare-and-set** (`UPDATE … SET v = new WHERE id = 1 AND v = old`, check rows affected = 1); in replicated systems, siblings plus merge or CRDTs. → *More:* Ch 7 — Transactions

Q109. ⋆ Optimistic vs pessimistic locking — when to use each? Pessimistic locking assumes conflicts: lock the row before you change it (`SELECT … FOR UPDATE`), others wait. Good when conflicts are frequent or retries are expensive. **Optimistic** locking assumes conflicts are rare: read a version number, then write with `WHERE version = ?` (or let the database detect conflicts at commit, as SSI does); on conflict, abort and retry. Good for low contention and long user "think time", where holding locks would be wasteful. Under high contention optimistic schemes waste work on retries.

```
UPDATE account SET balance = 80, version = version + 1
WHERE id = 7 AND version = 3;   -- 0 rows updated => someone else won, retry
```

→ *More:* Ch 7 — Transactions

Q110. ⋆ What is write skew? Give the on-call doctors example. Write skew happens when two transactions read the same data, each decides something based on it, and then they write to *different* rows — so neither overwrites the other, yet together they break an invariant. Example: the rule "at least one doctor on call". Alice and Bob are both on call and both feel sick; each transaction checks "are at least 2 doctors on call?" (yes), then sets *their own* row to off call. Both commit, and now no one is on call. Snapshot isolation does not prevent this, because the transactions update different objects. Other examples: double-booking a meeting room, two users claiming the same username, double-spending. → *More:* Ch 7 — Transactions

Q111. ⋆ What is a phantom? How do you prevent write skew without full serializability? A **phantom** is when a write in one transaction changes the result of a search query in another — typically, a row that did not exist yet when you checked "are there any bookings for this room at 12:00?". You cannot lock a row that does not exist with `SELECT … FOR UPDATE`. Options: lock the rows the check depends on (when they exist, like the doctors' rows); **materialize the conflict** — create a table of rows to lock (for example one per room per 15-minute slot) — an ugly last resort; add a **unique or exclusion constraint** (PostgreSQL `EXCLUDE USING gist (room_id WITH =, during WITH &&)` for overlapping time ranges; the `=` on a plain column needs the `btree_gist` extension); or simply use serializable isolation. → *More:* Ch 7 — Transactions

Q112. ⋆ What are the three ways to implement serializability?

1. **Actual serial execution**: run transactions one at a time on a single thread (VoltDB/H-Store, Redis, Datomic) — works if transactions are short, data fits in memory and are submitted as **stored procedures** (no interactive back-and-forth). 2. **Two-phase locking (2PL)**: readers and writers block

each other with shared and exclusive locks (MySQL InnoDB and SQL Server serializable). 3. **Serializable snapshot isolation (SSI)**: snapshot isolation plus detection of dangerous conflicts at commit (PostgreSQL serializable since 9.1, CockroachDB, FoundationDB). → *More:* Ch 7 — Transactions

Q113. ⋆ How does two-phase locking (2PL) work? Is it the same as two-phase commit? No — 2PL is about **isolation** on one database, 2PC is about **atomic commit** across several nodes (Q150). In 2PL, a transaction takes a **shared lock** to read and an **exclusive lock** to write; a writer waits for all readers and readers wait for writers. Locks are held until the transaction ends — the "two phases" are acquiring locks while running and releasing them all at the end. Phantoms are prevented with **predicate locks** or, in practice, **index-range (next-key) locks** that lock a range of the index. Costs: much lower throughput, unpredictable latency, and frequent **deadlocks** that the database detects and resolves by aborting one transaction. → *More:* Ch 7 — Transactions

Q114. ⋆ How does serializable snapshot isolation (SSI) work? SSI is **optimistic**: transactions run on a snapshot without blocking each other, and the database tracks what each one read and wrote. At commit it checks whether the transaction's premise is still valid: did its snapshot ignore another transaction's write that was still uncommitted at the time and has since committed (**stale MVCC read**), or did it read data that a concurrent transaction later wrote (**writes that affect prior reads**)? If such a dangerous pattern of dependencies is found, one transaction is aborted with a serialization failure and must be retried. It avoids 2PL's blocking, but high contention causes many aborts, so keep read-write transactions short. → *More:* Ch 7 — Transactions

Q115. What is the difference between serializability and linearizability? Serializability is an *isolation* property of *transactions* over many objects: the result equals *some* serial order, not necessarily the real-time order. **Linearizability** is a *recency* guarantee on *single objects*: once a write completes, every later read sees it, matching real time. A database providing both is **strict serializable** (strong one-copy serializability): implementations based on actual serial execution or 2PL typically are; SSI is not linearizable by design, because it reads from a snapshot. → *More:* Ch 9 — Consistency and Consensus

Q116. What does `SELECT … FOR UPDATE` do, and what are its pitfalls? It reads rows and takes exclusive (write) locks on them, so other transactions trying to lock or update them wait until you commit — preventing lost updates in a read-modify-write. Pitfalls: it cannot lock rows that do not exist yet (phantoms, unless the database uses next-key locks); locking rows in different orders causes deadlocks (always lock in a consistent order); and holding locks while waiting on the user or a network call kills throughput. `SKIP LOCKED` and `NOWAIT` variants help build job queues and fail-fast behaviour. → *More:* Ch 7 — Transactions

Q117. What is a deadlock in a database and how is it handled? A **deadlock** is two (or more) transactions each waiting for a lock held by the other, so none can continue. Databases detect the cycle in the waits-for graph (or use a timeout) and abort one transaction as the victim; the application must retry it. Reduce them by keeping transactions short, touching rows in a consistent order, and using the right indexes so fewer rows are locked. → *More:* Ch 7 — Transactions

Q118. ⋆ How would you implement a money transfer between two accounts correctly? In one database: one transaction that debits and credits, with a check that the balance does not go negative — `UPDATE accounts SET balance = balance - 100 WHERE id = A AND balance >= 100` (check 1 row affected), then credit B, commit; lock rows in ID order to avoid deadlocks. Better still, store transfers as an append-only **ledger** of entries and derive balances. Across services or databases, avoid distributed transactions: write the transfer request with a unique ID, process it idempotently in each step, and use a saga (Q187) or a log-based approach (Q189). Always make the API idempotent with a client-supplied request ID. → *More:* Ch 7 — Transactions

Q119. Why is "repeatable read" confusing across databases? The SQL standard's definition is based on the 1975 System R design and does not describe snapshot isolation, so vendors use the name differently. PostgreSQL's repeatable read is snapshot isolation (and detects lost updates). MySQL InnoDB's repeatable read uses a snapshot for plain reads but locking reads see the latest data, and it does not detect lost updates. Oracle calls its snapshot isolation "serializable". IBM DB2 uses "repeatable read" for serializability. Always ask which anomalies are actually prevented. → *More:* Ch 7 — Transactions

Q120. Why do stored procedures matter for serial execution, and what are their downsides? With one thread executing transactions, an interactive transaction (waiting for the app to send the next query

over the network) would stall everything. So serial-execution systems require the whole transaction to be submitted up front as a **stored procedure**. Downsides: vendor-specific languages (though VoltDB uses Java, Redis uses Lua), harder to debug, version and test, and a badly written procedure can hurt the whole database. To scale, the data is partitioned so each CPU core runs its own partition serially — but cross-partition transactions are much slower. → *More:* Ch 7 — Transactions

The trouble with distributed systems

Q121. ⋆ What is a partial failure, and why does it make distributed systems hard? On a single computer, things usually either work or crash completely. In a distributed system, some parts can be broken in unpredictable ways while others work fine — a **partial failure**. Worse, partial failures are **nondeterministic**: a request may or may not have succeeded, and you may not even know. So distributed software must assume faults and build reliability from unreliable parts. → *More:* Ch 8 — The Trouble with Distributed Systems

Q122. ⋆ You send a request and get no response. What could have happened? The request may have been lost; it may be waiting in a queue; the remote node may have crashed or paused (for example a long garbage-collection pause); the node may have processed the request but the response was lost or is delayed. The sender cannot tell these apart. The usual tool is a **timeout**, but after one you still do not know whether the action happened — which is why retries need **idempotency**. → *More:* Ch 8 — The Trouble with Distributed Systems

Q123. ⋆ How do you choose a timeout for failure detection? Too long: users wait and the system is down for longer before it fails over. Too short: a node that is merely slow (load spike, GC pause) is declared dead, its work moves to others, adding load that can cascade into more false "deaths" — and its action may run twice. Network delays are unbounded in practice (queueing in switches, OS, VMs, TCP retransmissions), so measure the distribution of round-trip times and set timeouts adaptively, as Cassandra and Akka do with the **Phi accrual failure detector**, which outputs a suspicion level rather than a yes/no. → *More:* Ch 8 — The Trouble with Distributed Systems

Q124. Why are datacentre networks asynchronous rather than synchronous like telephone circuits? A telephone call reserves fixed bandwidth along the whole path (a **circuit**), so delay is bounded. Internet and datacentre networks use **packet switching** optimised for **bursty** traffic: no reservation, packets share links and queue when links are busy, so delay is unbounded but utilisation is high. Bounded delay is possible (circuits, quality-of-service, admission control) but costs utilisation and money, so we accept variable delays and design for them. → *More:* Ch 8 — The Trouble with Distributed Systems

Q125. ⋆ Time-of-day clock vs monotonic clock — which do you use for measuring a duration? A **time-of-day (wall) clock** (`System.currentTimeMillis()`, `clock_gettime(CLOCK_REALTIME)`) returns the current date and time; NTP can step it forward or *backward*, so differences between two readings are unreliable. A **monotonic clock** (`System.nanoTime()`, `CLOCK_MONOTONIC`) always moves forward; only differences on the same machine are meaningful. So: monotonic for timeouts and durations, wall clock only when you need an actual date — and never compare wall-clock timestamps across machines for ordering. → *More:* Ch 8 — The Trouble with Distributed Systems

Q126. ⋆ Why can't we rely on synchronized clocks to order events across machines? Quartz clocks drift (Google assumes 200 ppm — about 17 seconds per day if not resynced); NTP synchronisation is limited by network delay (tens of milliseconds over the internet), can be misconfigured or firewalled, and steps clocks; VMs pause; leap seconds cause bugs. So two events on different nodes can get timestamps that contradict the order in which they really happened, and LWW then silently drops the later write. Use **logical clocks** (Lamport timestamps, version vectors) to order events instead. → *More:* Ch 8 — The Trouble with Distributed Systems

Q127. How does Google Spanner use TrueTime to get consistent snapshots? TrueTime returns an interval `[earliest, latest]` that is guaranteed to contain the real current time, using GPS receivers and atomic clocks in every datacentre to keep the uncertainty around 7 ms. When committing a read-write transaction, Spanner picks a timestamp and then deliberately **waits out the uncertainty** ("commit wait") before making the result visible. That guarantees that if transaction A finished before B started, A's timestamp is lower — so timestamps match real-time order and snapshots across datacentres are

consistent. The smaller the uncertainty, the shorter the wait. → *More:* Ch 8 — The Trouble with Distributed Systems

Q128. ⋆ What are process pauses, and why are they dangerous for leases and locks? A thread can be paused for seconds at any point: garbage-collection "stop-the-world" pauses, a VM being suspended or live-migrated, a laptop lid closed, OS context switches, swapping to disk, a `SIGSTOP`. The process does not notice. Danger: a node checks "my lease is valid for 10 more seconds", then pauses for 15 seconds, wakes up and writes as if it were still the leader — while another node has already taken over. You cannot fix this with more careful checking; you need fencing tokens (Q129). → *More:* Ch 8 — The Trouble with Distributed Systems

Q129. ⋆ What is a fencing token? Why isn't a distributed lock with a timeout safe on its own? A **fencing token** is a number that increases every time the lock service grants a lock or lease. Every write to the protected storage must include it, and the storage rejects any write with a lower token than one it has already seen. So if client 1 pauses, its lease expires, client 2 gets token 34 and writes, and client 1 wakes up and writes with token 33, the storage rejects it. A lock with a timeout alone (for example a basic Redis lock) cannot stop a paused client from writing after expiry. ZooKeeper's `zxid` or node version can serve as the token. → *More:* Ch 8 — The Trouble with Distributed Systems

Q130. What is a Byzantine fault? Do typical backend systems need to tolerate them? A **Byzantine fault** is a node that behaves arbitrarily — sending corrupted or contradictory messages, or lying on purpose. Byzantine fault-tolerant protocols (needing more than two-thirds of nodes honest) are used in aerospace and blockchains with mutually untrusting parties. Typical datacentre systems assume nodes are honest but may crash or be slow, because all nodes are controlled by one organisation; BFT would be too costly. Still, guard against weak forms of lying: checksums for corrupted packets, input validation for user data, multiple NTP servers. → *More:* Ch 8 — The Trouble with Distributed Systems

Q131. What system models do distributed algorithms assume? Timing models: **synchronous** (bounded delays, pauses and clock error — unrealistic), **partially synchronous** (usually behaves synchronously, sometimes exceeds bounds — the realistic one), **asynchronous** (no timing assumptions, no clocks at all). Node failure models: **crash-stop** (a node that fails is gone forever), **crash-recovery** (nodes may come back with their durable storage intact; the usual one), **Byzantine** (arbitrary). Algorithms are proved correct within a model; real systems also need to handle cases the model leaves out. → *More:* Ch 8 — The Trouble with Distributed Systems

Q132. What is the difference between safety and liveness properties? A **safety** property says nothing bad ever happens (for example, uniqueness of fencing tokens: no two clients get the same token); if violated, you can point to the moment it broke, and the damage cannot be undone. A **liveness** property says something good eventually happens (for example, a request eventually gets a response); it may not hold right now but could later. Distributed algorithms usually require safety to hold always, even if every node crashes, while liveness is only guaranteed under some conditions (for example a majority of nodes up and the network recovered). → *More:* Ch 8 — The Trouble with Distributed Systems

Q133. "The truth is defined by the majority" — what does that mean? A single node's view of the world can be wrong: after a pause or a network cut, it may still believe it leads while everyone else has already written it off. So decisions — who is leader, whether a node is dead, whether a write is committed — are made by a **quorum**, usually a majority of nodes. A majority is used because there can be only one majority at a time, so two conflicting decisions cannot both win. A node that the quorum declares dead must step down even if it feels fine. → *More:* Ch 8 — The Trouble with Distributed Systems

Q134. What is a network partition, and how common are they really? A **network partition** (netsplit) is when part of the network is cut off from the rest while the nodes on each side keep running. Studies cited in the book found frequent network faults even in carefully managed datacentres — about 12 per month in one mid-sized datacentre, misconfigured switches, even sharks biting undersea cables. So your software must have a defined behaviour during a partition, and you should test it (for example with Jepsen-style fault injection). Note a partition is different from a node crash: both sides keep running and may keep acting. → *More:* Ch 8 — The Trouble with Distributed Systems

Consistency and consensus

Q135. ⋆ What is linearizability? Linearizability (atomic consistency, strong consistency) makes a replicated system behave as if there were **only one copy of the data** and every operation took effect atomically at some instant between its start and end. Its key **recency guarantee**: once any read has returned a new value (or a write has completed), all later reads — by anyone — return that value or a newer one, never an older one. It is what you need for leader election, locks, uniqueness constraints and "is this seat still free" checks. → *More:* Ch 9 — Consistency and Consensus

Q136. ⋆ Which replication methods can be linearizable? Single-leader: potentially, if you read from the leader (or synchronously updated followers) and know for sure who the leader is — but split brain or async failover breaks it. **Consensus algorithms** (ZooKeeper, etcd): yes, they are designed for it (with care on reads). **Multi-leader**: no — concurrent writes in several places. **Leaderless (Dynamo-style)**: probably not; LWW with clocks breaks it, and even strict quorums with w + r > n can show non-linearizable behaviour unless readers do synchronous read repair and writers read a quorum first (and even then compare-and-set needs consensus). → *More:* Ch 9 — Consistency and Consensus

Q137. ⋆ Explain the CAP theorem. Why is "pick two of three" misleading? CAP says that when a **network partition** happens, a system must choose between **consistency** (here meaning linearizability) and **availability** (every request to a non-failed node gets a non-error response). Without a partition you can have both. "Pick 2 of 3" is misleading because partitions are not optional — they happen whether you like it or not — so the real choice is "**C or A when partitioned**". The book also points out CAP's definitions are narrow (one consistency model, one kind of fault) and it says nothing about latency or crashed nodes, so it is of little practical help for design; it mattered historically for opening up the design space. → *More:* Ch 9 — Consistency and Consensus

Q138. What is PACELC? PACELC extends CAP: if there is a **P**artition, choose **A**vailability or **C**onsistency; **E**lse (normal operation), choose **L**atency or **C**onsistency. It captures the cost CAP ignores — even without failures, strong consistency costs latency because replicas must coordinate. Examples: Cassandra and Dynamo-style stores are PA/EL; Spanner and traditional synchronous systems are PC/EC; MongoDB is often described as PA/EC by default. → *More:* Ch 0a — System Design Fundamentals

Q139. Why do many systems choose not to be linearizable even without partitions? Performance. Linearizability requires coordination, so response time is at least proportional to the uncertainty of network delays — Attiya and Welch proved this. The same reasoning applies inside one machine: CPU cores work against their own caches, so shared memory is not linearizable unless code adds memory barriers. So systems give it up for speed; partitions are a secondary concern. → *More:* Ch 9 — Consistency and Consensus

Q140. ⋆ Strong vs eventual vs causal consistency — explain the spectrum. Eventual consistency: if writes stop, replicas eventually converge; until then, any read may be stale and reads may go back in time. **Causal consistency**: operations that are causally related (a reply after the message it answers; a write after the read it was based on) are seen by everyone in that order, while concurrent operations may be seen in different orders. **Linearizability (strong)**: a single total order matching real time. Causal consistency is the strongest model that stays available during network partitions, making it an attractive middle ground; session guarantees (read-your-writes, monotonic reads) are pieces of it. → *More:* Ch 9 — Consistency and Consensus

Q141. ⋆ What are Lamport timestamps? How are they different from vector clocks? A **Lamport timestamp** is a pair (counter, node ID). Each node increments its counter on each event, and every message carries the sender's maximum counter; a receiver that sees a bigger counter jumps its own counter forward to it. Ordering by (counter, then node ID) gives a **total order consistent with causality**: if A happened before B, A's timestamp is smaller. But from two timestamps you cannot tell whether events were concurrent or causally related. **Vector clocks / version vectors** keep one counter per node and *can* detect concurrency, at the cost of more metadata. → *More:* Ch 9 — Consistency and Consensus

Q142. Why are Lamport timestamps not enough to enforce a uniqueness constraint? To decide "who gets username *alice*", a node needs to know it has seen *every* request with a smaller timestamp — but another node might have issued a smaller timestamp that has not arrived yet, so you would have to ask every node, and you are stuck if one is unreachable. The total order only becomes known *after the fact*. To

decide in real time you need **total order broadcast** (or consensus). → *More:* Ch 9 — Consistency and Consensus

Q143. What is total order broadcast and why is it equivalent to consensus? **Total order broadcast** (atomic broadcast) is a protocol for sending messages to all nodes with two guarantees: **reliable delivery** (if a message is delivered to one node, it is delivered to all) and **totally ordered delivery** (every node delivers messages in the same order). It is exactly what a replicated log needs: **state machine replication** — every replica applies the same deterministic writes in the same order. Solving total order broadcast lets you solve consensus (decide the first message in the log) and vice versa (run consensus for each log slot). ZooKeeper's Zab and etcd's Raft implement it. → *More:* Ch 9 — Consistency and Consensus

Q144. How can you build a linearizable compare-and-set (e.g. a unique username) on top of total order broadcast? Append a message "I claim username X" to the log; read the log until your message comes back; if the *first* claim for X in the log is yours, you got it — commit (and confirm to the user); otherwise abort. Every node sees the same order, so they all agree who won. Reads can be made linearizable by appending a "read" marker to the log and serving the read when it is delivered, or by waiting until you have caught up to the latest log position (etcd's quorum reads, ZooKeeper's `sync()`). → *More:* Ch 9 — Consistency and Consensus

Q145. ⋆ What is the consensus problem and where is it needed? **Consensus** means getting several nodes to agree on one value, with these properties: **uniform agreement** (no two nodes decide differently), **integrity** (no node decides twice), **validity** (the value decided was proposed by some node), and **termination** (every non-crashed node eventually decides). Needed for **leader election** (exactly one leader, no split brain), **atomic commit** (all nodes commit or all abort), **uniqueness constraints**, locks and leases, and **membership** (which nodes are alive). Consensus can tolerate up to f failures with $2f + 1$ nodes (a majority must be alive). → *More:* Ch 9 — Consistency and Consensus

Q146. ⋆ Explain how Raft works. Raft keeps a replicated log across (usually 3 or 5) servers. **Leader election**: time is split into numbered **terms**; a follower that hears no heartbeat within a randomized election timeout becomes a candidate, increments the term and asks for votes; a node votes for at most one candidate per term, and only for a candidate whose log is at least as up to date as its own; a majority of votes makes a leader. **Log replication**: the leader appends client commands to its log and sends them to followers (AppendEntries); an entry is **committed** once stored on a majority, then applied to the state machine. (Subtlety: a leader only counts replicas for entries of its *own* term; entries left over from earlier terms become committed indirectly when a newer entry after them commits.) A leader that sees a higher term steps down. Used by etcd, Consul, CockroachDB, TiKV, Kafka's KRaft and MongoDB's replica sets (a Raft-like protocol). → *More:* Ch 9 — Consistency and Consensus

Q147. Raft vs Paxos? Both solve consensus with majorities and an increasing number (Paxos **ballot/proposal numbers**, Raft **terms**; the book's general name is **epoch**). **Paxos** (Lamport) is the original; basic Paxos agrees on one value, Multi-Paxos on a log, and it is known for being hard to understand and leaving many practical details open. **Raft** (2014) was designed for understandability: a strong leader, explicit log-matching rules and a full spec including membership changes, so it became the common choice for new systems. **Zab** (ZooKeeper) and Viewstamped Replication are similar. All have the same core: a leader proposes, a quorum votes, and the epoch number prevents an old leader from overwriting newer decisions. → *More:* Ch 9 — Consistency and Consensus

Q148. Why do consensus clusters have an odd number of nodes? Consensus needs a **majority** to make progress. With 3 nodes the majority is 2, so 1 can fail; with 5, 2 can fail. A 4-node cluster also needs 3 for a majority and still tolerates only 1 failure — the extra node adds cost and latency but no fault tolerance (and in a 2+2 network split, neither side has a majority). So clusters use 3 or 5 nodes, rarely 7; more nodes make every write slower. → *More:* Ch 9 — Consistency and Consensus

Q149. What are the limits of consensus? Every decision needs a vote from a majority (synchronous replication), which costs latency. You need a strict majority alive, so a network partition can leave the minority side unable to work. Most algorithms assume a fixed set of nodes (dynamic membership is harder). Failure detection relies on timeouts, so in networks with highly variable delay you get frequent false leader elections, which can stall progress. Raft has also shown edge cases where one unreliable link causes leadership to bounce continuously. → *More:* Ch 9 — Consistency and Consensus

Q150. ⋆ How does two-phase commit (2PC) work? A **coordinator** (transaction manager) runs the commit across several **participants** (databases). **Phase 1 — prepare**: the coordinator asks every participant "can you commit?"; a participant that says *yes* must have made the writes durable and promises to commit if asked — it gives up the right to abort. **Phase 2 — commit/abort**: if all said yes, the coordinator writes the decision to its own log on disk (the **commit point**) and tells everyone to commit; if any said no or timed out, it tells everyone to abort. The coordinator retries the decision forever until every participant has received it. → *More:* Ch 9 — Consistency and Consensus

Q151. ⋆ What is the main problem with 2PC? What does "in doubt" mean? If the coordinator crashes after participants voted *yes* but before they hear the decision, those participants are **in doubt** (uncertain): they cannot commit or abort on their own, because the others might have done the opposite. They must hold their **row locks** and wait until the coordinator recovers — possibly for hours, blocking other transactions. So 2PC is a **blocking** atomic commit protocol, and the coordinator is effectively a single point of failure. If its log is lost, an admin may have to decide manually (**heuristic decisions**), which can break atomicity. → *More:* Ch 9 — Consistency and Consensus

Q152. Why doesn't three-phase commit (3PC) fix 2PC in practice? 3PC is non-blocking only if you assume bounded network delay and response times, so that a timeout reliably means a node has crashed. In real networks with unbounded delays, a timeout does not mean a crash, so 3PC cannot guarantee atomicity. A truly non-blocking atomic commit needs a perfect failure detector — which we do not have — or a consensus algorithm to replicate the coordinator's decision. → *More:* Ch 9 — Consistency and Consensus

Q153. Is 2PC the same as consensus? How do they differ? They are related but different. In 2PC, *every* participant must vote yes for commit — a single "no" or a single unreachable participant forces abort or blocking; consensus only needs a *majority*, and any proposed value can win. 2PC's coordinator is not elected and is a single point of failure; consensus algorithms elect a new leader automatically. Modern systems combine them: Spanner and CockroachDB run 2PC across shards where each participant (and the coordinator's state) is itself a Raft or Paxos group, so no single crash blocks the protocol. → *More:* Ch 9 — Consistency and Consensus

Q154. What are XA transactions and why are they unpopular? XA (X/Open eXtended Architecture) is a standard C API for 2PC across **heterogeneous** systems — for example a database and a message broker in one transaction (Java's JTA uses it). Problems: the coordinator often lives inside the application server, so its log becomes critical state and that server is a single point of failure; XA cannot detect deadlocks across systems and does not work with SSI; it is a lowest common denominator; and the performance cost is high (MySQL distributed transactions were reported as 10× slower). In-doubt transactions hold locks until resolved. So most teams prefer log-based integration and idempotency instead. → *More:* Ch 9 — Consistency and Consensus

Q155. ⋆ What does ZooKeeper (or etcd) actually provide? A small, fault-tolerant key-value store built on total order broadcast, designed to hold small amounts of coordination data in memory. Writes are linearizable; ZooKeeper serves reads from the local server, so they can be stale unless you call `sync()` first (etcd reads are linearizable by default). It offers: **linearizable atomic operations** (compare-and-set) for locks; **total ordering of operations** (the `zxid`, usable as a fencing token); **failure detection** via sessions and **ephemeral nodes** that disappear when a client's session dies; and **change notifications** (watches). Use cases: leader election, assigning partitions to nodes, service discovery, membership and configuration. You outsource consensus to it rather than implementing your own. Kafka 4.0 (2025) removed ZooKeeper and runs its own Raft-based controller quorum (KRaft) instead. → *More:* Ch 9 — Consistency and Consensus

Q156. How would you implement leader election in your service? Do not invent your own consensus. Use a coordination service: in ZooKeeper, every candidate creates an **ephemeral sequential** node under `/election`; the lowest sequence number is leader; each other candidate watches the node just ahead of it and checks again when it disappears. In etcd, use a lease plus a key with a transaction (or its built-in election API). In Kubernetes, use a `Lease` object. Then handle the hard part: the old leader may still be running after a pause, so pass a **fencing token** (term/epoch/zxid) with every write and have storage reject stale ones. → *More:* Ch 9 — Consistency and Consensus

Batch processing

Q157. ⋆ Services vs batch vs stream processing — what's the difference? Online services wait for a request and answer quickly; the measure is response time and availability. **Batch processing (offline)** runs a job over a large, **bounded** input (for example yesterday's logs) and produces output; the measure is **throughput**, and jobs typically run for minutes to days on a schedule. **Stream processing (near-real-time)** consumes **unbounded** input events shortly after they happen, with lower latency than batch. Batch and stream jobs produce **derived data** — data you could rebuild from the source of truth. → *More:* Ch 10 — Batch Processing

Q158. What is the Unix philosophy and how does MapReduce inherit it? Unix tools each do one thing well, read from stdin and write to stdout with a **uniform interface** (lines of text), don't modify their input, and can be composed with pipes (`cat log | awk … | sort | uniq -c | sort -rn | head`). Errors are easy to recover from because input is immutable. MapReduce is "Unix tools distributed across thousands of machines": mappers and reducers are composable functions, the distributed filesystem (HDFS) is the uniform interface, and input files are never modified. The difference: MapReduce writes intermediate output to files instead of streaming it through pipes. → *More:* Ch 10 — Batch Processing

Q159. ⋆ Explain how a MapReduce job works.

1. Input files on HDFS are split into blocks; the scheduler runs a **mapper** on each block, ideally on the machine that stores it (**putting the computation near the data**). 2. The mapper extracts key-value pairs from each record. 3. The **shuffle**: map output is partitioned by a hash of the key, sorted, and copied to **reducers**, so all values for one key arrive at the same reducer, in sorted order. 4. The reducer processes all values per key and writes the output to HDFS. Jobs are chained into **workflows** (Airflow, Oozie) where one job's output is the next one's input. → *More:* Ch 10 — Batch Processing

Q160. ⋆ How are joins done in batch processing? Sort-merge vs broadcast hash join? Reduce-side (sort-merge) join: mappers emit the join key from both datasets, the shuffle brings all records with the same key to one reducer (with a **secondary sort** so, say, the user record comes before their events), and the reducer joins them. Works on any data but is expensive (sorting and copying everything). **Map-side joins** avoid the shuffle when you know something about the data: **broadcast hash join** — the small dataset is loaded into an in-memory hash table on every mapper; **partitioned hash join** — both inputs are already partitioned the same way, so each mapper joins one matching pair of partitions; **map-side merge join** — both are also sorted. → *More:* Ch 10 — Batch Processing

Q161. How do you handle skew (a hot key) in a batch join? If one key has millions of records (a celebrity), the reducer that gets it takes far longer than the rest, and the job waits for it (a **straggler**). Fixes: **skewed join** (Pig) — sample to find the hot keys, spread their records over several reducers and send the other side of the join for those keys to all of them; Hive's skewed join stores hot keys separately and uses a map-side join for them; or a two-stage aggregation — first aggregate with a random suffix to spread the load, then combine. → *More:* Ch 10 — Batch Processing

Q162. Why should batch jobs write output to new files instead of writing directly into a production database? Writing one record at a time from thousands of tasks to a database is slow and can overload it, hurting online users; and if a task fails and reruns, the partial writes are already visible (side effects). Better: build a brand-new database file or index inside the job (like building Voldemort read-only stores, or a search index), write it to the distributed filesystem, then atomically switch the serving system to the new files. Output becomes immutable and reproducible — **human fault tolerance**: if you ship a bad job, rerun the fixed one or roll back to the old output. → *More:* Ch 10 — Batch Processing

Q163. Why did Spark (and Flink, Tez) replace MapReduce? MapReduce writes every intermediate result to HDFS (replicated) and every job must finish all mappers before reducers start, which wastes a lot of I/O and time in multi-step workflows. **Dataflow engines** (Spark, Flink, Tez) treat the whole workflow as one job — a graph of operators — keep intermediate state in memory or local disk, only sort when needed, schedule tasks with locality, and pipeline data between operators. Fault tolerance: Spark recomputes lost partitions from **lineage** (the recorded transformations of each RDD), Flink uses checkpoints. They also offer declarative high-level APIs (DataFrames, SQL) with a query optimizer. → *More:* Ch 10 — Batch Processing

Q164. How does a batch engine recover from a failed task, and why does determinism matter? A failed task is simply rerun on another machine from its input (MapReduce reruns from materialized intermediate files; Spark recomputes lost partitions from lineage). This only produces correct results if operators are **deterministic** — same input always gives same output. Nondeterminism (iterating a hash table in random order, using the current time or random numbers without a fixed seed) can make a recomputed partition differ from the original, so downstream tasks that already consumed the old version become inconsistent and must be rerun too. → *More:* Ch 10 — Batch Processing

Q165. Hadoop/data lake vs MPP database — what is the difference in philosophy? MPP databases (Teradata, Greenplum) require data to be loaded into a structured schema first and focus on running SQL fast. Hadoop takes the "**sushi principle**: raw data is better" approach — dump raw files of any format into the distributed filesystem quickly and decide how to interpret them later (schema-on-read), and run arbitrary code (machine learning, image processing), not just SQL. MapReduce also tolerates the failure of individual tasks at a fine granularity, which suits shared clusters where low-priority jobs get preempted. Today the two have converged (Spark SQL, Trino, lakehouses). → *More:* Ch 10 — Batch Processing

Q166. How are graph algorithms like PageRank run in batch? What is the Pregel model? Iterative graph algorithms repeat until convergence, which is very inefficient as a chain of MapReduce jobs (each iteration rereads the whole graph). The **Pregel (bulk synchronous parallel)** model: each vertex keeps state and in each iteration (**superstep**) processes messages sent to it and sends messages along edges; the framework handles partitioning, message delivery and checkpointing. Used in Apache Giraph, Spark GraphX and Flink Gelly. Cross-machine messages dominate the cost, so a graph that fits on one machine is often faster to process there. → *More:* Ch 10 — Batch Processing

Stream processing, CDC and event sourcing

Q167. ⋆ Message queue (RabbitMQ) vs log-based broker (Kafka) — what's the difference? A traditional **message broker** (RabbitMQ, ActiveMQ, Amazon SQS) assigns each message to a consumer and **deletes it once acknowledged**; consumers process in parallel without order guarantees across them, which is good for task queues where each message is expensive to process. A **log-based broker** (Kafka, Amazon Kinesis, Apache Pulsar) **appends** messages to a partitioned, persisted log; consumers read sequentially and track an **offset**; messages stay until retention expires, so you can **replay** them and many consumer groups read the same data independently. Order is guaranteed only within a partition, and parallelism is limited to the number of partitions. → *More:* Ch 11 — Stream Processing

Q168. ⋆ How does Kafka guarantee ordering, and how do you scale consumers? A Kafka **topic** is split into **partitions**; each partition is an ordered, append-only log. Messages with the same **key** go to the same partition, so events for one entity stay ordered. A **consumer group** shares the partitions: each partition is read by exactly one consumer in the group at a time, so the maximum parallelism equals the number of partitions (extra consumers sit idle). If processing one message is slow, it holds up the rest of its partition (**head-of-line blocking**). To scale, add partitions — but that changes the key→partition mapping for new messages. → *More:* Ch 11 — Stream Processing

Q169. What happens when consumers can't keep up with producers? Three options: **drop** messages; **buffer** them in a queue (and decide what happens when the buffer fills — memory, disk, or crash); or apply **backpressure / flow control** (block the producer, like Unix pipes and TCP do). Kafka's log is a large bounded disk buffer: a slow consumer just falls behind (its **consumer lag** grows) without affecting other consumers, until its offset points to data already deleted by retention. So you monitor consumer lag and alert on it. → *More:* Ch 11 — Stream Processing

Q170. ⋆ What is change data capture (CDC) and why use it instead of dual writes? CDC observes all changes written to a database (usually by reading its replication log — PostgreSQL logical decoding, MySQL binlog, MongoDB oplog) and publishes them as a stream that other systems (search index, cache, warehouse) consume in the same order. With **dual writes** — the application writes to the database and then to the search index — you get race conditions (two clients' writes arrive in different orders at the two systems, leaving them permanently inconsistent) and partial failures (one write succeeds, the other fails). CDC makes the database the single leader and everything else a follower. Tools: Debezium, Maxwell, LinkedIn Databus, Kafka Connect. → *More:* Ch 11 — Stream Processing

Q171. How do you bootstrap a new consumer of a CDC stream when the log doesn't go back to the beginning? Take a **consistent snapshot** of the database, remembering its exact position (offset) in the change log, load the snapshot into the new system, then apply the changes from that offset onward. Debezium does this automatically with its initial snapshot. Alternatively, use **log compaction** (Q172), so the log itself always holds the latest value of every key and a new consumer can start from offset 0. → *More:* Ch 11 — Stream Processing

Q172. What is Kafka log compaction? With **log compaction**, Kafka periodically removes older messages that have the same key as a newer one, keeping only the latest value per key; a message with a null value (a **tombstone**) means the key was deleted and is eventually removed too. The log then needs space proportional to the current database size, not the full history, and a new consumer can rebuild a full copy of the data by reading the topic from the start. It is how Kafka Streams stores state changelogs and how CDC topics can serve as a durable source of the latest state. → *More:* Ch 11 — Stream Processing

Q173. ⋆ What is event sourcing? How is it different from CDC? Event sourcing stores all changes to application state as an append-only log of **immutable events** that record *what happened* at the application level ("SeatReserved", "ItemRemovedFromCart"), and derives current state by replaying them. CDC captures low-level row changes from a mutable database that the application uses normally; event sourcing is designed in from the start and the events express user intent. Event sourcing distinguishes **commands** (requests that may be rejected after validation) from **events** (facts that have already happened and cannot be rejected). Snapshots of state avoid replaying the full history every time. → *More:* Ch 11 — Stream Processing

Q174. What is CQRS and how does it relate to event sourcing? CQRS (command query responsibility segregation) separates the model you write to from the model(s) you read from. Writes go to a log (or a normalized write store); one or more **read views** (search index, cache, denormalized tables) are derived from it and optimised for queries. Event sourcing is a natural fit because the event log is the write model and every read view is derived from it — and you can add a new view later by replaying the log. The cost: read views are updated asynchronously, so users may read stale data (handle read-your-writes, see Q71). → *More:* Ch 11 — Stream Processing

Q175. What are the advantages and challenges of keeping data as an immutable event log? Advantages: a full audit trail (accounting ledgers work this way), easier recovery from bugs (you can see and replay what happened), more information than current state alone (a cart item added then removed is useful for analytics), and multiple read views derived from the same log. Challenges: the log grows without bound (compaction and snapshots help), reading your own write in a derived view is asynchronous, and some data must truly be deleted — privacy rules such as GDPR, or illegal content — which needs special mechanisms like Datomic's **excision** or encrypting each user's data and destroying the key (crypto-shredding). → *More:* Ch 11 — Stream Processing

Q176. Event time vs processing time — why does it matter? Event time is when the event actually happened (recorded by the device); **processing time** is when the stream processor handles it. They can differ by seconds to days — queueing, network faults, mobile devices that were offline, a consumer restart, or reprocessing old data. Windowing by processing time produces wrong results (for example, an apparent spike in request rate after a backlog clears). Use event time for correctness, but then you must deal with clocks on devices being wrong: the book suggests logging the device's send time and the server's receive time to estimate and correct the device clock offset. → *More:* Ch 11 — Stream Processing

Q177. ⋆ What are the types of windows in stream processing? Tumbling window: fixed length, non-overlapping (every event in exactly one 1-minute window). **Hopping window**: fixed length, overlapping by a fixed step (5-minute windows every 1 minute). **Sliding window**: all events within some interval of each other (any 5-minute span). **Session window**: no fixed length; groups a user's events that are close together and ends after a period of inactivity (for example 30 minutes) — common for website analytics. → *More:* Ch 11 — Stream Processing

Q178. ⋆ How do you handle late-arriving (straggler) events? What is a watermark? You can never be sure all events for a window have arrived. Options: **ignore** late events (track how many you drop, alert if many), or **publish a correction** — an updated value for the window, retracting the earlier result. Frameworks like Flink and Beam use a **watermark**: a marker in the stream that says "I believe no more events with event time earlier than T will arrive"; when it passes the end of a window, the window is emitted, and an **allowed lateness** setting decides how long late events can still update it. (Watermark

is the frameworks' name; the book describes the same idea as declaring a window ready after a timeout.) → *More:* Ch 11 — Stream Processing

Q179. What are the types of stream joins? Stream–stream (window) join: match events from two streams within a time window — for example, join search events with click events on the same session ID within one hour; the processor keeps state of recent events from both sides. **Stream–table (enrichment) join**: enrich each event with data from a database table, for example add user profile fields to activity events; keep a local copy of the table updated by a CDC stream instead of querying the database remotely each time. **Table–table (materialized view maintenance) join**: both inputs are changelogs and the output is a continuously updated view — like Twitter's timeline cache, joining tweets with follows. → *More:* Ch 11 — Stream Processing

Q180. What is a slowly changing dimension, and why does it make stream joins nondeterministic? In a stream–table join, if the table changes over time (a user changes country, a tax rate changes), the result depends on whether the event is joined with the old or new version — and if events from the two streams interleave differently on a rerun, the output differs. The fix is to give each version of the record a unique ID (version the table), so each event joins with the version that was valid at its event time. Data warehouses call such tables **slowly changing dimensions (SCD)**. The downside is that you can no longer compact old versions away. → *More:* Ch 11 — Stream Processing

Q181. How do stream processors achieve fault tolerance? They cannot just rerun a failed task from the beginning — the stream never ends. Approaches: **microbatching** — split the stream into small batches (about 1 second) and treat each like a mini batch job (Spark Streaming); **checkpointing** — periodically write a consistent snapshot of operator state to durable storage, triggered by **barriers** flowing through the stream, and on failure restart from the last checkpoint (Flink); **rebuilding state** from a compacted changelog topic (Kafka Streams, Samza) or by replaying input. Inside the framework this gives exactly-once *effect*; side effects to external systems need extra care (Q183). → *More:* Ch 11 — Stream Processing

Q182. What are the main uses of stream processing? Complex event processing (CEP): search for patterns of events, like a stored query that runs continuously (fraud patterns). **Stream analytics**: aggregates and statistics over windows — rates, rolling averages, trends — often using probabilistic structures such as HyperLogLog for distinct counts. **Maintaining materialized views**: keeping caches, search indexes and derived tables in sync with a database. **Search on streams**: match each new document against stored queries (alerts for new listings matching a saved search). Plus event-driven workflows between services. → *More:* Ch 11 — Stream Processing

Q183. ⋆ At-most-once, at-least-once, exactly-once — what do they mean? At-most-once: each message is processed zero or one times — commit the offset first, then process; a crash loses messages. **At-least-once**: process first, then commit the offset; a crash after processing but before committing means the message is processed again — duplicates. **Exactly-once** (more precisely **effectively-once**): the final *effect* is as if each message was processed once — messages may physically be processed more than once, but duplicates leave no trace. You get it by making the output and the offset commit atomic, or by making processing **idempotent**. → *More:* Ch 11 — Stream Processing

Q184. ⋆ How does Kafka provide exactly-once semantics? Three pieces. **Idempotent producer** (`enable.idempotence=true`, the default since Kafka 3.0 together with `acks=all`, unless conflicting settings such as `acks=1` are configured): the broker uses a producer ID and per-partition sequence numbers to drop duplicate retries. **Transactions** (`transactional.id`): a producer atomically writes to several partitions *and* commits the consumer offsets in the same transaction — the read-process-write cycle becomes atomic. **Consumers with** `isolation.level=read_committed` see only committed transactional messages. Kafka Streams wraps this as `processing.guarantee=exactly_once_v2`. Limits: it covers Kafka-to-Kafka; writing to an external database still needs idempotent writes or storing the offset in that database's transaction. These features came after the book, which describes the same idea of keeping the atomic commit internal to the stream system. → *More:* Ch 11 — Stream Processing

Exactly-once, idempotency and data integration

Q185. ⋆ How do you make an operation idempotent? An **idempotent** operation has the same effect whether applied once or many times. Some are naturally idempotent (set a key to a value, delete by ID). For others (charge a card, increment a counter), have the client generate a unique **request ID / idempotency**

key (a UUID) once per logical operation and send it with every retry; the server stores processed IDs — ideally in the same transaction as the effect, with a unique constraint — and returns the stored result for duplicates. Stripe-style APIs work this way. For stream consumers, store the last processed offset or event ID together with the output.

```
INSERT INTO payments (request_id, account, amount) VALUES ($1, $2, $3)
ON CONFLICT (request_id) DO NOTHING;   -- retry = no second payment
```

→ *More:* Ch 12 — The Future of Data Systems

Q186. ⋆ What is the transactional outbox pattern? It solves the dual-write problem when a service must update its database *and* publish an event. In the **same local transaction** as the business change, insert the event into an `outbox` table. A separate relay process (polling, or CDC via Debezium reading the outbox table) publishes the outbox rows to the broker and marks them sent. Either both the change and the event are committed, or neither; publishing is at-least-once, so consumers must be idempotent (dedupe by event ID). The book does not use the name "outbox" but describes the same principle: write once to a log-like source of truth, derive everything else from it. → *More:* Ch 12 — The Future of Data Systems

Q187. ⋆ What is a saga, and when do you use it instead of 2PC? A **saga** splits a business transaction across services into a sequence of local transactions, each with a **compensating action** that semantically undoes it (refund the payment, release the seat) if a later step fails. It can be **orchestrated** (a central coordinator tells each service what to do) or **choreographed** (each service reacts to events). Use it across microservices where 2PC is unavailable or would block; the cost is no isolation — other transactions can see intermediate states — so design for it (pending states, idempotent steps, retries). The book's equivalent idea is multi-stage processing of a request through a log (Q189). → *More:* Ch 0a — System Design Fundamentals

Q188. Lambda vs Kappa architecture? Lambda architecture runs two pipelines on the same immutable input: a batch layer (Hadoop/Spark) that produces correct results slowly, and a speed layer (stream processor) that produces approximate results quickly; queries merge both. Problems: you maintain the same logic twice in two frameworks, and merging outputs is hard. **Kappa architecture** uses only a stream processor and a replayable log: to reprocess, start a new job reading the log from the beginning, then switch over. The book favours unifying batch and stream in one system that can do both. → *More:* Ch 12 — The Future of Data Systems

Q189. ⋆ How can you enforce a constraint like "a username must be unique" in a log-based (event-driven) system? Partition the log by the value that must be unique (the username). A single-threaded processor for each partition reads requests in log order; the first request for a username wins and the processor emits "success" or "rejected" events. Because there is a total order per partition, no consensus beyond the log itself is needed. Similarly, a transfer between accounts can be one request event, processed in stages (debit A's partition, credit B's partition), each idempotent by request ID — giving correctness without an atomic multi-partition commit. → *More:* Ch 12 — The Future of Data Systems

Q190. What is the end-to-end argument for databases? Even if each component (TCP, the database's transactions, the stream processor) is exactly-once internally, the system as a whole can still produce duplicates — for example, a user's browser resubmits a form after a timeout, and the server cannot know it is the same request. Duplicate suppression must therefore be done **end to end**: a unique ID generated by the client, carried through every layer and checked at the final destination. Low-level guarantees help performance but cannot replace application-level checks. → *More:* Ch 12 — The Future of Data Systems

Q191. What is the difference between timeliness and integrity? Timeliness means users see the system in an up-to-date state; violations are temporary (stale reads that eventually catch up). **Integrity** means no corruption — no lost or contradictory data, no false information; violations are permanent and need manual repair. The book argues integrity matters much more than timeliness — a card statement missing yesterday's purchase is normal, but a statement whose total does not match its transactions is a serious bug — and that event-based systems can guarantee integrity without coordination, accepting weak timeliness — while traditional ACID transactions tie both together. → *More:* Ch 12 — The Future of Data Systems

Q192. Why do systems often check constraints after the fact rather than before? Many business constraints can tolerate a short violation if you can **apologise and compensate** — airlines overbook flights, warehouses oversell and backorder, banks charge overdraft fees. Checking strictly up front needs coordination (linearizability or consensus), which costs latency and availability. So you may process optimistically, detect violations asynchronously, and run a compensating action — as long as integrity (no lost or duplicated money) is preserved. → *More:* Ch 12 — The Future of Data Systems

Q193. What does "trust, but verify" mean for data systems? Do not assume hardware and software never corrupt data: disks have bit rot, databases have bugs (Jepsen has found consistency bugs in many products), and backups may not be restorable. Build **auditing** in: checksums, periodic comparison of replicas or derived data with the source, end-to-end integrity checks, and actually testing restores. Event-sourced systems make auditing easier because you can replay the log and check that derived state matches; Merkle trees and cryptographic logs help verify large datasets cheaply. → *More:* Ch 12 — The Future of Data Systems

Q194. What is "unbundling the database"? A database bundles features internally: secondary indexes, materialized views, replication logs, caches. "Unbundling" means building the same features across *different* specialised systems (a database, a search index, a cache, a warehouse) connected by a durable, ordered log (Kafka) and stream processors that keep derived views up to date — like a database's internal index maintenance, but at the scale of the whole organisation. The log is the source of truth; derived systems can be rebuilt by replaying it; and **dataflow** replaces dual writes. → *More:* Ch 12 — The Future of Data Systems

Scenario and design questions

Q195. ⋆ Your read replicas are lagging and users see stale data after updating their profile. What do you do? First measure: replication lag metrics and their cause (a long transaction, heavy writes, a slow replica, network). Short-term product fix: **read-your-writes** — route reads of a user's own data to the leader for a short time after they write, or track the write's log position and only read from replicas that have caught up. Longer term: reduce write load on the leader (batching, removing unused indexes), scale replicas, or for critical paths read from the leader. Also check monotonic reads (pin users to one replica). → *More:* Ch 5 — Replication

Q196. ⋆ Design a distributed key-value store (like Dynamo or Cassandra). Clarify: size, read/write ratio, consistency needs, latency. Core design: **partition** with consistent hashing and virtual nodes; **replicate** each key to $n = 3$ nodes on the ring (across racks/zones); **leaderless writes** with tunable quorums (w, r); **conflict handling** with version vectors and siblings, or LWW for append-only data; **sloppy quorum and hinted handoff** for availability; **read repair and anti-entropy** with Merkle trees; **gossip** for membership and failure detection (phi accrual); each node's storage engine is an LSM-tree with WAL, memtable, SSTables, Bloom filters and compaction. Discuss trade-offs: AP vs CP, what happens during a partition, hot keys. → *More:* Ch 5 — Replication

Q197. ⋆ Which database would you choose for X? (how to answer) Do not name a product first. Ask about the **access patterns** (key lookups vs range scans vs ad-hoc analytics vs graph traversals), **read/write ratio**, **consistency and transaction needs** (money, inventory), **data size and growth**, **latency targets**, and the team's operational experience. Then map: relational (PostgreSQL) as a strong default for OLTP with joins and transactions; document stores for self-contained aggregates; wide-column/LSM (Cassandra) for very high write throughput with known queries; a column store or warehouse for analytics; a search engine for text; Redis for caching; a graph DB for deep relationship queries. Mention polyglot persistence kept in sync by CDC. → *More:* Ch 2 — Data Models and Query Languages

Q198. ⋆ Design a system that keeps a search index (or cache) consistent with the primary database. Avoid dual writes from the application. Make the database the source of truth and capture its changes with **CDC** (Debezium → Kafka), partitioned by primary key so updates to a row stay ordered. A consumer applies changes to Elasticsearch/Redis idempotently (upsert by ID, ignore events with an older version or log position). Bootstrap with a snapshot plus the log offset. To rebuild or change the index mapping, replay from a compacted topic into a new index and switch an alias. Monitor consumer lag and run periodic reconciliation jobs that compare counts or checksums. → *More:* Ch 11 — Stream Processing

Q199. ⋆ How would you prevent double-booking of a seat/room/appointment? Options in increasing complexity: a **unique constraint** on (resource, slot) so the second insert fails; `SELECT … FOR UPDATE` on the resource row before checking availability; an **exclusion constraint** for overlapping time ranges in PostgreSQL; **serializable** isolation with retries; for high demand (ticket sales), a short **hold/reservation** with an expiry written atomically, plus a queue in front; in a log-based design, partition requests by resource and process them serially. Name the anomaly — write skew with phantoms — to show you understand why snapshot isolation alone fails. → *More:* Ch 7 — Transactions

Q200. ⋆ A cron job runs on two servers at once after a failover and processes everything twice. How do you fix it? The root cause is two nodes believing they are the leader (a pause or network partition plus timeout-based failover). Fixes: run the scheduler with **leader election** from a coordination service (ZooKeeper, etcd, a Kubernetes Lease); protect the side effects with **fencing tokens** so the stale leader's writes are rejected; and make the job **idempotent** — for example, record each processed item with a unique key in the same transaction as its effect, so a second run does nothing. Never rely on "lock with TTL" alone. → *More:* Ch 8 — The Trouble with Distributed Systems

Q201. ⋆ Design a pipeline that counts page views per minute and is correct under failures and late data. Clients send events with an **event-time** timestamp and a unique **event ID** to an ingestion service that writes to Kafka (partitioned by page ID). A stream processor (Flink / Kafka Streams) does **tumbling 1-minute windows by event time** with a watermark and allowed lateness; late events produce corrected counts (upserts). State is checkpointed; output goes to a store with **idempotent upserts** keyed by (page, minute), so replays do not double count; dedupe on event ID if the client may resend. For exact historical numbers, a nightly batch job recomputes from the raw log and overwrites. Mention approximate distinct counts with HyperLogLog if needed. → *More:* Ch 11 — Stream Processing

Q202. Your database is too slow for writes at peak. Walk through how you'd scale it. Measure first: which queries, locks, I/O or CPU dominate. Then, roughly in order: fix queries and indexes (remove unused indexes — each one slows writes); batch writes and use group commit; move non-critical work to asynchronous queues; scale up the machine (faster disks/SSD); separate reads onto replicas so the primary only takes writes; use a write-optimised (LSM) store for append-heavy data like logs and events; and finally **partition** by a good shard key. State the cost of each step — sharding loses easy joins and cross-shard transactions. → *More:* Ch 6 — Partitioning

Q203. How would you migrate from one database to another with zero downtime?

1. Set up CDC from the old database and do an initial snapshot load into the new one. 2. Keep it in sync from the change stream and verify continuously (row counts, checksums, **shadow reads** comparing results). 3. Switch reads gradually (per feature or percentage) while writes still go to the old system. 4. Switch writes — briefly pause or use a routing flag — and optionally stream changes back to the old database for rollback. 5. Decommission after a safe period. Schema differences are handled in the transformation step; encoding compatibility (Q51) matters throughout. → *More:* Ch 12 — The Future of Data Systems

Q204. ⋆ Explain how you'd make a payments flow safe against retries, timeouts and crashes. Client generates an **idempotency key** per payment attempt; the API stores it with a unique constraint and the request's result. Write the payment intent and an outbox event in one local transaction. Call the payment provider with the same idempotency key (providers support this), and on a timeout **query the status** instead of blindly retrying. Use a state machine (created → authorised → captured → refunded) with only valid transitions, a double-entry ledger, and a reconciliation job that compares your ledger with the provider's reports. This is the end-to-end argument in practice. → *More:* Ch 12 — The Future of Data Systems

Q205. ⋆ What happens to a Raft/ZooKeeper cluster of 5 nodes when the network splits 3–2? The side with 3 nodes is a majority: it keeps (or elects) a leader and continues serving writes and linearizable reads. The 2-node side cannot form a majority, so it cannot elect a leader or commit writes; a leader stranded there stops being able to commit and steps down when it sees a higher term after the partition heals, or sooner if the implementation uses a check-quorum rule as etcd does (its uncommitted entries are discarded). Clients connected to the minority get errors or timeouts — the system chose **C over A** for them. When the network heals, the minority nodes catch up from the leader's log. → *More:* Ch 9 — Consistency and Consensus

Q206. A senior wrap-up: what are the most important trade-offs in data-intensive systems? Reads vs writes (indexes, LSM vs B-tree, normalization vs denormalization); consistency vs latency and availability (linearizability, CAP/PACELC, sync vs async replication); isolation vs throughput (read committed vs snapshot vs serializable); coordination vs scalability (consensus and 2PC vs partitioned logs and idempotency); timeliness vs integrity; and simplicity vs features (one database vs many derived systems). A strong candidate names the trade-off, the workload facts that decide it, and how they would detect and recover when the chosen weakness shows up. → *More:* Ch 12 — The Future of Data Systems

Sources of the question lists

Question wording and frequency were checked against these public lists and discussions (titles only; all answers above are written for this book):

- LeetCode Discuss — "How is *Designing Data Intensive Applications* for Sys Design interview" and system-design / backend interview-experience posts (leetcode.com/discuss)
- donnemartin/system-design-primer — sections on CAP, consistency patterns, availability patterns, replication, sharding (github.com/donnemartin/system-design-primer)
- Public Notion and study-note pages for *Designing Data-Intensive Applications* (chapter-by-chapter notes and question lists)
- DesignGurus — "Explain LSM tree vs B-tree"; PracHub — "B+ Tree vs LSM Tree for Database Interviews"
- TechInterview.org — "What isolation levels really guarantee", MVCC and LSM-tree internals posts
- Educative and CoddyKit — transaction isolation levels; dirty, non-repeatable and phantom reads
- DataVidhya and DataDriven — Kafka / streaming data-engineer interview questions (exactly-once, consumer groups, CDC)
- System Design Handbook — "Distributed systems design interview questions"; DEV Community — consensus algorithms for system design interviews
- Five version facts were checked against official sources in October 2026: ZooKeeper removal in Kafka 4.0 (Kafka 4.0.0 release announcement), the idempotent-producer default since Kafka 3.0 (KIP-679 and the producer config reference), MongoDB's implicit `w: "majority"` default since 5.0 and the map-reduce deprecation in 5.0 (MongoDB manual), and `uuidv7()` in PostgreSQL 18 (PostgreSQL 18 docs). The isolation-anomaly behaviour in Q103, Q108 and Q110 was tested on PostgreSQL 16. Other version and product statements follow the book or general knowledge and were not each re-checked against vendor docs.